

Teoría de Autómatas y Lenguajes Formales

Alvaro E. Campos
Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Marzo 1995

Contents

0	PROLOGO	5
0.1	¿Qué es un Lenguaje?	5
0.2	Sintaxis versus Semántica	5
0.3	Los Problemas a Estudiar	6
0.4	Aplicación a Otros Problemas	6
0.5	Clases de Lenguajes	7
0.6	Otros Problemas a Estudiar	8
0.7	Problemas No Decidibles	8
1	MATEMÁTICAS BÁSICAS	9
1.1	Conjuntos	9
1.1.1	Operaciones con Conjuntos	10
1.1.2	Conjuntos Infinitos	11
1.2	Inducción Matemática	12
1.2.1	Otras Bases	13
1.2.2	Inducción Completa	14
1.2.3	Definiciones Inductivas	15
1.3	Grafos y Árboles	17
1.3.1	Grafos Dirigidos	17
1.3.2	Árboles	18
1.4	Relaciones Binarias	19
1.4.1	Propiedades	20
1.4.2	Relaciones de Equivalencia	21
1.4.3	Clausuras	22
2	LENGUAJES FORMALES	25
2.1	Símbolos y Alfabetos	25
2.2	Palabras	25
2.2.1	Longitud de una Palabra	26
2.2.2	Concatenación	27
2.2.3	Subpalabras, Prefijos y Sufijos	28
2.2.4	Reverso	28
2.3	Lenguajes	29
2.3.1	Concatenación de Lenguajes	30
2.3.2	Clausuras	31
2.3.3	Representación de Lenguajes	32
2.4	Autómatas	33

3	ACEPTACIÓN Y GENERACIÓN DE LENGUAJES REGULARES	35
3.1	Autómatas Finitos Determinísticos	35
3.2	Autómatas Finitos No Determinísticos	39
3.3	Autómatas Finitos con Transiciones en Vacío	44
3.4	Teorema de Myhill-Nerode.	47
3.5	Minimización de Autómatas Finitos.	50
3.6	Traductores de Estado Finito	54
3.7	Expresiones Regulares	56
3.8	Aplicaciones de los Lenguajes Regulares	65
4	PROPIEDADES DE LOS LENGUAJES REGULARES	67
4.1	Lema de Bombeo para Conjuntos Regulares	67
4.2	Propiedades de Clausura	69
4.3	Algoritmos de Decisión	72
5	ACEPTACIÓN Y GENERACIÓN DE LENGUAJES LIBRES DE CONTEXTO	75
5.1	Autómatas Apiladores	75
5.2	Definiciones	76
5.3	Gramáticas Libres de Contexto	80
5.4	Configuración de las Gramáticas Libres de Contexto	81
5.5	Árboles de Derivación	83
5.6	Simplificación de Gramáticas	88
5.7	Formas Normales	91
5.8	Equivalencia entre LLC y Autómatas Apiladores	96
5.9	Ambigüedad Inherente	101
6	PROPIEDADES DE LOS LENGUAJES LIBRES DE CONTEXTO	105
6.1	Lema de Bombeo para Lenguajes Libres de Contexto	105
6.2	Propiedades de Clausura	109
6.3	Algoritmos de Decisión	113
7	ACEPTACIÓN Y GENERACIÓN DE LENGUAJES ENUMERABLES RECURSIVAMENTE Y LENGUAJES RECURSIVOS	117
7.1	Algoritmos	117
7.2	Modelo de la Máquina de Turing	118
7.3	Técnicas para la construcción de Máquinas de Turing	120
7.3.1	Almacenamiento en el Control Finito	120
7.3.2	Pistas Múltiples	121
7.3.3	Marcas Símbolos	121
7.3.4	Correr Símbolos	122
7.3.5	Subrutinas	122
7.4	Lenguajes y Funciones Computables	122
7.5	Extensiones al Modelo	124
7.5.1	Cinta Infinita en Ambas Direcciones	124
7.5.2	Máquinas de Turing con Varias Cintas	126
7.5.3	Movidas No Determinísticas	127
7.5.4	Máquinas Multidimensionales	127
7.5.5	Máquinas de Varias Cabezas	128
7.5.6	Máquinas Off-Line	128
7.6	Hipótesis de Church	129
7.7	Máquinas de Turing como Generadores	130

8	PROPIEDADES DE LOS LENGUAJES ENUMERABLES RECURSIVAMENTE Y RECURSIVOS	133
8.1	Algunas Propiedades	133
8.2	Máquina de Turing Universal	135
9	INDECIDIBILIDAD	139
9.1	Problemas	139
9.2	Otros Problemas No Decidibles	140

Chapter 0

PROLOGO

Como lo sugiere el nombre de estos apuntes: *Teoría de Autómatas y Lenguajes Formales*, el tema central a estudiar en ellos será el concepto de lenguaje. En particular, este concepto se analizará desde el punto de vista de su aplicación a problemas computacionales y se verán distintos dispositivos y algoritmos para trabajar con ellos. Se estudiarán lenguajes de distinto grado de complejidad, los cuales requieren de dispositivos cada vez más sofisticados para manejarlos. El objetivo de esta pequeña introducción es mostrar, en forma muy somera y general, los distintos aspectos en que se concentrarán estos apuntes.

0.1 ¿Qué es un Lenguaje?

Enfrentados a esta pregunta, se trata, en lo posible, de encontrar una respuesta que presente una definición amplia del concepto de lenguaje. De tal forma que ella sea suficiente para abarcar los lenguajes naturales como Castellano, Inglés y Japonés; los lenguajes de programación, como COBOL, Pascal y PROLOG; y además, cualquier otro lenguaje conocido, como las fórmulas bien formadas del cálculo de predicados de primer orden, o como las ecuaciones que representan reacciones químicas posibles.

Un lenguaje (*formal*) se define como un conjunto, ya sea finito o infinito, de sentencias construidas a partir de un conjunto finito de elementos llamados símbolos. Cada una de las sentencias de un lenguaje es una secuencia con un número finito de estos símbolos.

Todos los lenguajes naturales, ya sea en su forma hablada o escrita, son lenguajes según esta definición. Cada sentencia de ellos está construida por un número finito de elementos, sean éstos fonemas, palabras, letras u otros símbolos. Aún cuando en principio hay un número infinito de sentencias posibles, cada sentencia se puede representar por una secuencia finita de esos elementos.

En forma similar, las sentencias posibles en un lenguaje de programación, es decir, los programas escritos en ese lenguaje, se construyen de palabras reservadas, letras, dígitos y otros símbolos especiales. Cada programa contiene un número finito de ellos, aunque hay un número infinito de programas posibles de ser escritos en cada lenguaje.

0.2 Sintaxis versus Semántica

La noción intuitiva de lenguaje, que se ha formalizado en forma simple más arriba, tiene dos componentes básicos:

Sintaxis Principios y procesos que permiten combinar los símbolos para formar las sentencias de un lenguaje particular. Corresponde a la pregunta: ¿Qué es gramaticalmente correcto?

Semántica Mecanismo subyacente a través del cual se le asigna un significado a las sentencias de un lenguaje particular. Corresponde a las preguntas: ¿Qué significa esta sentencia? ¿Qué sentencias tienen sentido?

Es claro que la noción de lo que es gramaticalmente correcto (sintaxis), es independiente de si la sentencia tiene sentido o no (semántica). Sin embargo, para que una sentencia tenga sentido, ella debe ser gramaticalmente correcta. Por ejemplo, considere las siguientes sentencias en Castellano y Pascal:

1. Las manzanas ultravioletas duermen velozmente.
2. Ultravioletas velozmente las duermen manzanas.
3. `X := SQRT(ch)` {ch es de tipo char }
4. `:= SQRT ) ch X (`

Las sentencias (1) y (3) son gramaticalmente correctas en Castellano y Pascal respectivamente, es decir, están construidas de acuerdo a las reglas sintácticas de dichos lenguajes. Empero, ninguna de ellas tiene sentido. Por el contrario, las sentencias (2) y (4) no son gramaticalmente correctas, porque no tienen la estructura de una sentencia en Castellano o Pascal. Por supuesto, tampoco es posible asignarles un significado dentro de sus respectivos lenguajes.

0.3 Los Problemas a Estudiar

Estos apuntes se referirán exclusivamente a la sintaxis de los lenguajes. El problema fundamental al analizar cualquier lenguaje, ya sea Castellano, Pascal u otro, será separar las sentencias gramaticalmente correctas: las sentencias del lenguaje, de aquellas que no son correctas: las que no pertenecen al lenguaje.

Desde este punto de vista, hay dos preguntas distintas, pero íntimamente relacionadas, que es conveniente hacer sobre un lenguaje dado:

Aceptación: Dado un lenguaje, ¿qué clase de dispositivo mecánico puede construirse, ya sea una máquina o un algoritmo, de manera que lea secuencias de símbolos — candidatos a sentencias— e indique si son o no sentencias del lenguaje?

Cuando las personas escuchan hablar a alguien o cuando leen un libro, intuitivamente pueden determinar si una sentencia es, o no, gramaticalmente correcta. El analizador sintáctico de un compilador determina mecánicamente las sentencias correctas de un lenguaje de programación, es decir, los programas sintácticamente válidos.

Generación: Dado un lenguaje, ¿es posible construir un dispositivo que liste todas las sentencias de ese lenguaje y solamente éstas? O equivalentemente, ¿existe un modelo matemático que especifique cómo generar todas y sólo esas sentencias, de forma que pueda implementarse como un programa de computador que liste sólo sentencias correctas; el cual, dándole el tiempo suficiente, llegue a producir cualquier sentencia en particular?

El problema de generación no corresponde realmente a hablar, como el problema de aceptación corresponde a escuchar o leer; ni siquiera corresponde a escribir sentencias. Es claro que para escribir programas o frases en su lenguaje nativo, la gente no lista todas las sentencias válidas hasta obtener la que desea. Sin embargo, un mecanismo de generación es una manera formal de especificar el lenguaje mismo. Describir la sintaxis de un lenguaje de programación como Pascal, usando simplemente Castellano, requiere de un libro completo. Pero si se conoce el simbolismo matemático de un mecanismo de generación es posible describirla en sólo algunas páginas, como sucede si, por ejemplo, se usa BNF.

0.4 Aplicación a Otros Problemas

Hasta aquí se ha visto que al estudiar los mecanismos de aceptación y generación, es posible lograr un mejor manejo del análisis de los lenguajes naturales, de programación o de otro tipo. Sin embargo, es posible que el

uso de la palabra lenguaje sea, a primera vista, demasiado restrictiva. Las ideas enunciadas para lenguajes tienen también otras derivaciones de importancia.

Por ejemplo, de acuerdo con la definición de lenguaje en uso, es posible definir el siguiente lenguaje:

$$L_+ = \{X\#Y\#Z/ X, Y \text{ y } Z \text{ son enteros no negativos tales que } Z = X + Y\}$$

Nótese que L_+ es un conjunto infinito de sentencias. Cada una de ellas tiene longitud finita y está construida por elementos tomados del conjunto finito de símbolos: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \#\}$, es decir, L_+ es un lenguaje. En realidad, el lenguaje L_+ expresa la función suma entre números enteros no negativos.

Esta misma idea puede extenderse a cualquier función binaria. Para cada función binaria, f , es posible definir el lenguaje:

$$L_f = \{X\#Y\#Z/ X, Y \text{ y } Z \text{ son enteros no negativos tales que } Z = f(x, y)\}$$

Y, en general, también es posible extender esta idea a funciones de uno o más argumentos, con tan solo usar el número apropiado de símbolos “#” como separadores. Más aún, también es posible extenderla a dominios que no sean el de los números enteros no negativos.

Por sobre todo, la definición de los lenguajes L_f y la construcción de mecanismos de aceptación para ellos, es una forma de estudiar algoritmos para estudiar la función f . En particular, si se tiene un dispositivo que acepta todas y sólo las sentencias de un lenguaje L_f , ese dispositivo debe incluir la noción de un algoritmo para calcular la función f .

0.5 Clases de Lenguajes

Los lenguajes se pueden clasificar según el tipo de dispositivos de aceptación y generación que existen para ellos. Estas clases corresponden a lenguajes de distinta complejidad que, a su vez, representan problemas de complejidad diferente. En particular se estudiarán las siguientes tres clases, las de la clásica jerarquía de Chomsky, además de algunas subclases de ellas:

- Lenguajes Regulares.
- Lenguajes Libres de Contexto.
- Lenguajes Enumerables Recursivamente.

Para cada clase hay un tipo de dispositivo de aceptación para todas y sólo las sentencias de esos lenguajes: autómatas de distinto grado de complejidad. También existe, para cada clase, un tipo de gramática que genera todas y sólo las sentencias de esos lenguajes.

Los dispositivos de generación de los lenguajes regulares y de los lenguajes libres de contexto, son ampliamente usados como modelos para expresar la sintaxis de los lenguajes de programación. Sus mecanismos de aceptación forman la base para el diseño de los analizadores léxicos y sintácticos de los compiladores. En la actualidad, la aplicación de estas técnicas ha permitido que esas fases de los compiladores sean generadas en forma automática por programas que utilizan dichos modelos como base de trabajo.

Las máquinas de Turing, dispositivos que aceptan y que también pueden generar los lenguajes enumerables recursivamente, fueron formuladas originalmente como un modelo de un computador de propósito general, esto fue aún antes de que existieran los computadores electrónicos modernos. Hoy en día, aún se consideran un modelo apropiado de la capacidad de un computador, siempre que no se considere la cantidad de recursos necesarios, ni la eficiencia de implementación. Así los lenguajes enumerables recursivamente se consideran los lenguajes más generales que pueden ser generados por un proceso implementable en un computador. Es decir, la máquina de Turing es un modelo razonable de la capacidad de un computador, aunque obviamente no del hardware o software real, ya que por la simplicidad del modelo, las máquinas de Turing trabajan muy ineficientemente; pero aún así, ellas pueden hacer cualquier cosa que es posible hacer en un computador.

0.6 Otros Problemas a Estudiar

Adicionalmente a los dispositivos de aceptación y mecanismos de generación para cada una de estas clases de lenguajes, también se estudiarán los siguientes tipos de problemas:

Propiedades de Clausura: ¿Qué operaciones es legítimo realizar con lenguajes de estas clases manteniéndose dentro de ella? Esto tiene importancia por el concepto de modularidad, es decir, dividir una tarea en partes realizables individualmente. Una vez dividida la tarea, el problema es : ¿es posible re-ensamblar las partes componentes y obtener un lenguaje que requiera el mismo tipo de dispositivos, o es necesario recurrir a dispositivos más poderosos?

Problemas de Decisión: ¿Qué propiedades de un lenguaje, o de sus sentencias, pueden ser decididas por un algoritmo que inspeccione un dispositivo de aceptación o generación? Por ejemplo, dada una gramática de algún tipo, ¿es posible determinar si es útil?, es decir, ¿es posible saber si define un lenguaje no vacío?

0.7 Problemas No Decidibles

La impresión generalizada de la gente, es que los computadores pueden, en principio, realizar todo trabajo que se desee, provisto que no importe el costo, ni el tiempo que pueda tomar. A lo más, la gente estaría dispuesta a aceptar que a lo mejor no se conoce un algoritmo para realizar cierto trabajo, pero no que haya ciertas tareas que no se pueden realizar.

Sin embargo, es posible demostrar que hay ciertas tareas que las máquinas de Turing, y por lo tanto los computadores, *no pueden hacer*. La primera vez que se conoce esta realidad es muy difícil de creerla; incluso, la gente trata de sobrellevar el choque que le produce, pensando que se trata de tareas muy rebuscadas, que nadie estaría interesado en ejecutar en la práctica. Desgraciadamente, este razonamiento tampoco es válido; hay muchas tareas que sería bueno poder hacer, pero que simplemente no se pueden realizar. Un ejemplo es el siguiente:

Problema de Detención: Dada una máquina de Turing y sus datos de entrada, ¿se detendrá en algún momento y dará su respuesta? O en otros términos, ¿es posible saber si un programa tiene un error que lo haga entrar en un ciclo infinito?

La respuesta es *no*. No es posible escribir un algoritmo (un programa) tal que dado, por ejemplo, cualquier programa en FORTRAN y sus datos de entrada, diga si este último se detendrá o no al ser ejecutado con esos datos. Es claro que es posible saber si un programa se detiene tras una cierta cantidad de tiempo, basta usar un cronómetro y verificarlo. La idea detrás de la no-decidibilidad de problemas es que no hay un programa único que pueda resolver el problema en todos los casos y para todos los datos de entrada. Es posible que algunos casos especiales o problemas limitados puedan ser resueltos.

Chapter 1

MATEMÁTICAS BÁSICAS

Este capítulo resume los principales conceptos matemáticos necesarios para el estudio de los lenguajes formales. Entre ellos se incluyen nociones generales como conjuntos, inducción matemática, grafos, árboles y relaciones binarias. Los conceptos más generales serán tratados someramente, suponiendo un conocimiento previo de la materia y con el exclusivo fin de fijar un lenguaje común y recordar los aspectos más importantes para estos apuntes.

1.1 Conjuntos

Un *conjunto* es, simplemente, una colección de objetos. Por ejemplo, la colección de los dígitos binarios 0 y 1 es un conjunto y se denota por $\{0, 1\}$. Los objetos que forman un conjunto son llamados sus *miembros* o *elementos*. Por ejemplo, 0 es un elemento del conjunto L definido anteriormente; este hecho se expresa como “ $0 \in L$ ”, y se lee como “0 pertenece a L ”. Es usual referirse a esto con frases como “0 está en L ” o “ L contiene a 0”. Por otro lado, el dígito decimal 2 no es un elemento de L , lo que se denota por $2 \notin L$, y se lee “2 no pertenece a L ”.

En un conjunto, cada objeto sólo puede estar o no estar; no interesan las repeticiones de un objeto. Es decir, el conjunto $\{a, b, a\}$ es el mismo conjunto que $\{a, b\}$. Similarmente, tampoco interesa el orden de los elementos; por ejemplo, $\{0, 1, 2\}$, $\{2, 0, 1\}$ y $\{1, 2, 0\}$ son exactamente el mismo conjunto. En resumen, dos conjuntos son iguales (son el mismo conjunto) si y sólo si tienen exactamente los mismos elementos.

Hay un conjunto que no tiene miembros. Por supuesto, sólo puede haber un conjunto con esta característica: se le denomina el *conjunto vacío* y se le denota usualmente por el símbolo $\emptyset$. De cualquier otro conjunto se dice que es *no vacío*, para indicar que sí tiene elementos.

Hasta aquí, ha sido posible definir los conjuntos listando todos sus elementos, separados por comas y encerrados entre llaves. Algunos conjuntos no pueden ser descritos de esta manera porque son *infinitos*, es decir, tienen un número infinito de elementos. Por ejemplo, el conjunto de los números naturales es un conjunto infinito. De todo conjunto que no es infinito, se dice que es *finito*.

Para describir conjuntos infinitos se hace necesario utilizar un *constructor de conjuntos*, de la forma:

$$\{x / P(x)\},$$

o también

$$\{x \in A / P(x)\}.$$

El primero representa al conjunto de todos los objetos para los cuales la proposición P se cumple. En el segundo caso, se especifica que esos objetos deben ser miembros del conjunto A , y es equivalente a la definición:

$$\{x / P(x) \text{ y } x \in A\}.$$

Ejemplo 1 El conjunto de los número enteros pares se puede definir utilizando el siguiente constructor de conjuntos:

$$\{i/i \text{ es un entero y existe un entero } j \text{ tal que } i = 2j\}$$

□

Si cada elemento de un conjunto A es también miembro de un conjunto B , se dice que A es un *subconjunto* de B ($A \subseteq B$), o que B *incluye* a A ($B \supseteq A$). De acuerdo con esto, todo conjunto es un subconjunto de sí mismo. Si A es un subconjunto de B , pero es distinto de B , entonces A es un subconjunto *propio* de B , y se denota por $A \subset B$. También se dice que dos conjuntos son *disjuntos* cuando no tienen ningún elemento en común.

1.1.1 Operaciones con Conjuntos

Varias operaciones permiten combinar dos conjuntos para formar un tercer conjunto, tal como los números se pueden combinar con las operaciones aritméticas para obtener otro. Las operaciones más usuales entre conjuntos son las siguientes:

1. La unión de A y B :

$$A \cup B = \{x/x \in A \text{ o } x \in B\}$$

2. La intersección de A y B :

$$A \cap B = \{x/x \in A \text{ y } x \in B\}$$

3. La diferencia de A y B :

$$A - B = \{x \in A \text{ y } x \notin B\}$$

4. El producto cartesiano de A y B :

$$A \times B = \{(x, y)/x \in A \text{ e } y \in B\}$$

5. El conjunto potencia de A :

$$2^A = \{S/S \subseteq A\}$$

Ejemplo 2 Sea A el conjunto $\{a, b\}$ y sea B el conjunto $\{b, c\}$, entonces las operaciones antes definidas producen los siguientes conjuntos:

$$A \cup B = \{a, b, c\}$$

$$A \cap B = \{b\}$$

$$A - B = \{a\}$$

$$A \times B = \{(a, b), (a, c), (b, b), (b, c)\}$$

$$2^A = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$$

□

Es interesante notar que si A y B son conjuntos finitos que tienen n y m miembros respectivamente, $A \cup B$ tiene a lo más $n + m$ elementos, $A \cap B$ tiene a lo más el mínimo entre n y m elementos y $A - B$ tiene a lo más n elementos; pero, en general, el número de elementos de estos conjuntos puede ser menor, como se aprecia en el Ejemplo 2. Sin embargo, $A \times B$ tiene exactamente $n * m$ elementos y 2^A tiene exactamente 2^n elementos, sin importar cuáles sean los conjuntos originales.

1.1.2 Conjuntos Infinitos

Una propiedad básica de los conjuntos finitos es su tamaño, es decir, el número de miembros que contiene. Algunos hechos sobre el tamaño de los conjuntos finitos son tan evidentes, que difícilmente necesitan demostración. Uno de ellos es que si A es un subconjunto de B , el tamaño de A es menor o igual al de B ; y que en caso de ser un subconjunto propio, el tamaño es simplemente menor.

Sin embargo, si se extiende la noción de tamaño a los conjuntos infinitos, tratando de seguir un camino intuitivo, siempre se producirán dificultades. Por ejemplo, ¿Hay más cubos perfectos $(0, 1, 8, 27, \dots)$ que múltiplos de 13 $(0, 13, 26, 39, \dots)$? Se puede especular con la respuesta, pero se ha demostrado que lo único razonable es suponer que tienen el mismo tamaño.

Se dice que dos conjuntos, A y B , son *equinumerosos* (tienen la misma *cardinalidad* o, simplemente, el mismo número de elementos), si hay una función $f : A \rightarrow B$ que sea biyectiva. Así, por ejemplo, los cubos perfectos y los múltiplos de 13 son equinumerosos; la biyección está dada por $f(13n) = n^3$, para todo número natural n .

Ejemplo 3 Sea A el conjunto de los enteros pares y B el conjunto de todos los enteros. Obviamente, A es un subconjunto propio de B . Sin embargo, A y B tienen la misma cardinalidad: son equinumerosos. La función:

$$f(i) = 2i \quad \text{para todo entero } i,$$

es una biyección entre los enteros y los números pares. Similarmente, se puede demostrar que los impares son, también, equinumerosos con los enteros.

□

En general, un conjunto es finito si es equinumeroso con el conjunto $\{1, \dots, n\}$, para algún número natural n . Un conjunto es infinito si no es finito. Por ejemplo, el conjunto de los números naturales es infinito; también son infinitos el conjunto de los números enteros, los reales, los cubos perfectos y muchos otros. Empero, no todos los conjuntos infinitos son equinumerosos entre sí: hay diferentes cardinalidades entre ellos.

Un conjunto se dice *infinito contable* si tiene la misma cardinalidad que los números naturales y se dice *contable* si es finito o infinito contable. Un conjunto que no es contable es *incontable*. El conjunto de los cubos perfectos, los enteros y los racionales son algunos ejemplos de conjuntos infinitos contables; los irracionales, los reales y los complejos, son incontables.

Ejemplo 4 Se demostrará que el conjunto potencia de N , el conjunto de todos los subconjuntos de números naturales, es incontable. Es decir, que hay más subconjuntos de números naturales que números naturales mismos.

Suponga que 2^N es un conjunto infinito contable —ciertamente no es finito, ya que N es infinito— es decir, suponga que hay una biyección $f : N \rightarrow 2^N$. Luego, 2^N puede listarse como:

$$2^N = \{S_0, S_1, S_2, \dots\},$$

en que para cada número natural i , $f(i) = S_i$. Considere ahora el conjunto:

$$D = \{n \in N / n \notin S_n\},$$

el conjunto de los números naturales que no pertenecen al subconjunto que enumeran. Claramente, D es un subconjunto de N ; y como tal, debe ser S_k para algún número natural k . La pregunta que es necesario hacer, es: ¿Pertenece k a S_k ?

- Suponga que la respuesta es sí, que $k \in S_k$. Entonces, por la definición de D , $k \notin D$. Pero $D = S_k$, por lo tanto, $k \notin S_k$. Una contradicción.

- Suponga que la respuesta es no, que $k \notin S_k$. Entonces, por la definición de D , $k \in D$. Pero $D = S_k$, por lo tanto, $k \in S_k$. Otra contradicción.

Luego, en ambos casos se llega a una contradicción. Como no hay una tercera alternativa, se concluye que la hipótesis de que existe un número natural k , tal que $D = S_k$, es errónea; es decir, que D , que es un subconjunto de los números naturales, no corresponde a ninguno de los S_i . En otras palabras, hay más subconjuntos de números naturales que números naturales mismos y, por lo tanto, el conjunto potencia de los números naturales es incontable, como se quería mostrar.

□

El método usado en el Ejemplo 4, es conocido con el nombre de *diagonalización*. Es una técnica muy utilizada que se basa en el uso de los números en un doble papel; como ocurre con el número n en la definición del conjunto D de ese ejemplo, en que se usa para representar a uno de los subconjuntos de N y, simultáneamente, a los números que no pertenecen a ese subconjunto específico. El nombre del método proviene de representar el proceso como una tabla en que, para este ejemplo, las filas representan a los subconjuntos de N y las columnas, a los números naturales, de tal modo que en el casillero (i, j) haya un 1 si el número j pertenece al i -ésimo subconjunto, y un cero si no es así; al hacer esto, el conjunto D queda definido por los valores en la diagonal de la tabla y en general se le conoce como el *conjunto diagonal* en estas demostraciones.

1.2 Inducción Matemática

En estos apuntes, muchas proposiciones se demuestran usando el llamado *Principio de Inducción Matemática*. Este principio indica que para probar que una cierta proposición $P(n)$ es válida para todo número natural n , es suficiente probar que se cumple para *cero* y, además, probar que si se cumple para algún número natural, se cumple también para el número siguiente. Es decir, basta establecer:

- $P(0)$, y que
- para todo número natural n : $P(n)$ implica $P(n + 1)$.

La primera parte, $P(0)$, es llamada la *base* y normalmente es la más simple de probar. La segunda parte es llamada el *paso inductivo* o la *inducción*; su antecedente, $P(n)$, es conocido como la *hipótesis de inducción* o *hipótesis inductiva*, y es un hecho que puede emplearse, sin necesidad de prueba, al hacer la demostración de $P(n + 1)$, la conclusión deseada en la inducción.

El principio de inducción es equivalente a otro principio matemático, conocido como el *principio del menor entero*, y expresa, fundamentalmente, la noción de que un número natural es el número cero, o es el sucesor de otro número natural. Es decir, expresa la idea intuitiva de que cualquier número natural puede ser formado a partir del número cero en un número finito de pasos, en un proceso que, en cada uno de sus pasos, agrega uno al número formado hasta el paso anterior.

Se le ha llamado inducción a este proceso porque primero debe decidirse, por algún otro método, cuál es la proposición que va a ser probada, y sólo entonces puede utilizarse para, en realidad, demostrar la validez de la suposición. Este principio no permite deducir cuál es la proposición a ser probada; ella debe obtenerse por otros métodos con anterioridad. En realidad, el concepto es muy diferente del llamado *razonamiento inductivo*, empleado por los científicos para crear una hipótesis, a partir de un número de observaciones de la realidad.

Ejemplo 5 Se prueba que la fórmula $1 + 2 + \cdots + n = n(n + 1)/2$, se cumple para todo número natural n . La demostración es por inducción en n , sobre los números naturales.

Base ($n = 0$): La suma del lado izquierdo es cero, pues no hay nada que sumar. La expresión del lado derecho queda $0(0 + 1)/2$, que también es cero, tal como se quería.

Inducción ($n \geq 0$): La hipótesis de inducción asegura que $1 + 2 + \cdots + n = n(n+1)/2$. Se desea mostrar que la fórmula se cumple también para $n+1$; es decir, que $1 + 2 + \cdots + n + (n+1) = (n+1)(n+2)/2$. Se tiene:

$$\begin{aligned} 1 + 2 + \cdots + n + (n+1) &= (1 + 2 + \cdots + n) + (n+1) && \text{(asociatividad de la suma)} \\ &= n(n+1)/2 + (n+1) && \text{(hipótesis de inducción)} \\ &= (n+1)(n/2 + 1) && \text{(factorizando)} \\ &= (n+1)(n+2)/2 \end{aligned}$$

como se deseaba mostrar.

Luego, por el principio de inducción matemática, se concluye que la fórmula se cumple para todos los números naturales. □

Una forma de comprender intuitivamente la validez del principio de inducción matemática, es a través de una analogía entre los números naturales y una serie infinita (pero contable) de cartas de un juego de dominó, dispuestas de forma tal que una carta al caer pueda botar a la carta siguiente. En esta analogía, cada carta corresponde a uno de los números naturales, y el hecho que una carta caiga, corresponde a que la proposición se cumple para el número natural respectivo. Al demostrar la inducción, se está probando que si cualquiera de las cartas cae, la siguiente carta también caerá. La base, por el contrario, establece un hecho concreto: la carta número cero cae. Ambas cosas son, obviamente, suficientes para concluir que todas las cartas caerán, y que, en realidad, cada carta caerá después de un lapso finito de tiempo. Es decir, para concluir que la proposición es válida para todos los números naturales.

1.2.1 Otras Bases

Si se quiere mostrar que una proposición $P(n)$ se cumple para todos los números naturales mayores o iguales a un cierto número natural k , también se puede emplear el principio de inducción matemática. En este caso se debe aplicar de forma que la base corresponda a $P(k)$ y, además, en la inducción se puede considerar que el número n es mayor o igual a k . Es decir, basta establecer:

- $P(k)$, y que
- para todo número natural $n \geq k$: $P(n)$ implica $P(n+1)$.

Esta formulación expresa la noción de que cualquier número natural mayor o igual a k , puede ser formado a partir del número k , en un número finito de pasos; en que en cada paso, se agrega uno al número formado hasta el paso anterior.

Ejemplo 6 Se demuestra que $2^n > n^3$, para todo número natural mayor o igual a 10. La demostración es por inducción en n , sobre los números naturales, a partir del número 10.

Base ($n = 10$): En este caso se tiene, $2^n = 2^{10} = 1024$ y, por otro lado, $n^3 = 10^3 = 1000$. Es decir, para $n = 10$, $2^n > n^3$, como se quería probar.

Inducción ($n \geq 10$): La hipótesis de inducción asegura que $2^n > n^3$ cuando $n \geq 10$. Se desea mostrar que esta desigualdad también se cumple para $n+1$; es decir, que $2^{n+1} > (n+1)^3$. Entonces, se tiene:

$$\begin{aligned} 2^n &> n^3 && = nn^2 && \text{(hipótesis de inducción)} \\ &> 9n^2 && = 3n^2 + 3n^2 + 3n^2 && \text{(porque } n \geq 10\text{)} \\ &> 3n^2 + 3n + 1 && && \text{(porque } n \text{ es positivo)} \end{aligned}$$

Utilizando nuevamente la hipótesis de inducción y sumándola a la última desigualdad obtenida, se tiene:

$$2^n + 2^n > n^3 + 3n^2 + 3n + 1$$

de donde se concluye, usando la expresión para el cubo de un binomio, que:

$$2^{n+1} > (n+1)^3$$

como se quería mostrar.

Luego, por el principio de inducción matemática, se concluye que la desigualdad se cumple para todos los números naturales mayores o iguales a 10. Es interesante destacar que en el paso inductivo, no sólo se hizo uso de la hipótesis de inducción, sino que también se utilizó la condición que indica que N es mayor o igual a 10 en este caso.

□

Nuevamente, la analogía con las cartas del juego de dominó sirve para explicar, al menos intuitivamente, esta formulación del principio de inducción. En estas aplicaciones, el hecho concreto establecido por la base es que la carta número k cae. Este hecho, sumado a lo probado con la inducción —en la que además es legítimo suponer que n es mayor o igual que k , pues son esas las cartas que interesan— es suficiente para establecer que todas las cartas, a partir de la carta número k , caerán. Y por lo tanto se puede concluir que la proposición se cumple para todo número natural mayor o igual al número k .

1.2.2 Inducción Completa

Existen muchas otras formas de expresar el principio de inducción. Una generalización bastante útil, es la llamada *inducción completa*. Ella expresa, en una de sus formas, que para demostrar que una cierta proposición $P(n)$ es válida para todos los números naturales, es suficiente probar que se cumple para cero y, además, probar que si se cumple para todos los naturales entre la base y un número natural n cualquiera, se cumple también para el número siguiente a éste: $n+1$. Es decir, basta establecer:

- $P(0)$, y que
- para todo número natural n : $P(0), P(1), \dots$ y $P(n)$ implican $P(n+1)$.

La diferencia con el principio enunciado anteriormente, radica en que la hipótesis de inducción es mucho más fuerte en este caso, ya que permite suponer que la proposición se cumple no sólo para n , sino que en general, para cualquier número menor que $n+1$, y mayor o igual a la base. La posibilidad de utilizar esta hipótesis hace que las demostraciones sean, algunas veces, mucho más sencillas y cortas que si se usara el enunciado original; aún cuando la demostración sería igualmente posible, ya que la inducción completa no es un principio nuevo, sino que una consecuencia del principio original.

Ejemplo 7 Se demuestra que todo número natural n , mayor o igual a dos, se puede escribir como el producto de números primos¹. Un número primo es un número natural mayor que uno, que no tiene divisores exactos, excepto 1 y el número mismo. La demostración es por inducción completa en n , sobre los números naturales, a partir del número dos.

Base ($n=2$): El número 2 se puede escribir como el producto de números primos en que el único factor es el número 2 mismo. Claramente 2 es un primo, ya que es mayor que 1 y sólo es divisible, en forma exacta, por 1 y por 2, el número mismo.

Inducción ($n \geq 2$): La hipótesis de inducción asegura que todo número natural k entre 2 y n , ambos inclusive, se puede escribir como el producto de números primos. Se desea demostrar que el número $n+1$ también puede descomponerse en esta forma.

¹Ésta es una parte del llamado Teorema Fundamental de la Aritmética, que indica que todo número natural mayor que uno, puede expresarse en forma única como el producto de números primos. La unicidad se refiere a que hay un único conjunto de primos envueltos en ese producto, y a que cada número primo tiene multiplicidad fija en él.

Hay dos alternativas posibles, $n + 1$ es un número primo o no lo es. En el primer caso, el número puede escribirse directamente como el producto en que el único factor es el mismo y, por lo tanto, puede escribirse como un producto de primos. En el segundo caso, si $n + 1$ no es un número primo, quiere decir que existen dos números naturales, p y q , cuyo producto es $n + 1$ y en que tanto p como q están entre 2 y n , ya que si no fuese así, $n + 1$ sería un número primo. Pero por la hipótesis de inducción se sabe que tanto p como q se pueden escribir como el producto de números primos, ya que ambos están entre 2 y n , por lo tanto, el número $n + 1$ puede escribirse como el producto de esos primos en conjunto. Es decir, en ambos casos se ha probado que el número $n + 1$ puede ser escrito como el producto de números primos, lo que completa el paso inductivo.

Por lo tanto, por el principio de inducción matemática, se concluye que todo número natural puede escribirse como el producto de números primos. Este ejemplo pone de manifiesto la utilidad de la inducción completa; en caso de haber tratado de usar el principio de inducción en su forma original, la hipótesis de inducción sólo habría dicho que el número n puede escribirse como el producto de primos, lo cual es insuficiente para la demostración, ya que los números p y q son, en general, distintos de n y de ellos sólo se sabe que son menores que $n + 1$ y mayores o iguales a 2.

□

Usando una vez más —pero por última vez— la analogía con las cartas de dominó, es fácil justificar intuitivamente la inducción completa. En este caso, la inducción prueba que si todas las cartas entre la carta base y una carta cualquiera caen, la carta siguiente también caerá. Como la base prueba que la carta número cero cae, se puede concluir que la carta número uno cae; pero como ahora se sabe que tanto la número cero como la número uno caen, se concluye que cae la carta número dos, y así sucesivamente se puede demostrar que todas las cartas caerán. Es decir, se puede concluir que la proposición se cumple para todos los números naturales.

1.2.3 Definiciones Inductivas

Empleando las mismas ideas que inspiran al principio de inducción matemática, es posible definir ciertos conjuntos de objetos en forma inductiva. En estas definiciones, los objetos se definen empleando instancias más simples del mismo tipo de entes. Además, se hace necesario considerar una instancia básica como un caso especial, de manera que esta especie de recursión infinita termine eventualmente.

Ejemplo 8 La serie de números de Fibonacci (0, 1, 1, 2, 3, 5, 8, ...) se puede definir, en forma inductiva, a través de las siguientes reglas:

$$\begin{aligned} f_0 &= 0 \\ f_1 &= 1 \\ f_n &= f_{n-1} + f_{n-2}, \text{ para todo número natural } n > 1 \end{aligned}$$

Se aprecia en esta definición que un número de Fibonacci es la suma de los dos números anteriores en la secuencia y, por lo tanto, se hace necesario definir en forma especial los dos primeros valores de la serie, de manera que ella pueda comenzar.

□

Es importante destacar que en este tipo de definiciones, es la definición misma la que indica como obtener cada uno de estos entes. En el caso del Ejemplo 8, un número de Fibonacci, f_i , cualquiera se obtiene como se indica a continuación. Si i es cero o uno, se aplica el caso básico correspondiente, y el número se conoce de inmediato; si i es mayor que uno, se aplica la tercera regla y el problema se descompone en dos problemas más sencillos del mismo tipo, los que pueden, a su vez, descomponerse sucesivamente, hasta llegar a alguno de los casos básicos. Es decir, estas definiciones sugieren un algoritmo para determinar cada uno de los entes que definen; el que dependiendo de la definición misma, puede ser más o menos eficiente.

Ejemplo 9 Se calcula los seis primeros números de la serie de Fibonacci $(f_0, \dots, f_5)$, empleando la definición dada en el Ejemplo 8 para esta secuencia.

$$\begin{array}{lll} f_0 & = & 0 \\ f_1 & = & 1 \\ f_2 & = & f_1 + f_0 = 1 \end{array} \qquad \begin{array}{lll} f_3 & = & f_2 + f_1 = 2 \\ f_4 & = & f_3 + f_2 = 3 \\ f_5 & = & f_4 + f_3 = 5 \end{array}$$

□

Es usual que cuando se trata de probar propiedades de entes que han sido definidos inductivamente, lo más conveniente sea utilizar, precisamente, el principio de inducción. Esto se debe a que la definición coincide apropiadamente con la división que se hace entre el caso básico y el paso inductivo en este método, facilitando, en consecuencia, la demostración.

Ejemplo 10 Se demuestra que la siguiente relación, entre números de Fibonacci, se cumple para todo número natural n , mayor o igual a uno:

$$f_n^2 = f_{n-1} * f_{n+1} + (-1)^{n+1}$$

La demostración se hará por inducción completa en n , sobre los números naturales, a partir del número uno.

Base ($n = 1$): En este caso se tiene: $f_{n-1} * f_{n+1} + (-1)^{n+1} = f_0 * f_2 + (-1)^2 = 0 * 1 + 1 = 1$. Por otro lado, $f_n^2 = f_1^2 = 1$. Es decir, para $n = 1$ la relación se cumple, como se quería probar.

Inducción ($n \geq 1$): La hipótesis de inducción asegura que $f_k^2 = f_{k-1} * f_{k+1} + (-1)^{k+1}$, para todo número natural k , entre 1 y n , ambos inclusive. Se desea mostrar que esta desigualdad también se cumple para $n + 1$; es decir, que

$$f_{n+1}^2 = f_n * f_{n+2} + (-1)^{n+2}$$

La demostración del paso inductivo se hará en dos partes. Primero se verá el caso en que $n = 1$ y, posteriormente, el caso en que $n \geq 2$.

Caso 1 ($n = 1$): En este caso se tiene: $f_n * f_{n+2} + (-1)^{n+2} = f_1 * f_3 + (-1)^3 = 1 * 2 - 1 = 1$. Por otro lado, se tiene $f_2^2 = 1$, también; como se deseaba probar.

Caso 2 ($n \geq 2$): En este caso se tiene,

$$\begin{aligned} f_{n+1}^2 &= (f_n + f_{n-1})^2 && \text{(definición, pues } n + 1 > 1) \\ &= f_n^2 + 2f_n f_{n-1} + f_{n-1}^2 && \text{(cuadrado de binomio)} \\ &= f_n^2 + 2f_n f_{n-1} + f_{n-2} f_n + (-1)^n && \text{(hipótesis, con } k = n - 1 \geq 1) \\ &= f_n(f_n + f_{n-1} + f_{n-1} + f_{n-2}) + (-1)^n \\ &= f_n(f_{n+1} + f_n) + (-1)^n && \text{(definición, pues } n + 1 > n > 1) \\ &= f_n f_{n+2} + (-1)^{n+2} && \text{(definición, pues } n + 2 > 1) \end{aligned}$$

como se deseaba mostrar.

Luego, por el principio de inducción matemática, se concluye que la relación se cumple para todos los números naturales mayores o iguales a 1. Es importante destacar que fue necesario dividir el paso inductivo en dos partes, pues cuando $n = 1$ no es lícito hacer referencia a f_{n-2} , ya que no existe, ni tampoco es aplicable la hipótesis de inducción para f_{n-1} , pues sólo es aplicable entre la base y n , no para f_0 , valor para el cual no tiene sentido por lo demás.

□

En un análisis más profundo, toda demostración de una propiedad de los números enteros se basa, de una u otra manera, en el principio de inducción matemática, ya que si se va a los conceptos básicos, los números enteros mismos están definidos esencialmente en forma inductiva. Aunque no se mencionó explícitamente, ésto ha ocurrido también en las pruebas por inducción de los primeros ejemplos de esta sección. En forma implícita, se han usado definiciones inductivas de la suma, producto, potencia y de los números naturales mismos, lo que contribuye a efectuar estas demostraciones por inducción.

1.3 Grafos y Árboles

Un *grafo* (finito), denotado como $G = (V, R)$, es una estructura que consta de un conjunto finito de *vértices* V , también llamados *nodos*; y de un conjunto de pares no ordenados de vértices, R , llamados las *ramas* del grafo. La forma usual de representar grafos es a través de un diagrama en que los nodos se grafican como puntos y las ramas, como líneas entre los vértices que forman el par respectivo.

Ejemplo 11 En la Figura 1.1 se muestra la representación gráfica usual para el grafo $G = (V, R)$, cuyas componentes están definidas por:

$$V = \{1, 2, 3, 4, 5\}$$

$$R = \{(i, j)/i + j = 4 \text{ o } |i - j| = 3\}$$

Hay cinco nodos y cuatro ramas en dicho grafo; éstas últimas son: $(1, 3)$, $(1, 4)$, $(2, 2)$, $(2, 5)$.

□

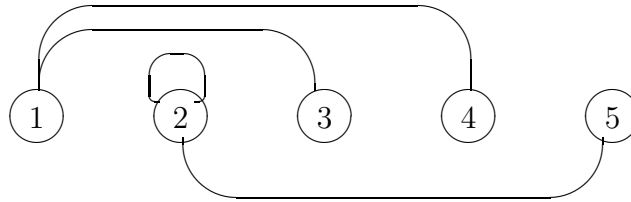


Figure 1.1: Representación gráfica del grafo G

Un *camino* en un grafo, es una secuencia de vértices $v_1, v_2, \dots, v_n$, con $n \geq 1$, en que hay una rama (v_i, v_{i+1}) por cada i tal que $1 \leq i \leq n$; los nodos v_1 y v_n son llamados el vértice *inicial* y *final*, respectivamente, y se dice que el camino es de v_1 a v_n . Un camino es entonces una secuencia de vértices, tal que es posible pasar de uno de ellos al siguiente en la secuencia, a través de una rama del grafo. La *longitud* del camino es $n - 1$, es decir, el número de ramas que lo forman. Por ejemplo, $3-1-4$ es un camino en el grafo de la Figura 1.1 y tiene longitud 2; también lo es el nodo 4, o cualquier otro vértice por sí solo, estos últimos tienen longitud 0, por supuesto.

Un camino de longitud al menos 1, en que los vértices inicial y final corresponden al mismo nodo, es llamado un *circuito* y equivale a un lazo cerrado en el diagrama para el grafo. Así, por ejemplo, $2-2$, es un circuito de longitud 1 en el grafo de la Figura 1.1.

1.3.1 Grafos Dirigidos

Un *grafo dirigido* (finito), denotado por $G = (V, A)$, consta de un conjunto finito de vértices, V ; y de un conjunto de pares *ordenados* de vértices A , llamados *arcos*. Un arco (u, v) se denota por $u \rightarrow v$ y se dice que es un arco de u a v ; el nodo u es un *predecesor* del nodo v y v es un *sucesor* de u en el grafo. Los diagramas que representan grafos dirigidos son similares a los usados para grafos, pero los arcos, que son ramas dirigidas, se dibujan como líneas con un sentido definido —normalmente como flechas— dirigidas desde el nodo predecesor al nodo sucesor en el arco.

Ejemplo 12 La Figura 1.2 muestra el diagrama correspondiente al grafo $G = (V, A)$, cuyas componentes quedan definidas por:

$$V = \{1, 2, 3, 4\}$$

$$A = \{i \rightarrow j / i < j\}$$

En este grafo, el vértice 3 es un sucesor de los vértices 1 y 2, y un predecesor del vértice 4.

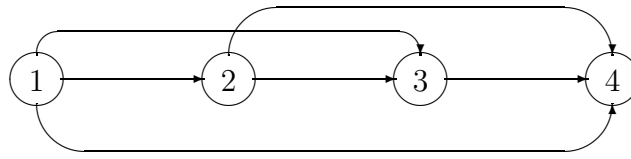


Figure 1.2: Representación gráfica del grafo G

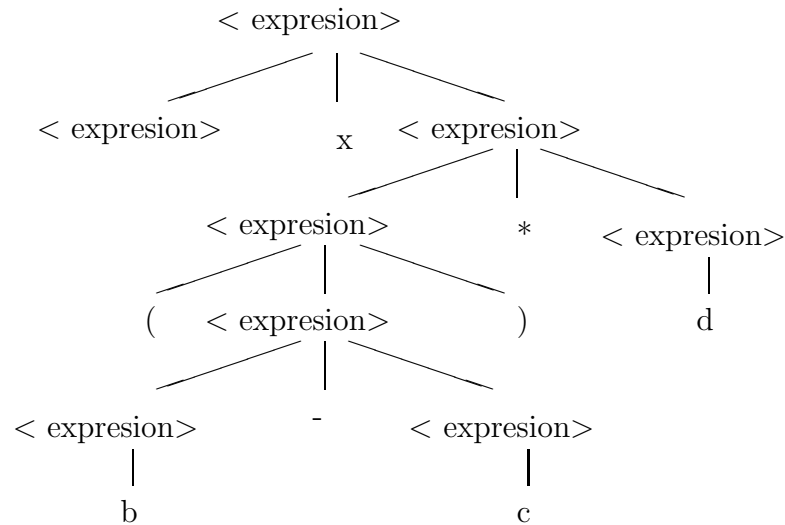
□

Un *camino* en un grafo dirigido es una secuencia de vértices $v_1, v_2, \dots, v_n$, con $n \geq 1$, en que hay un arco $v_i \rightarrow v_{i+1}$ por cada i tal que $1 \leq i \leq n$; en este caso se dice que el camino es de v_1 a v_n y que pasa a través de los nodos $v_2, \dots, v_{n-1}$. La *longitud* de un camino se define en forma análoga que para los grafos y mide el número de arcos involucrados en él. Por ejemplo, 1–2–4 es un camino de 1 a 4 en el grafo de la Figura 1.2, y tiene longitud 2. Un *circuito* es un camino de longitud mayor o igual a 1, que va de un nodo a sí mismo. El grafo de la Figura 1.2 no tiene circuitos.

1.3.2 Árboles

Un *árbol*, o más exactamente un árbol dirigido ordenado, es un grafo dirigido con las siguientes propiedades adicionales:

- Hay un vértice, llamado la *raíz*, que no tiene predecesores y desde el cual hay un camino a cada nodo del árbol.
- Cada vértice tiene exactamente un predecesor, con la única excepción de la raíz, que no tiene predecesores.
- Los sucesores de cada vértice están ordenados. Ordenamiento que se conoce como orden de izquierda a derecha.

Figure 1.3: Diagrama sintáctico para la expresión $a + (b - c) * d$

Al representar gráficamente los árboles, es usual poner la raíz arriba y todos los arcos apuntando hacia abajo; con esta convención es posible dibujar los arcos como simples ramas no dirigidas, ya que se subentiende

que su dirección es hacia abajo. Los sucesores de cada vértice se dibujan de izquierda a derecha, de acuerdo al orden definido para ellos.

Ejemplo 13 La Figura 1.3 muestra el diagrama de un árbol que corresponde al “diagrama sintáctico” de la expresión aritmética $a + (b - c) * d$. En este caso no se muestran los nombres de los nodos, sino las etiquetas asociadas a ellos. Varios nodos tienen la misma etiqueta.

La raíz de ese árbol es el nodo con etiqueta $\langle \text{expresión} \rangle$ que aparece más arriba que todos los demás; desde ella hay un camino a los otros dieciséis vértices, y no tiene predecesores. Sus tres sucesores son, ordenados de izquierda a derecha, los nodos con etiquetas $\langle \text{expresión} \rangle$, $+$ y $\langle \text{expresión} \rangle$ que aparecen bajo él.

□

Existe una terminología especial para árboles basada en la genealogía, que difiere de la terminología general para grafos dirigidos arbitrarios. En un árbol, un sucesor de un nodo se llama un *hijo* y el predecesor es llamado el *padre*. Si hay un camino de un vértice u a un vértice v , se dice que u es un *ancestro* de v y que v es un *descendiente* de u ; ambos nodos pueden ser el mismo vértice y, por lo tanto, todo nodo es un ancestro y descendiente de sí mismo. Un vértice que no tiene hijos es una *hoja* y todos los demás, incluida la raíz, son llamados *nodos interiores*.

Ejemplo 14 En el árbol de la Figura 1.3, el nodo con etiqueta $+$ es un hijo de la raíz, y este último nodo es su padre. El vértice con etiqueta d es un descendiente de sí mismo y de otros tres nodos del árbol; la raíz es un ancestro de todos los nodos del árbol. Los nodos con etiqueta $\langle \text{expresión} \rangle$ son todos nodos interiores, los demás son las hojas.

□

Es posible extender el orden que existe entre los hijos de cada nodo, a un ordenamiento de izquierda a derecha entre todas las hojas de un árbol. En realidad, se puede extender a dos vértices cualesquiera, siempre que ninguno de ellos sea un ancestro del otro y, obviamente, una hoja no es nunca ancestro de otra hoja. La extensión del orden a dos nodos cualesquiera que cumplan con esta condición se hace de la siguiente manera. Dados dos nodos n_1 y n_2 en el árbol, se trazan los caminos —invertidos— desde cada uno de ellos hacia la raíz, hasta que se encuentran en algún vértice v . Sean h_1 y h_2 los hijos de v en los caminos hacia n_1 y n_2 , respectivamente. Si n_1 no es ancestro de n_2 , o viceversa, h_1 y h_2 son nodos distintos y, por lo tanto, uno de ellos está a la izquierda del otro como hijos de v . Si h_1 está a la izquierda de h_2 , entonces n_1 está a la izquierda de n_2 ; si no, n_2 está a la izquierda de n_1 .

Ejemplo 15 En el árbol de la Figura 1.3, el nodo con etiqueta c está a la izquierda del nodo con etiqueta d . Los caminos desde ellos hacia la raíz se encuentran en el nodo con etiqueta $\langle \text{expresión} \rangle$ que es el hijo de más a la derecha de la raíz del árbol. El nodo con etiqueta c está en el camino que pasa por el hijo de más a la izquierda de ese vértice, y el con etiqueta d , en el que pasa por el hijo de más a la derecha. Obviamente el primero está a la izquierda del segundo en el orden para esos nodos, por lo que se concluye que el nodo con etiqueta c está a la izquierda del nodo con etiqueta d , en el orden extendido.

□

1.4 Relaciones Binarias

Una *relación binaria* es un conjunto de pares ordenados; es decir, es un subconjunto del producto cartesiano de dos conjuntos. Si ambos conjuntos son el mismo conjunto, S , se le denomina relación *en* S . Intuitivamente, es el conjunto de todos los pares de objetos en S entre los que la relación se cumple. Si $\mathcal{R}$ es una relación y el par (a, b) pertenece a ella, se acostumbra escribir $a\mathcal{R}b$ indicando que el elemento a está en relación $\mathcal{R}$ con b ; en forma similar, cuando $(a, b) \notin \mathcal{R}$, se escribe $a\not\mathcal{R}b$.

Una relación binaria $\mathcal{R}$ en un conjunto finito S , puede representarse por un grafo dirigido en que cada nodo del grafo corresponde a un elemento de S , y en que hay un arco de un vértice v_1 a un vértice v_2 , si y sólo si $v_1 \mathcal{R} v_2$. Una relación binaria $\mathcal{R}$ en un conjunto finito S se representa, entonces, por el grafo dirigido $G = (S, \mathcal{R})$. A la inversa, cualquier grafo dirigido $G = (V, A)$ puede interpretarse como la representación de una relación binaria A en el conjunto V , de sus nodos.

Ejemplo 16 Sea $\mathcal{R} = \{(1, 2), (1, 4), (2, 1), (2, 3), (3, 3), (3, 1), (4, 3)\}$ una relación binaria definida en el conjunto $S = \{1, 2, 3, 4\}$. El grafo dirigido que la representa se muestra en la Figura 1.4.

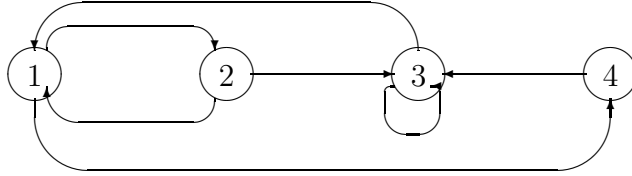


Figure 1.4: Representación gráfica para la relación $\mathcal{R}$

□

1.4.1 Propiedades

Es posible definir muchas propiedades que las relaciones binarias pueden o no cumplir. Algunas de las propiedades más usadas y sus definiciones para una relación $\mathcal{R}$ en S , son las siguientes:

1. Reflexividad: $\mathcal{R}$ es refleja si y sólo si

$$a \mathcal{R} a, \text{ para todo } a \in S.$$

2. Irreflexividad: $\mathcal{R}$ es irrefleja si y sólo si

$$a \not\mathcal{R} a, \text{ para todo } a \in S.$$

3. Simetría: $\mathcal{R}$ es simétrica si y sólo si

$$a \mathcal{R} b \text{ implica } b \mathcal{R} a, \text{ para todo } a \text{ y } b \in S.$$

4. Asimetría: $\mathcal{R}$ es asimétrica si y sólo si

$$a \mathcal{R} b \text{ implica } b \not\mathcal{R} a, \text{ para todo } a \text{ y } b \in S.$$

5. Antisimetría: $\mathcal{R}$ es antisimétrica si y sólo si

$$a \mathcal{R} b \text{ y } b \mathcal{R} a \text{ implica } a = b, \text{ para todo } a \text{ y } b \in S.$$

6. Transitividad: $\mathcal{R}$ es transitiva si y sólo si

$$a \mathcal{R} b \text{ y } b \mathcal{R} c \text{ implica } a \mathcal{R} c, \text{ para todo } a, b \text{ y } c \in S.$$

Es conveniente hacer notar que según estas definiciones, toda relación asimétrica debe ser irrefleja. Por el contrario, una relación antisimétrica puede ser refleja, irrefleja o no tener ninguna de esas dos propiedades.

Ejemplo 17 La relación “ancestro de”, sobre el conjunto de personas, y de acuerdo a lo que intuitivamente se entiende por este concepto, es irrefleja, ya que nadie es ancestro de sí mismo; asimétrica —y también antisimétrica—, ya que si una persona es ancestro de otra, esta última no puede ser ancestro de la primera. Y, finalmente, es una relación transitiva, pues si una persona es ancestro de otra y ésta, a su vez, es ancestro de una tercera persona, la primera es un ancestro de esta última.

Nótese, sin embargo, que la relación “ancestro de”, sobre el conjunto de los nodos de un árbol, de acuerdo a las definiciones dadas en la sección anterior, es una relación refleja, antisimétrica y transitiva, lo que en realidad difiere del concepto intuitivo.

□

Las relaciones simétricas pueden representarse simplemente, empleando grafos. Es sabido que en estos casos, si (a, b) está en la relación, también lo estará el par (b, a) y, por lo tanto, no es necesario retener la información sobre el orden de los pares. Cualquier grafo $G = (V, R)$ puede entenderse como la representación de una relación simétrica, R , en el conjunto de vértices V . A su vez, cualquier relación simétrica $\mathcal{R}$ en un conjunto finito S , puede representarse por el grafo $G = (S, \mathcal{R})$. Sin embargo, esta forma de representación no será utilizada en estos apuntes.

1.4.2 Relaciones de Equivalencia

Una relación binaria que es refleja, simétrica y transitiva se denomina *relación de equivalencia*. El nombre se debe a que dos objetos relacionados por una relación de equivalencia son esencialmente equivalentes —cumplen el mismo papel— en cuanto al propósito de la relación.

Una propiedad muy importante de una relación de equivalencia $\mathcal{R}$ en un conjunto S , es que divide a este último en varios subconjuntos no vacíos y disjuntos entre sí, llamados *clases de equivalencia*. La unión de todas estas clases, cuyo número puede ser infinito, forma el conjunto S . Cada elemento de cualquiera de estas clases, está en la relación $\mathcal{R}$ con todos los otros miembros de ese conjunto; sin embargo, miembros de clases diferentes no están nunca en relación. Es decir, una relación de equivalencia $\mathcal{R}$ en un conjunto S , define subconjuntos no vacíos $S_1, S_2, \dots$ que cumplen las siguientes propiedades:

- $S = S_1 \cup S_2 \cup \dots$
- Si $i \neq j$, $S_i \cap S_j = \emptyset$
- Para todo a y $b \in S_i : a\mathcal{R}b$
- Si $i \neq j$, para todo $a \in S_i$ y $b \in S_j : a\not\mathcal{R}b$

Ejemplo 18 Un ejemplo de relación de equivalencia es congruencia módulo un entero k y se escribe $i \equiv j \pmod k$, si y sólo si $i - j$ es divisible por k . Es simple demostrar que esta relación en los números enteros es una relación de equivalencia, es decir, que es refleja, simétrica y transitiva. Las clases de equivalencia que define son los siguientes k conjuntos, cada uno de ellos es un conjunto infinito contable:

$$\{\dots, -2k, -k, 0, k, 2k, \dots\}$$

$$\{\dots, -2(k-1), -(k-1), 1, k+1, 2k+1, \dots\}$$

.....

$$\{\dots, -(k+1), -1, k-1, 2k-1, 3k-1, \dots\}$$

□

Tal como una relación de equivalencia en S particiona a este conjunto en sus clases de equivalencia, también es cierto que toda partición² de un conjunto, induce una relación de equivalencia en él. En efecto, la relación de equivalencia inducida es:

$$\{(a, b)/a \text{ y } b \text{ pertenecen al mismo subconjunto en la partición}\}.$$

1.4.3 Clausuras

Si $\mathcal{P}$ es un conjunto de propiedades de relaciones, la *clausura- $\mathcal{P}$* de una relación $\mathcal{R}$, es la relación más pequeña que incluye a $\mathcal{R}$ y que tiene las propiedades en $\mathcal{P}$. No cualquier conjunto de propiedades es posible. Por ejemplo, no cualquier relación tiene clausura irrefleja. Sin embargo, toda relación binaria tiene clausura con respecto a cualquier combinación de reflexividad, simetría y transitividad.

Dos clausuras serán especialmente importantes para estos apuntes. La primera de ellas, la *clausura transitiva* de una relación $\mathcal{R}$, denotada como $\mathcal{R}^+$, se puede definir de la siguiente manera:

- Si $a\mathcal{R}b$, entonces $a\mathcal{R}^+b$.
- Si $a\mathcal{R}^+b$ y $b\mathcal{R}^+c$, entonces $a\mathcal{R}^+c$.
- Nada más pertenece a $\mathcal{R}^+$.

Es simple mostrar que la relación así definida es en realidad la relación más pequeña que incluye a $\mathcal{R}$ y es transitiva, es decir, que es su clausura transitiva.

La segunda clausura que interesará es la *clausura refleja y transitiva* de una relación $\mathcal{R}$ en un conjunto S . Esta relación, denominada $\mathcal{R}^*$, se puede definir en forma análoga a la anterior. Sin embargo, es también fácil ver que corresponde al conjunto:

$$\mathcal{R}^* = \mathcal{R}^+ \cup \{(a, a)/a \in S\}$$

Ejemplo 19 Sea $\mathcal{R} = \{(a, b), (b, b), (b, c)\}$, una relación en el conjunto $S = \{a, b, c\}$. Entonces sus clausuras transitiva, refleja y transitiva son:

$$\mathcal{R}^+ = \{(a, b), (a, c), (b, b), (b, c)\}$$

$$\mathcal{R}^* = \{(a, a), (a, b), (a, c), (b, b), (b, c), (c, c)\}$$

Los grafos dirigidos que representan las relaciones $\mathcal{R}$, $\mathcal{R}^+$, y $\mathcal{R}^*$, se muestran, en ese orden, en la Figura 1.5.

□

²Una partición de un conjunto A es un subconjunto de su conjunto potencia, esto es, un conjunto de sus subconjuntos. Cada subconjunto de A en una partición es no vacío. Además, dos subconjuntos distintos cualesquiera son disjuntos y la unión de todos ellos forma el conjunto A . Es decir, cada elemento de A está en exactamente uno de los subconjuntos, y cada uno de ellos contiene al menos un elemento de A .

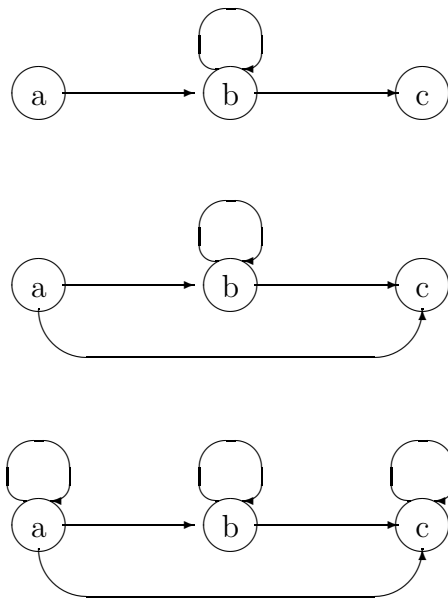


Figure 1.5: Representación gráfica para las relaciones $\mathcal{R}$, $\mathcal{R}^+$ y $\mathcal{R}^*$

Chapter 2

LENGUAJES FORMALES

La teoría de computación es el estudio, desde un punto de vista matemático, de los computadores y sus capacidades. No se trata de estudiar algún computador en particular, sino de generalizar el concepto y formalizar la noción de lo que es computable. Para hacer este estudio, se requiere definir un modelo de los objetos manipulados por los computadores. Como modelo matemático para los datos, en el sentido amplio que incluye a todos los objetos que los computadores manejan —ya sean programas o datos propiamente tales— se utilizan secuencias finitas de símbolos.

Este capítulo presenta conceptos propios del tema a tratar en estos apuntes, como son las nociones de palabra y de lenguajes abstractos, concentrándose principalmente en su definición y en algunas propiedades y operaciones básicas entre ellos.

2.1 Símbolos y Alfabetos

Un *símbolo* es una entidad abstracta que no se definirá formalmente, tal como el concepto de punto no se define en geometría. Las letras y los dígitos son ejemplos típicos de símbolos que se usan frecuentemente; aún cuando cualquier objeto puede considerarse un símbolo. Un conjunto finito de símbolos será llamado un *alfabeto*. Un ejemplo de alfabeto, conocido por toda la gente, es el *alfabeto Romano*, cuyos símbolos son cada una de las letras usadas en el lenguaje Castellano: $\{a, b, c, \dots, z\}$. Un alfabeto particularmente relacionado con los computadores actuales, es el denominado *alfabeto binario*: $\{0, 1\}$. En realidad cualquier objeto puede pertenecer a un alfabeto, ya que cualquier objeto puede ser un símbolo. Desde un punto de vista formal, un alfabeto es simplemente un conjunto finito con cualquier tipo de componentes. Sin embargo, por simplicidad, se usará como símbolos sólo las letras, los dígitos y algunos otros caracteres comúnmente usados, como \$, #, /, etcétera.

2.2 Palabras

Una *palabra* sobre un cierto alfabeto es una secuencia finita de símbolos tomados de él. Este concepto, que también se conoce por el vocablo inglés *string*, representa lo que intuitivamente se entiende por palabra, frase o sentencia, si se incluye en estas últimas todos los símbolos que sirven para contruirlas; esto es, no sólo las letras y símbolos de puntuación, sino que también los espacios en blanco usados como separadores. El concepto de palabra que aquí se presenta difiere del sentido usual, en que no se le asigna significado ni representación a estas secuencias de símbolos, y sólo se está interesado en cómo se construyen a partir de los símbolos del alfabeto¹. Los vocablos *frase* y *sentencia* se usarán como sinónimo de palabra en estos apuntes. Cualquier símbolo del alfabeto puede aparecer cero o más veces en una palabra. No es necesario

¹Tampoco debe confundirse esta noción con el concepto homónimo que refiere a la parte de la memoria de un computador que es normalmente manipulada en forma conjunta por la unidad central de proceso.

que todos ellos estén en cada palabra, ni que cada símbolo que aparezca lo haga una sola vez. En lugar de escribir la secuencia como una lista de símbolos separados por comas y encerrada entre paréntesis, como se acostumbra escribir en otras secuencias en matemática, simplemente se escribirán los símbolos yuxtapuestos.

Ejemplo 20 *clase* es una palabra sobre el alfabeto romano.

01101 es una palabra sobre el alfabeto binario.

\$105.0 es una palabra sobre el alfabeto $\{0, 1, 5, ., \$\}$.

□

Utilizando el isomorfismo natural que existe entre los símbolos de un alfabeto y las palabras sobre ese alfabeto que están compuestas por un único símbolo, se acostumbra identificar esas palabras con el símbolo que las forma. Por lo tanto, se considera que, por ejemplo, el símbolo a es lo mismo que la palabra a . Una palabra puede no tener símbolos. En ese caso se le conoce como *palabra vacía* o *palabra nula*, y se la representa por ε . Debe quedar claro que ε no es un símbolo, sino que es el nombre de una palabra. Lo que sucede es que esa palabra, la palabra vacía, consta de ningún símbolo y, por lo tanto, no se puede representar por la secuencia de los símbolos que la componen, requiriéndose el uso de un nombre para referirse a ella. En general se usarán letras, como U, V, W, X, Y y Z , o sus minúsculas, y algunas letras griegas para denotar palabras es decir, como nombres para ellas. Es obvio que por claridad y para evitar confusiones, no es conveniente utilizar ε o cualquier otro caracter usado como nombre de una palabra, como un símbolo del alfabeto. Formalmente, una palabra sobre un alfabeto Σ cualquiera, se define inductivamente a través de las siguientes reglas:

- ε es una palabra sobre Σ .
- Si x es una palabra sobre Σ y a es un símbolo en Σ , ax es una palabra sobre Σ .

La primera regla asegura que la palabra vacía es una palabra sobre el alfabeto Σ . La segunda regla indica como construir una palabra a partir de otra, basta anteponer cualquier símbolo del alfabeto a los símbolos de la palabra original. Una definición alternativa es suponer que las palabras crecen hacia el lado derecho. Es interesante destacar que ε es una palabra sobre cualquier alfabeto; en realidad, es la única palabra que se puede construir a partir de los símbolos de cualquier alfabeto.

2.2.1 Longitud de una Palabra

Tal como en el caso de los números naturales o, como se vio anteriormente, en el caso de los conjuntos, hay también varias funciones, operaciones y relaciones interesantes de estudiar en las palabras. En este punto y los siguientes, se verán algunas de ellas. Una de las cosas que interesa conocer es el largo de una palabra. Para ello se define la *longitud* de una palabra x , usualmente denotada por $|x|$, como el número de símbolos que componen la palabra. Esta es una función que asigna a cada palabra un número natural. La palabra vacía, al no tener ningún símbolo, tiene longitud cero; es la única palabra, cualquiera sea el alfabeto, con longitud cero.

Ejemplo 21 Considere, por ejemplo, las siguientes palabras

$ clase $	=	5
$ 101 $	=	3
$ \varepsilon $	=	0

□

Formalmente, la función longitud puede definirse inductivamente empleando las dos reglas siguientes:

- $|\varepsilon| = 0$

- Si x es una palabra sobre Σ y $a \in \Sigma : |ax| = 1 + |x|$.

Es decir, la palabra nula tiene longitud cero, y cualquier palabra construida al anteponer un símbolo del alfabeto a otra palabra, tiene una longitud superior en uno, a la longitud de esta última; diferencia que corresponde al símbolo que se está agregando.

2.2.2 Concatenación

Dos palabras sobre un mismo alfabeto pueden ser combinadas para formar una tercera palabra, utilizando la operación conocida como *concatenación*. La concatenación de dos palabras x e y sobre un alfabeto Σ , escrita como $x \circ y$, o simplemente xy , es la palabra formada al escribir los símbolos de la primera, x , seguidos inmediatamente por los símbolos de la segunda, y .

Ejemplo 22 Sean u y v las siguientes palabras sobre el alfabeto romano: $u = ca$ y $v = sa$. Entonces la concatenación de u y v es:

$$u \circ v = uv = casa,$$

y la concatenación de v con u es:

$$v \circ u = vu = saca.$$

□

Formalmente, la operación de concatenación se define inductivamente a través de las siguientes reglas:

- Para toda palabra y sobre $\Sigma : \varepsilon \circ y = y$.
- Para todo símbolo $a \in \Sigma$ y palabras x e y sobre $\Sigma : (ax) \circ y = a(x \circ y)$.

La primera regla indica que la concatenación de la palabra nula con cualquier otra palabra resulta ser, simplemente, esta última palabra. La segunda regla indica cómo hacer la concatenación cuando el primer operando es una palabra compuesta por la anteposición de un símbolo del alfabeto a otra palabra. El Ejemplo 22 dejó en claro que la concatenación de palabras no es conmutativa. Sin embargo, es posible demostrar que sí se trata de una operación asociativa. Es decir, para toda palabra w , x e y , sobre un alfabeto Σ cualquiera,

$$w \circ (x \circ y) = (w \circ x) \circ y.$$

También se puede demostrar que, en realidad, la palabra nula es el elemento neutro en la operación de concatenación de palabras. Es decir, para toda palabra w , sobre un alfabeto cualquiera, se cumple que:

$$w \circ \varepsilon = \varepsilon \circ w = w.$$

Además se cumple la siguiente propiedad que relaciona la función longitud con la operación de concatenación. Para todo par de palabras x e y sobre Σ ,

$$|x \circ y| = |x| + |y|.$$

La notación w^k se usa para representar la concatenación consecutiva de k copias de una misma palabra w . Es decir,

$$w^k = w \circ w \circ \dots \circ w \quad (k \text{ veces})$$

Debe notarse que en la expresión anterior no es necesario emplear paréntesis, puesto que la operación de concatenación es asociativa.

2.2.3 Subpalabras, Prefijos y Sufijos

Una palabra es una *subpalabra* de otra palabra, cuando sus símbolos aparecen entre los símbolos de la segunda, en forma consecutiva y en el mismo orden; es decir cuando un trozo contiguo de la segunda es igual a la primera. Formalmente, una palabra v se dice una subpalabra de otra palabra w , si y sólo si existen dos palabras x e y sobre el alfabeto, tales que $w = x \circ v \circ y$. En la definición anterior, cualquiera de las palabras x e y , o ambas, puede ser la palabra nula. Por lo tanto, si $x = y = \varepsilon$, se concluye que toda palabra es una subpalabra de sí misma. También, si se considera que $x = w$ y que $v = y = \varepsilon$, se ve que la palabra nula es una subpalabra de todas las palabras. Cuando la subpalabra es tal que sus símbolos aparecen al comienzo de la otra palabra, se le llama un *prefijo* de ésta última. Si aparecen al final, se le llama un *sufijo*. Intuitivamente, una palabra es un prefijo de otra, cuando ésta comienza con aquélla; será un sufijo cuando termina con ella. Formalmente, si $w = u \circ v$ para alguna palabra u sobre el alfabeto, v se dice un sufijo de w . En forma similar, si $w = u \circ v$ para alguna palabra v , u se dice un prefijo de w . Según estas definiciones, cada palabra es un sufijo, prefijo y subpalabra de sí misma. Un sufijo, prefijo o subpalabra que no sea la palabra misma se llama un sufijo, prefijo o subpalabra *propia*. Debe notarse que la palabra vacía es un sufijo, prefijo y subpalabra propia de todas las palabras, excepto de sí misma.

Ejemplo 23 PUES es un prefijo de PUESTOS.

PUES es un sufijo de DESPUES.

PUES es una subpalabra de PUESTOS, DESPUES y también de APUESTA.

□

Es interesante notar que en una palabra puede haber varias ocurrencias de una misma subpalabra. Por ejemplo, la palabra binaria 101010 tiene tres ocurrencias de las subpalabras 0, 1 y 10, y dos ocurrencias de las subpalabras 01, 010, 101, 1010. Una ocurrencia incluye, además de la subpalabra que interesa, una posición dentro de la palabra que indica donde comienza esa instancia de la subpalabra.

2.2.4 Reverso

Otra función interesante entre palabras sobre un alfabeto, es la función conocida como el *reverso* de una palabra. Intuitivamente, esta función transforma la palabra dada como argumento, en la palabra formada por los mismos símbolos pero en orden inverso. Formalmente, el reverso de una palabra w , denotada como w^r , se define por las siguientes reglas:

- $\varepsilon^r = \varepsilon$
- Para todo símbolo $a \in \Sigma$ y toda palabra x sobre Σ : $(ax)^r = x^r \circ a$

Éste es otro ejemplo de una definición inductiva. La primera regla define cuál es el reverso de la palabra nula; la segunda, indica cómo determinar el reverso de una palabra compuesta por la anteposición de un símbolo a otra palabra sobre el alfabeto². En esta última regla, a aparece como un símbolo en el lado izquierdo y como una palabra en el lado derecho de la igualdad. El Ejemplo 24 muestra como es posible aprovechar las definiciones inductivas para demostrar ciertas propiedades de las palabras, utilizando el principio de inducción matemática. En particular, se prueba que el reverso de la concatenación de dos palabras es lo mismo que la concatenación, en orden inverso, de los reversos de las palabras originales.

Ejemplo 24 Se muestra que para todo par de palabras x e y , sobre un alfabeto Σ cualquiera, se cumple la siguiente relación:

$$(x \circ y)^r = y^r \circ x^r.$$

²Nótese el parecido de la definición con la forma en que se podría implementar esta función utilizando el lenguaje de programación LISP: (defun reverso (x)(cond ((null x) x)(t (append (reverso (cdr x))(list (car x))))))

Sea y una palabra cualquiera sobre el alfabeto. La demostración se hará por inducción en el largo de la palabra x .

Base ($|x| = 0$): La única palabra con longitud cero es la palabra vacía. Es decir, en este caso $x = \varepsilon$ y, por lo tanto, se cumple que:

$$\begin{aligned}
 (x \circ y)^r &= (\varepsilon \circ y)^r && \text{(porque } x = \varepsilon) \\
 &= y^r && \text{(definición de concatenación)} \\
 &= y^r \circ \varepsilon && \text{(elemento neutro)} \\
 &= y^r \circ \varepsilon^r && \text{(definición del reverso)} \\
 &= y^r \circ x^r && \text{(porque } x = \varepsilon)
 \end{aligned}$$

como se deseaba mostrar.

Inducción ($|x| \geq 0$): La hipótesis de inducción asegura que si la palabra x tiene longitud $n \geq 0$, entonces se cumple que $(x \circ y)^r = y^r \circ x^r$. Se debe demostrar que esta relación también se cumple cuando la longitud de x es $n + 1$. Sea x una palabra de longitud $n + 1 \geq 1$, entonces $x = au$ para algún símbolo $a \in \Sigma$ y alguna palabra u sobre Σ , en que $|u| = n$. Por lo tanto:

$$\begin{aligned}
 (x \circ y)^r &= ((au) \circ y)^r && \text{(porque } x = au) \\
 &= (a(u \circ y))^r && \text{(definición de concatenación)} \\
 &= (u \circ y)^r \circ a && \text{(definición del reverso)} \\
 &= (y^r \circ u^r) \circ a && \text{(hipótesis de inducción)} \\
 &= y^r \circ (u^r \circ a) && \text{(asociatividad)} \\
 &= y^r \circ ((au)^r) && \text{(definición del reverso)} \\
 &= y^r \circ x^r && \text{(porque } x = au)
 \end{aligned}$$

como se quería mostrar.

Por el principio de inducción matemática, ya que se ha mostrado la base y la inducción, se puede concluir que para todo par de palabras x e y , sobre un alfabeto Σ cualquiera, se cumple la relación $(x \circ y)^r = y^r \circ x^r$.

□

2.3 Lenguajes

En esta sección se definirá y estudiará el concepto de lenguaje, noción sobre la que girarán estos apuntes. Un *lenguaje* (formal) sobre un alfabeto, es un conjunto de palabras sobre ese alfabeto. Esta simple definición permite formalizar la idea intuitiva de lenguaje, de forma que abarque los lenguajes naturales, de programación y de otros tipos. En los lenguajes naturales los símbolos son fonemas, letras u otros símbolos, y las palabras son las frases y sentencias que se pueden expresar en ese idioma. En los lenguajes de programación, los símbolos son las palabras reservadas, caracteres y símbolos especiales del lenguaje; las palabras son los programas escritos en dicho lenguaje.

El conjunto vacío, $\emptyset$ y el conjunto cuyo único elemento es la palabra vacía, $\{\varepsilon\}$, tienen la característica de ser lenguajes sobre cualquier alfabeto. Es importante hacer notar que ellos son dos lenguajes absolutamente diferentes. El primero no tiene elementos, mientras que el segundo lenguaje tiene un único elemento: la palabra nula.

Ejemplo 25 El conjunto de palíndromos sobre el alfabeto romano es un lenguaje infinito. Los palíndromos son palabras que tienen la característica de ser iguales a su reverso. Algunos de los elementos de este lenguaje son: ABBA, C, PRZHZRP, RADAR, RECONOCER. La palabra nula, ε , también pertenece a este lenguaje.

Es conveniente destacar que cualquier palabra formada por símbolos del alfabeto, y que se lee igual de derecha a izquierda que de izquierda a derecha, pertenece a este lenguaje. No sólo aquellas que tienen algún significado, ya sea en Castellano o en cualquier otro idioma.

□

Un lenguaje de importancia es aquél formado por todas las palabras que se pueden construir con los símbolos de un alfabeto Σ dado. A este lenguaje se le denota por Σ^* . Hablar de una palabra sobre Σ o de una palabra en Σ^* será, entonces, enteramente equivalente. Es claro que cualquier lenguaje sobre un alfabeto Σ , es un subconjunto de Σ^* y que la clase de todos los lenguajes posibles sobre Σ , es el conjunto potencia 2^{Σ^*} .

Ejemplo 26 Si el alfabeto es $\Sigma = \{a\}$ —un alfabeto con un solo símbolo— el lenguaje de todas las palabras que se pueden construir sobre él es el lenguaje:

$$\Sigma^* = \{\varepsilon, a, aa, aaa, aaaa, \dots\}$$

□

Como se vio anteriormente, un símbolo se identifica con la palabra compuesta sólo por ese símbolo; en consecuencia, cualquier alfabeto Σ es, a su vez, un lenguaje. Visto como tal, este lenguaje tiene las características de ser finito y de estar compuesto únicamente por palabras de longitud uno.

Por otra parte, como los lenguajes son conjuntos, ellos pueden ser combinados por las operaciones usuales para conjuntos, como son la unión, intersección y diferencia. En general, cuando el alfabeto Σ se subentiende, se acostumbra hablar del complemento de A , A^c , en lugar de la diferencia $\Sigma^* - A$. En los próximos dos puntos se definen algunas otras operaciones que se pueden realizar específicamente con lenguajes.

2.3.1 Concatenación de Lenguajes

La *concatenación* de lenguajes es el lenguaje resultante de concatenar las palabras de los lenguajes originales. Si L_1 y L_2 son lenguajes sobre un alfabeto Σ (es decir, $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$), su concatenación es el lenguaje L sobre Σ , definido por:

$$L = L_1 \circ L_2 = L_1 L_2 = \{x \circ y / x \in L_1 \text{ e } y \in L_2\}.$$

Este lenguaje está compuesto por todas las palabras que se forman al concatenar una palabra de L_1 con una palabra de L_2 , en ese orden.

Ejemplo 27 Sean L_1 y L_2 los siguientes lenguajes finitos sobre el alfabeto binario:

$$L_1 = \{01, 1\} \quad L_2 = \{101, 1010\}$$

entonces,

$$L_1 \circ L_2 = \{01101, 011010, 1101, 11010\}$$

y

$$L_2 \circ L_1 = \{10101, 1011, 101001\}$$

□

Como en el caso de las palabras, la concatenación de lenguajes no es conmutativa. El Ejemplo 27 deja esto de manifiesto. Otra observación interesante de hacer es que si L_1 y L_2 son lenguajes finitos con n y m palabras respectivamente, entonces el lenguaje resultante al concatenarlos tiene a lo sumo $n * m$ elementos pero, en general, puede tener menos.

Es posible demostrar que la concatenación de lenguajes es una operación asociativa, igual que lo que sucede con la concatenación de palabras. De hecho, la asociatividad de la concatenación de lenguajes es

producto de la asociatividad de la concatenación de palabras. Es decir, para todo lenguaje L_1 , L_2 y L_3 sobre un alfabeto Σ cualquiera,

$$L_1 \circ (L_2 \circ L_3) = (L_1 \circ L_2) \circ L_3$$

También se cumplen las siguientes propiedades de la concatenación de lenguajes. Para todo lenguaje L sobre un alfabeto Σ cualquiera,

$$L \circ \emptyset = \emptyset \circ L = \emptyset$$

$$L \circ \{\varepsilon\} = \{\varepsilon\} \circ L = L$$

En forma similar al caso de las palabras, la notación L^i se utiliza para representar la concatenación, i veces, de un lenguaje L consigo mismo. Formalmente esta operación se puede definir para todo número natural i , a través de las siguientes reglas inductivas: Para todo lenguaje L :

- $L^0 = \{\varepsilon\}$
- Para todo número natural $i \geq 1$: $L^i = L \circ L^{i-1}$

La idea intuitiva es que para todo número natural i , el lenguaje L^i está formado por la concatenación de exactamente i palabras del lenguaje L . Estas palabras pueden ser distintas o no, no hay restricciones al respecto. Es bueno hacer notar que se cumplen las siguientes propiedades de esta operación.

- Para todo lenguaje L : $L^1 = L$
- $\emptyset^0 = \{\varepsilon\}$
- Para todo número natural $n \geq 1$: $\emptyset^n = \emptyset$

Ejemplo 28 Si L es el lenguaje $\{a, b\}$ sobre el alfabeto romano, se pueden formar los siguientes lenguajes a partir de él:

$$L^0 = \{\varepsilon\}$$

$$L^1 = \{a, b\}$$

$$L^2 = \{aa, ab, ba, bb\}$$

$$L^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$$

□

2.3.2 Clausuras

Una vez definida la concatenación de lenguajes, es posible definir una nueva función sobre los lenguajes, llamada *clausura de Kleene*, o simplemente *clausura*. La clausura de un lenguaje L se define como el lenguaje

$$L^* = \bigcup L^i, \text{ para todo } i \geq 0.$$

También se define la *clausura positiva* de un lenguaje L , como el lenguaje:

$$L^+ = \bigcup L^i, \text{ para todo } i \geq 1.$$

La clausura de un lenguaje L , L^* , denota el lenguaje formado al concatenar cualquier número de palabras de L , incluyendo la posibilidad de cero. La clausura positiva, L^+ , es similar, pero en este caso no se aceptan concatenaciones de cero palabras. El nombre *clausura* para estas funciones proviene del hecho que la clausura, o la clausura positiva, de un lenguaje L , es un lenguaje que incluye a L y es cerrado bajo concatenación; es decir, que al concatenar dos palabras cualesquiera de ellos, la palabra resultante también está en ese lenguaje.

Ejemplo 29 Sea L el lenguaje $\{a, b\}$ sobre el alfabeto romano. La clausura y clausura positiva de este lenguaje, son los lenguajes:

$$L^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

$$L^+ = \{a, b, aa, ab, ba, bb, aaa, aab, aba, \dots\}$$

□

Esta definición de las clausuras hace que las siguientes propiedades se cumplan para lenguajes sobre cualquier alfabeto:

- Para todo lenguaje $L : L \subseteq L^+ \subseteq L^*$.
- $\varepsilon \in L^+$ si y sólo si $\varepsilon \in L$.
- Para todo lenguaje $L : \varepsilon \in L^*$.

En particular, para el lenguaje vacío y para aquél que sólo consta de la palabra nula se cumplen las siguientes propiedades:

$$\emptyset^* = \{\varepsilon\}^* = \{\varepsilon\}$$

Nótese que el uso de Σ^* para denotar el conjunto de todas las palabras sobre Σ es consistente con la notación de la clausura del alfabeto Σ , visto como el lenguaje finito que es.

Ejemplo 30 En este ejemplo se muestra como es posible usar las definiciones ya vistas, para obtener algunas conclusiones sobre los lenguajes. En particular, considere el lenguaje

$$L = \{w \in \{0, 1\}^* / w \text{ tiene distinto número de ceros (0) que de unos (1)}\}.$$

Se mostrará que $L^* = \{0, 1\}^*$.

Primero, nótese que por la definición de la clausura de Kleene, se tiene que para cualquier par de lenguajes L_1 y L_2 : si $L_1 \subseteq L_2$, entonces $L_1^* \subseteq L_2^*$.

Ya que tanto la palabra 0 como la palabra 1 tienen diferente número de ceros que de unos, se sabe que $\{0, 1\} \subseteq L$ y, por lo tanto, que $\{0, 1\}^* \subseteq L^*$.

Pero por la definición de L , también se sabe que $L^* \subseteq \{0, 1\}^*$. Y, por lo tanto, ya que cada uno es subconjunto del otro, se concluye que $L^* = \{0, 1\}^*$.

□

2.3.3 Representación de Lenguajes

Un problema central en la teoría de la computación es la representación de lenguajes empleando especificaciones finitas. Naturalmente, cualquier lenguaje finito es representable por la enumeración explícita de todas y cada una de las palabras en el lenguaje. El problema de la representación finita se hace interesante sólo en la medida que se consideran lenguajes infinitos. Pero, ¿qué es una especificación finita de un lenguaje? ¿qué características debe cumplir para ser aceptable como tal? Lo primero que se puede decir es que debe, a su vez, ser una palabra, es decir, una secuencia finita de símbolos tomados de algún alfabeto. En segundo lugar, interesa que sean tales que lenguajes diferentes tengan representaciones diferentes, de otra forma difícilmente se les podría llamar representación del lenguaje.

El problema es que estos dos requisitos ya implican que las posibles especificaciones finitas están seriamente limitadas. El conjunto Σ^* de palabras sobre un alfabeto Σ es infinito contable, por lo que el número de posibles representaciones de lenguajes es, a su vez, infinito contable. Pero, por otro lado, el conjunto de todos los posibles lenguajes sobre un alfabeto Σ —esto es, 2^{Σ^*} — es incontable, puesto que 2^N y, por lo tanto, el conjunto potencia de cualquier conjunto infinito contable, es incontable. Al tener sólo un número

contable de representaciones y un número incontable de cosas por representar, no debe extrañar que no sea posible representar en forma finita a todos los lenguajes. En realidad, a lo más que se puede aspirar es a encontrar una representación finita, de algún tipo, para al menos algunos de los lenguajes más interesantes.

Éste es el primer resultado importante de la teoría de computación que se ha obtenido en estos apuntes: No importando cuán poderosos puedan ser los métodos para describir lenguajes, sólo un número contable de lenguajes puede ser representado usando especificaciones finitas. Como hay un número incontable de lenguajes, un número también incontable de ellos quedará irremediabilmente fuera de cualquier esquema de representación finita. No es posible hablar de todos los lenguajes que existen, simplemente no hay suficientes nombres para referirse a cada uno de ellos.

En el resto de estos apuntes se estudiarán diversas formas para describir y representar lenguajes, cada una más poderosa que la anterior, en el sentido de que es capaz de describir lenguajes indescriptibles por la forma previa. Esta jerarquía de esquemas no contradice el hecho que todas ellas son inevitablemente limitadas en los lenguajes que pueden representar. Al final de los apuntes, se verán formas de exhibir lenguajes particulares que no pueden ser representados por ninguno de los métodos que se estudiarán. Se sabe que el mundo de los lenguajes está plagado por un vasto número de estos lenguajes, sin embargo puede ser muy difícil encontrar uno en particular y demostrarlo como tal. La técnica de diagonalización será muy útil para estos efectos.

2.4 Autómatas

Un autómata finito es un modelo matemático de un sistema con entrada y salida discretas. El sistema puede estar en cualquiera de un conjunto finito de configuraciones internas o *estados*. El estado de un sistema resume la información de las entradas pasadas, pero sólo en lo que es necesario para determinar el comportamiento del sistema en las entradas por venir.

Ejemplos de sistemas de estado finito son el mecanismo de control de un ascensor, la unidad de control de un computador, ciertos editores de texto y los analizadores léxicos de los compiladores.

Los analizadores léxicos procesan los símbolos que componen un programa de computador para ubicar los que corresponden a identificadores, números, palabras reservadas, etc. En este proceso sólo es necesario recordar una cantidad finita de información, como por ejemplo qué tan largo ha sido el prefijo de una palabra reservada que ya se ha visto.

El computador mismo puede ser visto como una máquina de estado finito. Teóricamente, el estado de la CPU, memoria y almacenamiento secundario es, en cada instante, uno de un conjunto muy grande, pero finito, de estados posibles; provisto, por supuesto, que hay un número fijo de discos, cintas, etc., y que la memoria no puede agrandarse indefinidamente. Sin embargo, este modelo no resulta muy útil pues impone un límite artificial en la capacidad de memoria y, por lo tanto, impide notar la esencia de lo que es una computación.

Antes de estudiar más formalmente los sistemas de estado finito, se presentará un ejemplo de ellos.

Ejemplo 31 Un hombre, un lobo, una cabra y un repollo están en la orilla izquierda de un río. Existe un bote con capacidad para transportar al hombre y sólo una de las otras tres cosas. El hombre quiere cruzar a la otra orilla con todos y es capaz de acarrear a cualquiera de ellos en el bote. Sin embargo, si el hombre deja al lobo y a la cabra juntos en una orilla, el lobo comerá a la cabra. Algo similar sucede si la cabra y el repollo quedan en una orilla sin la presencia del hombre. El problema es saber si es posible y cómo puede el hombre pasar a todos a la otra orilla.

El problema se modela observando que la información que interesa son los ocupantes en cada orilla, después de cada cruce del hombre en bote. Hay 16 subconjuntos del hombre (H), lobo (L), cabra (C) y repollo (R). Un *estado* corresponde al subconjunto que está en la orilla izquierda (en la derecha está el complemento).

Los nombres de los estados corresponden entonces a pares como $LR-HC$, en que los símbolos a la izquierda del guión forman el subconjunto que está en la orilla izquierda del río. Algunos de los estados, como $CR-HL$, son fatales y el sistema nunca debe llegar a ellos.

Las entradas al sistema son las acciones que el hombre toma. Puede cruzar solo (h), con el lobo (l), con la cabra (c) o con el repollo (r). La configuración inicial es el estado $HLCR-\emptyset$ y el estado final deseado es $\emptyset-HLCR$. El diagrama de transición de este sistema se muestra en la Figura 2.1.

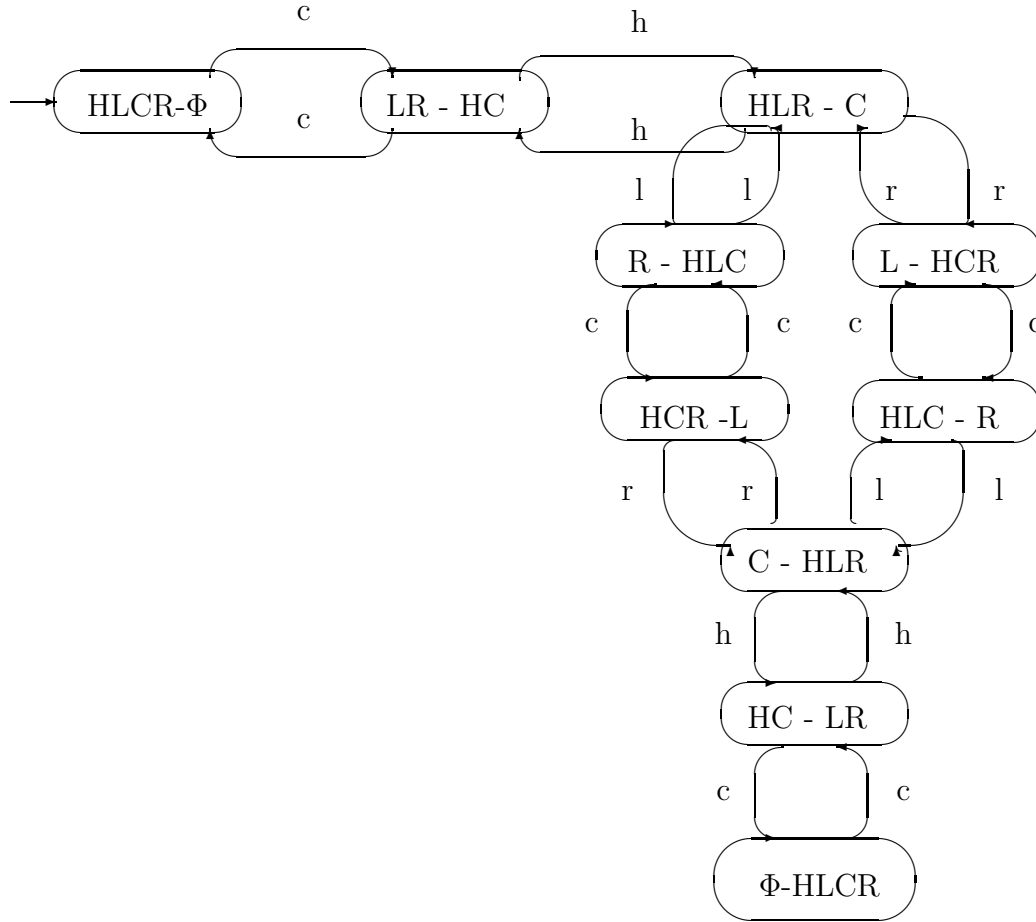


Figure 2.1: Diagrama de transición para el sistema H L C R

Hay dos soluciones igualmente cortas al problema, según puede verse al buscar caminos entre el estado inicial y final (representado con línea doble). En realidad hay una infinidad de soluciones distintas al problema, pero sólo dos no emplean ciclos inútiles. Este sistema de estado finito puede verse como la definición de un lenguaje infinito, el conjunto de todos los strings que son etiquetas de caminos entre el estado inicial y el final. Es decir el conjunto de todas las secuencias de acciones que puede emprender el hombre para resolver su problema.

Debe notarse que hay al menos dos características de este sistema que lo hacen atípico. Primero, hay sólo un estado final, cuando en general puede haber varios. Segundo, sucede que, para cada transición hay una transición invertida en el mismo símbolo, lo que no es necesario que suceda en general.

También es conveniente notar que el término “estado final”, aunque tradicional, no implica una detención del proceso. El sistema puede seguir haciendo transiciones, por ejemplo, al estado $HC-LR$ en el caso anterior.

□

Chapter 3

ACEPTACIÓN Y GENERACIÓN DE LENGUAJES REGULARES

En este capítulo se estudiarán los lenguajes regulares, sus dispositivos de aceptación y de generación.

3.1 Autómatas Finitos Determinísticos

Un *autómata finito* (AF) consta de un conjunto finito de *estados* y un conjunto de *transiciones* de estado a estado, que ocurren en símbolos tomados de un alfabeto Σ . Por cada símbolo hay exactamente una transición desde cada estado. Un estado, usualmente denominado q_0 , es el *estado inicial* en el que el autómata comienza; algunos estados se designan como *estados finales* o *de aceptación*.

Un grafo dirigido, llamado *diagrama de transición*, es asociado con un AF como se indica a continuación. Los vértices del grafo corresponden a los estados del AF. Si hay una transición del estado q al estado p en símbolo a , entonces hay un arco con etiqueta a , desde el estado q al estado p en el diagrama de transición. El AF acepta un string x si y sólo si la secuencia de transiciones que corresponden a los símbolos de x , llevan desde el estado inicial a uno de los estados de aceptación.

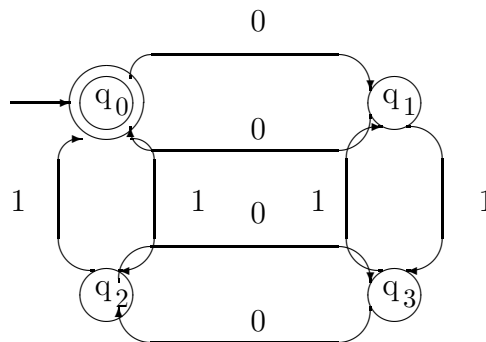


Figure 3.1: Autómata que acepta los strings binarios con número par de 0's y número par de 1's

Ejemplo 32 En la Figura 3.1 el estado inicial q_0 está indicado por la flecha. Hay sólo un estado final, también q_0 en este caso, indicado por el círculo doble. Este autómata acepta todos los strings binarios en que hay un número par de 0's y un número par de 1's.

□

Formalmente, un *autómata finito determinístico* es una quintupla:

$$(Q, \Sigma, \delta, q_0, F)$$

en que Q es un conjunto finito de *estados*, Σ es un *alfabeto de entrada*, $q_0 \in Q$ es el *estado inicial*, $F \subseteq Q$ es el conjunto de *estados finales* y δ es la *función de transición* que va de $Q \times \Sigma$ a Q . Esto es, $\delta(q, a) \in Q$ para todo $q \in Q$ y $a \in \Sigma$.

Como se aprecia en la Figura 3.2, un autómata finito se visualiza como un *control finito*, que está en alguno de los estados de Q , leyendo una secuencia de símbolos de Σ escritos sobre una cinta.

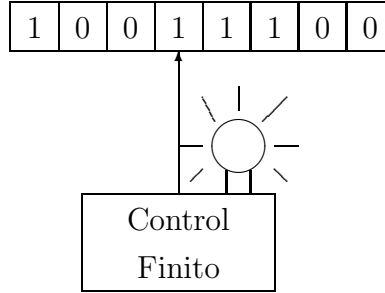


Figure 3.2: Representación de un autómata finito

En una *movida*, el AF en estado q y viendo el símbolo a , entra al estado $\delta(q, a)$ y mueve su *cabeza lectora* un símbolo hacia la derecha sobre la cinta. Si $\delta(q, a)$ es un estado de aceptación, el AF habría aceptado el prefijo del string escrito en la cinta, a la izquierda del símbolo sobre el cual recién llegó la cabeza lectora, sin incluirlo. Si la cabeza lectora se ha salido del final de la cinta (por la derecha), entonces acepta el string completo. Nótese que mientras se mueve sobre el string, el AF puede aceptar (o no) muchos prefijos diferentes.

Para describir formalmente el comportamiento de un AF en un string, es necesario extender la función de transición δ , de forma tal que actúe sobre un string y un estado, en lugar de un estado y un símbolo. Es decir, se quiere una función $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$. La intención es que $\hat{\delta}(q, w)$ sea el estado en que el AF estaría leyendo w a partir del estado q . Dicho de otra forma, $\hat{\delta}(q, w)$ es el estado p (único) tal que hay un camino de q a p en el diagrama de transición y en que las etiquetas de sus arcos forman w .

Formalmente:

- $\hat{\delta}(q, \varepsilon) = q$
- Para todo string $w \in \Sigma^*$ y símbolo $a \in \Sigma$, $\delta(\hat{\delta}(q, w), a)$

La primera parte de esta definición asegura que el AF no puede cambiar de estado sin leer símbolos. La segunda, indica cómo encontrar el estado en que quedará después de leer un string no vacío wa .

Dado que $\hat{\delta}(q, a) = \delta(\hat{\delta}(q, \varepsilon), a) = \delta(q, a)$, no hay diferencia entre δ y $\hat{\delta}$ para aquellos argumentos en que ambas están definidas. Por lo tanto, por conveniencia, se usará δ en lugar de $\hat{\delta}$, siempre.

En general, se tratará de usar los mismos símbolos para significar las mismas cosas a través de todo el material para autómatas finitos. En particular se usarán los siguientes:

- Q es un conjunto de estados. Los símbolos p y q , con o sin subíndice serán estados. El estado inicial será q_0 .
- Σ es un alfabeto de símbolos de entrada. Los símbolos a y b , con o sin subíndice, y los dígitos, serán símbolos de entrada.
- δ es la función de transición de un AF.

- F es el conjunto de estados finales de un AF.
- w , x , y y z , con o sin subíndice, serán strings de símbolos de entrada.

Se dice que un string x es aceptado por un autómata finito $M = (Q, \Sigma, \delta, q_0, F)$ si y sólo si

$$\delta(q_0, x) \in F$$

El lenguaje aceptado por M , llamado $L(M)$, es el conjunto

$$\{x / \delta(q_0, x) \in F\}$$

Un lenguaje es un *conjunto regular* o, simplemente, es *regular* si es el conjunto aceptado por algún AF.

Debe notarse que al hablar del conjunto aceptado por un autómata finito, se está refiriendo específicamente al conjunto $L(M)$ y no a cualquier conjunto de strings aceptados por M que, en general, será sólo un subconjunto.

Ejemplo 33 Considere el autómata finito descrito por el diagrama de transición del ejemplo anterior (véase Figura 3.1). Su descripción formal es $M = (Q, \Sigma, \delta, q_0, F)$, en que

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = q_0$$

$$F = \{q_0\}$$

y δ es la función descrita por la siguiente tabla de transición:

$Q \setminus \Sigma$	0	1
q_0	q_1	q_2
q_1	q_0	q_3
q_2	q_3	q_0
q_3	q_2	q_1

Suponga que el input a M es el string binario 110101, entonces

$$\delta(q_0, 11) = \delta(\delta(q_0, 1), 1) = \delta(q_2, 1) = q_0$$

es decir, el prefijo 11 del input pertenece a $L(M)$, la ampolleta del autómata se enciende al procesarlo; sin embargo interesa el string completo y así,

$$\delta(q_0, 110) = \delta(\delta(q_0, 11), 0) = \delta(q_0, 0) = q_1$$

$$\delta(q_0, 1101) = \delta(\delta(q_0, 110), 1) = \delta(q_1, 1) = q_3$$

$$\delta(q_0, 11010) = \delta(\delta(q_0, 1101), 0) = \delta(q_3, 0) = q_2$$

$$\delta(q_0, 110101) = \delta(\delta(q_0, 11010), 1) = \delta(q_2, 1) = q_0 \in F$$

es decir, la secuencia de estados es:

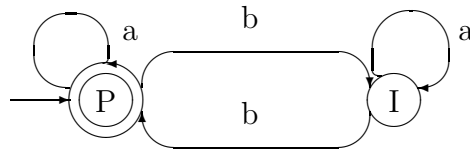
$$\begin{array}{cccccc} 1 & 1 & 0 & 1 & 0 & 1 \\ q_0 & q_2 & q_0 & q_1 & q_3 & q_2 & q_0 \end{array}$$

y el string $110101 \in L(M)$.

□

Ejemplo 34 Un autómata finito que acepte todos los strings sobre $\Sigma = \{a, b\}$, que tengan un número par de b 's. (Ver Figura 3.3)

Formalmente el autómata es $M = (Q, \Sigma, \delta, q_0, F)$, en que

Figure 3.3: Autómata finito que acepta los strings con número par de b 's

$$Q = \{P, I\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = P$$

$$F = \{P\}$$

y la función δ :

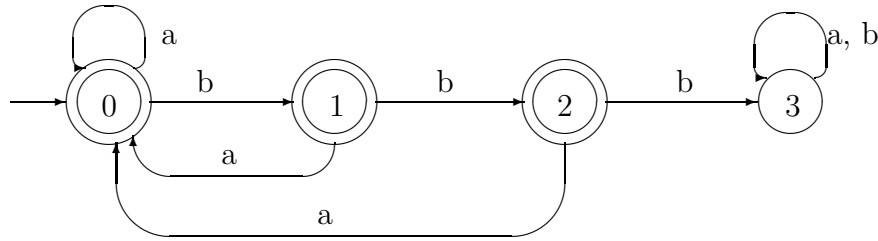
$Q \setminus \Sigma$	a	b
P	P	I
I	I	P

El autómata pasa de P a I y de I a P al leer una b ; los símbolos a son esencialmente ignorados al permanecer en el mismo estado. Es decir, M cuenta las b 's en módulo 2 y como P es el estado inicial y único estado final, M acepta los strings que tienen un número par de b 's.

□

Ejemplo 35 Un autómata finito (ver Figura 3.4) que acepta el lenguaje

$$L(M) = \{w/w \in \{a, b\}^* \text{ y } w \text{ no tiene tres } b\text{'s consecutivas}\}$$

Figure 3.4: Autómata finito que acepta strings que no tienen tres b 's consecutivas

formalmente, $M = (Q, \Sigma, \delta, q_0, F)$, en que

$$Q = \{0, 1, 2, 3\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = 0$$

$$F = \{0, 1, 2\}$$

y la función δ :

$Q \setminus \Sigma$	a	b
0	0	1
1	0	2
2	0	3
3	3	3

□

3.2 Autómatas Finitos No Determinísticos

En esta sección se introduce el concepto de no-determinismo en un autómata finito. Como se verá más adelante, cualquier conjunto aceptado por un autómata finito no determinístico es aceptado por un autómata finito determinístico. Sin embargo, la no-determinación es un concepto útil para probar teoremas y simplificar la descripción de los autómatas. Más aún, el concepto de indeterminación es central en la teoría de lenguajes y computación y es útil entenderlo en un contexto simple como el de estos autómatas. Después se verán autómatas con versiones determinísticas y no-determinísticas que, se sabe, no son equivalentes o en que la equivalencia está todavía no resuelta.

Considere una modificación del modelo de autómatas finitos que permita cero, una o más transiciones desde un estado en un mismo símbolo del alfabeto. Este modelo es llamado un *autómata finito no determinístico* (AFND). Un diagrama de transición para un AFND se muestra a continuación.

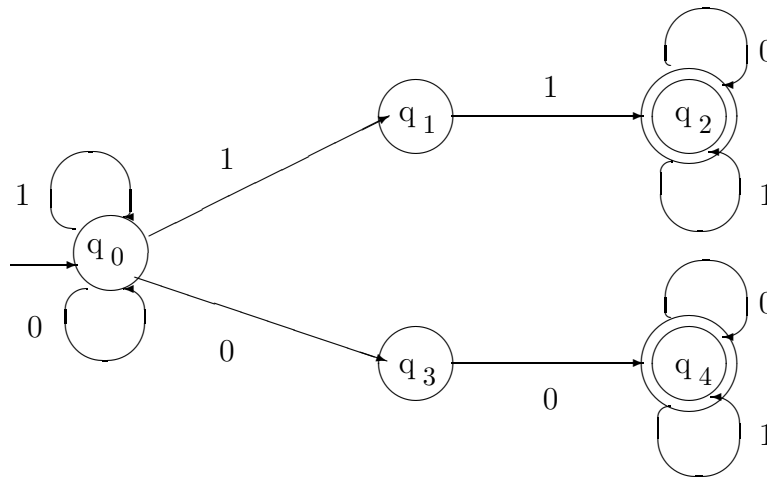


Figure 3.5: Diagrama de transición para un AFND

Ejemplo 36 Considérese el autómata de la Figura 3.5. En él hay dos arcos con etiqueta 0 que salen desde q_0 , uno vuelve a q_0 y el otro va al estado q_3 .

Un string de símbolos $a_1a_2 \dots a_n$ es aceptada por un autómata finito no determinístico si existe una secuencia de transiciones, correspondientes al string, que lleve desde el estado inicial a algún estado final. Por ejemplo, 01001 es aceptado por el AFND de la Figura 3.5, porque hay una secuencia de transiciones, a través de $q_0, q_0, q_0, q_3, q_4, q_4$, cuyas etiquetas son 01001, que van del estado inicial q_0 al estado final q_4 . Nótese que, el que haya una secuencia (por ejemplo $q_0, q_0, q_0, q_0, q_0, q_1$) que no conduce a un estado final no importa; es decir, el no determinismo no molesta, basta que haya una secuencia para que el string sea aceptado. El AFND del ejemplo acepta todos los strings binarios que tienen dos 1's ó dos 0's consecutivos.

□

El autómata finito de la Sección 3.1, es un caso especial del AFND, en que todos los estados tienen una transición única en cada símbolo. Es decir, en un AFD, por cada string w y estado q , hay exactamente un camino con etiqueta w que comienza en q . Para saber si el AFD acepta w , es suficiente revisar ese camino. Para un AFND, en cambio, puede haber muchos caminos posibles y todos deben revisarse, en general, para saber si al menos uno conduce a un estado final.

Un AFND también puede ser visto como un control finito que lee una cinta. Sin embargo, en este caso, el control finito puede estar, a cada instante, en cualquiera de un grupo de estados. Cuando es posible escoger el próximo estado, se puede imaginar que se producen copias del autómata. Por cada próximo estado posible, hay una copia del autómata cuyo control finito está en ese estado. La Figura 3.6 muestra este proceso para el AFND del Ejemplo 36, cuando lee el string 01001.

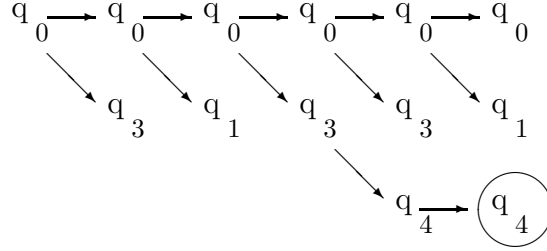


Figure 3.6: Secuencia de pasos al procesar el string 01001

Formalmente, un autómata finito no determinístico es una quintupla:

$$(Q, \Sigma, \delta, q_0, F)$$

en que Q , Σ , q_0 y F tienen el mismo significado que para el autómata finito determinístico, pero δ es una función que va de $Q \times \Sigma$ a 2^Q , es decir:

$$\delta : Q \times \Sigma \rightarrow 2^Q$$

La idea es que $\delta(q, a)$ es el conjunto de todos los estados a los que hay una transición desde q con etiqueta a . Recuerde que 2^Q es el conjunto potencia de Q , el conjunto de todos los subconjuntos de Q .

Ejemplo 37 La función de transición para el AFND de la Figura 3.5 está dada por:

$Q \setminus \Sigma$	0	1
q_0	$\{q_0, q_3\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$

□

Nuevamente es posible extender la función de transición δ a la función

$$\hat{\delta} : Q \times \Sigma^* \rightarrow 2^Q$$

para reflejar el comportamiento de un AFND en un string:

- $\hat{\delta}(q, \varepsilon) = \{q\}$
- $\hat{\delta}(q, wa) = \{p / \text{para algún estado } r \in \hat{\delta}(q, w), p \in \delta(r, a)\}$

La primera condición impide cambios sin procesar símbolos. La segunda, indica que comenzando en estado q y leyendo el string w , seguido del símbolo a , es posible estar en un estado p , si y sólo si r es uno de los estados en que se puede estar luego de leer w , y desde r es posible ir a p leyendo a .

Nótese que $\hat{\delta}(q, a) = \delta(q, a)$, para todo $a \in \Sigma$ y $q \in Q$. Por lo tanto, nuevamente se usará δ en lugar de $\hat{\delta}$.

También es útil extender δ a argumentos en $2^Q \times \Sigma^*$ a través de

$$\delta(P, w) = \bigcup_{q \in P} \delta(q, w)$$

$\forall P \in 2^Q, w \in \Sigma^*$ es decir, es el conjunto de todos los estados a los que se puede llegar, partiendo de algún estado en P , al leer el string w .

El lenguaje aceptado por un AFND, $M = (Q, \Sigma, \delta, q_0, F)$, es el conjunto:

$$L(M) = \{x / F \cap \delta(q_0, x) \neq \emptyset\}$$

Ejemplo 38 Para el AFND de la Figura 3.5 considere el string 01001.

$$\delta(q_0, 0) = \{q_0, q_3\}$$

$$\begin{aligned} \delta(q_0, 01) &= \delta(\delta(q_0, 0), 1) = \delta(\{q_0, q_3\}, 1) \\ &= \delta(q_0, 1) \cup \delta(q_3, 1) = \{q_0, q_1\} \end{aligned}$$

similarmente,

$$\delta(q_0, 010) = \{q_0, q_3\}$$

$$\delta(q_0, 0100) = \{q_0, q_3, q_4\}$$

y, finalmente,

$$\delta(q_0, 01001) = \{q_0, q_1, q_4\}$$

Nótese que

$$F \cap \delta(q_0, 01001) = \{q_4\} \neq \emptyset$$

□

Ejemplo 39 Un autómata finito no determinístico, M (ver Figura 3.7), que acepte el lenguaje:

$$L(M) = \{w / w \in \{a, b\}^* \text{ y } w \text{ tiene tres } b\text{'s consecutivas} \}$$

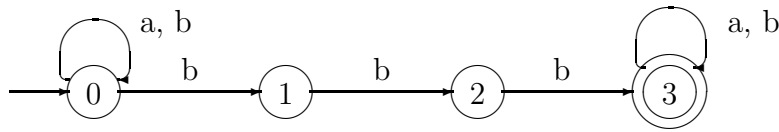


Figure 3.7: Autómata finito que acepta strings con tres b 's consecutivas

Formalmente, $M = (Q, \Sigma, \delta, q_0, F)$, en que

$$Q = \{0, 1, 2, 3\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = 0$$

$$F = \{3\}$$

y la función de transición δ :

$Q \setminus \Sigma$	a	b
0	$\{0\}$	$\{0, 1\}$
1	$\emptyset$	$\{2\}$
2	$\emptyset$	$\{3\}$
3	$\{3\}$	$\{3\}$

□

Dos autómatas finitos M_1 y M_2 se dicen *equivalentes* si y sólo si ellos aceptan el mismo lenguaje, es decir, si y sólo si

$$L(M_1) = L(M_2)$$

no importa qué método usen para reconocer el lenguaje, son equivalentes si aceptan el mismo lenguaje.

Ejemplo 40 El AFD de la Figura 3.8 es equivalente al AFND del Ejemplo 39.

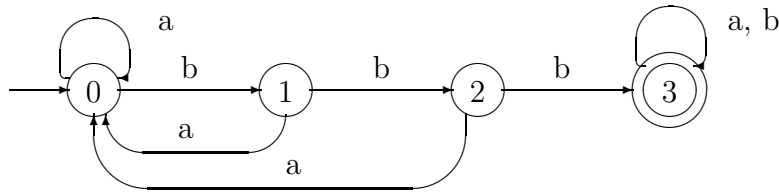


Figure 3.8: AFD que acepta strings con tres b 's consecutivas

□

Como todo AFD es un AFND, es claro que la clase de lenguajes aceptados por los AFND incluye a los lenguajes regulares (aceptados por los AFD). Pero hay más, sucede que estos son los únicos lenguajes aceptados por los AFND. La prueba se basa en mostrar que los AFD pueden simular a los AFND; ésto es, por cada AFND es posible construir un AFD equivalente.

La forma de simular un AFND con un AFD es permitir que los estados del AFD correspondan a conjuntos de estados del AFND, de manera que el AFD pueda almacenar en su control finito todos aquellos estados en que el AFND podría estar, habiendo leído el mismo prefijo del input. La construcción formal se incluye en la demostración del siguiente teorema:

Teorema 1 Sea L un lenguaje aceptado por un autómata finito no determinístico. Existe un autómata finito determinístico que acepta L .

Demostración : Sea $M = (Q, \Sigma, \delta, q_0, F)$ el AFND que acepta L . Defina un autómata finito determinístico $M' = (Q', \Sigma, \delta', q_0', F')$ como sigue: Los estados de M' son todos los subconjuntos del conjunto de estados de M , es decir, $Q' = 2^Q$. M' tendrá en sus estados la información de todos los estados en que M podría estar. F' es el conjunto de todos los estados en Q' que contienen al menos un estado final de M . Un estado en Q' se denotará por $[q_1, q_2, \dots, q_i]$ en que $\{q_1, q_2, \dots, q_i\} \in Q$. Nótese que $[q_1, q_2, \dots, q_i]$ es un solo estado del AFD M' , sólo que su nombre es compuesto. También se tiene que $q_0' = [q_0]$. Y se define

$$\delta'([q_1, q_2, \dots, q_i], a) = [p_1, p_2, \dots, p_j]$$

si y sólo si

$$\delta([q_1, q_2, \dots, q_i], a) = \{p_1, p_2, \dots, p_j\}.$$

Es decir, δ' aplicado a un estado $[q_1, q_2, \dots, q_i]$ de Q' , se calcula aplicando δ a cada estado de Q representado por el estado $[p_1, p_2, \dots, p_j]$ en Q' , el que es el valor de esta aplicación de la función.

Es fácil mostrar, por inducción en la longitud del string x , que

$$\delta'(q_0', x) = [q_1, q_2, \dots, q_i]$$

si y sólo si

$$\delta(q_0, x) = \{q_1, q_2, \dots, q_i\}$$

Base ($|x| = 0$): Entonces $x = \varepsilon$ y se tiene

$$\delta'(q_0', x) = \delta'(q_0', \varepsilon) = q_0' = [q_0]$$

Inducción: Asuma que la hipótesis se cumple para strings de largo n y considere xa , un string de largo $n + 1$, con $|x| = n$, $x \in \Sigma^*$ y $a \in \Sigma$. Entonces:

$$\delta'(q_0', xa) = \delta'(\delta'(q_0', x), a)$$

pero por la hipótesis

$$\delta'(q_0', x) = [p_1, p_2, \dots, p_j]$$

si y sólo si

$$\delta(q_0, x) = \{p_1, p_2, \dots, p_j\}$$

pero por la definición de δ' ,

$$\delta'([p_1, p_2, \dots, p_j], a) = [r_1, r_2, \dots, r_k]$$

si y sólo si

$$\delta([p_1, p_2, \dots, p_j], a) = \{r_1, r_2, \dots, r_k\}.$$

Por lo tanto,

$$\delta'(q_0', xa) = [r_1, r_2, \dots, r_k]$$

si y sólo si

$$\delta(q_0, xa) = \{r_1, r_2, \dots, r_k\}$$

como se quería demostrar. Sólo falta agregar que $\delta'(q_0', x) \in F'$ exactamente cuando $\delta(q_0, x)$ contiene un estado de Q que está en F . Por lo tanto

$$L(M) = L(M')$$

□

Ejemplo 41 Considere el AFND que reconoce los strings que tienen tres b 's consecutivas, visto en el ejemplo anterior. Se construirá un AFD, a partir de él, usando el método implícito en el teorema 1. (Ver Figura 3.9)

Es conveniente comenzar con $[q_0]$ y agregar estados sólo a medida que aparecen como transiciones desde otros ya incluidos, porque la mayoría de los estados (en general) no son accesibles desde $[q_0]$ y, por lo tanto, son inútiles.

Nótese que el AFD anterior acepta el mismo lenguaje que el AFND del cual se partió y también que otro AFD visto anteriormente para el mismo lenguaje. Todos ellos son equivalentes.

□

Ejemplo 43 La función de transición para el AFND- ε anterior está dada por

$Q \setminus \Sigma \cup \{\varepsilon\}$	0	1	2	ε
q_0	$\{q_0\}$	$\emptyset$	$\emptyset$	$\{q_1\}$
q_1	$\emptyset$	$\{q_1\}$	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$	$\{q_2\}$	$\emptyset$

□

Nuevamente es conveniente extender la función de transición a una nueva función

$$\hat{\delta} : Q \times \Sigma^* \rightarrow 2^Q$$

de tal forma que $\hat{\delta}(q, w)$ contenga todos los estados a los que se puede llegar desde q por caminos con etiqueta w ; sin descartar la posible inclusión entre éstos de arcos con etiqueta ε .

Para definir $\hat{\delta}$, es importante calcular el conjunto de todos los estados alcanzables desde algún estado q , sin consumir input, sólo por transiciones en vacío. Esto es equivalente a encontrar el conjunto de vértices alcanzables desde un vértice dado en un grafo dirigido; el vértice es el estado q y el grafo dirigido es el diagrama de transición con todos y sólo los arcos que tienen etiqueta ε , hacia ellos desde q se le denotará por $clausura-\varepsilon(q)$, la clausura vacía de q .

Ejemplo 44 En el AFND- ε anterior, se tiene:

$$clausura-\varepsilon(q_0) = \{q_0, q_1, q_2\}$$

$$clausura-\varepsilon(q_1) = \{q_1, q_2\}$$

$$clausura-\varepsilon(q_2) = \{q_2\}$$

□

Es natural extender la clausura vacía a un conjunto de estados como sigue:

$$clausura-\varepsilon(P) = \bigcup_{q \in P} clausura-\varepsilon(q) \quad \forall P \subseteq Q$$

Ahora es posible definir la función de transición extendida a strings, $\hat{\delta}$:

- $\hat{\delta}(q, \varepsilon) = clausura-\varepsilon(q)$
- Para todo $w \in \Sigma^*$, $a \in \Sigma$ y $q \in Q$

$$\hat{\delta}(q, wa) = clausura-\varepsilon(P),$$

en que:

$$P = \{p / \exists r \in \hat{\delta}(q, w) \text{ y } p \in \delta(r, a)\}$$

Nuevamente es conveniente extender δ y $\hat{\delta}$ a conjuntos de estados, a través de:

- $\delta(P, a) = \bigcup_{q \in P} \delta(q, a) \quad \forall P \subseteq Q \text{ y } a \in \Sigma \cup \{\varepsilon\}$
- $\hat{\delta}(P, w) = \bigcup_{q \in P} \hat{\delta}(q, w) \quad \forall P \subseteq Q \text{ y } w \in \Sigma^*$

Nótese que a diferencia de los casos anteriores, $\hat{\delta}(q, a)$ no es necesariamente igual a $\delta(q, a)$, ya que el primero incluye los estados alcanzables desde q por caminos con etiqueta a (incluyendo posiblemente arcos con etiqueta ε), mientras que el segundo incluye sólo aquellos estados alcanzables desde q por un arco con etiqueta a . Similarmente, $\hat{\delta}(q, \varepsilon)$ es distinto de $\delta(q, \varepsilon)$. Por lo tanto, si se está hablando de un AFND- ε es necesario distinguir entre δ y $\hat{\delta}$.

El lenguaje aceptado por un AFND- ε , $M = (Q, \Sigma, \delta, q_0, F)$ es el conjunto

$$L(M) = \{x / F \cap \hat{\delta}(q_0, x) \neq \emptyset\}$$

Ejemplo 45 Para el AFND- ε anterior considere el string 01.

$$\hat{\delta}(q_0, \varepsilon) = \text{clausura} - \varepsilon(q_0) = \{q_0, q_1, q_2\}$$

$$\begin{aligned} \hat{\delta}(q_0, 0) &= \text{clausura} - \varepsilon(\delta(\hat{\delta}(q_0, \varepsilon), 0)) \\ &= \text{clausura} - \varepsilon(\delta(\{q_0, q_1, q_2\}, 0)) \\ &= \text{clausura} - \varepsilon(\delta(\{q_0\}, 0) \cup \delta(\{q_1\}, 0) \cup \delta(\{q_2\}, 0)) \\ &= \text{clausura} - \varepsilon(\{q_0\} \cup \emptyset \cup \emptyset) \\ &= \text{clausura} - \varepsilon(\{q_0\}) \\ &= \text{clausura} - \varepsilon(q_0) \\ &= \{q_0, q_1, q_2\} \end{aligned}$$

luego,

$$\begin{aligned} \hat{\delta}(q_0, 01) &= \text{clausura} - \varepsilon(\delta(\hat{\delta}(q_0, 0), 1)) \\ &= \text{clausura} - \varepsilon(\delta(\{q_0, q_1, q_2\}, 1)) \\ &= \text{clausura} - \varepsilon(q_1) \\ &= \{q_1, q_2\} \end{aligned}$$

es decir, el AFND- ε acepta el string 01 ya que

$$\hat{\delta}(q_0, 01) \cap F = \{q_1, q_2\} \cap \{q_2\} = \{q_2\} \neq \emptyset$$

□

Como todo AFND es un AFND- ε , es claro que la clase de lenguajes aceptados por los AFND- ε incluye a los lenguajes aceptados por los AFND, los lenguajes regulares. Pero hay más, sucede que éstos son los únicos lenguajes aceptados por los AFND- ε . La prueba se basa en mostrar que los AFND pueden simular los AFND- ε ; esto es: por cada AFND- ε , es posible construir un AFND equivalente.

Teorema 2 Sea L un lenguaje aceptado por un autómata finito no determinístico con transiciones en vacío. Existe un autómata finito no determinístico que acepta L .

Demostración : Sea $M = (Q, \Sigma, \delta, q_0, F)$ el AFND- ε que acepta L . Se define un autómata finito no determinístico $M' = (Q, \Sigma, \delta', q_0, F')$ en que:

$$F' = \begin{cases} F \cup \{q_0\} & \text{ssi } \text{clausura} - \varepsilon(q_0) \text{ contiene un estado de } F \ (\varepsilon \in L) \\ F & \text{en otro caso} \end{cases}$$

y $\delta'(q, a)$ es $\hat{\delta}$ para todo $q \in Q$ y $a \in \Sigma$.

Nótese que M' no tiene transiciones en vacío y se puede entonces usar δ' en lugar de $\hat{\delta}$.

Se quiere probar, por inducción en $|x|$, que $\delta'(q_0, x) = \hat{\delta}(q_0, \varepsilon) = \text{clausura} - \varepsilon(q_0)$. Sin embargo, esto puede no ser cierto para $x = \varepsilon$, ya que $\delta'(q_0, \varepsilon) = \{q_0\}$, mientras que $\delta(q_0, \varepsilon) = \text{clausura} - \varepsilon(q_0)$. Por lo tanto la inducción empieza con $|x| = 1$.

Base ($|x| = 1$): Entonces x es un símbolo $a \in \Sigma$ y por la definición de δ' ,

$$\delta'(q_0, a) = \hat{\delta}(q_0, a)$$

Inducción: Sea $x = wa$ para un símbolo $a \in \Sigma$, entonces (con $|w| \geq 1$).

$$\delta'(q_0, wa) = \delta'(\delta'(q_0, w), a)$$

pero, por la hipótesis de inducción

$$\delta'(q_0, w) = \hat{\delta}(q_0, w)$$

basta mostrar entonces que

$$\delta'(\hat{\delta}(q_0, w), a) = \hat{\delta}(q_0, wa)$$

pero

$$\begin{aligned} \delta'(\hat{\delta}(q_0, w), a) &= \bigcup_{q \in \hat{\delta}(q_0, w)} \delta'(q, a) = \bigcup_{q \in \hat{\delta}(q_0, w)} \hat{\delta}(q, a) \\ &= \hat{\delta}(\hat{\delta}(q_0, w), a) \\ &= \hat{\delta}(q_0, wa) \end{aligned}$$

como se quería. Para completar la prueba, se mostrará que $\delta'(q_0, x)$ contiene un estado de F' si y sólo si $\hat{\delta}(q_0, x)$ contiene un estado de F . Si $x = \varepsilon$ ésto es cierto por la definición de F' ; es decir, $\delta'(q_0, \varepsilon) = \{q_0\}$ y $q_0 \in F'$ cuando $\hat{\delta}(q_0, \varepsilon) \in F$. Si $x \neq \varepsilon$ entonces $x = wa$ para algún $a \in \Sigma$. Si $\hat{\delta}(q_0, x)$ contiene un estado de F , con toda seguridad $\delta'(q_0, x)$ contiene el mismo estado en F' . Si $\delta'(q_0, x)$ contiene un estado en F' que no sea q_0 , $\hat{\delta}(q_0, x)$ lo contiene en F . Si $\delta'(q_0, x)$ contiene a q_0 y $q_0 \notin F$, entonces como $\hat{\delta}(q_0, x)$ es igual a la *clausura* $-\varepsilon(\delta(\hat{\delta}(q_0, w), a))$, los estados en *clausura* $-\varepsilon(q_0)$ y en F deben estar en $\hat{\delta}(q_0, x)$.

□

Ejemplo 46 Considere el AFND- ε cuyo diagrama de transición se muestra en la Figura 3.10. Se construirá un AFND usando el método implícito en la demostración del teorema anterior, a partir de él.

$$\text{clausura} - \varepsilon(q_0) = \{q_0, q_1, q_2\}$$

incluye a $q_2 \in F$, por lo tanto

$$F' = F \cup \{q_0\} = \{q_0, q_2\}$$

$$\hat{\delta}(q, a) = \delta'(q, a)$$

$Q \setminus \Sigma$	0	1	2
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$	$\{q_2\}$

y el diagrama del AFND resultante queda:

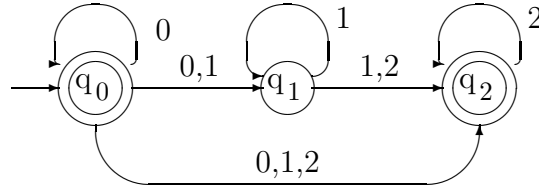


Figure 3.11: AFND obtenido, equivalente al AFND- ε

□

3.4 Teorema de Myhill-Nerode.

Con cualquier lenguaje L es posible asociar una relación de equivalencia R_L definida por

$$XR_LY \quad \text{si y sólo si} \quad (XZ \in L \text{ ssi } YZ \in L) \quad \forall Z \in \Sigma^*$$

En el peor caso, cada string está en una clase de equivalencia por sí solo, pero es posible que haya menos clases de equivalencia. En particular, el *índice* (número de clases de equivalencia) es siempre finito si L es un lenguaje regular.

Ejemplo 47 Considere el conjunto L compuesto por strings de paréntesis correctamente balanceados, entonces

$\begin{array}{lll}) & R_L &)(\\ () & R_L & ()(()) \\ (& R_L & () \end{array}$

□

Ejemplo 48 Sea L el conjunto de strings binarios que tienen un número par de ceros y un número par de unos, entonces

$\begin{array}{lll} 00 & R_L & 0101 \\ 10 & R_L & 1011 \\ 0 & R_L & 11 \end{array}$

□

Existe también una relación de equivalencia natural asociada con un AFD. Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AFD. La relación R_M , se define por

$$xR_M y \quad \text{si y sólo si} \quad \delta(q_0, x) = \delta(q_0, y)$$

Esta relación divide al conjunto Σ^* en clases de equivalencia, una por cada estado que es alcanzable desde q_0 . Además se cumple que

$$xR_M y \Rightarrow xzR_M yz \quad \forall z \in \Sigma^*$$

ya que $\delta(q_0, xz) = \delta(\delta(q_0, x), z) = \delta(\delta(q_0, y), z) = \delta(q_0, yz)$.

Ejemplo 49 Sea L el conjunto de strings binarios que tienen un número par de ceros y un número par de unos, que es aceptado por

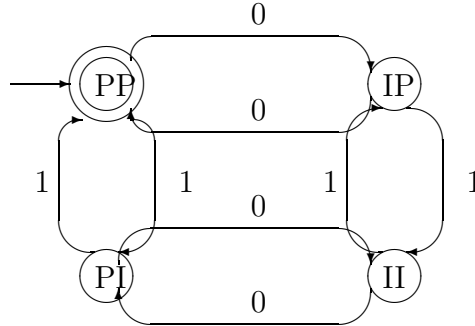


Figure 3.12: AFD que acepta strings binarios con número par de ceros y unos

Las clases de equivalencia para R_M son

$$\begin{array}{ll}
 PP &= \{x/\delta(q_0, x)\} = PP \\
 IP &= \{x/\delta(q_0, x)\} = IP \\
 II &= \{x/\delta(q_0, x)\} = II \\
 PI &= \{x/\delta(q_0, x)\} = PI
 \end{array}$$

□

Una relación de equivalencia R , tal que se cumple

$$xRy \Rightarrow xzRyz \quad \forall z$$

es llamada *invariante por la derecha* (con respecto a la concatenación). Así, todo autómata finito induce una equivalencia invariante por la derecha, la relación R_M definida anteriormente, en el conjunto de sus strings de entrada.

Teorema 3 Las siguientes tres aserciones son equivalentes:

1. El conjunto $L \subseteq \Sigma^*$ es aceptado por un AF.
2. L es la unión de algunas de las clases de equivalencia de una relación de equivalencia invariante por la derecha, de índice finito.
3. Sea R_L una relación de equivalencia definida por xR_Ly ssi para todo $z \in \Sigma^*$, $xz \in L$ precisamente cuando $yz \in L$. Entonces R_L tiene índice finito.

Demostración : Se probará que $1 \Rightarrow 2$, $2 \Rightarrow 3$ y $3 \Rightarrow 1$, demostrando la equivalencia de las tres aserciones.

($1 \Rightarrow 2$) Asuma que L es aceptado por un AFD, $M = (Q, \Sigma, \delta, q_0, F)$. Sea R_M la relación de equivalencia xR_My si y sólo si $\delta(q_0, x) = \delta(q_0, y)$. R_M es invariante por la derecha ya que para todo z , si $\delta(q_0, x) = \delta(q_0, y)$ entonces $\delta(q_0, xz) = \delta(q_0, yz)$. El índice de R_M es finito ya que es, a lo sumo, el número de estados en Q . Además L es la unión de aquellas clases de equivalencia tales que incluyen un string w con $\delta(q_0, w) \in F$, esto es, las clases que corresponden a estados finales.

($2 \Rightarrow 3$) Se muestra que cualquier relación de equivalencia E , que satisface 2 es un refinamiento de R_L ; es decir, cada clase de equivalencia de E está enteramente contenida en alguna de las clases de equivalencias de R_L . Por lo tanto el índice de R_L no puede ser mayor que el de E y, por lo tanto, es finito.

Asuma que xEy ; entonces, ya que E es invariante por la derecha, para cada $z \in \Sigma^*$, $xzEyz$ y, por lo tanto, $yz \in L$ si y sólo si $xz \in L$. Por lo tanto, xR_Ly y entonces la clase de equivalencia que contiene a x en E , está contenida en la clase de equivalencia de x en R_L . Se concluye que cada clase de equivalencia de E está contenida completamente por una de las clases de equivalencia de R_L .

($3 \Rightarrow 1$) Primero se mostrará que R_L es invariante por la derecha. Suponga que xR_Ly y sea w un string en Σ^* . Se debe probar que xwR_Lyw ; esto es, para todo $z \in \Sigma^*$, $xwz \in L$ precisamente cuando $ywz \in L$. Pero ya que xR_Ly , se sabe por la definición de R_L que para todo v , $xv \in L$, precisamente cuando $yv \in L$. En particular, sea $v = wz$ para probar que R_L es invariante por la derecha.

Sea Q' el conjunto finito de clases de equivalencia de R_L y sea $[x]$ el elemento de Q' que contiene al string x . Defina $\delta'([x], a) = [xa]$. La definición es consistente ya que R_L es invariante por la derecha. Si se hubiese elegido y en lugar de x de la clase $[x]$, se obtendría $\delta'([x], a) = [ya]$. Pero xR_Ly , por lo tanto $xz \in L$ precisamente cuando $yz \in L$. En particular, si $z = az'$, $xaz' \in L$ precisamente cuando $yaz' \in L$, es decir, xaR_Lya y $[xa] = [ya]$. Sea $q'_0 = [\varepsilon]$ y sea $F' = \{[x] / x \in L\}$. El AF $M' = (Q', \Sigma, \delta', q'_0, F')$ acepta L ya que $\delta'(q'_0, x) = [x]$ y por lo tanto $x \in L(M')$ si y sólo si $[x]$ está en F' .

□

Ejemplo 50 Sea L el lenguaje 0^*10^* . L es aceptado por el siguiente AFD, M .

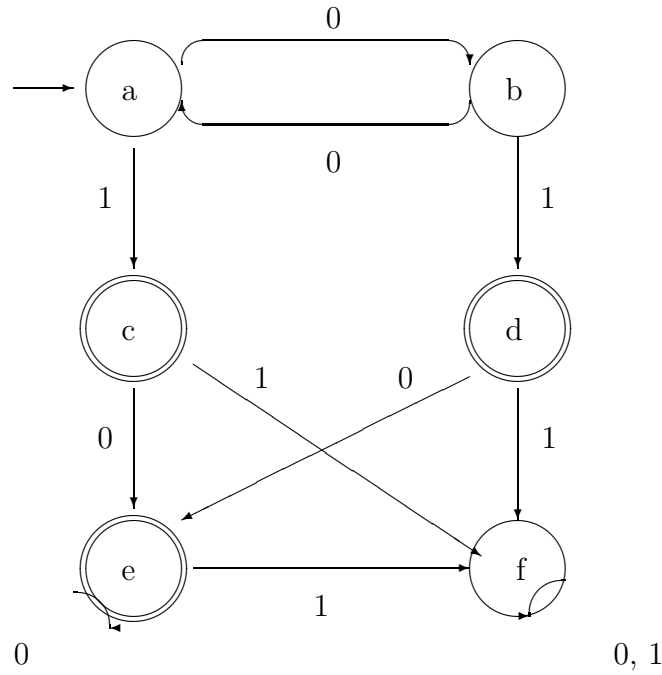
Considere la relación R_M definida por M . Como todos los estados son alcanzables desde el estado inicial, R_M tiene seis clases de equivalencia:

$$\begin{array}{ll} C_a = (00)^* & C_d = (00)^*01 \\ C_b = (00)^*0 & C_e = 0^*100^* \\ C_c = (00)^*1 & C_f = 0^*10^*1(0+1)^* \end{array}$$

El lenguaje L es la unión de C_c , C_d y C_e .

La relación R_L para el lenguaje L tiene tres clases de equivalencia; xR_Ly si y sólo si

- x y y no tienen 1's, ambos.
- x y y tienen un solo 1, cada uno.
- x y y tienen más de un 1, ambos.

Figure 3.13: AFD que acepta el lenguaje 0^*10^*

estas clases de equivalencia pueden describirse por

$$C_1 = 0^*$$

$$C_2 = 0^*10^*$$

$$C_3 = 0^*10^*1(0+1)^*$$

La relación entre las clases de equivalencia de R_M y las de R_L se describe en la Figura 3.14.

A partir de R_L se puede construir un AFD como sigue. Elija representantes para C_1 , C_2 y C_3 , por ejemplo, ε , 1 y 11. La máquina se construye de acuerdo al método implícito en la tercera parte de la demostración del teorema anterior.

Por ejemplo, $\delta'([1], 0) = [1]$ porque si w es cualquier string en $[1]$ (es decir, en C_2), supongamos 0^i10^j , entonces $w0 = 0^i10^{j+1}$ también pertenece a $C_2 = 0^*10^*$.

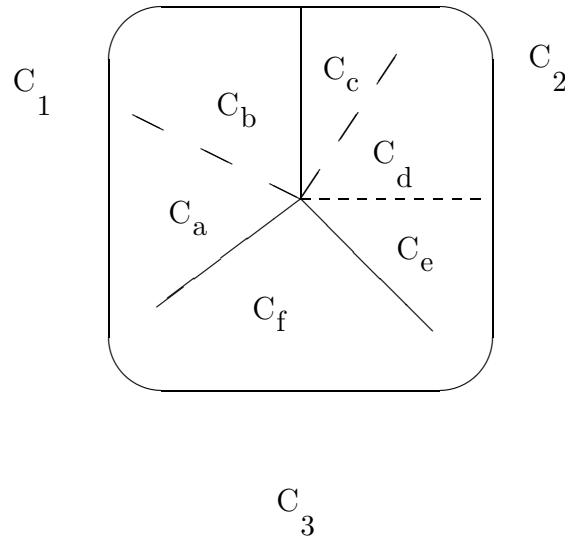
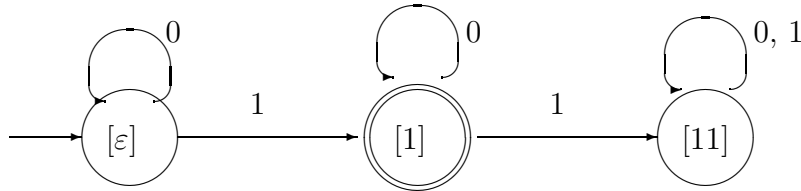
□

3.5 Minimización de Autómatas Finitos.

El teorema de Myhill-Nerode tiene, entre otras consecuencias, la implicación de que existe esencialmente un único AFD con mínimo número de estados, por cada conjunto regular.

Teorema 4 *El AFD con un mínimo número de estados que acepta un lenguaje L , es único hasta un isomorfismo (renombrado de los estados) y está dado por M' de la demostración del teorema anterior.*

Demostración : En la demostración de dicho teorema se vio que todo AFD, $M = (Q, \Sigma, \delta, q_0, F)$ que acepta L , define una relación de equivalencia que es un refinamiento de R_L . Por lo tanto, el número de estados de M es mayor o igual al número de estados de M' construido, como en la demostración del teorema anterior, a partir de las clases de equivalencia de R_L . Si tienen el mismo número de estados, cada estado de M se puede identificar con un estado de M' . Esto es, sea q un estado de M . Debe haber algún $x \in \Sigma^*$, tal que $\delta(q_0, x) = q$, si no q podría ser removido de Q y un autómata más pequeño resultaría. Se identifica q con

Figure 3.14: Relación entre clases de equivalencia R_M y R_L Figure 3.15: AFD con mínimo número de estados, para el lenguaje 0^*10^*

el estado $\delta'(q'_0, x)$ de M' . Esta identificación será consistente, pues, por la prueba del teorema anterior, si $\delta(q_0, x) = \delta(q_0, y) = q$, x e y están en la misma clase de equivalencia de R_L y, por lo tanto, $\delta'(q'_0, x) = \delta'(q'_0, y)$.

□

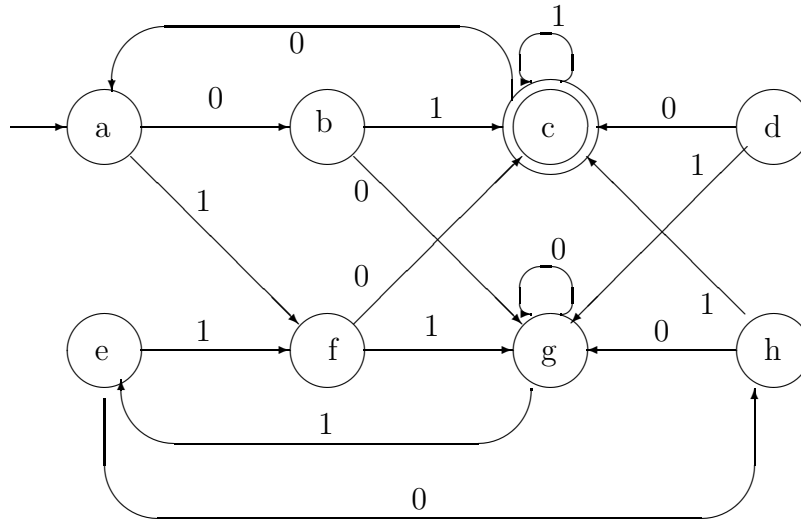
Hay un método simple para encontrar el AFD, M' , con el mínimo número de estados y equivalente a un AFD $M = (Q, \Sigma, \delta, q_0, F)$ dado. Sea $\equiv$ la relación de equivalencia en los estados de M tal que $p \equiv q$ si y sólo si para todo string $x \in \Sigma^*$, $\delta(p, x) \in F$ si y sólo si $\delta(q, x) \in F$. Obviamente, hay un isomorfismo entre las clases de equivalencia de $\equiv$ que contienen un estado alcanzable desde q_0 para algún string y los estados de M' . Si $p \equiv q$ se dice que p es equivalente a q ; se dice que p es distinguible de q si existe un string x tal que $\delta(p, x) \in F$ y $\delta(q, x) \notin F$ o viceversa.

Ejemplo 51 Sea M el AFD siguiente:

A continuación se muestra una tabla con una entrada por cada par de estados distintos. Se pone una $\times$ en la tabla cuando se descubre que un par de estados son distinguibles.

Inicialmente se pone una $\times$ en todas las entradas de la tabla que corresponden a un estado final y a uno no final. En este caso se pone una $\times$ en (a, c) , (b, c) , (c, d) , (c, e) , (c, f) , (c, g) y (c, h) .

A continuación, para cada par de estados p y q , que aún no se sabe si son distinguibles, se consideran los pares de estados $r = \delta(p, a)$ y $s = \delta(q, a)$, para cada símbolo a . Si r y s son distinguibles por algún string x , entonces p y q son distinguibles por ax . Por lo tanto, si en la entrada (r, s) hay una $\times$, se pone una $\times$ en (p, q) . Si la entrada (r, s) no tiene una $\times$ aún, el par (p, q) se pone en una lista asociada a (r, s) . En el futuro, si (r, s) recibe una $\times$, cada par en su lista asociada también la recibe.



- $(a, b) : (\delta(a, 1), \delta(b, 1)) = (f, c) \Rightarrow (a, b) \text{ recibe } \times$
 $(a, d) : (\delta(a, 0), \delta(d, 0)) = (b, c) \Rightarrow (a, d) \text{ recibe } \times$
 $(a, e) : (\delta(a, 0), \delta(e, 0)) = (b, h) \Rightarrow (a, e) \text{ se pone en lista } (b, h)$
 $(a, e) : (\delta(a, 1), \delta(e, 1)) = (f, f) \Rightarrow \text{No ayuda}$
 $(a, f) : (\delta(a, 0), \delta(f, 0)) = (b, c) \Rightarrow (a, f) \text{ recibe } \times$
 $(a, g) : (\delta(a, 0), \delta(g, 0)) = (a, g) \Rightarrow (a, g) \text{ se pone en lista } (b, g)$
 $(b, g) : (\delta(b, 1), \delta(g, 1)) = (c, e) \Rightarrow (b, g) \text{ y } (a, g) \text{ reciben } \times$

y así sucesivamente, se obtiene la tabla que aparece en la Figura 3.16. De ella, se concluye que los siguientes pares de estados son equivalentes

$$a \equiv e; b \equiv h; d \equiv f$$

El autómata finito con el mínimo número de estados se presenta en la Figura 3.17

□

El algoritmo para marcar los pares de estados que son distinguibles es el siguiente:

```

begin
(1)   FOR  $p$  en  $F$  y  $q$  en  $Q - F$  DO mark  $(p, q)$ ;
(2)   FOR cada par de estados distintos  $(p, q)$  en  $F \times F$  o  $(Q - F) \times (Q - F)$  DO
(3)     IF para algun  $a \in \Sigma$   $(\delta(p, a), \delta(q, a))$  esta marcado THEN BEGIN
(4)       mark  $(p, q)$ 
(5)       Marque recursivamente todos los pares no marcados de la lista  $(p, q)$ 
        y de las listas de elementos marcados
      END ELSE (* ningun  $(\delta(p, a), \delta(q, a))$  esta marcado
(6)     FOR todo  $a \in \Sigma$  DO
(7)       Ponga  $(p, q)$  en la lista de  $(\delta(p, a), \delta(q, a))$  a menos que  $\delta(p, a) = \delta(q, a)$ 
end

```

Lema 1 Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AFD. Entonces p es distinguible de q si y sólo si la entrada (p, q) está marcada después de aplicar el algoritmo anterior.

Demostración : Asuma que p es distinguible de q y sea x el string más corto que los distingue. Se prueba, por inducción en la longitud de x que la entrada (p, q) es marcada por el algoritmo. Si $x = \varepsilon$, entonces

b	×						
c	×	×					
d	×	×	×				
e		×	×	×			
f	×	×	×		×		
g	×	×	×	×	×	×	
h	×		×	×	×	×	×
	a	b	c	d	e	f	g

Figure 3.16: Tabla auxiliar en la construcción del AFD con mínimo número de estados

exactamente uno de p y q es final y es marcado en la línea (1). Suponga que la hipótesis es verdadera para $|x| < i$ con $i \geq 1$ y sea $|x| = i$. Entonces $x = ay$ y sean $t = \delta(p, a)$ y $u = \delta(q, a)$. Ahora y distingue t de u y $|y| = i - 1$, por inducción, el par (t, u) será marcado eventualmente. Si esto ocurre después que (p, q) ha sido considerado, entonces ya sea (p, q) está marcado al considerar (t, u) o bien (p, q) está en la lista de (t, u) y es marcado en la línea (5). Si (p, q) se considera después que (t, u) , (p, q) es marcado al ser considerado. En cualquiera de los dos casos (p, q) es marcado. Una inducción similar en el número de pares marcados muestra que si (p, q) es marcado, p y q son distinguibles.

□

El algoritmo mostrado es más eficiente que el algoritmo más obvio; empero, no es el más eficiente posible. Si Σ tiene k símbolos y Q tiene N estados, línea (1) toma $\vartheta(N^2)$ pasos. El loop de líneas (2) a (7) se ejecuta $\vartheta(N^2)$ veces, a lo más una vez por cada par de estados. El tiempo en líneas (2) a (4), (6) y (7) es $\vartheta(kN^2)$. El tiempo en línea (5) es la suma de los largos de las listas. Pero cada par (r, s) se pone en, a lo más, k listas en línea (7). Por lo tanto, el tiempo ocupado en línea (5) es $\vartheta(kN^2)$. Es decir, el tiempo total es $\vartheta(kN^2)$.

Teorema 5 *El AFD construido por el algoritmo anterior, con estados inaccesibles removidos, es el AFD con mínimo número de estados para ese lenguaje.*

Demostración : Sean $M = (Q, \Sigma, \delta, q_0, F)$ el AFD al que se le aplica el algoritmo y $M' = (Q', \Sigma, \delta', [q_0], F')$ el AFD construido. Esto es,

$$Q' = \{[q] / q \text{ es accesible } q_0\}$$

$$F' = \{[q] / q \in F\}$$

$$\delta'([q], a) = [\delta(q, a)]$$

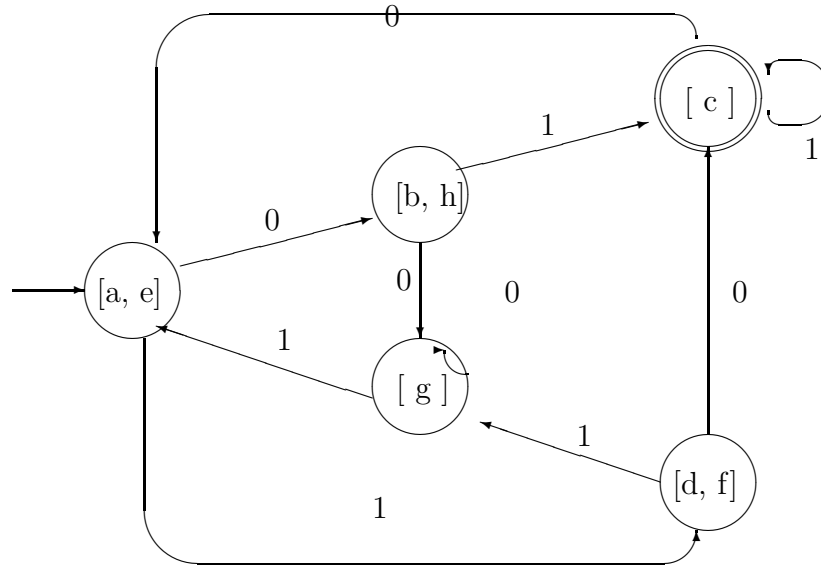


Figure 3.17: AFD buscado, con mínimo número de estados

Es fácil ver que δ' está definida en forma consistente, ya que si $q \equiv p$, entonces $\delta(q, a) \equiv \delta(p, a)$. Esto es, si $\delta(q, a)$ se distingue de $\delta(p, a)$ por el string x , entonces ax distingue q de p . Es también fácil mostrar que $\delta'([q_0], w) = [\delta(q_0, w)]$ por inducción en $|w|$. Por lo tanto $L(M) = L(M')$.

Se debe ahora mostrar que M' no tiene más estados que el índice de R_L , en que $L = L(M)$. Supóngase que tuviera más estados, entonces habría dos estados accesibles, q y $p \in Q$, tales que $[q] \neq [p]$; pero también hay x e y tales que $\delta(q_0, x) = q$ y $\delta(q_0, y) = p$, con $xR_L y$. Entonces debe ser $p \equiv q$ porque si no, algún $w \in \Sigma^*$ distingue p de q . Pero entonces $xwR_L yw$ es falso pues si $z = \varepsilon$ exactamente uno de xwz y ywz pertenece a L . Pero R_L es invariante por la derecha, así que $xwR_L yw$ es verdadero. Por lo tanto, q y p no existen y M' no tiene más estados que el índice de R_L . Es decir, M' es el AFD mínimo para L .

□

3.6 Traductores de Estado Finito

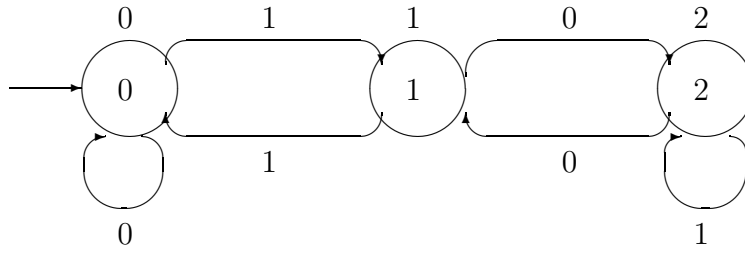
Una restricción de los autómatas finitos, tal como han sido definidos en este capítulo, es que su salida de información está limitada a una señal binaria: acepta / no acepta. En esta sección se considerará modelos en que la salida se escoge de algún otro alfabeto. Hay dos enfoques diferentes; la salida está asociada con el estado (llamado una *Máquina de Moore*) o con las transiciones (llamado una *Máquina de Mealy*).

Una *Máquina de Moore* es una séxtupla $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, en que Q , Σ y q_0 son como en los autómatas finitos determinísticos. Δ es el alfabeto de salida y λ es una función de $Q \rightarrow \Delta$, indicando el output asociado a cada estado.

El output de estas máquinas en respuesta a un string de entrada $a_1 a_2 \dots a_N$, $N \geq 0$, es $\lambda(q_0)\lambda(q_1) \dots \lambda(q_N)$, en que $q_1 q_2 \dots q_N$ es la secuencia de estados tales que $\delta(q_{i-1}, a_i) = q_i$, para $1 \leq i \leq N$. Nótese que toda máquina de Moore da output $\lambda(q_0)$ en respuesta al string ε .

Un AFD puede ser visto como un caso especial de una máquina de Moore, en que el alfabeto de salida, Δ , es $\{0, 1\}$ y un estado q es de aceptación si y sólo si $\lambda(q) = 1$.

Ejemplo 52 Suponga que se desea determinar el resto en módulo 3 de cada string binario, tratado como un entero. Observe que si i , escrito en binario, es seguido por un 0, el string tiene valor $2i$; si el binario i es



seguido por un 1, su valor es $2i + 1$. Además, si el resto de $i/3$ es p , el resto de $2i/3$ es $2p \bmod 3$. Si $p = 0$, 1 ó 2 , $2p \bmod 3$ es 0 , 2 ó 1 , respectivamente.

$$\Delta = \{0, 1, 2\} \quad \lambda(i) = i$$

Si el string de entrada es 1010, el autómata entra a los estados 0–1–2–2–1 y produce el output 01221. Esto es, ε (que se ha supuesto, vale cero), tiene residuo 0, 1 tiene residuo 1, 2 decimal (10 binario) tiene residuo 2, 101 (5 en decimal) tiene residuo 2 y, finalmente, 1010 (10 en decimal) tiene residuo 1.

□

Una *máquina de Mealy* es también una séxtupla $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, en que todo es como en las máquinas de Moore, excepto que λ va de $Q \times \Sigma$ a Δ . Es decir, $\lambda(q, a)$ es el output asociado con la transición desde el estado q en símbolo a .

El output de estas máquinas en respuesta al string de entrada $a_1 a_2 \dots a_N$ es $\lambda(q_0, a_0) \lambda(q_1, a_1) \dots \lambda(q_N, a_N)$, donde $q_1 q_2 \dots q_N$ es la secuencia de estados tales que $\delta(q_{i-1}, a_i) = q_i$ ($1 \leq i \leq N$). Obsérvese que el string de salida tiene longitud N , y no $N + 1$ como en la máquina de Moore; y que si el string de entrada es ε , una máquina de Mealy tiene salida ε .

Ejemplo 53 Considere el lenguaje $(0+1)^*(00+11)$ de todos los strings binarios cuyos últimos dos símbolos son iguales. En el próximo capítulo se verán técnicas que permiten demostrar que 5 estados son necesarios para un AFD que lo acepte. Sin embargo se puede definir una máquina de Mealy con 3 estados, que usa sus estados para recordar el último símbolo leído y que emite una S cuando el símbolo actual es igual al previo, en otro caso, emite una N . La secuencia de S 's y N 's emitida corresponde a la secuencia de estados de aceptación y no-aceptación en los que entraría un AFD. hay una diferencia, la máquina de Mealy no emite antes de ver un input, mientras el AFD habría rechazado el string ε con $q_0 \notin F$.

□

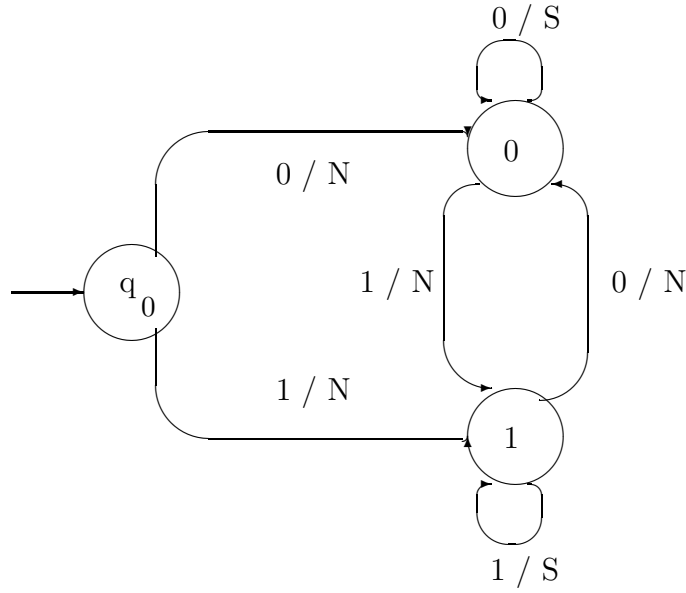
Sea M una máquina de Mealy o de Moore y definimos $T_M(w)$ como el output producido por M si el string de entrada es w . Es claro que no puede haber identidad exacta entre las funciones T_M y $T_{M'}(w)$ si una de M o M' es una máquina de Mealy (M) y la otra de Moore (M'), ya que uno de los string de salida será más corto. Sin embargo, es posible despreciar la respuesta de la máquina de Moore si la entrada es ε , y decir que una máquina de Mealy, (M), y una máquina de Moore, (M'), son equivalentes si para todos los strings de entrada w , $bT_M(w) = T_{M'}(w)$, en que b es el output de M' en su estado inicial. Es posible, entonces, probar los siguientes teoremas que igualan ambos modelos:

Teorema 6 Si $M_1 = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$ es una máquina de Moore, hay una máquina de Mealy, M_2 , equivalente a M_1 .

Demostración : Sea $M_2 = (Q, \Sigma, \Delta, \delta, \lambda', q_0)$ y defínase la función λ' como

$$\lambda'(q, a) = \lambda(\delta(q, a))$$

para todo estado $q \in Q$ y símbolo $a \in \Sigma$. Entonces M_1 y M_2 pasan por la misma secuencia de estados, en iguales inputs y, en cada transición, M_2 emite el output que M_1 asocia con el estado al que entra.



□

Teorema 7 Sea $M_1 = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$ una máquina de Mealy. Entonces existe una máquina de Moore, M_2 , equivalente a M_1 .

Demostración : Sea $M_2 = (Q \times \Delta, \Sigma, \Delta, \delta', \lambda', [q_0, b_0])$ en que b_0 es un miembro arbitrario de Δ . Los estados de M_2 son pares $[q, b]$ que consisten en un estado de M_1 y un símbolo de salida. Se define

$$\delta'([q, b], a) = [\delta(q, a), \lambda(q, a)]$$

y

$$\lambda'([q, b], a) = b$$

La segunda componente de un estado $[q, b]$ de M_2 es el output de M_1 en alguna transición a q . Sólo las primeras componentes de los estados de M_2 determinan las movidas hechas por M_2 .

Es simple probar, por inducción en N , que si M_1 entra a los estados $q_1 q_2 \dots q_N$ en el input $a_1 a_2 \dots a_N$ y emite el string $b_1 b_2 \dots b_N$, entonces M_2 entra a estados $[q_0, b_0], [q_1, b_1], \dots, [q_N, b_N]$ y emite $b_0 b_1 b_2 \dots b_N$.

□

Ejemplo 54 Se construye una máquina de Moore equivalente a la de Mealy del ejemplo anterior.

Nótese que $[q_0, S]$, que pudo ser elegido como estado inicial, es inútil y puede eliminarse.

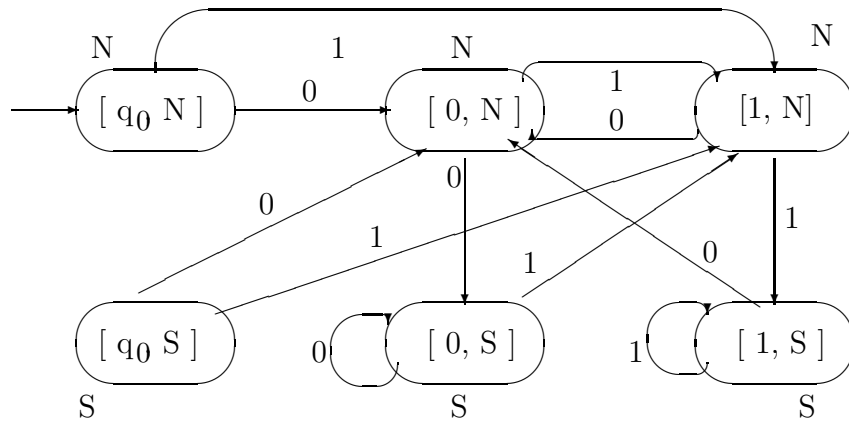
□

3.7 Expresiones Regulares

Los lenguajes aceptados por los autómatas finitos son fácilmente descritos por expresiones simples llamadas *expresiones regulares* quienes les dan el nombre de *conjuntos regulares* a dichos lenguajes.

Sea Σ un alfabeto; las expresiones regulares sobre Σ , y los conjuntos que ellas representan, se definen como sigue:

- $\emptyset$ es una expresión regular y denota el conjunto vacío.



- ε es una expresión regular y denota el conjunto $\{\varepsilon\}$
- Por cada $a \in \Sigma$, a es una expresión regular y denota el conjunto $\{a\}$
- Si r y s son expresiones regulares que denotan los lenguajes R y S , respectivamente, entonces:

$(r + s)$ es una expresión regular y denota $R \cup S$.

(rs) es una expresión regular y denota RS .

(r^*) es una expresión regular y denota R^* .

Al escribir expresiones regulares, se acostumbra omitir muchos de los paréntesis, asumiendo que $*$ tiene la precedencia más alta, seguida por la concatenación y, finalmente, por $+$.

$$((0(1^*)) + 0) = 01^* + 0$$

También se acostumbra utilizar la siguiente abreviación:

$$rr^* = r^+$$

Cuando es necesario distinguir entre una expresión regular, r , y el lenguaje denotado por r , se usa $L(r)$ para el lenguaje.

Ejemplo 55 Considere las siguientes expresiones regulares:

00	representa $\{00\}$
$(0 + 1)^*$	representa todos los strings binarios
$(0 + 1)^*00(0 + 1)^*$	representa todos los strings binarios con al menos un par de 0's consecutivos
$(0 + 1)^*011$	representa todos los strings binarios que terminan en 011

□

Ejemplo 56 $(1 + 10)^*$ representa todos los strings binarios que comienzan con un 1 y no tienen dos ceros consecutivos.

Es fácil probar por inducción en i que $(1 + 10)^i$ no tiene strings con dos ceros consecutivos. Más aún, dado cualquier string que comienza con un 1 y que no tiene dos 0's consecutivos, es posible dividirlo en substrings compuestos de un 1 seguido, posiblemente, de un cero, si los hay. Por ejemplo 10110111010 se divide como 10-1-10-1-1-10-10. Esta división prueba que todos estos strings están en $(1 + 10)^i$, con i igual al número de 1's.

La expresión regular $(0 + \varepsilon)(1 + 10)^*$ representa a todos los strings binarios que no tienen ceros consecutivos.

□

Ejemplo 57 La expresión regular $0^*1^*2^*$ representa cualquier número de ceros, seguidos de cualquier número de 1's, seguidos de cualquier número de 2's. Este es el lenguaje aceptado por el AFND- ε cuyo diagrama de transición aparece al comienzo de la sección anterior. Véase Figura 3.10

La expresión regular $00^*11^*22^*$ denota aquellos strings en $0^*1^*2^*$ con al menos uno de cada símbolo. Es posible abreviarlo como $0^+1^+2^+$, en lugar de $00^*11^*22^*$.

□

Se probará ahora, que los lenguajes aceptados por los autómatas finitos son, precisamente, los lenguajes descritos por las expresiones regulares. Esta equivalencia es la razón por la que dichos lenguajes son llamados conjuntos regulares. Para ello, es necesario probar dos teoremas. El primero mostrará que por cada expresión regular es posible construir un AFND- ε que acepte el mismo lenguaje que ella describe. El segundo, que por cada AFD es posible construir una expresión regular que describa el mismo lenguaje que él acepta. En conjunto con los dos teoremas demostrados anteriormente, éstos mostrarán que los cuatro mecanismos de definición de lenguajes mostrados en este capítulo, son esencialmente equivalentes, sirven para definir la misma clase de lenguajes: los conjuntos regulares. En la Figura 3.18 se muestran las construcciones vistas o por ver; un arco de A a B ($A \rightarrow B$), indica que por cada descriptor de tipo A es posible construir uno equivalente de tipo B :

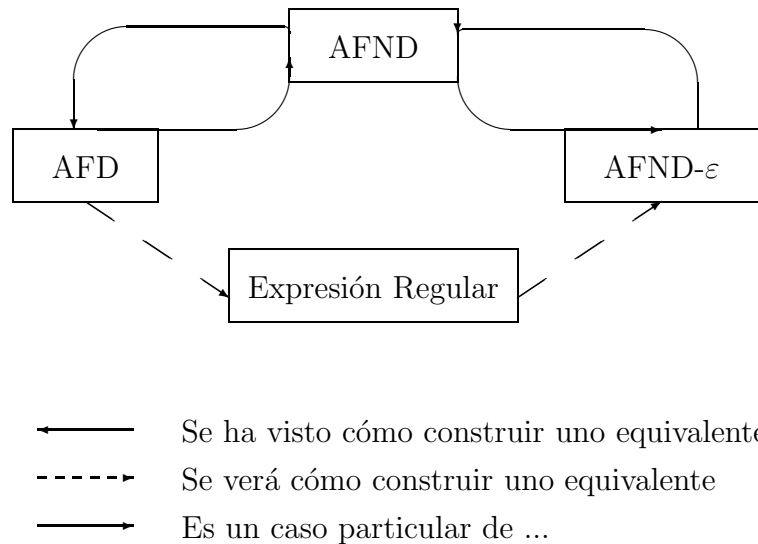


Figure 3.18: Equivalencias entre lenguajes aceptados por distintos mecanismos

Teorema 8 Sea r una expresión regular. Existe un AFND- ε que acepta el lenguaje $L(r)$.

Demostración : Se muestra, por inducción en el número de operadores de la expresión regular r , que existe un AFND- ε , M , con un solo estado final, sin transiciones que salgan de él, tal que $L(M) = L(r)$.

Base (Cero operadores): La expresión regular debe ser ε , $\emptyset$ o a , para algún $a \in \Sigma$, los autómatas siguientes satisfacen las condiciones en estos casos:

Inducción: Se asume que el teorema se cumple para expresiones regulares con N o menos operadores. Sea r una expresión regular con $N + 1$ operadores; hay tres casos que dependen de la forma de r .

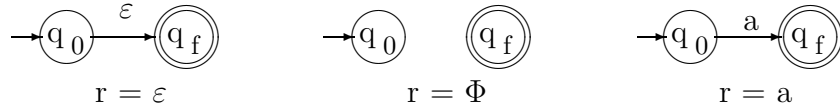


Figure 3.19: Expresiones regulares y sus correspondientes autómatas

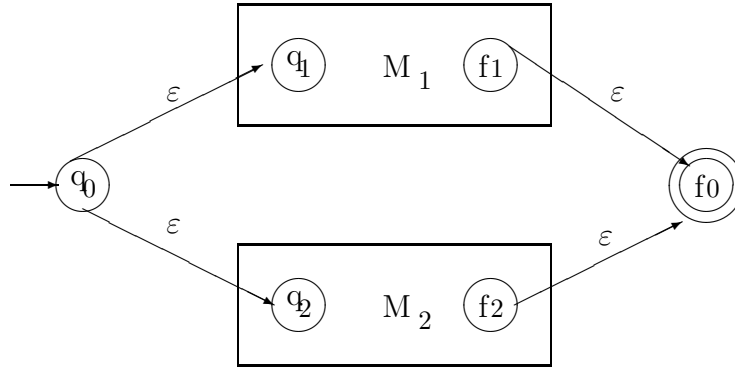
Caso 1: $r = (r_1 + r_2)$. Tanto r_1 como r_2 tienen N o menos operadores, por lo tanto, por la hipótesis de inducción, existen AFND- ε , $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, \{f_1\})$ y $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, \{f_2\})$ con $L(r_1) = L(M_1)$ y $L(r_2) = L(M_2)$. Ya que los estados pueden renombrarse, se puede asumir que Q_1 y Q_2 son disjuntos. Sean q_0 y f_0 nuevos estados. Se construye

$$M = (Q_1 \cup Q_2 \cup \{q_0, f_0\}, \Sigma_1 \cup \Sigma_2, \delta, q_0, \{f_0\})$$

en que δ queda definido por:

- $\delta(q_0, \varepsilon) = \{q_1, q_2\}$
- $\delta(q, a) = \delta_1(q, a) \quad \forall q \in Q_1 - \{f_1\}, a \in \Sigma_1 \cup \{\varepsilon\}$
- $\delta(q, a) = \delta_2(q, a) \quad \forall q \in Q_2 - \{f_2\}, a \in \Sigma_2 \cup \{\varepsilon\}$
- $\delta(f_1, \varepsilon) = \delta(f_2, \varepsilon) = \{f_0\}$

recuérdese que por la hipótesis de inducción no hay transiciones que salgan de f_1 o f_2 , por lo tanto todas las transiciones de M_1 y M_2 están en M . La construcción conduce al diagrama de transiciones de la Figura 3.20.

Figure 3.20: Diagrama de transición correspondiente a la operación $+$ aplicada a expresiones regulares

Cualquier camino entre q_0 y f_0 debe comenzar yendo a q_1 o a q_2 en ε . Si se va a q_1 , debe seguir un camino en M_1 de q_1 a f_1 y luego ir a f_0 en ε . Similarmente, los caminos que comienzan yendo a q_2 pueden seguir cualquier camino a f_2 en M_2 y luego ir a f_0 en M . Por lo tanto, hay un camino con etiqueta x en M de q_0 a f_0 , si y sólo si hay un camino con etiqueta x , de q_1 a f_1 en M_1 , o de q_2 a f_2 en M_2 . Por lo tanto, $L(M) = L(M_1) \cup L(M_2)$, como se quería mostrar.

Caso 2: $r = (r_1 r_2)$. Sean M_1 y M_2 , como en el caso anterior. Se construye

$$M = (Q_1 \cup Q_2, \Sigma_1 \cup \Sigma_2, \delta, q_1, \{f_2\})$$

con δ definido por:

- $\delta(q, a) = \delta_1(q, a) \quad \forall q \in Q_1 - \{f_1\}, a \in \Sigma_1 \cup \{\varepsilon\}$
- $\delta(f_1, \varepsilon) = \{q_2\}$
- $\delta(q, a) = \delta_2(q, a) \quad \forall q \in Q_2 - \{f_2\}, a \in \Sigma_2 \cup \{\varepsilon\}$

el diagrama de transición para M es, entonces, el que se muestra en la Figura 3.21.

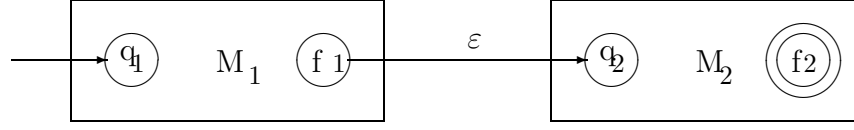


Figure 3.21: Diagrama de transición correspondiente a la concatenación de expresiones regulares

Cada camino de q_1 a f_2 en M está etiquetado por algún string x de q_1 a f_1 (en M_1), seguido por un arco de f_1 a q_2 en ε , seguido por un camino etiquetado por un string y de q_2 a f_2 (en M_2). Por lo tanto,

$$L(M) = \{xy / x \in L(M_1) \text{ e } y \in L(M_2)\}$$

es decir, $L(M) = L(M_1)L(M_2)$, como se quería mostrar.

Caso 3: $r = (r_1^*)$. Sea M_1 como en los casos anteriores. Se construye

$$M = (Q_1 \cup \{q_0, f_0\}, \Sigma_1, \delta, q_0, \{f_0\})$$

en que δ queda definido por:

- $\delta(q_0, \varepsilon) = \{q_1, f_0\}$
- $\delta(q, a) = \delta_1(q, a) \quad \forall q \in Q_1 - \{f_1\}, a \in \Sigma_1 \cup \{\varepsilon\}$
- $\delta(f_1, \varepsilon) = \{q_1, f_0\}$

el diagrama de transición para M es, entonces, el que se muestra en la Figura 3.22.

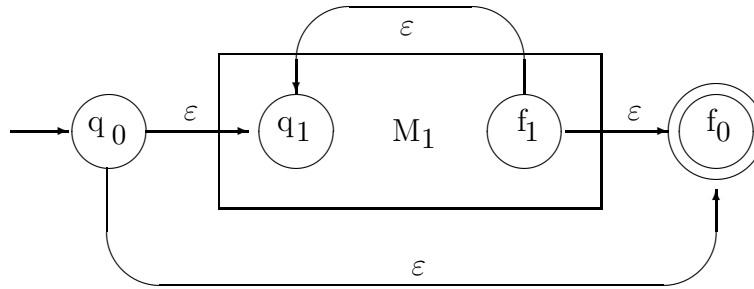


Figure 3.22: Diagrama de transición correspondiente a la operación $*$ aplicada a expresiones regulares

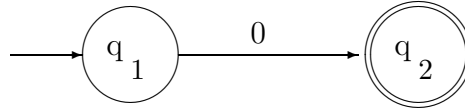
Cada camino de q_0 a f_0 en M consiste, ya sea de un arco directo de q_0 a f_0 (en ε), seguido de algún número (posiblemente cero) de caminos de q_1 a f_1 con un arco de vuelta a q_1 en ε , cada uno con etiqueta que corresponde a un string en $L(M_1)$, seguido de un camino de q_1 a f_1 en un string de $L(M_1)$ y, finalmente, de f_1 a f_0 en ε . Por lo tanto hay un camino de q_0 a f_0 con etiqueta x en M , si y sólo si $x = x_1x_2 \dots x_k$ ($k \geq 0$), tal que cada $x_i \in L(M_1)$. Es decir, $L(M) = L^*(M_1) = L(r^*)$, como se quería mostrar.

□

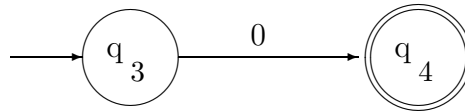
Ejemplo 58 Se construye un AFND- ε que acepta el lenguaje descrito por la expresión regular $01^* + 0$. Por las reglas de precedencia, ya se vio que la expresión regular es realmente:

$$r = ((0(1^*)) + 0)$$

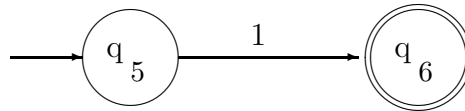
es decir, es de la forma $r_1 + r_2$, en que $r_1 = 01^*$ y $r_2 = 0$. El autómata para r_2 es simple:



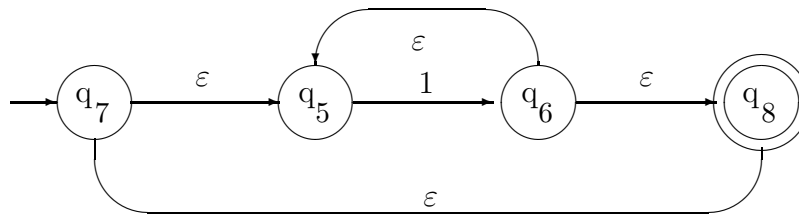
La expresión regular r_1 puede anotarse como $r_3 r_4$, en que $r_3 = 0$ y $r_4 = 1^*$. El autómata para r_3 es también simple:



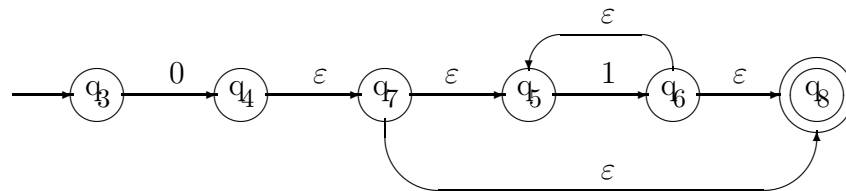
A su vez, r_4 es r_5^* , en que $r_5 = 1$, cuyo autómata es



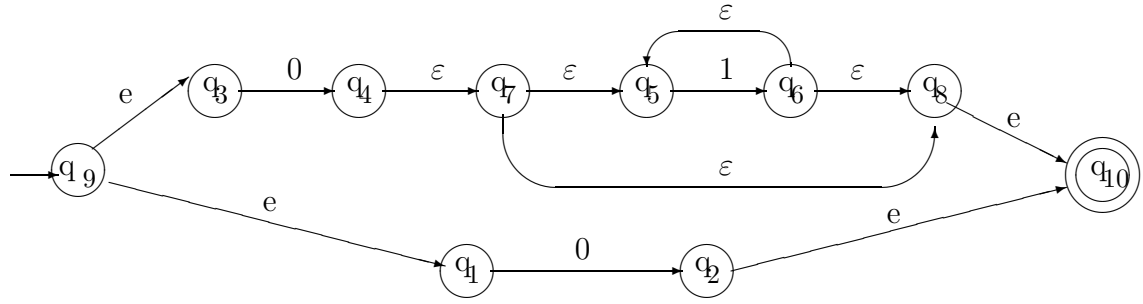
Para construir el autómata para r_4 , se usa el caso 3 del teorema anterior, obteniéndose:



Para $r_1 = r_3 r_4$, se usa el caso 2:



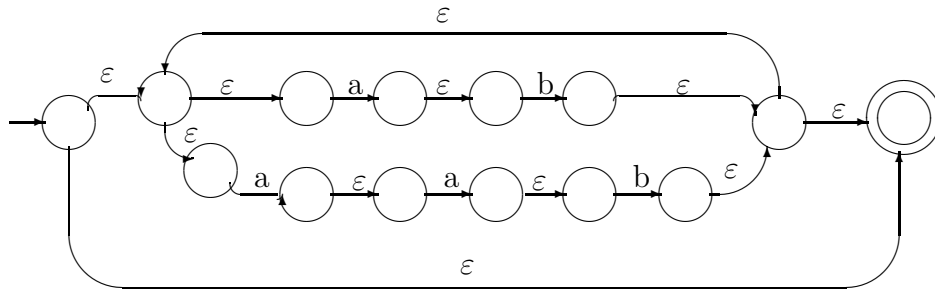
Finalmente, usando el caso 1, se construye el autómata para $r = r_1 + r_2$



un autómata finito determinístico con transiciones en vacío que acepta el lenguaje descrito por la expresión regular $01^* + 0$

□

Ejemplo 59 Un AFND- ε equivalente a la expresión regular $(ab + aab)^*$.



□

La demostración del teorema anterior contiene un algoritmo para convertir una expresión regular en un autómata finito (no determinístico con transiciones en vacío), asumiendo que la expresión regular esté totalmente parentizada.

Teorema 9 Sea L un lenguaje aceptado por un AFD. Hay una expresión regular que lo representa.

Demostración : Sea L un lenguaje aceptado por un AFD $M = (\{q_1, \dots, q_n\}, \Sigma, \delta, q_1, F)$. Se construirá una expresión regular que describe $L(M)$.

Sea R_{ij}^k el conjunto de todos los strings x , tales que

$$\delta(q_i, x) = q_j$$

y que si $\delta(q_i, y) = q_l$, para cualquier y prefijo de x (que no sea x o ε), entonces $l \leq k$.

Esto es, R_{ij}^k es el conjunto de todos los strings que llevan al AFD de q_i a q_j , sin pasar por ningún estado con número (sub-índice) mayor que k . Por pasar se entiende entrar y salir. Por lo tanto i, j o ambos pueden ser mayores que k .

Ya que no hay estados con numeración mayor que N , R_{ij}^N denota todos los strings que llevan al AFD de q_i a q_j .

Es posible definir R_{ij}^k de la siguiente manera formal:

$$R_{ij}^k = R_{ik}^{k-1}(R_{kk}^{k-1})^*R_{kj}^{k-1} \cup R_{ij}^{k-1} \quad (\forall 1 \leq k \leq N)$$

$$R_{ij}^0 = \begin{cases} \{a/\delta(q_i, a) = q_j\} & \text{si } i \neq j \\ \{a/\delta(q_i, a) = q_j\} \cup \{\varepsilon\} & \text{si } i = j \end{cases}$$

Informalmente, la definición anterior para R_{ij}^k significa que los strings que hacen que el AFD vaya de q_i a q_j , sin pasar por un estado mayor que q_k , son de dos tipos:

- están en R_{ij}^{k-1} , es decir, no pasan ni siquiera por q_k
- están compuestos de un string en R_{ik}^{k-1} , que lleva a M de q_1 a q_k por primera vez, seguido por cero o más strings en R_{kk}^{k-1} , que lleva a M de q_k a q_k sin pasar ni por q_k , ni por un estado mayor, seguido finalmente por un string en R_{kj}^{k-1} , que lleva a M de q_k a q_j .

Se debe demostrar que para cada i , j y k , existe una expresión regular r_{ij}^k , que representa al lenguaje R_{ij}^k . La prueba es por inducción en k .

Base ($k = 0$): R_{ij}^0 es un conjunto finito de strings, cada uno de los cuales es ε o un solo símbolo del alfabeto. Por lo tanto, r_{ij}^0 puede ser escrito como $a_1 + a_2 + \dots + a_p$ (o $a_1 + a_2 + \dots + a_p + \varepsilon$ si $i = j$), en que $\{a_1, \dots, a_p\}$ es el conjunto de todos los símbolos a , tales que $\delta(q_i, a) = q_j$. Si no los hay, entonces $\emptyset$ (o ε si $i = j$) sirve como r_{ij}^0 .

Inducción: La fórmula recursiva para R_{ij}^k envuelve sólo las operaciones : unión, concatenación y clausura. Por la hipótesis, para cada l y m existe una expresión regular r' , tal que

$$L(r_{lm}^{k-1}) = R_{lm}^{k-1}$$

Por lo tanto, para r_{ij}^{k-1} se puede usar la expresión regular

$$(r_{lm}^{k-1})(r_{kk}^{k-1})^*(r_{kj}^{k-1}) + r_{ij}^{k-1}$$

lo que completa la prueba por inducción.

Para terminar la demostración del teorema, basta con observar que

$$L(M) = \cup_{q_j \in F} R_{1j}^N$$

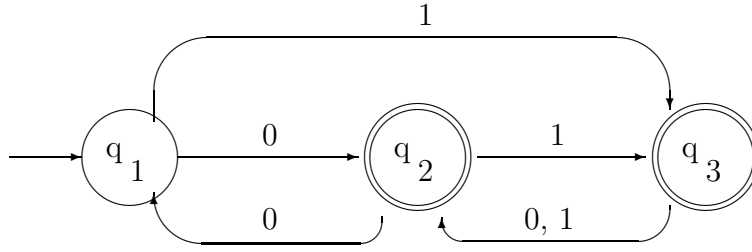
dado que R_{1j}^N denota las etiquetas de los caminos de q_1 , el estado inicial, a q_j . Por lo tanto, $L(M)$ se puede representar por la expresión regular

$$r_{1j_1}^N + r_{1j_2}^N + \dots + r_{1j_p}^N$$

en que $F = \{q_{j_1}, q_{j_2}, \dots, q_{j_p}\}$

□

Ejemplo 60 Se construirá una expresión regular que describe el lenguaje aceptado por el siguiente AFD



Interesa $r = r_{12}^3 + r_{13}^3$

$$r_{12}^3 = r_{12}^2 + r_{13}^2 (r_{33}^2)^* r_{32}^2$$

$$r_{12}^2 = r_{12}^1 + r_{12}^1 (r_{22}^1)^* r_{22}^1$$

$$\begin{aligned} r_{12}^1 &= r_{12}^0 + r_{11}^0 (r_{11}^0)^* r_{12}^0 \\ &= 0 + \varepsilon(\varepsilon)^* 0 = 0 \end{aligned}$$

$$r_{22}^1 = r_{22}^0 + r_{21}^0 (r_{11}^0)^* r_{12}^0 = \varepsilon + 0\varepsilon^* 0 = \varepsilon + 00$$

$$r_{12}^2 = 0 + 0(\varepsilon + 00)^*(\varepsilon + 00) = 0 + 0(\varepsilon + 00)^+ = 0(00)^*$$

$$r_{13}^2 = r_{13}^1 + r_{13}^1 (r_{22}^1)^* r_{23}^1$$

$$r_{13}^1 = r_{13}^0 + r_{11}^0 (r_{11}^0)^* r_{13}^0 = 1 + \varepsilon(\varepsilon)^* 1 = 1$$

$$r_{23}^1 = r_{23}^0 + r_{21}^0 (r_{11}^0)^* r_{13}^0 = 1 + 0(\varepsilon)^* 1 = 1 + 01$$

$$\begin{aligned} r_{13}^2 &= 1 + 0(\varepsilon + 00)^*(1 + 01) = 1 + 0(00)^* 1 = \varepsilon \\ &= 1 + 00^* 1 = 0^* 1 \end{aligned}$$

$$r_{33}^2 = r_{33}^1 + r_{32}^1 (r_{22}^1)^* r_{23}^1$$

$$r_{33}^1 = r_{33}^0 + r_{31}^0 (r_{11}^0)^* r_{13}^0 = \varepsilon + \emptyset(\varepsilon)^* 1 = \varepsilon$$

$$r_{32}^1 = r_{32}^0 + r_{31}^0 (r_{11}^0)^* r_{12}^0 = (0 + 1) + \emptyset(\varepsilon)^* 0 = 0 + 1$$

$$r_{33}^2 = \varepsilon + (0 + 1)(\varepsilon + 00)^*(1 + 01) = \varepsilon + (0 + 1)0^* 1$$

$$\begin{aligned} r_{32}^2 &= r_{32}^1 + r_{32}^1 (r_{22}^1)^* r_{22}^1 = (0 + 1) + (0 + 1)(\varepsilon + 00)^*(\varepsilon + 00) \\ &= 0 + 1 + (0 + 1)(00)^* = (0 + 1)(00)^* \end{aligned}$$

luego

$$\begin{aligned} r_{12}^3 &= 0(00)^* + 0^* 1(\varepsilon + (0 + 1)0^* 1)^*(0 + 1)(00)^* \\ &= 0(00)^* + 0^* 1((0 + 1)0^* 1)^*(0 + 1)(00)^* \end{aligned}$$

similarmente,

$$r_{13}^3 = r_{13}^2 + r_{13}^2(r_{33}^2)^*r_{33}^2$$

$$= 0^*1 + 0^*1(\varepsilon + (0+1)0^*1)^*(\varepsilon + (0+1)0^*1)$$

$$= 0^*1 + 0^*1(\varepsilon + (0+1)0^*1)^+$$

$$= 0^*1 + (\varepsilon + (0+1)0^*1)^*$$

$$= 0^*1((0+1)0^*1)^*$$

Por lo tanto

$$r = r_{12}^3 + r_{13}^3$$

$$= 0(00)^* + 0^*1((0+1)0^*1)^*(0+1)(00)^* + 0^*1((0+1)0^*1)^*$$

□

3.8 Aplicaciones de los Lenguajes Regulares

Hay una cantidad de problemas de diseño de software que son simplificados por la conversión automática de la notación de expresiones regulares a una eficiente implementación en computador del autómata finito correspondiente.

Los *tokens* en un lenguaje de programación son, casi sin excepción, expresables como conjuntos regulares. Por ejemplo, los identificadores de Pascal pueden expresarse como

$$\text{letra}(\text{letra} + \text{dígito})^*$$

en que

$$\text{letra} \equiv a + b + \dots + z + A + B + \dots + Z$$

y

$$\text{dígito} \equiv 0 + 1 + 2 + \dots + 9$$

y los identificadores de FORTRAN, con un límite de seis símbolos y sólo mayúsculas, como

$$\text{letra}(\varepsilon + \text{letra} + \text{dígito})^5$$

en que, ahora,

$$\text{letra} \equiv A + B + \dots + Z$$

Una cantidad de generadores de analizadores léxicos toman como datos una secuencia de expresiones regulares, describiendo los tokens, y producen un único autómata finito que reconoce cualquiera de ellos.

Usualmente, las expresiones regulares son convertidas a un AFND- ε y de ahí, directamente, a un AFD, sin eliminar primero las transiciones en vacío. Cada estado final indica el token particular que se ha reconocido, así que el autómata puede, en realidad, considerarse una máquina de Moore.

La función de transición del AFD se puede almacenar de diversas maneras para que ocupe menos espacio que representada como un arreglo de dos dimensiones con la tabla de transición. El analizador léxico producido por el generador es un programa fijo que interpreta esas tablas codificadas, junto con la tabla particular que representa al AFD que reconoce los tokens. (Ver Figura 3.23) Este analizador léxico, así generado, puede ser usado como un módulo de un compilador.

Algunos editores de texto y programas similares permiten la sustitución por un string dado, de cualquier string representado por una expresión regular, también dada.

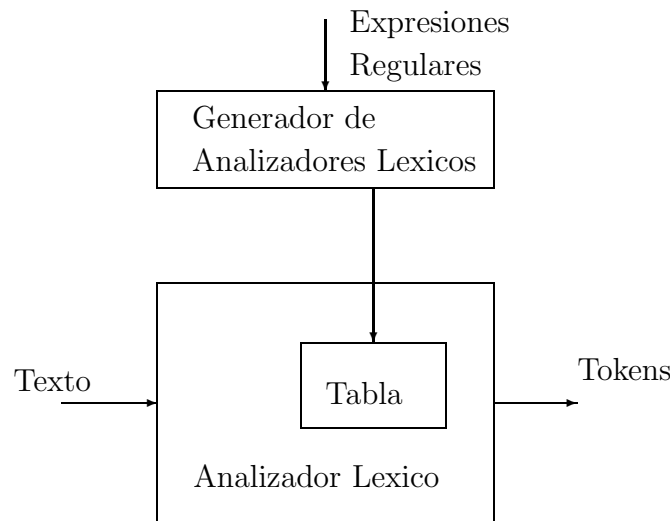


Figure 3.23: Construcción de analizadores léxicos

Por ejemplo, el editor de texto de UNIX permite un comando como:

s/ b b b / b/*

que sustituye por un solo blanco el primer string con dos o más blancos que se encuentre en una línea.

Si *TODO* denota la expresión $a_1 + a_2 + \dots + a_n$ en que los a_i 's son todos los caracteres del computador, excepto el de cambio de línea (newline), es posible convertir una expresión regular r a un AFD que acepte $TODO*r$. La presencia de $TODO*$ permite reconocer un miembro de $L(r)$ que comience en cualquier parte de una línea. Sin embargo, la conversión de la expresión regular a un AFD toma, en la mayoría de los casos, mucho más tiempo que el que toma revisar una línea usando el AFD y, además, el AFD puede tener un número de estados que es exponencial en la longitud de la expresión regular.

Lo que realmente sucede en el editor de texto de UNIX, es que la expresión regular $TODO*r$ es convertida en un AFND- ϵ , el que es simulado directamente. A medida que se revisa la línea, una lista de estados posibles (o actuales según se mire), es mantenida, la que inicialmente es la *clausura* $-\epsilon$ del estado inicial. Si a es el próximo carácter en la línea, se crea una nueva lista de todos los estados con una transición en a desde algunos de los estados de la lista antigua. La lista antigua se descarta y se computa la clausura vacía de la nueva. Si no hay estados finales en la lista nueva, se repite el proceso con el próximo símbolo.

Chapter 4

PROPIEDADES DE LOS LENGUAJES REGULARES

En este capítulo se estudiarán propiedades de clausura y problemas de decisión para los lenguajes regulares. Hay varias preguntas que se pueden hacer respecto de los conjuntos regulares. Una pregunta es: dado un lenguaje L , especificado en alguna forma, ¿Es L regular? También es posible preguntarse si los lenguajes descritos por expresiones regulares distintas son el mismo lenguaje

4.1 Lema de Bombeo para Conjuntos Regulares

En esta sección se verá un resultado básico, llamado el *Lema de Bombeo* (o *Pumping Lemma*), que es un instrumento muy poderoso para demostrar que ciertos lenguajes no son regulares. También es útil para el desarrollo de algoritmos que respondan preguntas tales como si un AF acepta un lenguaje finito o no.

Si un lenguaje es regular, es aceptado por un AFD, $M = (Q, \Sigma, \delta, q_0, F)$ con algún número particular (y finito) de estados, N . Considérese ahora un string de entrada con más de N símbolos (o N):

$$a_1 a_2 \dots a_M \quad (M \geq N)$$

y para $i = 1, 2, \dots, M$ sea

$$\delta(q_0, a_1 a_2 \dots a_i) = q_i \quad (1 \leq i \leq M)$$

No es posible que los $N + 1$ estados $(q_0, q_1, \dots, q_N)$ sean todos diferentes ya que hay sólo N estados distintos. Por lo tanto hay dos enteros j y k (con $0 \leq j < k \leq N$) tales que $q_j = q_k$. El camino con etiqueta $a_1 a_2 \dots a_M$ se ilustra en la siguiente figura:

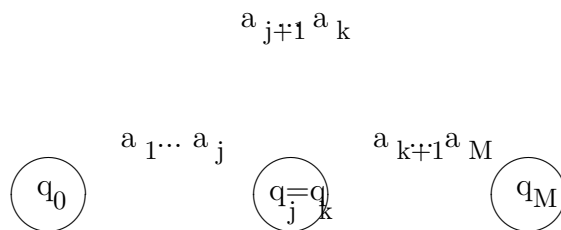


Figure 4.1: Esquema explicativo del Lema de Bombeo

Dado que $j < k$, el string $a_{j+1} \dots a_k$ es de longitud 1 a lo menos y como $k \leq N$, su longitud no es mayor a N .

Si $q_M \in F$, esto es, $a_1 a_2 \dots a_M \in L(M)$, entonces $a_1 a_2 \dots a_j a_{k+1} \dots a_M$ también pertenece a $L(M)$ ya que hay un camino que va de q_0 a q_M , pasando por q_j pero no por el loop con etiqueta $a_{j+1} \dots a_k$.

Formalmente

$$\begin{aligned} \delta(q_0, a_1 \dots a_j a_{k+1} \dots a_M) &= \delta(\delta(q_0, a_1 \dots a_j), a_{k+1} \dots a_M) \\ &= \delta(q_j, a_{k+1} \dots a_M) \\ &= \delta(q_k, a_{k+1} \dots a_M) \\ &= q_M \in F \end{aligned}$$

En forma similar, es posible reconocer el loop más de una vez, de hecho, tantas veces como se desee. Es decir:

$$a_1 \dots a_j (a_{j+1} \dots a_k)^i a_{k+1} \dots a_M$$

está en $L(M)$ para cualquier $i \geq 0$. Lo que se ha demostrado es que dado un string suficientemente largo, aceptado por un AF, se puede encontrar un substring cerca del comienzo del string, el que puede ser bombeado, es decir repetido, cuantas veces se desee y el string resultante también será aceptado por el AF.

Lema 2 Sea L un conjunto regular. Entonces hay una constante N tal que si $z \in L$ y $|z| \geq N$, se puede escribir $z = uvw$, de tal forma que $|uv| \leq N$ y $|v| \geq 1$ y, además, para todo $i \geq 0$ $uv^i w \in L$. Además, N no es mayor que el número de estados del más pequeño AF que acepta L .

Demostración : Ver la discusión anterior al enunciado del lema. En ella $z = a_1 a_2 \dots a_M$; $u = a_1 a_2 \dots a_j$; $v = a_{j+1} \dots a_k$ y $w = a_{k+1} \dots a_M$.

□

Nótese que el lema de bombeo indica que si un lenguaje regular contiene un string suficientemente largo, z , entonces contiene un conjunto infinito de strings de la forma $uv^i w$. El lema *no* establece que cada string suficientemente largo de un conjunto regular sea de la forma $uv^i w$ para algún valor de i . De hecho, $(0+1)^*$ contiene strings arbitrariamente largos en que ningún substring aparece tres veces consecutivas.

El lema de bombeo es muy útil para probar que ciertos conjuntos *no* son lenguajes regulares. La metodología usual es un “argumento adverso” del siguiente tipo:

- Seleccione el lenguaje L que se desea probar no es regular.
- El “adversario” elige N , la constante que se menciona en el lema de bombeo. Este puede ser cualquier valor entero finito, pero una vez elegido, el adversario no lo puede cambiar.
- Seleccione un string $z \in L$. La elección del string puede depender del valor de N .
- El adversario divide z en u , v y w , sujeto a que $|uv| \leq N$ y que $|v| \geq 1$.
- Se obtiene una contradicción con el lema de bombeo, mostrando que para cualquier u , v y w elegidos por el adversario, existe un entero i para el cual $uv^i w$ no pertenece a L . Se puede entonces concluir que L no es regular. La selección de i puede depender de N , u , v y w .

Es interesante notar que las selecciones propias corresponden a los cuantificadores universales y las selecciones del adversario, a los cuantificadores existenciales en una presentación formal del lema de bombeo:

$$\begin{aligned} &(\text{Para todo lenguaje regular } L) \\ &\quad (\text{Existe un entero positivo } N) \\ &\quad \quad (\text{Para todo string } z \in L \text{ con } |z| \geq N) \\ &\quad \quad \quad (\text{Existen } u, v \text{ y } w \text{ con } z = uvw, |uv| \leq N, |v| \geq 1) \\ &\quad \quad \quad \quad (\text{Para todo } i \text{ no negativo } uv^i w \in L) \end{aligned}$$

Ejemplo 61 $L = \{0^i/i \geq 1\}$ no es regular. Asuma que L es regular y sea N la constante del lema de bombeo. Considere:

$$z = 0^{N^2} \in L$$

Por el lema de bombeo z puede ser reescrito como uvw , en que $|uv| \leq N$, $|v| \geq 1$ y $uv^i w$ debiera pertenecer a L , para todo $i \geq 0$. En particular considere $i = 2$, entonces como

$$N^2 < |uv^2 w| \leq N^2 + N < (N + 1)^2$$

esto es, la longitud de $uv^2 w$ está entre N^2 y $(N + 1)^2$ y, por lo tanto, no es un cuadrado perfecto; quiere decir que $uv^2 w$ no pertenece a L . Una contradicción. Se concluye entonces que L no es regular. □

Ejemplo 62 $L = \{a_i b_i / i \geq 1\}$ no es regular. Asuma que L es regular y sea N la constante del lema de bombeo. Considere:

$$z = a^N b^N \in L$$

Por el lema de bombeo, z puede ser reescrito como uvw , en que $|uv| \leq N$ y $|v| \geq 1$, es decir v es un string de a 's de la forma

$$v = a_k \quad \text{con } 1 \leq k \leq N$$

Según el lema de bombeo, el string

$$z' = uv^2 w$$

debiera pertenecer a L . Sin embargo,

$$z' = a^{N+k} b^N \quad (1 \leq k \leq N)$$

y, por lo tanto, no tiene igual número de a 's que de b 's, es decir, no pertenece a L . Una contradicción. Se concluye que L no es un lenguaje regular. □

4.2 Propiedades de Clausura

Hay muchas operaciones entre lenguajes que conservan a los lenguajes regulares, en el sentido que la operación aplicada a lenguajes regulares produce un lenguaje regular.

Por ejemplo, la unión de dos conjuntos regulares es un conjunto regular, ya que si r_1 y r_2 son expresiones regulares describiendo los lenguajes regulares L_1 y L_2 , entonces $r_1 + r_2$ describe $L_1 \cup L_2$, por lo tanto la unión es también regular. Similarmente, la concatenación de conjuntos regulares y la clausura de Kleene de un lenguaje regular es regular.

Si una clase de lenguajes es cerrada bajo una cierta operación, ese hecho es llamado una *propiedad de clausura* de esa clase de lenguajes. Se está particularmente interesado en propiedades de clausura efectivas, en que dado descriptores de los lenguajes en la clase, hay un algoritmo para construir una representación para el lenguaje que resulta de aplicar la operación a esos lenguajes. Por ejemplo, se acaba de dar un algoritmo para construir expresiones regulares para la unión de dos lenguajes descritos por expresiones regulares, por lo tanto, la clase de conjuntos regulares es efectivamente cerrada bajo la unión.

Debe observarse que las equivalencias entre autómatas finitos de distinto tipo y expresiones regulares, mostradas en el capítulo anterior, fueron equivalencias efectivas en el sentido que se dieron algoritmos para pasar de una representación a otra.

Teorema 10 *Los conjuntos regulares son cerrados bajo unión, concatenación y clausura de Kleene.*

Demostración : Inmediata de la definición de expresiones regulares. □

Teorema 11 *La clase de los conjuntos regulares es cerrada bajo complementación. Esto es, si L es regular y $L \subseteq \Sigma^*$, entonces $\Sigma^* - L$ es un conjunto regular.*

Demostración : Sea $M = (Q, \Sigma_1, \delta, q_0, F)$ un AFD que acepta $L \subseteq \Sigma^*$. Se puede asumir que $\Sigma_1 = \Sigma$ porque si hay símbolos en Σ_1 que no pertenecen a Σ es posible eliminar las transiciones de M en los símbolos que $\notin \Sigma$, el hecho que $L \subseteq \Sigma^*$ asegura que no se está cambiando $L(M)$. Si hay símbolos en Σ que no están en Σ_1 , ninguno de ellos puede aparecer en strings de L , por lo tanto se puede agregar un estado “sumidero” S en M con $\delta(q, a) = S$, para todo $q \in Q$ y $a \in \Sigma - \Sigma_1$ y con $\delta(S, a) = S$ para todo $a \in \Sigma$.

Para aceptar $\Sigma^* - L$ basta complementar los estados finales de M , esto es, sea $M' = (Q, \Sigma, \delta, q_0, Q - F)$, entonces M' acepta un string w si y sólo si M no lo acepta, es decir, si y sólo si $w \in \Sigma^* - L$. Nótese que es esencial en la construcción que M sea determinístico. □

Teorema 12 *La clase de los conjuntos regulares es cerrada bajo intersección.*

Demostración : De la teoría de conjuntos se sabe que la siguiente relación se cumple:

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

por lo tanto, la clausura bajo intersección es inmediata después de las clausuras bajo unión y complementación. □

Vale la pena notar que existe una construcción directa para el AFD que acepta la intersección de dos lenguajes regulares: Sean $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ y $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ dos AFD, se construye

$$M = (Q_1 \times Q_2, \Sigma, \delta, [q_1, q_2], F_1 \times F_2)$$

en que para todo $p_1 \in Q_1$, $p_2 \in Q_2$ y $a \in \Sigma$, se tiene

$$\delta([p_1, p_2], a) = [\delta_1(p_1, a), \delta_2(p_2, a)]$$

es fácil mostrar que $L_M = L(M_1) \cap L(M_2)$.

La clase de los lenguajes regulares tiene la propiedad de ser cerrada bajo sustitución en el siguiente sentido. Por cada símbolo a en el alfabeto de algún conjunto regular R , sea R_a un conjunto regular. Suponga que se reemplaza cada string en R , $a_1 a_2 \dots a_N$, por el conjunto de palabras de la forma $w_1 w_2 \dots w_N$ en que los w_i son palabras de R_{a_i} . El resultado es también un lenguaje regular.

Formalmente, una sustitución f es una función desde un alfabeto Σ a 2^{Δ^*} , para algún alfabeto Δ . Es decir, f asocia un lenguaje con cada símbolo de Σ . La sustitución se extiende a strings de la siguiente forma:

- $f(\varepsilon) = \varepsilon$
- $f(xa) = f(x)f(a)$

y se extiende a lenguajes por

- $f(L) = \cup_{w \in L} f(w)$

Ejemplo 63 Sea $f(0) = a$ y $f(1) = b^*$, entonces $f(010) = ab^*a$. También, si $L = 0^*(0+1)^*$ entonces

$$\begin{aligned} f(L) &= a^*(a+b^*)(b^*)^* \\ &= a^*b^* \end{aligned}$$

□

Teorema 13 La clase de los conjuntos regulares es cerrada bajo sustitución por conjuntos regulares.

Demostración : Sea $R \subseteq \Sigma^*$ un lenguaje regular y por cada $a \in \Sigma$ sea $R_a \subseteq \Delta^*$ un lenguaje regular.

Sea $f : \Sigma \rightarrow 2^{\Delta^*}$ una sustitución definida por $f(a) = R_a$, para todo $a \in \Sigma$.

Seleccione expresiones regulares denotando R y cada R_a , reemplace cada ocurrencia de un símbolo a en la expresión regular para R por la expresión regular para R_a . Claramente, el resultado es otra expresión regular.

Para probar que dicha expresión describe $f(R)$, basta observar que la sustitución de una unión, concatenación o clausura, es la unión, concatenación o clausura de la sustitución. Es decir, por ejemplo, $f(L_1 \cup L_2) = f(L_1) \cup f(L_2)$. Una simple inducción en el número de operadores de la expresión regular completa la demostración.

□

Un tipo de sustitución especial es el homomorfismo. Un homomorfismo h es una sustitución tal que para cada símbolo $a \in \Sigma$, $h(a)$ contiene sólo un string. Generalmente se considera que $h(a)$ es el string mismo más que el conjunto que sólo lo contiene a él.

Es también útil definir la imagen homomórfica inversa de un lenguaje L como

$$h^{-1}(L) = \{x/h(x) \in L\}$$

y también para un string w

$$h^{-1}(w) = \{x/h(x) = w\}$$

Ejemplo 64 Sea $h(0) = aa$ y $h(1) = aba$.

Entonces $h(010) = aaabaaa$. Si $L_1 = (01)^*$ entonces $h(L_1) = (aaaba)^*$.

Sea $L_2 = (ab+ba)^*a$, entonces $h^{-1}(L_2) = \{1\}$. Obsérvese que un string en L_2 que comienza con una b no puede ser $h(x)$ para ningún $x \in \{0,1\}^*$ ya que $h(0)$ y $h(1)$ comienzan con a . Por lo tanto si $h^{-1}(w)$ no es vacío y $w \in L_2$, entonces w comienza con a . Ahora, $w = a$ en cuyo caso $h^{-1}(w) = \emptyset$; o w es abw' para algún w' en $(ab+ba)^*a$. Se concluye que cada palabra en $h^{-1}(w)$ comienza con un 1 y, ya que $h(1) = aba$, w' debe comenzar con a . Si $w' = a$ se tiene $w = aba$ y $h^{-1}(w) = \{1\}$. Si $w' \neq a$ entonces $w' = abw''$ y por lo tanto $w = ababw''$. Pero ningún string x en $\{0,1\}^*$ tiene $h(x)$ comenzando con $abab$. Es decir, el único string en L_2 que tiene una imagen inversa bajo h es aba y, por lo tanto, $h^{-1}(L_2) = \{1\}$.

Obsérvese que $h(h^{-1}(L_2)) = \{aba\} \neq L_2$. Es fácil probar que $h(h^{-1}(L)) \subseteq L$ y $L \subseteq h^{-1}(h(L))$ para todo lenguaje L .

□

Teorema 14 La clase de los conjuntos regulares es cerrada bajo homomorfismos y el inverso de un homomorfismo.

Demostración : La clausura bajo homomorfismos es inmediata de la clausura bajo sustitución por conjuntos regulares, ya que todo homomorfismo es una sustitución por un conjunto regular en que cada $h(a)$ tiene un solo elemento.

Para probar la clausura bajo el inverso de un homomorfismo, sea $M = (Q, \Sigma, \delta, q_0, F)$ un AFD que acepte L y sea h un homomorfismo de $\Delta \rightarrow \Sigma^*$. Se construye un AFD, M' , que acepte $h^{-1}(L)$ leyendo un símbolo $a \in \Delta$ y simulando M en $h(a)$. Formalmente, sea $M' = (Q, \Sigma, \delta', q_0, F)$ y se define $\delta'(q, a)$, para todo $q \in Q$ y $a \in \Delta$, como $\delta(q, h(a))$. Nótese que $h(a)$ puede ser un string largo o ε , pero δ está definida sobre todos los strings por extensión. Es fácil mostrar, por inducción en $|x|$, que $\delta'(q_0, x) = \delta(q_0, h(x))$; es decir, M' acepta x si y sólo si M acepta $h(x)$. Esto es, $L(M') = h^{-1}(L(M))$.

□

Ejemplo 65 Como se vio en un ejemplo anterior, $\{a^N b^N / N \geq 1\}$ no es un lenguaje regular. Intuitivamente, $\{0^N 10^N / N \geq 1\}$ no es regular por las mismas razones. Si se tuviera un AF, M , que aceptara $\{0^N 10^N / N \geq 1\}$, se podría aceptar $\{a^N b^N / N \geq 1\}$ simulando M en 0 por cada a , al ver la primera b , simular M en 10 y luego simular M en 0 por cada b . Sin embargo, es necesario probar que $\{0^N 10^N / N \geq 1\}$ no es regular. Esto se puede hacer aplicando el lema de bombeo, pero es más simple utilizar operaciones que conservan regularidad para convertir $\{0^N 10^N / N \geq 1\}$ en $\{a^N b^N / N \geq 1\}$. Por lo tanto $\{0^N 10^N / N \geq 1\}$ no puede ser regular.

Sean h_1 y h_2 los homomorfismos

$$\begin{array}{ll} h_1(0) = 0 & h_2(0) = a \\ h_1(1) = 10 & h_2(1) = b \\ h_1(2) = 0 & h_2(2) = b \end{array}$$

Entonces

$$h_2(h_1^{-1}(\{0^N 10^N / N \geq 1\}) \cap 0^* 12^*) = \{a^N b^N / N \geq 1\}$$

porque

$$h_1^{-1}(\{0^N 10^N / N \geq 1\}) = (0 + 2)^* 1(0 + 2)^*$$

en que el número de símbolos después del 1 es uno menor que los anteriores al 1.

Por lo tanto

$$h_1^{-1}(\{0^N 10^N / N \geq 1\}) \cap 0^* 12^* = \{0^N 12^{N-1} / N \geq 1\}$$

Si $\{0^N 10^N / N \geq 1\}$ fuera regular, dado que el homomorfismo inverso de homomorfismos e intersección con un conjunto regular preservan la propiedad de ser regular, se concluiría que $\{a^N b^N / N \geq 1\}$ es regular, lo que es una contradicción. Por lo tanto $\{0^N 10^N / N \geq 1\}$ no puede ser regular.

□

4.3 Algoritmos de Decisión

El tipo de pregunta que nos preocupa incluye: ¿es un lenguaje regular dado vacío, finito o infinito?, ¿es un conjunto regular igual a otro?, etc. Para estos propósitos se supondrá que los lenguajes regulares están descritos por autómatas finitos.

Teorema 15 *El conjunto de strings aceptado por un autómata finito M con N estados es*

- *No vacío, si y sólo si M acepta un string de largo inferior a N .*
- *Infinito, si y sólo si M acepta un string de largo l , con $N \leq l < 2N$.*

Por lo tanto existe un algoritmo para determinar si un autómata finito acepta cero, un número finito o un número infinito de sentencias.

Demostración : Suponga que M acepta un conjunto no vacío. Sea w un string tan corto como cualquier otro aceptado. Por el lema de bombeo, $|w| < N$, porque si fuera $|w| \geq N$, entonces $w = uvv$ y uv sería aún más corto y estaría en el lenguaje. Una contradicción con el hecho que w es el string más corto. La otra dirección es obvia.

Si $w \in L(M)$ y $N \leq |w| < 2N$, por el lema de bombeo $L(M)$ es infinito. Esto es, $w = w_1 w_2 w_3$ y para todo $i \geq 0$, $w_1 w_2^i w_3 \in L$. Por el otro lado, si $L(M)$ es infinito, entonces existe w en $L(M)$ con $|w| \geq N$; si $|w| < 2N$ no hay problemas. Si ninguna palabra tiene longitud entre (N) y $(2N - 1)$, sea w de largo al menos $2N$, pero tan corta como cualquiera de longitud mayor o igual a $2N$. Por el lema de bombeo, se

puede escribir $w = w_1 w_2 w_3$ con $1 \leq |w_2| \leq N$ y $w_1 w_3 \in L(M)$. Por lo tanto, ya sea w no fue el más corto string de largo $2N$ o más, o $|w_1 w_3|$ está entre N y $2N - 1$, una contradicción en cualquier caso.

El algoritmo para decidir si $L(M)$ es vacío es: “Vea si algún string de longitud hasta N está en $L(M)$ ”. Es claro que este método tiene garantizado terminar. Para decidir si $L(M)$ es infinito: “Vea si algún string de largo entre N y $2N - 1$ está en $L(M)$ ”. Nuevamente, hay un procedimiento que está garantizado de terminar.

□

Debe notarse que los algoritmos sugeridos por este teorema son tremendamente ineficientes. Sin embargo, se puede verificar si un AFD acepta el conjunto vacío al eliminar de su diagrama de transición todos los estados no alcanzables desde el estado inicial. Si aún queda uno o más estados finales, el lenguaje es no vacío. Luego, sin cambiar el lenguaje aceptado, es posible eliminar todos los estados que no son finales y desde los cuales no se puede llegar a un estado final. El AFD acepta un lenguaje infinito si y sólo si el diagrama que resulta tiene un ciclo. El mismo método se puede usar para un AFND, pero hay que verificar que haya un ciclo con etiqueta distinta de ε .

Ahora se mostrará que hay un algoritmo para determinar si dos AF aceptan el mismo lenguaje.

Teorema 16 *Existe un algoritmo para determinar si dos autómatas finitos aceptan el mismo lenguaje (es decir, son equivalentes).*

Demostración: Sean M_1 y M_2 dos AF que aceptan los lenguajes L_1 y L_2 respectivamente. Por los teoremas anteriores, $(L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$ es aceptado por un AF, M_3 . Es fácil ver que M_3 acepta un string si y sólo si $L_1 \neq L_2$. Por lo tanto, por el teorema anterior, existe un algoritmo que determina si $L_1 = L_2$.

□

Chapter 5

ACEPTACIÓN Y GENERACIÓN DE LENGUAJES LIBRES DE CONTEXTO

Los lenguajes libres de contexto, como los conjuntos regulares, tienen gran importancia práctica, especialmente para definir lenguajes de programación, para formalizar la idea de “parsing”, simplificar la traducción de lenguajes de programación, etc.

En este capítulo estudiaremos los lenguajes libres de contexto, concentrándonos fundamentalmente en sus mecanismos de aceptación y generación.

Las primeras dos secciones abordan los mecanismos de aceptación. Tal como las expresiones regulares tienen un autómata equivalente, el autómata finito, las gramáticas libres de contexto, también tienen una máquina como contraparte: el autómata *apilador* (*pushdown*). La equivalencia es, en este caso, un poco menos satisfactoria, ya que el autómata apilador es un dispositivo no determinístico en que la versión determinística sólo acepta un subconjunto de los lenguajes libres de contexto. Por fortuna, este subconjunto incluye la sintaxis de la mayoría de los lenguajes de programación.

En las restantes secciones se estudian las gramáticas libres de contexto, como mecanismos de generación de lenguajes libres de contexto.

5.1 Autómatas Apiladores

El autómata apilador es básicamente un autómata finito con control no sólo sobre la cinta con el input, sino también sobre un stack con capacidad infinita.

Estos dispositivos pueden utilizarse para reconocer lenguajes no regulares. El conjunto $L = \{wcw^r / w \in \{0, 1\}^*\}$ es un lenguaje libre de contexto generado por la gramática

$$S \rightarrow 0S0|1S1|c$$

No es difícil probar que L no puede ser regular, es decir no puede ser aceptado por ningún autómata finito. Para aceptar L se hará uso de un control finito con dos estados, q_1 y q_2 , y de un stack en que se pondrán bolitas (símbolos) azules, verdes y rojas. El dispositivo tendrá las siguientes reglas de operación:

1. La máquina comienza con una bolita roja puesta en el stack y con el control finito en estado q_1 .
2. Si el input tiene un símbolo 0 y el autómata está en estado q_1 , se pone una bolita azul en el stack. Si el símbolo de entrada es un 1 y está en estado q_1 , se pone una bolita verde. En ambos casos el control permanece en estado q_1 .

3. Si el símbolo de entrada es una c y el control está en estado q_1 , el control cambia a estado q_2 sin modificar el stack.
4. Si el símbolo de entrada es un 0 y el dispositivo está en estado q_2 con una bolita azul (que representa un 0) en el tope del stack, la bolita es removida del stack. Si el símbolo de entrada es un 1 y el dispositivo está en estado q_2 con una bolita verde (que representa un 1) en el tope del stack, la bolita también es removida del stack. En ambos casos el control permanece en estado q_2 .
5. Si el dispositivo está en estado q_2 y hay una bolita roja en el tope del stack, la bolita es removida sin esperar input.
6. Para todos los casos no descritos anteriormente, el autómata no puede moverse.

Las reglas de operación precedentes están resumidas en la siguiente tabla:

Color de la Bolita en el Tope del Stack	Estado del Control Finito	Símbolo de Entrada		
		0	1	c
Azul	q_1	Poner bolita Azul Permanece en q_1	Poner bolita Verde Permanece en q_1	Cambiar a q_2
	q_2	Remover bolita Permanece en q_2		
Verde	q_1	Poner bolita Azul Permanece en q_1	Poner bolita Verde Permanece en q_1	Cambiar a q_2
	q_2		Remover bolita Verde Permanece en q_2	
Roja	q_1	Poner bolita Azul Permanece en q_1	Poner bolita Verde Permanece en q_1	Cambiar a q_2
	q_2	Sin esperar input remover bolita del Stack		

Se dice que el dispositivo así descrito acepta un string si al procesar el último símbolo del string, el stack de bolitas se vacía. Nótese que una vez vacío el stack, no más movidas son posibles.

Esencialmente el dispositivo anterior funciona de la siguiente forma. En estado q_1 el dispositivo construye una imagen de su input, poniendo una bolita azul por cada 0 y una verde por cada 1 en el string de entrada. Cuando el input es una c el autómata entra a estado q_2 . A continuación, el input es comparado con el stack al remover una bolita azul por cada 0 y una verde por cada 1. Si la bolita es de color equivocado respecto del símbolo de entrada, el autómata se detiene sin procesar más input. Si todas las bolitas corresponden, la bolita roja que está en el fondo del stack aparece, y es inmediatamente removida. El stack se vacía y el string es aceptado. Todas las bolitas serán removidas sólo si el string que sigue a la c es el reverso del prefijo anterior a la c .

5.2 Definiciones

En esta sección se formalizará el concepto de autómata apilador (AA o PDA por su nombre en inglés). Los AA tendrán una cinta de entrada, un control finito y un stack. El stack es un string de símbolos tomados de algún alfabeto. El símbolo de más a la izquierda es el que se considera al tope del stack. El dispositivo será no determinístico, teniendo algún número finito de alternativas en cada situación. Las movidas serán de dos tipos. El primer tipo utiliza símbolos de entrada; dependiendo del símbolo de entrada, del símbolo en el tope del stack y el estado del control finito, un número de alternativas es posible; cada alternativa consta de un próximo estado para el control finito y un (posiblemente vacío) string de símbolos para reemplazar el símbolo al tope del stack. Después de seleccionar una alternativa, la cabeza lectora avanza al próximo símbolo del string de entrada.

El segundo tipo de movida, llamado movida vacía (movida- ε), es similar a la anterior, con la excepción de que no se usa el símbolo de entrada y la cabeza lectora no se avanza. Este segundo tipo de movidas permite al AA manipular el stack sin consumir símbolos de entrada.

Finalmente se debe definir el lenguaje que acepta un AA. Hay dos formas naturales de hacerlo. La primera, que ya se ha sugerido, es definir el lenguaje aceptado como el conjunto de todos los inputs para el cual alguna secuencia de movidas hace que el autómata vacíe su stack. Este es el lenguaje aceptado por stack vacío.

La segunda forma de definir el lenguaje aceptado es similar a la forma en que un AF acepta strings. Esto es, se designa a algunos estados como estados finales y se define el lenguaje aceptado como el conjunto de todos los strings de entrada para los cuales alguna secuencia de movidas hace que el AA entre a un estado final.

Como se verá, las dos definiciones de aceptación son equivalentes en el sentido que si un conjunto es aceptado por stack vacío por algún AA, entonces es aceptado por estado final por algún otro AA, y viceversa.

Aceptación por estado final es la noción más común, pero es más fácil probar el teorema básico para los autómatas apiladores usando aceptación por stack vacío. Ese teorema dice que un lenguaje es aceptado por un AA si y sólo si es un lenguaje libre de contexto.

Formalmente, un autómata apilador M es una séxtupla $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ en que

Q es un conjunto finito de estados

Σ es el alfabeto de entrada

Γ es el alfabeto del stack

q_0 es el estado inicial ($q_0 \in Q$)

Z_0 es un símbolo especial del stack ($Z_0 \in \Gamma$), llamado símbolo inicial

$F \subseteq Q$ es el conjunto de estados finales

δ es una función de $Q \times (\Sigma \cup \varepsilon) \times \Gamma$ a $2^{Q \times \Gamma^*}$ (subconjuntos finitos de $Q \times \Gamma^*$)

Por convención se usarán letras minúsculas del comienzo del alfabeto para los símbolos de entrada y del final del alfabeto para strings de símbolos de entrada. Letras mayúsculas serán símbolos del stack y letras griegas indican strings de símbolos del stack.

La interpretación de

$$\delta(q, a, Z) = \{(p_1, \gamma_1), (p_2, \gamma_2), \dots, (p_M, \gamma_M)\}$$

en que q y p_i , ($1 \leq i \leq M$) son estados en Q , $a \in \Sigma$, $Z \in \Gamma$ y $\gamma_i \in \Gamma^*$, ($1 \leq i \leq M$), es que el autómata apilador en estado q , viendo el símbolo de entrada a y teniendo a Z en el tope del stack puede, para cualquier i , entrar a estado p_i , reemplazar el símbolo Z por el string γ_i en el stack y avanzar un lugar la cabeza de lectura. Se adopta la convención que el símbolo más a la izquierda en γ_i será el que queda al tope del stack. Nótese que no es posible elegir un estado p_i y un string γ_j , para $j \neq i$, en una sola movida.

La interpretación de

$$\delta(q, \varepsilon, Z) = \{(p_1, \gamma_1), (p_2, \gamma_2), \dots, (p_M, \gamma_M)\}$$

es que el autómata en estado q , independientemente del símbolo de entrada y teniendo Z al tope del stack, puede entrar al estado p_i y reemplazar Z por γ_i , para cualquier i , $1 \leq i \leq M$. En este caso, la cabeza lectora no es movida.

Ejemplo 66 Descripción formal del autómata apilador que acepta $\{w c w^r / w \in \{0, 1\}^*\}$ por stack vacío.

$$M = (\{q_1, q_2\}, \{0, 1, c\}, \{A, V, R\}, \delta, q_1, R, \emptyset)$$

con la función δ definida como sigue:

$$\begin{aligned}
\delta(q_1, 0, R) &= \{(q_1, AR)\} & \delta(q_1, 1, R) &= \{(q_1, VR)\} \\
\delta(q_1, 0, A) &= \{(q_1, AA)\} & \delta(q_1, 1, A) &= \{(q_1, VA)\} \\
\delta(q_1, 0, V) &= \{(q_1, AV)\} & \delta(q_1, 1, V) &= \{(q_1, VV)\} \\
\delta(q_1, c, R) &= \{(q_2, R)\} \\
\delta(q_1, c, A) &= \{(q_2, A)\} \\
\delta(q_1, c, V) &= \{(q_2, V)\} \\
\delta(q_2, 0, A) &= \{(q_2, \varepsilon)\} & \delta(q_2, 1, V) &= \{(q_2, \varepsilon)\} \\
\delta(q_2, \varepsilon, R) &= \{(q_2, \varepsilon)\}
\end{aligned}$$

Nótese que para cada movida en que el autómata escribe un símbolo en el tope del stack, δ tiene un valor (q, γ) en que $|\gamma| = 2$. Por ejemplo $\delta(q_1, 0, R) = \{(q_1, AR)\}$. Si γ fuera de longitud 1, el AA simplemente reemplazaría el símbolo al tope del stack por un nuevo símbolo, sin incrementar el tamaño del stack. Esto hace que si γ es ε , el resultado es un pop del stack.

Nótese también que la regla $\delta(q_2, \varepsilon, R) = \{(q_2, \varepsilon)\}$ significa que el AA en estado q_2 con R al tope del stack puede borrar esa R independientemente del símbolo de entrada. En este caso, la cabeza lectora no se avanza, y en realidad no es necesario que hubiese input adicional.

□

Para describir formalmente la configuración en que se encuentra un AA en un instante dado, se define una *descripción instantánea* (DI). La DI debe, por supuesto, registrar el estado y el contenido del stack; sin embargo es útil que además incluya el input aún no procesado. Por lo tanto una DI se define como una triple (q, w, γ) en que q es un estado, w un string de símbolos de entrada y γ un string de símbolos el stack.

Si $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ es un AA, se dice que $(q, aw, Z\alpha) \vdash_M^* (p, w, \beta\alpha)$ si $\delta(q, a, Z)$ contiene (p, β) . Nótese que a puede ser tanto ε como algún símbolo de entrada, en esta definición. Por ejemplo, en el AA del ejemplo anterior el hecho de que (q_1, AV) esté en $\delta(q_1, 0, V)$ asegura que

$$(q_1, 011, VVR) \vdash (q_1, 11, AVVR)$$

Se usa $\vdash_M^*$ para la clausura reflexiva y transitiva de $\vdash_M$. Esto es, $I \vdash_M^* I$ para toda DI I , y si $I \vdash_M^* J$ y $J \vdash_M^* K$ entonces $I \vdash_M^* K$. Se escribirá $I \vdash_M^i K$ si la descripción instantánea I se puede convertir a K después de exactamente i movidas.

Para un AA, $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, se define $L(M)$, el lenguaje aceptado por estado final a

$$\{w/(q_0, w, Z_0) \vdash_M^* (p, \varepsilon, \gamma) \text{ con } p \in F \text{ y } \gamma \in \Gamma^*\}$$

y se define $N(M)$, el lenguaje aceptado por stack vacío a

$$\{w/(q_0, w, Z_0) \vdash_M^* (p, \varepsilon, \varepsilon) \text{ con } p \in Q\}$$

Si la aceptación es por stack vacío, el conjunto de estados finales es irrelevante y normalmente, en esos casos, se define como el conjunto vacío.

Ejemplo 67 El siguiente autómata apilador acepta el lenguaje $\{ww^r/w \in \{0, 1\}^*\}$, por stack vacío.

$$M = (\{q_1, q_2\}, \{0, 1\}, \{R, A, V\}, \delta, q_1, R, \emptyset)$$

$$\begin{aligned}
\delta(q_1, 0, R) &= \{(q_1, AR)\} \\
\delta(q_1, 1, R) &= \{(q_1, VR)\} \\
\delta(q_1, 0, A) &= \{(q_1, AA), (q_2, \varepsilon)\} \\
\delta(q_1, 0, V) &= \{(q_1, AV)\} \\
\delta(q_1, 1, A) &= \{(q_1, VA)\} \\
\delta(q_1, 1, V) &= \{(q_1, VV), (q_2, \varepsilon)\} \\
\delta(q_2, 0, A) &= \{(q_2, \varepsilon)\} \\
\delta(q_2, 1, V) &= \{(q_2, \varepsilon)\} \\
\delta(q_1, \varepsilon, R) &= \{(q_2, \varepsilon)\} \\
\delta(q_2, \varepsilon, R) &= \{(q_2, \varepsilon)\}
\end{aligned}$$

En la tercera y sexta reglas, M tiene una elección de entre dos movidas. M puede decidir que ha encontrado la mitad del string y elegir la segunda alternativa: ir al estado q_2 y tratar de que el resto de los símbolos de entrada coincidan con los del stack. Si M adivina correctamente y el string de entrada era de la forma ww^r , entonces los símbolos van a coincidir, M va a vaciar su stack y por lo tanto aceptará el string. Igual que en los AF, un AA no determinístico M acepta un string si hay una secuencia de elecciones que lo hacen vaciar su stack. M siempre adivina (escoge) bien, porque una elección equivocada no causa el rechazo de un string. Un string se rechaza sólo si no hay elección correcta posible. La Figura 5.1 muestra las DI accesibles cuando M procesa el string 001100.

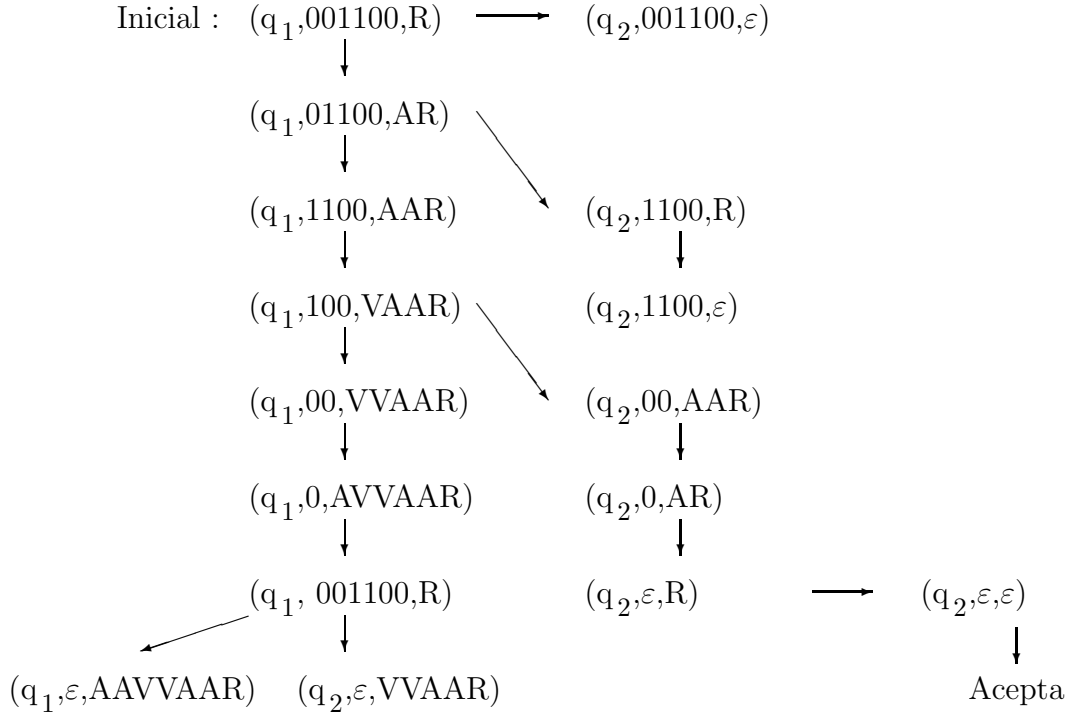


Figure 5.1: Descripciones instantáneas al procesar el string 001100

□

El autómata apilador del primer ejemplo es determinístico en el sentido que a lo más una sola movida es posible dada una DI. Formalmente, se dice que AA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ es determinístico ssi

1. Para cada $q \in Q$ y $Z \in \Gamma$, cuando $\delta(q, \varepsilon, Z)$ no es vacío, entonces $\delta(q, a, Z)$ es vacío para todo $a \in \Sigma$.
2. Para ningún $q \in Q$, $Z \in \Gamma$ y $a \in \Sigma \cup \{\varepsilon\}$, $\delta(q, a, Z)$ contiene más de un elemento.

La condición (1) previene la posibilidad de elegir entre una movida independiente del símbolo de entrada (movida- ε) y una movida que envuelva un símbolo. La condición (2) previene una elección en la movida para cualquier (q, a, Z) o para (q, ε, Z) .

Contrario al caso de los autómatas finitos, un autómata apilador se supone no determinístico. Para los AF, los modelos determinístico y no determinístico eran equivalentes respecto de los lenguajes aceptados. Esto no es cierto para los AA. De hecho, ww^r es aceptado por un AA no determinístico, pero no existe un AA determinístico que lo acepte.

5.3 Gramáticas Libres de Contexto

Una gramática libre de contexto es un conjunto finito de variables (también llamadas no-terminales o categorías sintácticas) cada una de las cuales representa un lenguaje. Estos lenguajes descritos por las variables se definen recursivamente en términos de otros y de símbolos llamados terminales. Las reglas que relacionan las variables son llamadas producciones. Una producción típica dirá que el lenguaje asociado a una variable está formado por strings generados al concatenar strings de los lenguajes de algunas otras variables y algunos terminales.

La motivación original para las gramáticas libres de contexto fue la descripción de lenguajes naturales. Por ejemplo, es posible escribir reglas como:

```

< sentencia >   → < sujeto >   < predicado >
< sujeto >      → < sujeto >   < adjetivo >
< sujeto >      → < artículo > < sustantivo >
< adjetivo >    → < roja >
< sustantivo >  → < casa >
< artículo >   → < la >

```

en que las categorías sintácticas están escritas entre paréntesis en ángulo (< >), y los terminales sin ellos. Por ejemplo, < sujeto > es una categoría sintáctica y casa es un terminal.

El significado de la regla

```

< sentencia > → < sujeto >   < predicado >

```

es que una manera de formar una sentencia (un string en el lenguaje de la categoría sintáctica < sentencia >) es tomar un sujeto y seguirlo de un predicado. El significado de la regla

```

< sustantivo > → < casa >

```

es que el string que consta del símbolo terminal casa, está en el lenguaje de la categoría < sustantivo >. Nótese que casa es un solo símbolo terminal en este caso, no un string de 4 símbolos.

Las gramáticas libres de contexto no se consideran, en general, apropiadas para la descripción de lenguajes regulares naturales como el Castellano. Por ejemplo, si se extienden las producciones anteriores a todo el Castellano, es posible derivar “frío” como un sujeto y “es caliente” como un predicado. Por lo tanto “frío es caliente” sería una sentencia, lo que no tiene sentido. Aún así, las gramáticas libres de contexto juegan un rol importante en lingüística computacional.

Mientras los lingüistas estudiaban gramáticas libres de contexto, los científicos de computación comenzaron a describir los lenguajes de programación con una notación llamada “*Backus-Naur Form* (BNF)”; que en realidad corresponde a la notación para gramáticas libres de contexto con algunos cambios menores y algunas abreviaciones en la descripción. Este uso de las gramáticas libres de contexto ha simplificado enormemente la definición de los lenguajes de programación y la construcción de compiladores. La razón de este éxito es debida, en parte, a la forma natural en que la mayoría de las construcciones de los lenguajes de programación se pueden describir usando gramáticas. Por ejemplo, considere el siguiente conjunto de producciones.

```

< expresion > → < expresion > + < expresion >
< expresion > → < expresion > * < expresion >
< expresion > → (< expresion >)
< expresion > → id

```

que define las expresiones aritméticas con operadores + y *, y operandos representados por el símbolo id. En ellas, < expresión > es la única variable y los terminales son los símbolos +, *, (,) e id.

Las dos primeras producciones indican que una expresión puede estar compuesta por dos expresiones conectadas por un signo de suma o multiplicación. La tercera indica que una expresión encerrada por paréntesis es también una expresión. La última indica que un operando es también una expresión.

Utilizando repetidamente las producciones, se pueden obtener expresiones cada vez más complicadas. Por ejemplo,

$$\begin{aligned}
< \text{expresion} > &\Rightarrow < \text{expresion} > * < \text{expresion} > \\
&\Rightarrow (< \text{expresion} >)* < \text{expresion} > \\
&\Rightarrow (< \text{expresion} >) * \text{id} \\
&\Rightarrow (< \text{expresion} > + < \text{expresion} >) * \text{id} \\
&\Rightarrow (< \text{expresion} > + < \text{id} >) * \text{id} \\
&\Rightarrow (< \text{id} > + < \text{id} >) * \text{id}
\end{aligned}$$

El símbolo $\Rightarrow$ denota derivación, esto es, el reemplazo de una variable por el lado derecho de una producción para esa variable. Así, la primera línea se obtiene por la segunda producción; la segunda línea se obtiene al reemplazar la primera $< \text{expresión} >$ de la línea anterior por el lado derecho de la tercera producción. Utilizando la cuarta, primera, cuarta y cuarta producción se obtienen las demás líneas. La última línea, $(<\text{id}> + <\text{id}>)*\text{id}$, contiene sólo terminales y es por lo tanto un string en el lenguaje de $< \text{expresión} >$.

5.4 Configuración de las Gramáticas Libres de Contexto

En esta sección se formalizará la noción intuitiva de gramática, presentada en la sección anterior.

Una gramática libre de contexto (CFG, por sus siglas en inglés: Context Free Grammar) o simplemente gramática, es una cuádrupla,

$$G = (V, T, P, S)$$

en que V y T son conjuntos finitos de variables y terminales respectivamente. Se asume que V y T son conjuntos disjuntos. P es un conjunto finito de producciones; cada producción es de la forma $A \rightarrow \alpha$ en que $A \in V$ y α es un string de símbolos sobre $(V \cup T)$. Por último, $S \in V$ es una variable especial llamada el símbolo inicial (start symbol).

Ejemplo 68 Si se usa E , en lugar de $< \text{expresión} >$, para la variable de la gramática anterior, es posible expresarla formalmente como

$$(\{E\}, \{+, *, (,), \text{id}\}, P, E)$$

en que P consta de las siguientes producciones,

$$\begin{aligned}
< E > &\rightarrow < E > + < E > \\
< E > &\rightarrow < E > * < E > \\
< E > &\rightarrow (< E >) \\
< E > &\rightarrow \text{id}
\end{aligned}$$

□

En la especificación de gramáticas se usarán las siguientes convenciones:

- Las letras mayúsculas, A, B, C, D, E y S representan variables; S será el símbolo inicial
- Las letras minúsculas a, b, c, d y e , los dígitos, símbolos y algunos strings como id , serán terminales
- Las letras mayúsculas X, Y y Z representarán símbolos que pueden ser terminales o variables
- Las letras minúsculas u, v, w, x, y y z representan strings de terminales
- Las letras griegas α, β y γ denotan strings de variables y terminales

Usando las convenciones anteriores, es posible deducir cuáles son las variables, terminales y símbolo inicial de una gramática con sólo examinar las producciones. Por lo tanto, normalmente una gramática se

presentará simplemente listando sus producciones. Si $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_N$, son producciones para la variable A de alguna gramática, es posible expresarlas usando la notación

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_N$$

en que $\mid$ es leído “o”. La gramática completa del ejemplo anterior puede escribirse como

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

Ahora se definirá formalmente el lenguaje generado por una gramática $G = (V, T, P, S)$. Para ello es necesario desarrollar una notación que represente las derivaciones.

Primero, se definen dos relaciones: $\xrightarrow{G}$ y $\xRightarrow{*}_G$, entre strings en $(V \cup T)^*$. Si $A \rightarrow B$ es una producción en P y α y γ son strings cualesquiera en $(V \cup T)^*$, entonces

$$\alpha A \gamma \xrightarrow{G} \alpha \beta \gamma$$

Se dice que la producción $A \rightarrow \beta$ se le aplica al string $\alpha A \gamma$ para obtener $\alpha \beta \gamma$, o que $\alpha A \gamma$ deriva directamente $\alpha \beta \gamma$ en la gramática G . Dos strings están relacionados por $\xrightarrow{G}$ exactamente cuando el segundo se obtiene del primero por una aplicación de alguna producción.

Suponga que $\alpha_1, \alpha_2, \dots, \alpha_M$ son strings en $(V \cup T)^*$, con $M \geq 1$, y que

$$\alpha_1 \xrightarrow{G} \alpha_2, \alpha_2 \xrightarrow{G} \alpha_3, \dots, \alpha_{M-1} \xrightarrow{G} \alpha_M$$

Entonces se dice que $\alpha_1 \xRightarrow{*}_G \alpha_M$ o que α_1 deriva α_M en la gramática G . Esto es, $\xRightarrow{*}_G$ es la clausura reflexiva y transitiva de $\xrightarrow{G}$. También, $\alpha \xRightarrow{*}_G \beta$ si β proviene de α por la aplicación de cero o más producciones de P . Nótese que $\alpha \xRightarrow{*}_G \alpha$, para todo string α . Usualmente, si es claro cuál es la gramática G , se usa $\Rightarrow$ en lugar de $\xrightarrow{G}$, y $\Rightarrow^*$ en lugar de $\xRightarrow{*}_G$. También, si α deriva β en exactamente i pasos, se dice que $\alpha \xRightarrow{i}_G \beta$.

El lenguaje generado por G , denotado por $L(G)$, es el conjunto

$$\{w/w \in T^* \text{ y } S \xRightarrow{*}_G w\}$$

esto es, un string está en $L(G)$ si y sólo si

- el string consiste sólo de terminales
- el string es derivable desde S

Un lenguaje se llamará *lenguaje libre de contexto* si es $L(G)$ para alguna gramática libre de contexto G . Un string de terminales y variables, α , es llamado una *forma sentencial* si $S \xRightarrow{*}_G \alpha$. Dos gramáticas se dicen equivalentes si $L(G_1) = L(G_2)$.

Ejemplo 69 Considere la gramática $G = (V, T, P, S)$, con $V = \{S\}$, $T = \{a, b\}$ y P dado por

$$S \rightarrow aSb$$

$$S \rightarrow ab$$

S es la única variable; a y b son terminales. Usando la primera producción $N - 1$ veces, seguidas de una aplicación de la segunda producción, se obtiene:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow \dots \Rightarrow a^{N-1}Sb^{N-1} \Rightarrow a^N b^N$$

Además, sólo strings de la forma $a^N b^N$ ($N \geq 1$) están en $L(G)$. Cada vez que $S \rightarrow aSb$ es usada, se mantiene el número de S 's. Después de usar la producción $S \rightarrow ab$, el número de S 's de la forma sentencial disminuye en uno. Por lo tanto, ya que se empieza con S y ya que ambas producciones son para S , el único orden en que ellas pueden ser usadas es empleando $S \rightarrow aSb$ algún número de veces seguidas por una aplicación de $S \rightarrow ab$. Por lo tanto,

$$L(G) = \{a^N b^N / N \geq 1\}$$

Este lenguaje es el ejemplo de un lenguaje libre de contexto que no es un lenguaje regular.

□

Ejemplo 70 Considere la gramática $G = (V, T, P, S)$, con $V = \{S, A, B\}$, $T = \{a, b\}$ y P dado por las siguientes producciones

$$\begin{array}{ll} S \rightarrow aB & A \rightarrow bAA \\ S \rightarrow bA & B \rightarrow b \\ A \rightarrow a & B \rightarrow bS \\ A \rightarrow aS & B \rightarrow aBB \end{array}$$

El lenguaje $L(G)$ es el conjunto de todos los strings en T^* que tienen el mismo número (≥ 1) de a 's y b 's. Se probará, por inducción en la longitud del string que

- $S \Rightarrow^* w$ si y sólo si w tiene tantas a 's como b 's
- $A \Rightarrow^* w$ si y sólo si w tiene una a más que b 's
- $B \Rightarrow^* w$ si y sólo si w tiene una b más que a 's

La hipótesis es obviamente cierta si $|w| = 1$, ya que $A \Rightarrow a$ y $B \Rightarrow b$ y ningún string de largo 1 de terminales es derivable de S . También, ya que todas las producciones, excepto $A \rightarrow a$ y $B \rightarrow b$ incrementan el largo de un string, ningún string de longitud 1, excepto a y b , son derivables de A y B , ni ninguno es derivable de S .

Suponga ahora que la hipótesis inductiva es verdadera para todo w de largo $k - 1$ ó menos. Se mostrará que se cumple para $|w| = k$. Si $S \Rightarrow^* w$ entonces la derivación debe comenzar con $S \rightarrow a$ o $S \rightarrow bA$. En el primer caso, $w = aw_1$ con $|w_1| = k - 1$ y $B \Rightarrow w_1$. Por la hipótesis inductiva, el número de b 's en w_1 es 1 más que el número de a 's; por lo tanto, w tiene igual número de b 's que de a 's. Un argumento similar es válido si la derivación comienza con $S \rightarrow bA$. Para la prueba en la otra dirección, esto es, si $|w| = k$ y w tiene tantas a 's como b 's, entonces $S \Rightarrow w$, considere que el primer símbolo de w es una a o una b . Suponga que $w = aw_1$; pero $|w_1| = k - 1$ y tiene una b más que a 's. Por la hipótesis inductiva entonces $B \Rightarrow w_1$. Luego $S \Rightarrow aB \Rightarrow^* aw_1 = w$. Un argumento similar es válido si el primer símbolo de w es una b .

Debe ahora probarse las aserciones para A y B , pero se hacen en forma similar a la de S .

Otra gramática posible para este mismo lenguaje es

$$\begin{array}{ll} S \rightarrow ab \\ S \rightarrow ba \\ S \rightarrow aSb \\ S \rightarrow bSa \\ S \rightarrow SS \end{array}$$

□

5.5 Árboles de Derivación

Es muy útil representar las derivaciones como árboles. Estos árboles, llamados *árboles de derivación* (o *de parse*) imponen una estructura en los strings de un lenguaje que es muy útil en aplicaciones como la compilación de lenguajes de programación.

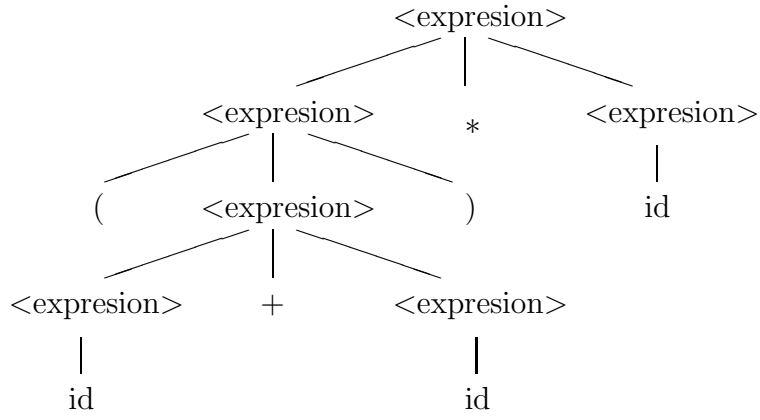
Los vértices o nodos de un árbol de derivación tienen etiquetas que son terminales, variables o el string nulo ε . Si un nodo interior n tiene etiqueta A y los hijos de n tienen etiquetas $X_1, X_2, \dots, X_k$ (de izquierda a derecha), entonces $A \rightarrow X_1X_2 \dots X_k$ debe ser una producción.

La Figura 5.2 muestra el árbol para la derivación de $(id + id) * id$ mostrada anteriormente.

Nótese que si se leen las hojas de izquierda a derecha, se obtiene el string $(id + id) * id$.

Más formalmente, sea $G = (V, T, P, S)$ una gramática libre de contexto. Un árbol es un árbol de derivación si

- Cada vértice tiene una etiqueta que es un símbolo en $V \cup T \cup \varepsilon$

Figure 5.2: Árbol de derivación de $(id + id) * id$

- La etiqueta de la raíz es S
- Si a es un nodo interior y tiene etiqueta A , debe cumplirse que $A \in V$
- Si n tiene etiqueta A y sus hijos de izquierda a derecha son $n_1, n_2, \dots, n_k$ con etiquetas $X_1, X_2, \dots, X_k$ respectivamente, entonces

$$A \rightarrow X_1 X_2 \dots X_k$$

debe ser una producción en P

- Si un vértice n tiene etiqueta ε , entonces n es una hoja y es el único hijo de su padre

Ejemplo 71 Considere la gramática $G = (\{S, A\}, \{a, b\}, P, S)$ en que P está compuesto por

$$S \rightarrow aAS|a$$

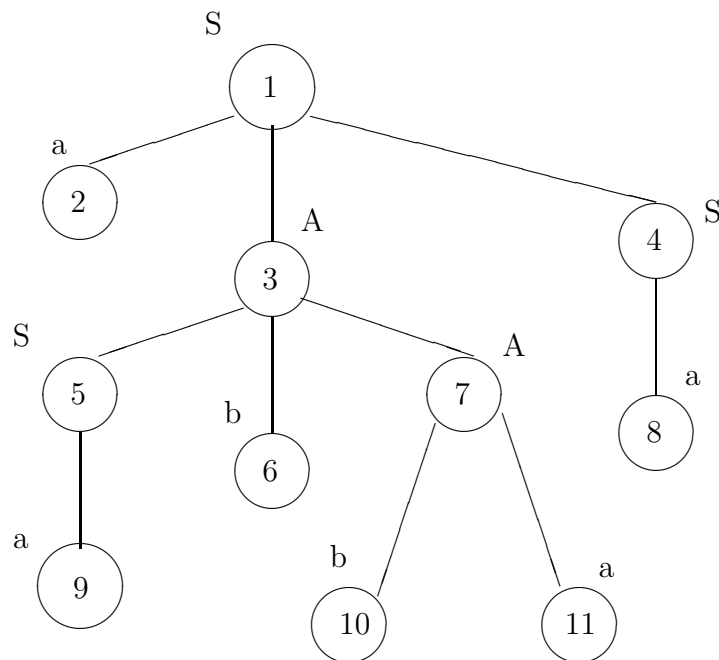
$$A \rightarrow SbA|SS|ba$$

y el árbol de la Figura 5.3.

Los vértices interiores son 1, 3, 4, 5 y 7. El vértice 1 tiene etiqueta S y sus hijos, de izquierda a derecha, tienen etiquetas a , A y S . Nótese que $S \rightarrow aAS$ es una producción en P . Igualmente, el nodo 3 tiene etiqueta A y las etiquetas de sus hijos son S , b y A (de izquierda a derecha). $A \rightarrow SbA$ también es una producción. Los vértices 4 y 5 tienen etiqueta S , sus únicos hijos tienen etiqueta a y $S \rightarrow a$ es una producción. Por último, el vértice 7 tiene etiqueta A y sus hijos, de izquierda a derecha, tienen etiquetas b y a . $A \rightarrow ba$ también es una producción. Por lo tanto, este árbol es un árbol de derivación para G .

□

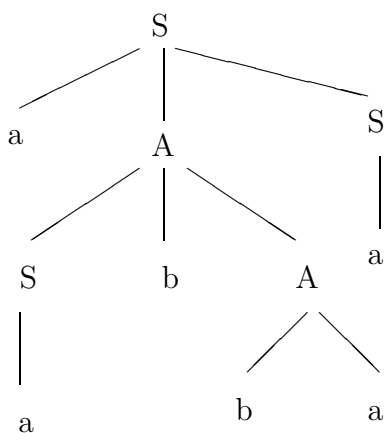
Es posible extender el orden de los hijos de un nodo a un ordenamiento de izquierda a derecha de todas las hojas. De hecho, dos vértices cualesquiera, ninguno de los cuales es un ancestro del otro, uno está a la izquierda del otro. Dados dos vértices v_1 y v_2 , se siguen los caminos de cada uno de ellos hacia la raíz, hasta que se encuentran en un vértice w . Sean X_1 y X_2 los hijos de w en los caminos desde v_1 y v_2 , respectivamente. Si v_1 no es ancestro de v_2 , o viceversa, $X_1 \neq X_2$. Si X_1 está a la izquierda de X_2 como hijos de w , entonces v_1 está a la izquierda de v_2 . Por ejemplo, en el árbol anterior, si $v_1 = 9$ y $v_2 = 11$, entonces $w = 3$, $X_1 = 5$, $X_2 = 7$; y como 5 está a la izquierda de 7, se deduce que 9 está a la izquierda de 11.

Figure 5.3: Árbol correspondiente a la gramática G

Se verá que un árbol de derivación es una descripción natural de la derivación de una forma sentencial de la gramática G . Si se leen las etiquetas de las hojas de izquierda a derecha, se obtiene una forma sentencial. Este string es llamado el *rédito* (yield) del árbol de derivación.

Se necesita también el concepto de subárbol. Un subárbol de un árbol de derivación es un cierto vértice, todos sus descendientes, los arcos que los conectan y sus etiquetas. Se ve igual que un árbol de derivación excepto que la etiqueta de su raíz puede no ser el símbolo inicial de la gramática. Si la variable A es la etiqueta de la raíz, se dice que ese subárbol es un árbol- A . Por lo tanto, árbol- S es un sinónimo para árbol de derivación si S es el símbolo inicial.

Ejemplo 72 Considere la gramática y el árbol de derivación del ejemplo anterior que se reproduce a continuación:

Figure 5.4: Árbol correspondiente a la gramática G

El rédito de ese árbol es: $aabbaa$. Nótese que en este caso todas las hojas tienen etiquetas que son terminales; pero esto no es necesario, podría haber hojas con etiqueta ε o con una variable.

Nótese que $S \xRightarrow{*}_G aabbaa$ por la derivación siguiente:

$$S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aabAS \Rightarrow aabbaS \Rightarrow aabbaa$$

La Figura 5.5 es un subárbol del árbol de derivación anterior; corresponde al vértice 3 el árbol original y sus descendientes.

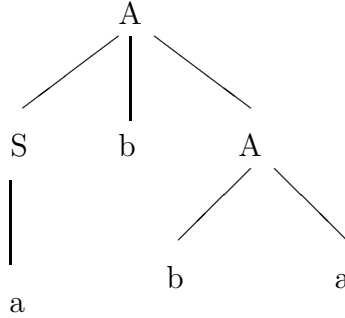


Figure 5.5: Vértice 3 del subárbol original

El rédito de este subárbol es $abba$. La etiqueta de su raíz es a y $A \xRightarrow{*}_G abba$ a través de la siguiente derivación:

$$A \Rightarrow SbA \Rightarrow abA \Rightarrow abba$$

□

Teorema 17 Sea $G = (V, T, P, S)$ una gramática libre de contexto. Entonces $S \xRightarrow{*}_G \alpha$ si y sólo si hay un árbol de derivación para G cuyo rédito sea α .

Demostración : Se probará algo un poco más general, que para cualquier $A \in V$, $A \xRightarrow{*}_G \alpha$ si y sólo si existe un árbol- A cuyo rédito sea α .

Suponga primero que α es el rédito de un árbol- A . Se prueba, por inducción en el número de vértices interiores en el árbol, que $A \xRightarrow{*}_G \alpha$. Si hay un solo nodo interior, el árbol debe lucir como el de la Figura 5.6.

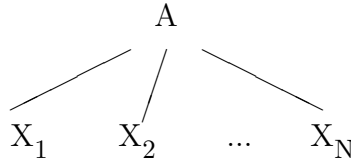


Figure 5.6: Árbol de derivación con un solo nodo interior

En ese caso, $X_1, X_2, \dots, X_n$ debe ser α y $A \rightarrow \alpha$ debe ser una producción de P , por la definición de un árbol de derivación. Luego, $A \Rightarrow \alpha$.

Supóngase ahora que el resultado es válido para árboles con hasta $k - 1$ nodos interiores. Sea α el rédito de un árbol- A con k nodos interiores, para algún $k > 1$. Considere los hijos de la raíz; no pueden ser todos hojas ya que $k > 1$. Sean las etiquetas de los hijos $X_1, X_2, \dots, X_n$, desde la izquierda. Entonces $A \rightarrow X_1X_2 \dots X_n$ es una producción en P . Note que en la discusión siguiente $n \geq 1$.

Si el i -ésimo hijo no es una hoja, es la raíz de un subárbol y $X_i \in V$. El subárbol debe ser un árbol- X_i y tendrá algún rédito α_i . Si el vértice i es una hoja, sea $\alpha_i = X_i$. Es fácil ver que si $j < i$, el vértice j y sus

descendientes están a la izquierda del vértice j y de todos sus descendientes. Por lo tanto, $\alpha = \alpha_1 \alpha_2 \dots \alpha_n$. Un subárbol debe tener menos vértices interiores que el árbol original, a menos que sean el mismo árbol. Por la hipótesis de inducción, para cada vértice i que no es una hoja $X_i \xRightarrow{*} \alpha_i$. Si $X_i = \alpha_i$, entonces $X_i \xRightarrow{*} \alpha_i$, trivialmente. Poniendo todas estas derivaciones parciales juntas,

$$A \Rightarrow X_1 X_2 \dots X_n \xRightarrow{*} \alpha_1 X_2 \dots X_n \xRightarrow{*} \alpha_1 \alpha_2 \dots X_n \xRightarrow{*} \dots \xRightarrow{*} \alpha_1 \alpha_2 \dots \alpha_n = \alpha$$

Por lo tanto $A \xRightarrow{*} \alpha$. Nótese que la anterior es sólo una de las posiblemente muchas derivaciones que se pueden obtener.

Suponga ahora que $A \xRightarrow{*} \alpha$. Se debe mostrar que existe un árbol- A con rédito α . Si $A \Rightarrow \alpha$, entonces $A \rightarrow \alpha$ está en P y hay un árbol con rédito α que tiene la forma de la Figura 5.7.

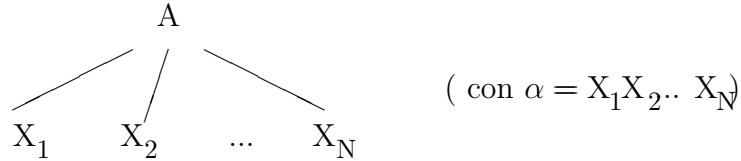


Figure 5.7: Árbol- A con rédito α

Supóngase ahora que para cualquier variable A , si $A \xRightarrow{*} \alpha$ en menos de k pasos, hay un árbol- A con rédito α . Suponga que $A \xRightarrow{*} \alpha$ por una derivación de k pasos. Sea $A \Rightarrow X_1 X_2 \dots X_n$ el primero de estos pasos; cualquier símbolo de α debe ser uno de $X_1, X_2, \dots, X_n$ o ser derivado de uno de ellos. También, la parte de α derivada de X_i debe estar a la izquierda de los símbolos derivados de X_j , si $i < j$. Por lo tanto es posible escribir α como $\alpha_1 \alpha_2 \dots \alpha_n$, en que para cada i entre 1 y n ,

- $\alpha_i = X_i$ si X_i es un terminal, y
- $X_i \xRightarrow{*} \alpha_i$ si X_i es una variable

Si X_i es una variable, la derivación de α_i desde ella debe tomar menos de k pasos. Por lo tanto, por la hipótesis de inducción, por cada X_i que es una variable, hay un árbol- X_i con rédito α_i , que se denominará T_i . Se construye un árbol- A con n hojas, con etiquetas $X_1, X_2, \dots, X_n$. Cada vértice con etiqueta $X_i \notin T$ se reemplaza por el árbol T_i . Si X_i es un terminal no se reemplaza el nodo.

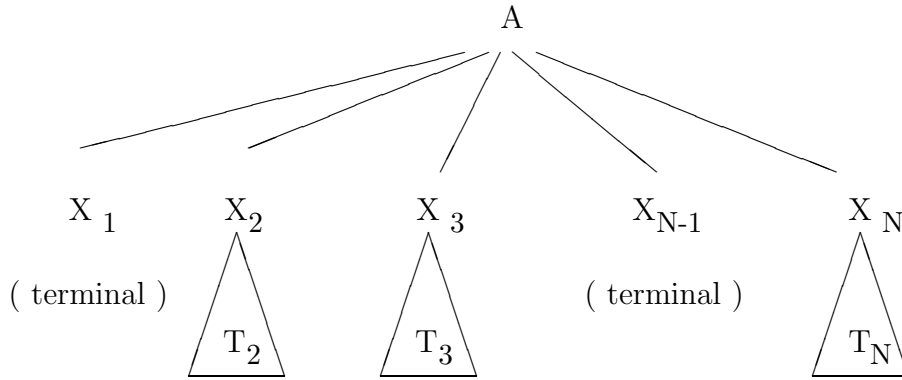


Figure 5.8: Construcción del árbol- A

El rédito del árbol así construido es α , como se quería.

□

Si en cada paso de una derivación se usa una producción para reemplazar la variable de más a la izquierda en la forma sentencial, se dice que esa es una derivación *por la izquierda*. Similarmente, si es la variable de más a la derecha, se dice que es una *derivación por la derecha*.

Si $w \in L(G)$ para alguna gramática libre de contexto G , entonces w tiene al menos un árbol de derivación; y correspondiente a un árbol de derivación en particular, w tiene una única derivación por la izquierda y una única derivación por la derecha. Por supuesto que w puede tener varias derivaciones por la izquierda y varias derivaciones por la derecha ya que puede haber más de un árbol de derivación para w . Sin embargo, es fácil mostrar que por cada árbol de derivación hay sólo una derivación por la izquierda y, también, una única derivación por la derecha.

Ejemplo 73 La derivación por la izquierda que corresponde al árbol del ejemplo anterior es

$$S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aabAS \Rightarrow aabbaS \Rightarrow aabbba$$

y la derivación por la derecha es

$$S \Rightarrow aAS \Rightarrow aAa \Rightarrow aSbAa \Rightarrow aSbbaa \Rightarrow aabbba$$

□

Una gramática G tal que algún string tiene dos árboles de derivación se dice que es *ambigua*. Una definición equivalente es que algún string tenga más de una derivación por la izquierda o más de una por la derecha. Un lenguaje libre de contexto para el cual toda gramática es ambigua, se dice que es un lenguaje *inherentemente ambiguo*. Más adelante se verá que existen lenguajes inherentemente ambiguos.

5.6 Simplificación de Gramáticas

Hay varias maneras en que se puede restringir el formato de las producciones, sin reducir el poder generador de las gramáticas libres de contexto. Si L es un lenguaje libre de contexto no vacío, entonces puede ser generado por una gramática libre de contexto, G , con las siguientes propiedades:

- Cada variable y cada terminal de G aparece en la derivación de algún string de L
- No hay producciones de la forma $A \rightarrow B$, en que A y B son variables

Más aún, si $\varepsilon \notin L$, no es necesario que haya producciones de la forma $A \rightarrow \varepsilon$.

Primero se verá cómo eliminar símbolos inútiles de una gramática. Sea $G = (V, T, P, S)$ una gramática. Un símbolo X es *útil* si existe una derivación

$$S \xRightarrow{*} \alpha X \beta \xRightarrow{*} w$$

para algún α, β y w , con $w \in T^*$. Si un símbolo no es útil, se dice que es *inútil*. Hay dos aspectos que considerar en esto de la utilidad. Primero, algún string de terminales debe ser derivable de X y, segundo, X debe ser parte de un string derivable de S . Pero no sólo eso, sino que además X debe ocurrir en alguna forma sentencial de la que es posible derivar un string del lenguaje.

Lema 3 Dada una gramática libre de contexto $G = (V, T, P, S)$, con $L(G) \neq \emptyset$, es posible encontrar efectivamente una gramática libre de contexto, $G' = (V', T, P', S)$, tal que para todo $A \in V'$ hay un $w \in T^*$ para el cual $A \xRightarrow{*} w$.

Demostración : Cada variable A con producciones $A \rightarrow w$ en P , pertenece a V' . Si $A \rightarrow X_1 X_2 \dots X_n$ es una producción en que cada X_i es un terminal o una variable que ya está en V' , entonces es posible derivar un string de terminales desde A por una derivación que comienza con $A \Rightarrow X_1 X_2 \dots X_n$ y, por lo tanto, $A \in V'$. El conjunto V' se puede calcular con el siguiente algoritmo:

- (1) OLDV := $\emptyset$;
- (2) NEWV := $\{A/A \rightarrow w \in P \text{ con } w \in T^*\}$;
- (3) while OLDV $\neq$ NEWV do begin
- (4) OLDV := NEWV;
- (5) NEWV := OLDV $\cup \{A/A \rightarrow w \in P \text{ con } \alpha \in (T \cup \text{OLDV})^*\}$
- end
- (6) V-PRIMA := NEWV

El algoritmo anterior encuentra todas las variables A que pertenecen a V' . Si A es puesto en NEWV en línea (2) ó (5) es porque deriva un string de terminales. Para demostrar que NEWV tendrá todas esas variables, se debe probar que si A deriva un string de terminales, w , entonces A será eventualmente puesto en NEWV. La prueba es por inducción en el largo de la derivación $A \xRightarrow{*} w$. Nótese que P' es el conjunto de todas las producciones cuyos símbolos están en $V' \cup T$.

Base: Si el largo de la derivación es 1, entonces $A \rightarrow w$ es una producción y A es puesto en NEWV en la línea (2).

Inducción: Sea $A \rightarrow X_1 X_2 \dots X_n \xRightarrow{*} w$ una derivación con k pasos. Entonces se puede escribir $w = w_1 w_2 \dots w_n$, en que $X_i \xRightarrow{*} w_i$, $1 \leq i \leq n$, por una derivación de menos de k pasos. Por la hipótesis de inducción los X_i que sean variables son eventualmente puestos en NEWV. La condición de la sentencia *while* en la línea (3), justo después que el último de los X_i se agrega a NEWV es falsa, ya que ese X_i no está en OLDV. Por lo tanto hay una iteración adicional (al menos), en la que A será agregada a NEWV en la línea (5). Sea V' el conjunto calculado en línea (6) y sea P' el conjunto de todas las producciones cuyos símbolos están en $V' \cup T$. Con toda seguridad $G' = (V', T, P', S)$ satisface la propiedad de que si $A \in V'$, entonces $A \xRightarrow{*} w$, para algún $w \in T^*$. También, como cada derivación en G' es una derivación de G , se sabe que $L(G') \subseteq L(G)$. Si hubiera algún $w \in L(G)$ y no en $L(G')$, entonces cualquier derivación de $w \in G$ debe incluir una variable en $V' - V$ o una producción en $P - P'$ (que implica que se usa una variable en $V - V'$). Pero entonces existe una variable en $V - V'$ que deriva un string de terminales, una contradicción.

□

Lema 4 Dada una gramática libre de contexto $G = (V, T, P, S)$, es posible encontrar efectivamente una gramática libre de contexto equivalente, $G' = (V', T', P', S)$, tal que por cada X en $V' \cup T'$ existen α y β en $(V' \cup T')^*$ tales que $S \xRightarrow{*} G' \alpha X \beta$.

Demostración : El conjunto $V' \cup T'$ de símbolos que aparecen en las formas sentenciales derivables de G se puede construir por un algoritmo iterativo. Ponga S en V' . Si A está en V' y $A \rightarrow \alpha_1 \alpha_2 \dots \alpha_n$, entonces agregue a V' todas las variables que aparezcan en $\alpha_1, \alpha_2, \dots$ o α_n , y a T' todos los terminales en $\alpha_1, \alpha_2, \dots, \alpha_n$. P' es el conjunto de producciones en P que sólo tienen símbolos de $V' \cup T'$.

□

Aplicando primero el lema anterior, y a continuación este último, es posible convertir una gramática en una equivalente sin símbolos inútiles. Es interesante notar que si se utilizan en el orden contrario es posible que aún queden símbolos inútiles.

Teorema 18 Todo lenguaje libre de contexto no vacío es generado por una gramática libre de contexto que no tiene símbolos inútiles.

Demostración : Sea $L = L(G)$ un lenguaje libre de contexto no vacío. Sea G_1 el resultado de usar el primer lema en G , y sea G_2 el resultado de aplicar la construcción del segundo lema a G_1 . Suponga que G_2 tiene un símbolo inútil X . Por el último lema, hay una derivación $S \xRightarrow{*} G_2 \alpha X \beta$. Ya que todos los símbolos de G_2 son símbolos de G_1 , del primer lema se sabe que $S \xRightarrow{*} G_1 \alpha X \beta \xRightarrow{*} G_1 w$ para algún string de terminales w . Por lo tanto, ningún símbolo en la derivación $\alpha X \beta \xRightarrow{*} G_1 w$ es eliminado por el segundo lema. Por lo tanto, X deriva un string de terminales en G_2 y no es inútil como se suponía.

□

Ejemplo 74 Considere la gramática

$$\begin{aligned} S &\rightarrow AB|a \\ A &\rightarrow a \end{aligned}$$

Por el primer lema, se nota que ningún string de terminales es derivable de B . Por lo tanto se elimina B y la producción $S \rightarrow AB$, con lo que queda

$$\begin{aligned} S &\rightarrow a \\ A &\rightarrow a \end{aligned}$$

Aplicándole el segundo lema, sólo S y a aparecen en formas sentenciales. Por lo tanto, $(\{S\}, \{a\}, \{S \rightarrow a\}, S)$ es una gramática equivalente sin símbolos inútiles.

Si se hubiera aplicado primero el segundo lema a la gramática original, se hubiera deducido que todos los símbolos aparecen en formas sentenciales. Aplicando luego el primer lema, se hubiese obtenido la segunda gramática, que aún tiene un símbolo inútil, A .

□

Se verá ahora cómo eliminar producciones de la forma $A \rightarrow \varepsilon$, llamadas producciones vacías (ε -productions). Es claro que si $\varepsilon \in L(G)$, no es posible eliminar todas las producciones vacías de G , pero si $\varepsilon \notin L(G)$, esto es posible. El método consiste en determinar, para cada variable A , si es posible que $A \xRightarrow{*} \varepsilon$, en cuyo caso se dice que A es anulable. Es posible reemplazar cada producción $B \rightarrow X_1 X_2 \dots X_n$ por todas las producciones que se forman al eliminar algún subconjunto de aquellos X_i 's que son anulables, pero sin incluir $B \rightarrow \varepsilon$, aún cuando todos los X_i sean anulables.

Teorema 19 Si $L = L(G)$ para alguna gramática libre de contexto $G = (V, T, P, S)$, entonces $L - \varepsilon$ es $L(G')$ para alguna gramática libre de contexto, G' , sin símbolos inútiles ni producciones vacías.

Demostración : Es posible determinar los símbolos anulables de G con el siguiente algoritmo. Si $A \rightarrow \varepsilon$ es una producción, entonces A es anulable. Si $B \rightarrow \alpha$ es una producción y todos los símbolos de α son anulables, entonces B es anulable. Este proceso se repite hasta que ningún otro símbolo anulable pueda ser encontrado.

El conjunto de producciones P' se construye como sigue. Si $A \rightarrow X_1 X_2 \dots X_n$ está en P , agregue a P' todas las producciones $A \rightarrow \alpha_1 \alpha_2 \dots \alpha_n$, donde

- si X_i no es anulable, entonces $\alpha_i = X_i$
- si X_i es anulable, entonces α_i es X_i o ε
- no todos los α_i 's son ε

Sea $G'' = (V, T, P, S)$. Se mostrará que para todo $A \in V$ y $w \in T^*$, $A \xRightarrow{*}_{G''} w$ ssi $w \notin \varepsilon$ y $A \xRightarrow{*}_G w$.

Sea $A \xRightarrow{*}_G w$ y $w \notin \varepsilon$. Se prueba, por inducción en i , que $A \xRightarrow{*}_{G'} w$. La base, $i = 1$, es trivial, pues $A \rightarrow w$ debe ser una producción en P . Dado que $w \notin \varepsilon$, también es una producción en P' . Para la inducción, sea $i > 1$. Entonces $A \xRightarrow{*}_G X_1 X_2 \dots X_n \xRightarrow{i-1}_G w$. Sea $w = w_1 w_2 \dots w_n$, tal que para cada j , $X_j \xRightarrow{*}_G w_j$ en menos de i pasos. Si $w_j \neq \varepsilon$ y X_j es una variable, entonces por la hipótesis de inducción se tiene $X_j \xRightarrow{*}_{G''} w_j$. Si $w_j = \varepsilon$ entonces X_j es anulable. Por lo tanto, $A \rightarrow \beta_1 \beta_2 \dots \beta_n$ es una producción en P' , con $\beta_j = X_j$ si $w_j \neq \varepsilon$ y $\beta_j = \varepsilon$ si $w_j = \varepsilon$. Como $w \neq \varepsilon$ no todos los β_j son ε . Por lo tanto se tiene una derivación

$$A \Rightarrow \beta_1 \beta_2 \dots \beta_n \xRightarrow{*} w_1 \beta_2 \dots \beta_n \xRightarrow{*} w_1 w_2 \dots \beta_n \xRightarrow{*} \dots \xRightarrow{*} w_1 w_2 \dots w_n = w$$

en G'' . Es decir, $A \xRightarrow{*}_{G''} w$.

Suponga ahora que $A \xRightarrow{*}_{G''} w$. Con toda seguridad $w \neq \varepsilon$ ya que G'' no tiene producciones vacías. Se muestra por inducción en i que $A \xRightarrow{*}_G w$. Para la base, $i = 1$, observe que $A \rightarrow w$ está en P' . Debe haber una

producción $A \rightarrow \alpha$ en P tal que al eliminar algunos símbolos anulables desde α , se obtiene w . Por lo tanto hay una derivación $A \xRightarrow{*}_G \alpha \xRightarrow{*}_G w$ en que $\alpha \xRightarrow{*}_G w$ envuelve el derivar ε de los símbolos anulables en α necesarios de eliminar para obtener w . Para la inducción, sea $i > 1$. Entonces $A \xRightarrow{*}_{G''} X_1 X_2 \dots X_n \xRightarrow{i-1}_{G''} w$. Debe haber alguna producción $A \rightarrow \beta$ en P , tal que $X_1 X_2 \dots X_n$ se logre al eliminar algunos símbolos anulables de β . Por lo tanto, $A \xRightarrow{*}_G X_1 X_2 \dots X_n$. Sea $w = w_1 w_2 \dots w_n$, tal que para todo j , $X_j \xRightarrow{*}_{G''} w_j$ en menos de i pasos. Por la hipótesis de inducción, $X_j \xRightarrow{*}_G w_j$ si X_j es una variable. Si X_j es un terminal, entonces $w_j = X_j$ y $X_j \xRightarrow{*}_G w_j$ se cumple trivialmente. Por lo tanto $A \xRightarrow{*}_G w$.

El último paso es aplicar el teorema anterior a G'' para obtener G' sin símbolos inútiles. Ya que las construcciones de los lemas no introducen producciones nuevas, G' no tiene símbolos inútiles ni producciones vacías. Además, $S \xRightarrow{*}_{G'} w$ si y sólo si $w \neq \varepsilon$ y $S \xRightarrow{*}_G w$. Esto es, $L(G') = L(G) = L(G) - \{\varepsilon\}$.

□

De aquí en adelante se asumirá que las gramáticas no tienen símbolos inútiles. Ahora se prestará atención a las producciones de la forma $A \rightarrow B$ cuyo lado derecho consiste sólo de una variable. Estas producciones son llamadas *producciones unitarias* (*unit productions*). Todas las otras producciones, incluyendo aquellas de la forma $A \rightarrow a$, o producciones vacías, son llamadas *producciones no unitarias* (*non unit*).

Teorema 20 *Todo lenguaje libre de contexto no vacío y sin ε es definido por una gramática sin símbolos inútiles, producciones vacías y producciones unitarias.*

Demostración: Sea L un lenguaje libre de contexto sin ε y $L = L(G)$ para alguna gramática $G = (V, T, P, S)$. Por el teorema anterior se puede asumir que G no tiene producciones vacías. Se construye un nuevo conjunto de producciones P' , incluyendo primero todas las producciones no unitarias de P . Luego, si $A \xRightarrow{*}_G B$, con $A, B \in V$, se agrega a P' todas las producciones de la forma $A \rightarrow \alpha$, en que $B \rightarrow \alpha$ es una producción no unitaria en P .

Observe que es fácil saber si $A \xRightarrow{*}_G B$, ya que G no tiene producciones vacías y si

$$A \xRightarrow{*}_G B_1 \xRightarrow{*}_G B_2 \xRightarrow{*}_G \dots \xRightarrow{*}_G B_M \xRightarrow{*}_G B$$

y alguna variable aparece dos veces en la secuencia, se puede encontrar una secuencia más corta de producciones unitarias que resulten en $A \xRightarrow{*}_G B$. Por lo tanto es suficiente considerar sólo aquellas secuencias de producciones unitarias que no repiten variables de G .

Suponga ahora que $w \in L(G)$ y considere una derivación por la izquierda para w en G .

$$S \Rightarrow \alpha_0 \xRightarrow{*}_G \alpha_1 \xRightarrow{*}_G \dots \xRightarrow{*}_G \alpha_N = w$$

Si, para $0 \leq i < N$, $\alpha_i \xRightarrow{*}_G \alpha_{i+1}$ por una producción no unitaria, entonces $\alpha_i \xRightarrow{*}_G \alpha_{i+1}$. O bien si $\alpha_i \xRightarrow{*}_G \alpha_{i+1}$ por una producción unitaria, pero $\alpha_{i-1} \xRightarrow{*}_G \alpha_i$ por una no unitaria, o $i = 0$, y además $\alpha_{i+1} \xRightarrow{*}_G \alpha_{i+2} \xRightarrow{*}_G \dots \xRightarrow{*}_G \alpha_j$, todas por producciones unitarias con $\alpha_j \xRightarrow{*}_G \alpha_{j+1}$ por una no unitaria; entonces $\alpha_{i+1} \alpha_{i+2} \dots \alpha_j$ todos tienen el mismo largo y ya que la derivación es por la izquierda, el símbolo reemplazado en cada una de ellas está en la misma posición. Pero entonces $\alpha_i \xRightarrow{*}_G \alpha_{j+1}$ por una de las producciones en $P' - P$. Por lo tanto, $L(G') = L(G)$. Para terminar la demostración, basta notar que G' no tiene producciones unitarias ni vacías. Si se usan los lemas anteriores para eliminar los símbolos inútiles no se agregan producciones, por lo tanto se obtiene una gramática como la pedida.

□

5.7 Formas Normales

En esta sección se verán dos formas normales para gramáticas libres de contexto. Se verá que para toda gramática libre de contexto existe una gramática equivalente con restricciones en la forma de sus producciones.

Teorema 21 (Forma Normal de Chomsky) *Todo lenguaje libre de contexto sin ε es generado por una gramática en que todas las producciones son de la forma $A \rightarrow BC$ o $A \rightarrow a$, en que A , B y C son variables y a es un terminal.*

Demostración : Sea G una gramática libre de contexto que genera un lenguaje que no contiene ε . Por el teorema anterior es posible encontrar una gramática equivalente, $G_1 = (V, T, P, S)$, tal que P no tiene producciones unitarias ni vacías. Por lo tanto, si una producción tiene un único símbolo a la derecha, ese símbolo es un terminal, y por lo tanto la producción está en una forma aceptable.

Considere una producción en P , de la forma $A \rightarrow X_1 X_2 \dots X_N$, con $N \geq 2$. Si X_i es un terminal a , se introduce una nueva variable C_a y una producción $C_a \rightarrow a$ que está en una de las formas permitidas. Luego se reemplaza X_i por C_a en la producción original. Sea V' el nuevo conjunto de variables y P' el nuevo conjunto de producciones. Considere la gramática $G_2 = (V', T, P', S)$, que no está aún en la forma normal de Chomsky. Si $\alpha \xrightarrow{*}_{G_1} \beta$, entonces $\alpha \xrightarrow{*}_{G_2} \beta$. Por lo tanto $L(G_1) \subseteq L(G_2)$. Se muestra, por inducción en el número de pasos de la derivación, que si $A \xrightarrow{*}_{G_2} w$, para $A \in V$ y w en T^* , entonces $A \xrightarrow{*}_{G_1} w$. El resultado es trivial para derivaciones de un paso. Supóngase que se cumple para derivaciones de k pasos. Sea $A \xrightarrow{*}_{G_2} w$ una derivación de $k+1$ pasos. El primer paso debe ser de la forma $A \rightarrow B_1 B_2 \dots B_N$, con $N \geq 2$. Se puede escribir $w = w_1 w_2 \dots w_N$, en que $B_i \xrightarrow{*}_{G_2} w_i$, con $1 \leq i \leq N$.

Si B_i es C_{a_i} , para algún terminal a_i , entonces w_i debe ser a_i . Por la construcción de P' , hay una producción $A \rightarrow X_1 X_2 \dots X_m$ de P , con $X_i = B_i$ si B_i está en V y con $X_i = a_i$ si B_i está en $V' - V$. Para los $B_i \in V$, se sabe que la derivación $B_i \xrightarrow{*}_{G_1} w_i$ toma no más de k pasos, luego, por la hipótesis de inducción, $X_i \xrightarrow{*}_{G_1} w_i$. Por lo tanto $A \xrightarrow{*}_{G_1} w$.

Se ha probado el resultado intermedio de que cualquier lenguaje libre de contexto puede ser generado por una gramática en que cada producción tiene la forma $A \rightarrow a$ o la forma $A \rightarrow B_1 B_2 \dots B_M$, ($M \geq 2$), en que A , B_1 , B_2 , $\dots$, B_M son variables y a es un terminal.

Considere una gramática de ese tipo, $G_2 = (V', T, P', S)$. Se modifica G_2 agregando algunos símbolos adicionales a V' y reemplazando algunas producciones de P' . Por cada producción de la forma $A \rightarrow B_1 B_2 \dots B_M$ en P' , con $M \geq 3$, se crean nuevas variables D_1 , D_2 , $\dots$, D_{M-2} y se reemplaza $A \rightarrow B_1 B_2 \dots B_M$ por el conjunto

$$A \rightarrow B_1 D_1, D_1 \rightarrow B_2 D_2, \dots, D_{M-2} \rightarrow B_{M-1} B_M$$

Sea V'' el nuevo conjunto de variables y P'' el nuevo conjunto de producciones. Sea $G_3 = (V'', T, P'', S)$. La gramática G_3 está en la forma normal de Chomsky. Es claro que si $A \xrightarrow{*}_{G_2} \beta$, entonces $A \xrightarrow{*}_{G_3} \beta$ y entonces $L(G_2) \subseteq L(G_3)$. Pero también se cumple que $L(G_3) \subseteq L(G_2)$, como puede demostrarse en esencialmente la misma forma en que se mostró que $L(G_2) \subseteq L(G_1)$.

□

Ejemplo 75 Considere la gramática $(\{S, A, B\}, \{a, b\}, P, S)$ con las producciones

$$\begin{aligned} S &\rightarrow bA|aB \\ A &\rightarrow bAA|aS|a \\ B &\rightarrow aBB|bS|b \end{aligned}$$

Las únicas producciones que ya están en la forma correcta son: $A \rightarrow a$ y $B \rightarrow b$. Luego, primero se transforma a la gramática

$$\begin{aligned} S &\rightarrow C_b A | C_a B \\ A &\rightarrow C_b A A | C_a S | a \\ B &\rightarrow C_a B B | C_b S | b \\ C_a &\rightarrow a \\ C_b &\rightarrow b \end{aligned}$$

En la segunda etapa se reemplaza por la gramática

$$\begin{aligned}
S &\rightarrow C_b A | C_a B \\
S &\rightarrow C_b D_1 | C_a S | a \\
B &\rightarrow C_a D_2 | C_b S | b \\
C_a &\rightarrow a \\
C_b &\rightarrow b \\
D_1 &\rightarrow AA \\
D_2 &\rightarrow BB
\end{aligned}$$

que está en la forma normal de Chomsky.

□

Ahora se verá otra forma normal que utiliza producciones cuyo lado derecho comienza con un terminal seguido, posiblemente, por variables. Primero se presentan dos lemas que dicen que es posible modificar las producciones de una gramática en ciertas formas, sin alterar el lenguaje que genera.

Lema 5 *Se define una producción-A como una producción que tiene la variable A en su lado izquierdo. Sea $G = (V, T, P, S)$ una gramática libre de contexto. Sea $A \rightarrow \alpha_1 B \alpha_2$ una producción en P y sean $B \rightarrow \beta_1 | \beta_2 | \dots | \beta_N$ todas las producciones-B de P . Sea $G_1 = (V, T, P_1, S)$, obtenida al eliminar la producción $A \rightarrow \alpha_1 B \alpha_2$ de P y agregando las producciones $A \rightarrow \alpha_1 \beta_1 \alpha_2 | \alpha_1 \beta_2 \alpha_2 | \dots | \alpha_1 \beta_N \alpha_2$. Entonces $L(G) = L(G_1)$.*

Demostración : Es claro que $L(G_1) \subseteq L(G)$, ya que si $A \rightarrow \alpha_1 \beta_i \alpha_2$ es usada en alguna derivación en G_1 , entonces $A \xrightarrow{G} \alpha_1 B \alpha_2 \xrightarrow{G} \alpha_1 \beta_i \alpha_2$ puede usarse en G . Para ver que $L(G) \subseteq L(G_1)$ basta notar que $A \rightarrow \alpha_1 B \alpha_2$ es la única producción de G que no está en G_1 . Sin embargo, si $A \rightarrow \alpha_1 B \alpha_2$ es usada en alguna derivación en G , la variable B debe ser reescrita posteriormente usando alguna de las producciones $B \rightarrow \beta_i$, ya que ellas son todas las producciones-B en P . Estos dos pasos pueden entonces reemplazarse por el paso $A \xrightarrow{G_1} \alpha_1 \beta_i \alpha_2$.

□

Lema 6 *Sea $G = (V, T, P, S)$ una gramática libre de contexto. Sean $A \rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_N$ el conjunto de producciones-A en que A es el símbolo de más a la izquierda en el lado derecho de la producción. Sean $A \rightarrow \beta_1 | \beta_2 | \dots | \beta_N$ las restantes producciones-A de P . Sea $G_1 = (V', T, P_1, S)$ la gramática formada al agregar la variable B a V ($V' = V \cup B$) y al reemplazar todas las producciones-A por las siguientes producciones*

$$\begin{aligned}
A &\rightarrow \beta_i & B &\rightarrow \alpha_i \\
A &\rightarrow \beta_i B & B &\rightarrow \alpha_i B \quad (1 \leq i \leq N)
\end{aligned}$$

Entonces $L(G_1) = L(G)$.

Demostración : En una derivación por la izquierda, una secuencia de producciones de la forma $A \rightarrow A\alpha_i$ debe eventualmente terminar con una de la forma $A \rightarrow \beta_j$. La secuencia de pasos en G ,

$$A \Rightarrow A\alpha_{i_1} \Rightarrow A\alpha_{i_2}\alpha_{i_1} \Rightarrow \dots \Rightarrow A\alpha_{i_l}\alpha_{i_{l-1}}\dots\alpha_{i_1} \Rightarrow \beta_j\alpha_{i_p}\alpha_{i_{p-1}}\dots\alpha_{i_1}$$

puede reemplazarse por la secuencia en G_1

$$\begin{aligned}
A &\Rightarrow \beta_j B \Rightarrow \beta_j \alpha_{i_p} B \Rightarrow \beta_j \alpha_{i_p} \alpha_{i_{p-1}} B \Rightarrow \dots \Rightarrow \\
&\beta_j \alpha_{i_p} \alpha_{i_{p-1}} \dots \alpha_{i_2} \Rightarrow \beta_j \alpha_{i_p} \alpha_{i_{p-1}} \dots \alpha_{i_1}
\end{aligned}$$

□

La transformación inversa también puede hacerse. Por lo tanto, $L(G) = L(G_1)$. La Figura 5.9 ilustra esta transformación usando árboles de derivación. Se ve que una cadena de A's extendiéndose hacia la izquierda en G se reemplaza por una de B's que se extiende hacia la derecha en G_1 .

Teorema 22 (Forma Normal de Greibach) *Todo lenguaje libre de contexto L , sin ε , puede ser generado por una gramática libre de contexto en que cada producción es de la forma $A \rightarrow a\alpha$, en que A es una variable, a es un terminal y α es un string (posiblemente vacío) de variables.*

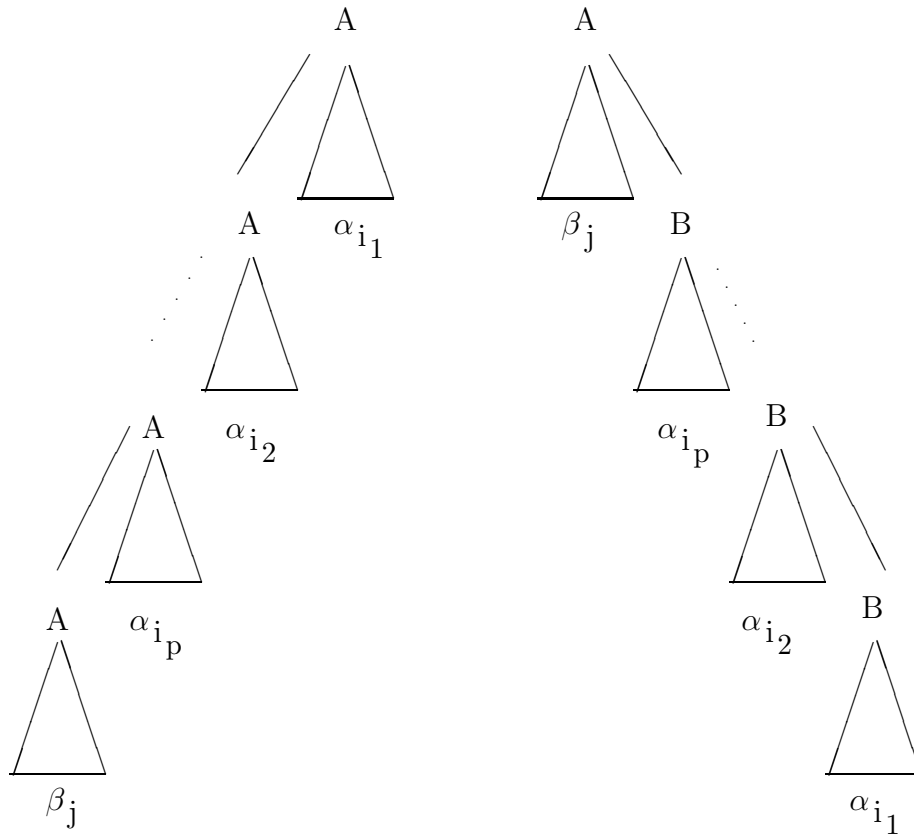


Figure 5.9: Transformación haciendo uso de árboles de derivación

Demostración : Sea $G = (V, T, P, S)$ una gramática libre de contexto en la forma normal de Chomsky, que genera L . Suponga que $V = \{A_1, A_2, \dots, A_M\}$. El primer paso en la construcción es modificar las producciones, de manera que si $A_i \rightarrow A_j \gamma$ es una producción, entonces $j > i$. Comenzando de A_1 y procediendo hacia A_M se asume que las producciones han sido modificadas de forma que, para $1 \leq i < k$, $A_i \rightarrow A_j \gamma$ es una producción sólo si $j > i$. Luego se modifican las producciones- A_k .

Si $A_k \rightarrow A_j \gamma$ es una producción, con $j < k$, se genera un nuevo conjunto de producciones sustituyendo por A_j el lado derecho de cada producción- A_j , de acuerdo al primero de los lemas previos. Repitiendo el proceso $k - 1$ veces a lo más, se obtienen producciones de la forma $A_k \rightarrow A_l \gamma$, con $l \geq k$. Las producciones con $l = k$ son entonces reemplazadas de acuerdo al segundo de esos lemas, introduciendo una nueva variable B_k . El algoritmo es el que sigue

```

(1)      for  $k := 1$  to  $M$  do begin
(2)          for  $j := 1$  to  $k - 1$  do
(3)              for cada produccion de la forma  $A_k \rightarrow A_j \alpha$  do begin
(4)                  for todas las producciones  $A_j \rightarrow \beta$  do
(5)                      agregue la produccion  $A_k \rightarrow \beta \alpha$ ;
(6)                      elimine  $A_k \rightarrow A_j \alpha$ 
(7)              end
(8)              for cada produccion de la forma  $A_k \rightarrow A_k \alpha$  do begin
(9)                  agregue producciones de la forma  $B_k \rightarrow \alpha$  y  $B_k \rightarrow \alpha B_k$ ;
(10)                 elimine  $A_k \rightarrow A_k \alpha$ 
(11)             end
(12)         end
(13)     end
(14)     for cada produccion de la forma  $A_k \rightarrow \beta$ 
(15)         en que  $\beta$  no empieza con  $A_k$  do
(16)             agregue la produccion  $A_k \rightarrow \beta B_k$ 
(17)         end

```

Repitiendo el proceso para cada variable original, se tienen sólo producciones de las formas

$$\begin{array}{ll}
 A_i \rightarrow A_j \gamma & j > i \\
 A_i \rightarrow a \gamma & a \in T \\
 B_i \rightarrow \gamma & \gamma \in (V \cup \{B_1, B_2, \dots, B_{i-1}\})^*
 \end{array}$$

Note que el símbolo de más a la izquierda en el lado derecho de alguna producción para A_M debe ser un terminal, ya que A_M es la variable con número mayor. El símbolo de más a la izquierda en el lado derecho de una producción para A_{M-1} debe ser A_M o un símbolo terminal. Cuando sea A_M , se puede generar nuevas producciones al reemplazar A_M por el lado derecho de las producciones para A_M , de acuerdo al primero de los lemas. Estas producciones deben tener lados derechos que comiencen con un símbolo terminal. Se procede entonces con las producciones para $A_{M-2}, \dots, A_2, A_1$, hasta que el lado derecho de cada producción, para algún A_i , comienza con un símbolo terminal.

Por último, se examinan las producciones para las variables nuevas $B_1 B_2 \dots B_M$. Ya que se comenzó con una gramática en la forma normal de Chomsky es fácil probar, por inducción en el número de aplicaciones de los lemas, que el lado derecho de cada producción- A_i , $1 \leq i \leq M$, comienza con un terminal o $A_j A_k$, para algún j y k . Por lo tanto α en línea (7) del algoritmo anterior nunca es vacío o comienza con algún B_j , es decir las producciones- B_i no pueden comenzar con otro B_j . Por lo tanto todas las producciones- B_i tienen lados derechos que comienzan con terminales o A_i 's; otra aplicación del primer lema para cada producción B_i completa la construcción.

□

Ejemplo 76 Se convertirá a la forma normal de Greibach la gramática $G = (\{A_1, A_2, A_3\}, \{a, b\}, P, A_1)$, en que P consiste de:

$$\begin{array}{ll}
 A_1 & \rightarrow A_2 A_3 \\
 A_2 & \rightarrow A_3 A_1 | b \\
 A_3 & \rightarrow A_1 A_2 | a
 \end{array}$$

Paso 1: ya que el lado derecho de las producciones para A_1 y A_2 comienzan con terminales o variables de número más alto, se comienza con la producción $A_3 \rightarrow A_1 A_2$. En lugar de A_1 se usa $A_2 A_3$, ya que $A_1 \rightarrow A_2 A_3$ es la única producción para A_1 . El resultado es

$$\begin{array}{ll}
 A_1 & \rightarrow A_2 A_3 \\
 A_2 & \rightarrow A_3 A_1 | b \\
 A_3 & \rightarrow A_2 A_3 A_2 | a
 \end{array}$$

Como el lado derecho de la producción $A_3 \rightarrow A_2 A_3 A_2$ empieza con una variable de menor número, se sustituye A_2 (su primera ocurrencia) tanto por $A_3 A_1$ como por b . El resultado es

$$\begin{aligned}
A_1 &\rightarrow A_2 A_3 \\
A_2 &\rightarrow A_3 A_1 | b \\
A_3 &\rightarrow A_3 A_1 A_3 A_2 | b A_3 A_2 | a
\end{aligned}$$

Se aplica ahora el segundo lema a las producciones- A_3 , con B_3 , una nueva variable. El resultado es

$$\begin{aligned}
A_1 &\rightarrow A_2 A_3 \\
A_2 &\rightarrow A_3 A_1 | b \\
A_3 &\rightarrow b A_3 A_2 B_3 | a B_3 | b A_3 A_2 | a \\
B_3 &\rightarrow A_1 A_3 A_2 | A_1 A_3 A_2 B_3
\end{aligned}$$

Paso 2: Ahora, todas las producciones- A_3 tienen un lado derecho que comienza con un terminal. Ellos se usan para reemplazar A_3 en $A_2 \rightarrow A_3 A_1$ y entonces las producciones para A_2 se usan para reemplazar A_2 en la producción $A_1 \rightarrow A_2 A_3$. El resultado es

$$\begin{aligned}
A_3 &\rightarrow b A_3 A_2 B_3 | a B_3 | b A_3 A_2 | a \\
A_2 &\rightarrow b A_3 A_2 B_3 A_1 | a B_3 A_1 | b A_3 A_2 A_1 | a A_1 | b \\
A_1 &\rightarrow b A_3 A_2 B_3 A_1 A_3 | a B_3 A_1 A_3 | b A_3 A_2 A_1 A_3 | a A_1 A_3 | b A_3 \\
B_3 &\rightarrow A_1 A_3 A_2 | A_1 A_3 A_2 B_3
\end{aligned}$$

Paso 3: Las dos producciones- B_3 se convierten a la forma adecuada, resultando 10 producciones. Se reemplaza el lado derecho de las 5 producciones- A_1 por la ocurrencia de A_1 como primer símbolo del lado derecho de las producciones- B_3 . El resultado es

$$\begin{aligned}
A_3 &\rightarrow b A_3 A_2 B_3 | a B_3 | b A_3 A_2 | a \\
A_2 &\rightarrow b A_3 A_2 B_3 A_1 | a B_3 A_1 | b A_3 A_2 A_1 | a A_1 | b \\
A_1 &\rightarrow b A_3 A_2 B_3 A_1 A_3 | a B_3 A_1 A_3 | b A_3 A_2 A_1 A_3 | a A_1 A_3 | b A_3 \\
B_3 &\rightarrow b A_3 A_2 B_3 A_1 A_3 A_3 A_2 | a B_3 A_1 A_3 A_3 A_2 | b A_3 A_2 A_1 A_3 A_3 A_2 | a A_1 A_3 A_3 A_2 | b A_3 A_3 A_2 \\
&\quad | b A_3 A_2 B_3 A_1 A_3 A_3 A_2 B_3 | a B_3 A_1 A_3 A_3 A_2 B_3 | b A_3 A_2 A_1 A_3 A_3 A_2 B_3 | a A_1 A_3 A_3 A_2 B_3 \\
&\quad | b A_3 A_3 A_2 B_3
\end{aligned}$$

una gramática en la forma normal de Greibach, que es equivalente a la original.

□

5.8 Equivalencia entre LLC y Autómatas Apiladores

En esta sección se probará el resultado fundamental que la clase de lenguajes regulares aceptados por los autómatas apiladores es precisamente la clase de los lenguajes libres de contexto.

Primero se verá que los lenguajes aceptados por un AA por estado final son exactamente los lenguajes aceptados por un AA por stack vacío. Luego se muestra que los lenguajes aceptados por stack vacío son exactamente los lenguajes libres de contexto.

Teorema 23 Si L es $L(M_2)$ para algún AA M_2 , entonces L es $N(M_1)$ para algún AA, M_1 .

Demostración : En resumen, se quiere que M_1 simule a M_2 , con la opción para M_1 de vaciar su stack cada vez que M_2 entre a un estado final. Se usa un estado q_e de M_1 para vaciar el stack y se usa un marcador del fondo del stack X_0 de M_1 , para que M_1 no acepte un string en forma accidental si M_2 vacía su stack en un estado no final. Sea $M_2 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un AA tal que $L = L(M_2)$. Sea

$$M_1 = (Q \cup \{q_e, q'_0\}, \Sigma, \Gamma \cup \{X_0\}, \delta', q'_0, X_0, \emptyset)$$

con δ' definida por

1. $\delta'(q'_0, \varepsilon, X_0) = \{(q_0, Z_0 X_0)\}$
2. $\delta'(q, a, Z)$ incluye los elementos de $\delta(q, a, Z)$, $\forall q \in Q$, $a \in \Sigma \cup \{\varepsilon\}$, $Z \in \Gamma$

3. $\forall q \in F$ y $Z \in \Gamma \cup \{X_0\}$, $\delta'(q, \varepsilon, Z)$ contiene (q_e, ε)
4. $\forall Z \in \Gamma \cup \{X_0\}$, $\delta'(q_e, \varepsilon, Z)$ contiene (q_e, ε)

La regla (1) hace que M_1 entre la descripción instantánea inicial de M_2 , excepto que M_1 tendrá su propio marcador, X_0 , al fondo del stack. La regla (2) le permite a M_1 simular las movidas de M_2 . Si M_2 entra alguna vez a un estado final, las reglas (3) y (4) le permiten a M_1 la elección de entrar al estado q_e y vaciar su stack (por lo tanto, aceptando el input) o de continuar simulando a M_2 . Se debe notar que M_2 podría vaciar su stack para algún string X que no está en $L(M_2)$. Por esta razón M_1 tiene una marca propia al fondo del stack. Si no M_1 , simulando a M_2 , podría vaciar su stack y aceptar X cuando no debiera.

Sea $X \in L(M_2)$. Entonces $(q_0, X, Z_0) \vdash^* (q, \varepsilon, \gamma)$ para algún $q \in F$. Considere M_1 con el string X . Por regla (1)

$$(q'_0, X, X_0) \vdash (q_0, X, Z_0 X_0)$$

por regla (2), todas las movidas de M_2 son legales en M_1 , por lo tanto

$$(q_0, X, Z_0) \vdash^* (q, \varepsilon, \gamma)$$

Si un AA puede hacer una secuencia de movidas desde una descripción instantánea dada, también puede hacer la misma secuencia de movidas desde cualquier DI obtenida de la primera insertando un string de símbolos del stack bajo el contenido original. Por lo tanto

$$(q'_0, X, X_0) \vdash (q_0, X, Z_0 X_0) \vdash (q, \varepsilon, \gamma X_0)$$

Por las reglas (3) y (4), porque $q \in F$,

$$(q, \varepsilon, \gamma X_0) \vdash^* (q_e, \varepsilon, \varepsilon)$$

Por lo tanto,

$$(q'_0, X, X_0) \vdash^* (q_e, \varepsilon, \varepsilon)$$

y así, M_1 acepta X por stack vacío, es decir, $X \in N(M_1)$.

En el otro sentido, si M_1 acepta X por stack vacío, es fácil mostrar que la secuencia de movidas debe ser una movida por regla (1), luego una secuencia por regla (2) en que M_1 simula la aceptación de X por M_2 , seguido del vaciamiento del stack de M_1 usando reglas (3) y (4). Por lo tanto X debe estar en $L(M_2)$.

□

Teorema 24 Si L es $N(M_1)$ para algún AA M_1 , entonces L es $L(M_2)$ para algún AA, M_2 .

Demostración : Ahora se quiere que M_2 simule a M_1 y pueda detectar cuando M_1 vacía su stack. La máquina M_2 entra a un estado final cuando y sólo cuando esto sucede. Sea $M_1 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$ un AA tal que $L = N(M_1)$. Sea

$$M_2 = (Q \cup \{q'_0, q_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta', q'_0, X_0, \{q_f\})$$

en que δ' se define como sigue

1. $\delta'(q'_0, \varepsilon, X_0) = \{(q_0, Z_0 X_0)\}$
2. $\forall q \in Q, a \in \Sigma \cup \{\varepsilon\}$ y $Z \in \Gamma$: $\delta'(q, a, Z) = \delta(q, a, Z)$
3. $\forall q \in Q, \delta'(q, \varepsilon, X_0)$ contiene (q_f, ε)

La regla (1) hace que M_2 entre la DI inicial de M_1 , excepto que M_2 tendrá su propio marcador X_0 , bajo los símbolos que M_1 tendría en su stack. La regla (2) permite que M_2 simule M_1 . Si alguna vez M_1 vaciara su stack completamente, entonces M_2 , al simular a M_1 , vaciará su stack excepto por el símbolo X_0 puesto al fondo. La regla (3) hace entonces que M_2 , al aparecer X_0 , entre a su estado final, aceptando el string. La prueba de que $L(M_2) = N(M_1)$ es similar a la del teorema anterior.

□

Teorema 25 *Si L es un lenguaje libre de contexto, existe un AA, M , tal que $L = N(M)$.*

Demostración : Se asume que $\varepsilon \notin L(G)$. La construcción es muy similar cuando $\varepsilon \in L(G)$. Sea $G = (V, T, P, S)$ una gramática libre de contexto en la forma normal de Greibach que genere L . Sea

$$M = (\{q\}, T, V, \delta, q, S, \emptyset)$$

en que $\delta(q, a, A)$ contiene (q, γ) si y sólo si $A \rightarrow a\gamma$ es una producción en P .

El AA, M , simula derivaciones por la izquierda en G . Como G está en la forma normal de Greibach, cada forma sentencial en una derivación por la izquierda consiste de un string de terminales X , seguido de un string de variables α . M almacena el sufijo α de la forma sentencial en su stack después de procesar el prefijo X .

Formalmente, se muestra que

$$S \xRightarrow{*} X\alpha \quad \text{por una derivación por la izquierda}$$

si y sólo si

$$(q, X, S) \vdash_M^* (q, \varepsilon, \alpha)$$

Primero, suponga que $(q, X, S) \vdash_M^i (q, \varepsilon, \alpha)$; se muestra, por inducción en i , que $S \xRightarrow{*} X\alpha$. La base, $i = 0$, es trivial ya que $X = \varepsilon$ y $\alpha = S$. Para la inducción se asume que $i \geq 1$ y sea $X = Ya$. Considérese el penúltimo paso:

$$(q, Ya, S) \vdash^{i-1} (q, a, \beta) \vdash (q, \varepsilon, \alpha)$$

si se remueve a desde el final del string de entrada en las primeras i DI's de la secuencia, se descubre que

$$(q, Y, S) \vdash^{i-1} (q, \varepsilon, \beta)$$

ya que a no puede afectar las movidas de M hasta que es realmente eliminado del input. Por la hipótesis de inducción, $S \xRightarrow{*} Y\beta$. La movida $(q, a, \beta) \vdash (q, \varepsilon, \alpha)$ implica que $\beta = A\gamma$ para algún $A \in V$, $A \rightarrow a\eta$ es una producción de G y $\alpha = \eta\gamma$. Por lo tanto

$$S \xRightarrow{*} Y\beta \Rightarrow Ya\eta\gamma = X\alpha$$

Ahora supóngase que $S \xRightarrow{i} X\alpha$ por una derivación por la izquierda. Se muestra, por inducción en i , que $(q, X, S) \vdash^i (q, \varepsilon, \alpha)$. La base, $i = 0$, es trivial nuevamente. Sea $i \geq 1$ y suponga que

$$S \xRightarrow{i-1} YA\gamma \Rightarrow Ya\eta\gamma$$

en que $X = Ya$ y $\alpha = \eta\gamma$. Por la hipótesis de inducción

$$(q, Y, S) \vdash^* (q, \varepsilon, A\gamma)$$

y por lo tanto $(q, Ya, S) \vdash^* (q, a, A\gamma)$. Ahora, como $A \rightarrow a\eta$ es una producción, se deduce que $\delta(q, a, A)$ contiene (q, η) . Por lo tanto

$$(q, X, S) \vdash^* (q, a, A\gamma) \vdash (q, \varepsilon, \alpha)$$

Esto concluye la demostración del teorema. Basta notar que si $\alpha = \varepsilon$, $S \xRightarrow{*} X$ si y sólo si $(q, X, S) \vdash^* (q, \varepsilon, \varepsilon)$. Esto es, $X \in L(G)$ ssi $X \in N(M)$.

□

Teorema 26 Si L es $N(M)$ para algún AA, M , entonces L es un lenguaje libre de contexto.

Demostración : Sea M el AA $Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset$. Sea $G = (V, \Sigma, P, S)$ una gramática libre de contexto en que V es un conjunto de objetos de la forma $[q, A, p]$, en que q y $p \in Q$ y $A \in \Gamma$, además de un nuevo símbolo S . P es el conjunto de producciones

1. $S \rightarrow [q_0, Z_0, q] \quad \forall q \in Q$
2. $[q, A, q_{M+1}] \rightarrow a [q_1, B_1, q_2] [q_2, B_2, q_3] \dots [q_M, B_M, q_{M+1}]$ para cada $q, q_1, q_2, \dots, q_{M+1} \in Q$, cada $a \in \Sigma \cup \{\varepsilon\}$ y $A, B_1, B_2, \dots, B_M \in \Gamma$ tales que $\delta(q, a, A)$ contiene $(q_1, B_1 B_2 \dots B_M)$. Si $M = 0$, la producción es $[q, A, q_1] \rightarrow a$.

Las variables y producciones de G se han definido de forma que una derivación por la izquierda de X en G es una simulación del AA, M , en input X . En particular las variables que aparecen en cualquier paso de una derivación por la izquierda en G , corresponden a los símbolos en el stack de M al momento en que M ha visto tanto del input como lo generado por la gramática. Puesto de otra forma, la intención es que $[q, A, p]$ derive X si y sólo si X hace que M elimine una A de su stack usando una secuencia de movidas que comienzan en el estado q y terminan en el estado p .

Para mostrar que $L(G) = N(M)$, se prueba por inducción en el número de pasos en una derivación de G o número de movidas de M , que

$$[q, A, p] \xrightarrow{G}^* X \text{ ssi } (q, X, A) \vdash_M^* (p, \varepsilon, \varepsilon)$$

Primero se muestra por inducción en i , que si $(q, X, A) \vdash^i (p, \varepsilon, \varepsilon)$ entonces $[q, A, p] \xrightarrow{*} X$. Si $i = 1$ entonces $\delta(q, X, A)$ debe contener (p, ε) . Aquí X es ε o un símbolo simple. Por lo tanto $[q, A, p] \rightarrow X$ es una producción de G . Si $i > 1$, sea $X = aY$ y

$$(q, aY, A) \vdash (q_1, Y, B_1 B_2 \dots B_N) \vdash^{i-1} (p, \varepsilon, \varepsilon)$$

el string Y puede escribirse $Y = Y_1 Y_2 \dots Y_N$ en que Y_j tiene el efecto de hacer pop de B_j desde el stack (posiblemente después de muchas movidas). Esto es, sea Y_1 el prefijo de Y al fin del cual el stack por primera vez llega a tener $N - 1$ símbolos. Sea Y_2 el substring de Y que sigue a Y_1 , tal que al final de Y_2 por primera vez el stack tiene $N - 2$ símbolos, y así sucesivamente.

Nótese que B_1 no es necesariamente el n -ésimo símbolo en el stack durante el tiempo en que Y_1 está siendo leído por M ; B_1 puede ser cambiado si está al tope del stack y ser reemplazado por uno o más símbolos. Sin embargo, ninguno de $B_2, B_3, \dots, B_N$ están nunca al tope mientras Y_1 está siendo leído, por lo tanto no pueden ser cambiados ni influenciar las movidas. En general B_j permanece sin cambios en el stack mientras $Y_1, \dots, Y_{j-1}$ es leído.

Existen estados $q_2, q_3, \dots, q_{N+1} = p$ tales que

$$(q_j, Y_j, B_j) \vdash^* (q_{j+1}, \varepsilon, \varepsilon)$$

en menos de i movidas de M (q_j es el estado al que se entra cuando por primera vez el stack tiene $n - j + 1$ símbolos). Por la hipótesis de inducción

$$[q_j, B_j, q_{j+1}] \xrightarrow{*} Y_j \quad (1 \leq j \leq N)$$

De la primera movida: $(q, aY, A) \vdash (q_1, Y, B_1 B_2 \dots B_N)$ se sabe que

$$[q, A, p] \Rightarrow a [q_1, B_1, q_2] [q_2, B_2, q_3] \dots [q_N, B_N, q_{N+1}]$$

y por lo tanto

$$[q, A, p] \xrightarrow{*} aY_1 Y_2 \dots Y_N = aY = X$$

Supóngase ahora que $[q, A, p] \xRightarrow{i} X$, se muestra, por inducción en i , que $(q, X, A) \vdash^*(p, \varepsilon, \varepsilon)$. La base, $i = 1$, es inmediata ya que $[q, A, p] \rightarrow X$ debe ser una producción de G y por lo tanto $\delta(q, X, A)$ debe contener (p, ε) . Nótese que X es ε o está en Σ .

Para la inducción suponga

$$[q, A, p] \Rightarrow a [q_1, B_1, q_2] \dots [q_N, B_N, q_{N+1}] \xRightarrow{i-1} X$$

en que $q_{N+1} = p$. Se puede escribir $X = aX_1X_2 \dots X_N$ en que $[q_j, B_j, q_{j+1}] \xRightarrow{*} X_j$, $(1 \leq j \leq N)$, y con cada derivación en menos de i -pasos. Por la hipótesis de inducción

$$(q_j, X_j, B_j) \vdash^*(q_{j+1}, \varepsilon, \varepsilon) \quad (1 \leq j \leq n)$$

Si se inserta $B_{j+1} \dots B_N$ al fondo del stack en la secuencia anterior de DI's, se ve que

$$(q, X, A) \vdash (q_1, X_1X_2 \dots X_N, B_1B_2 \dots B_N)$$

es una movida de M y, por lo tanto, usando la anterior para $j = 1, 2, \dots, N$, se tiene que

$$(q, X, A) \vdash^*(p, \varepsilon, \varepsilon)$$

La demostración concluye con la observación de que si $q = q_0$ y $A = Z_0$, se ha probado que

$$[q_0, Z_0, p] \xRightarrow{*} X \text{ ssi } (q_0, X, Z_0) \vdash^*(p, \varepsilon, \varepsilon)$$

Esta observación, junto con la primera regla para construir G , dicen que

$$S \xRightarrow{*} X \text{ ssi } (q_0, X, Z_0) \vdash^*(p, \varepsilon, \varepsilon)$$

para algún estado p . Es decir, $X \in L(G)$ ssi $X \in N(M)$

□

Ejemplo 77 Sea $M = (\{q_0, q_1\}, \{0, 1\}, \{X, Z_0\}, \delta, q_0, Z_0, \emptyset)$ con δ dada por

$$\begin{aligned} \delta(q_0, 0, Z_0) &= \{(q_0, XZ_0)\} \\ \delta(q_0, 0, X) &= \{(q_0, XX)\} \\ \delta(q_0, 1, X) &= \{(q_1, \varepsilon)\} \end{aligned}$$

$$\begin{aligned} \delta(q_1, 1, X) &= \{(q_1, \varepsilon)\} \\ \delta(q_1, \varepsilon, X) &= \{(q_1, \varepsilon)\} \\ \delta(q_1, \varepsilon, Z_0) &= \{(q_1, \varepsilon)\} \end{aligned}$$

Para construir una gramática libre de contexto, $G = (V, T, P, S)$, que genere $N(M)$, sea

$$\begin{aligned} V &= \{S, [q_0, X, q_0], [q_0, X, q_1], [q_1, X, q_0], [q_1, X, q_1], \\ &\quad [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1]\} \end{aligned}$$

y $\Gamma = \{0, 1\}$.

Para construir el conjunto de producciones con facilidad, es útil darse cuenta que algunas variables pueden no aparecer en derivaciones que comienzan con S . Se puede ahorrar algo de esfuerzo si se comienza con las producciones- S y se agregan aquellas para variables que aparecen en el lado derecho de alguna ya incluida en P .

Las producciones para S son

$$S \rightarrow [q_0, Z_0, q_0]$$

$$S \rightarrow [q_0, Z_0, q_1]$$

se agregan producciones para $[q_0, Z_0, q_0]$:

$$[q_0, Z_0, q_0] \rightarrow 0 [q_0, X, q_0] [q_0, Z_0, q_0]$$

$$[q_0, Z_0, q_0] \rightarrow 0 [q_0, X, q_1] [q_1, Z_0, q_0]$$

requeridas por $\delta(q_0, 0, Z_0) = \{(q_0, X Z_0)\}$ son

$$[q_0, Z_0, q_1] \rightarrow 0 [q_0, X, q_0] [q_0, Z_0, q_1]$$

$$[q_0, Z_0, q_1] \rightarrow 0 [q_0, X, q_1] [q_1, Z_0, q_1]$$

también requeridas por $\delta(q_0, 0, Z_0) = \{(q_0, X Z_0)\}$.

Las producciones para las variables y las movidas relevantes de M , son:

$$[q_0, X, q_0] \rightarrow 0 [q_0, X, q_0] [q_0, X, q_0]$$

$$\rightarrow 0 [q_0, X, q_1] [q_1, X, q_0]$$

$$[q_0, X, q_1] \rightarrow 0 [q_0, X, q_0] [q_0, X, q_1]$$

$$\rightarrow 0 [q_0, X, q_1] [q_1, X, q_1]$$

porque

$$\delta(q_0, 0, X) = \{(q_0, X X)\}$$

$$[q_0, X, q_1] \rightarrow 1$$

$$\text{porque } \delta(q_0, 1, X) = \{(q_1, \varepsilon)\}$$

$$[q_1, Z_0, q_1] \rightarrow \varepsilon$$

$$\text{porque } \delta(q_1, \varepsilon, Z_0) = \{(q_1, \varepsilon)\}$$

$$[q_1, X, q_1] \rightarrow \varepsilon$$

$$\text{porque } \delta(q_1, \varepsilon, X) = \{(q_1, \varepsilon)\}$$

$$[q_1, X, q_1] \rightarrow 1$$

$$\text{porque } \delta(q_1, 1, X) = \{(q_1, \varepsilon)\}$$

Debe notarse que no hay producciones para las restantes variables ($[q_1, X, q_0]$ y $[q_1, Z_0, q_0]$). Como todas las producciones para $[q_0, X, q_0]$ y $[q_0, Z_0, q_0]$ tienen $[q_1, X, q_0]$ o $[q_1, Z_0, q_0]$ a la derecha, ningún string de terminales puede derivarse de ellas. Eliminando las producciones en que aparecen esas variables, se llega a

$$S \rightarrow [q_0, Z_0, q_1]$$

$$[(q_1, Z_0, q_1) \rightarrow \varepsilon]$$

$$[q_0, Z_0, q_1] \rightarrow 0 [q_0, X, q_1] [q_1, Z_0, q_1]$$

$$[(q_1, X, q_1) \rightarrow \varepsilon]$$

$$[q_0, X, q_1] \rightarrow 0 [q_0, X, q_1] [q_1, X, q_1]$$

$$[(q_1, X, q_1) \rightarrow 1]$$

$$[q_0, X, q_1] \rightarrow 1$$

□

En resumen de esta sección, se puede concluir que las siguientes tres aserciones son equivalentes:

- L es un lenguaje libre de contexto.
- L es $N(M_1)$ para algún AA, M_1 .
- L es $L(M_2)$ para algún AA, M_2 .

5.9 Ambigüedad Inherente

Es muy fácil exhibir gramáticas libres de contexto que son ambiguas. Por ejemplo,

$$S \rightarrow A|B$$

$$A \rightarrow a$$

$$B \rightarrow a$$

en que el único string del lenguaje (a) tiene dos árboles de derivación.

Lo que no es tan simple es encontrar un lenguaje libre de contexto para el cual toda gramática sea ambigua. En esta sección se muestra que en realidad hay lenguajes libres de contexto que son inherentemente ambiguos. Se mostrará que el lenguaje

$$L = \{a^N b^N c^M d^M / N \geq 1, M \geq 1\} \cup \{a^N b^M c^M d^N / N \geq 1, M \geq 1\}$$

es inherentemente ambiguo, probando que el conjunto infinito de strings de la forma $a^N b^N c^N d^N$ ($N \geq 1$), deben tener dos derivaciones por la izquierda distintas.

Lema 7 Sean (N_i, M_i) , $1 \leq i \leq r$, pares de conjuntos de enteros (los conjuntos pueden ser finitos o infinitos). Sea

$$S_i = \{(n, m) / n \in N_i \text{ y } m \in M_i\}$$

y sea

$$S = S_1 \cup S_2 \cup \dots \cup S_r$$

Si cada par de enteros (n, m) está en S , para todo n y m , con $n \neq m$; entonces (n, n) está en S para todos, excepto un conjunto finito de n .

Demostración : Asuma que para todo n y m , con $n \neq m$, cada par $(n, m) \in S$, y que hay un número infinito de n tales que $(n, m) \notin S$. Sea $\bar{\delta}$ el conjunto de todos los n tales que (n, n) no está en S . Se contruye una secuencia de conjuntos $\bar{\delta}_r, \bar{\delta}_{r-1}, \dots, \bar{\delta}_1$, tales que

$$\bar{\delta} \supseteq \bar{\delta}_r \supseteq \bar{\delta}_{r-1} \dots \supseteq \bar{\delta}_1$$

Cada $\bar{\delta}_i$ será infinito y para cada n, m en $\bar{\delta}_i$, (n, m) no está en

$$S_i \cup S_{i+1} \cup \dots \cup S_r$$

Para $n \in \bar{\delta}$, o n no está en N_r o n no está en M_r ; sino, (n, m) estaría en S_r y por lo tanto en S . Hay, por lo tanto, un subconjunto infinito de $\bar{\delta}$, llamado $\bar{\delta}_r$, tal que para todo $n \in \bar{\delta}_r$, $n \notin N_r$, o para todo $n \in \bar{\delta}_r$, $n \notin M_r$. También, para n y $m \in \bar{\delta}_r$, (n, m) no está en S_r .

Asuma que $\bar{\delta}_r, \bar{\delta}_{r-1}, \dots, \bar{\delta}_{i-1}$ ha sido construido para $i \leq r-1$; $\bar{\delta}_i$ se construye como sigue. Por cada $n \in \bar{\delta}_{i+1}$, n no está en N_i o n no está en M_i ; si no (n, n) habría estado en S_i y por lo tanto en S , una contradicción ya que $\bar{\delta}_{i+1} \subseteq \bar{\delta}$. Por lo tanto, ya sea un subconjunto infinito de $\bar{\delta}_{i+1}$ no está en N_i o un subconjunto infinito de $\bar{\delta}_{i+1}$ no está en M_i . En cualquier caso, sea $\bar{\delta}_i$ ese conjunto infinito. Ahora, para todo n y m en $\bar{\delta}_i$, (n, m) no está en S_i y por lo tanto, no está en $S_i \cup S_{i+1} \cup \dots \cup S_r$.

Ya que $\bar{\delta}_1$ tiene un número infinito de elementos, existen n y m en $\bar{\delta}_1$, con $n \neq m$. Ahora, (n, m) no está en $S_1 \cup S_2 \cup \dots \cup S_r = S$, contradiciendo la hipótesis de que todo (n, m) , con $n \neq m$ está en S . Por lo tanto, (n, m) está en S para todos excepto un conjunto finito de n .

□

Lema 8 Sea G una gramática libre de contexto no ambigua. Entonces se puede construir efectivamente una gramática libre de contexto no ambigua, G' , equivalente a G , tal que G' no tiene símbolos inútiles, ni producciones unitarias, ni producciones vacías y en que para toda variable A , excepto posiblemente el símbolo inicial de G' , se tiene una derivación $A \xRightarrow{*}_{G'} X_1 A X_2$, en que X_1 y X_2 no son ambos ε .

Demostración : Las construcciones para remover símbolos inútiles no convierten una gramática no ambigua en una ambigua, ya que el conjunto de árboles de derivación no cambia. La construcción para remover producciones unitarias no puede introducir ambigüedades, ya que si se incluye la producción $A \rightarrow \alpha$, hay un único B , tal que $A \xRightarrow{*} B$ y $B \rightarrow \alpha$ es una producción, si no la gramática original era ambigua. Similarmente la construcción para remover producciones vacías, tampoco introduce ambigüedades.

Se asume por lo tanto, que G no tiene símbolos inútiles ni producciones unitarias ni producciones vacías. Suponga que para ningún X_1, X_2 , ambos no ε , $A \xRightarrow{*} X_1 A X_2$. Reemplace cada ocurrencia de A en el lado derecho de cada producción por todos los lados derechos de las producciones- A . Como no hay producciones unitarias ni producciones vacías ni símbolos inútiles, no puede haber una producción $A \rightarrow \alpha_1 A \alpha_2$, si no hay una derivación $A \xRightarrow{*} X_1 A X_2$ con X_1 y X_2 no ambos ε . El cambio descrito no altera el lenguaje generado, como se mostró en un lema anterior. Cada nueva producción viene de una única secuencia de producciones antiguas, si no G era ambigua. Por lo tanto la gramática resultante no era ambigua. A es ahora inútil y puede eliminarse. Después de remover la variables que violan la condición del lema de la manera descrita, la nueva gramática es equivalente a la original, es aún no ambigua y satisface el lema.

□

Teorema 27 *El lenguaje libre de contexto*

$$L = \{a^N b^N c^M d^M / N \geq 1 \text{ y } M \geq 1\} \cup \{a^N b^M c^M d^N / N \geq 1 \text{ y } M \geq 1\}$$

es inherentemente ambiguo.

Demostración : Asuma que hay una gramática no ambigua que genera L . Por el lema anterior, se puede construir una gramática no ambigua $G = (V, T, P, S)$, que genera L , que no tiene símbolos inútiles y en que por cada $A \in V - \{S\}$, $A \Rightarrow^* X_1 A X_2$ para algunos $X_1, X_2 \in T^*$, en que no son ambos ε .

Se hace notar que la gramática G debe tener las siguientes propiedades:

1. Si $A \Rightarrow^* X_1 A X_2$, entonces X_1 y X_2 consisten de un sólo tipo de símbolos (a, b, c o d); si no

$$S \Rightarrow^* w_1 A w_3 \Rightarrow^* w_1 X_1 X_1 A X_2 X_2 w_3 \Rightarrow^* w_1 X_1 X_1 w_2 X_2 X_2 w_3$$

para algunos w_1, w_2 y w_3 . El último string de terminales no pertenecería a L .

2. Si $A \Rightarrow^* X_1 A X_2$, entonces X_1 y X_2 tienen símbolos diferentes, si no en una derivación que usa A , se aumentaría el número de uno de los símbolos en una sentencia sin incrementar el número de ningún otro símbolo, generando sentencias que no están en L .
3. Si $A \Rightarrow^* X_1 A X_2$, entonces $|X_1|$ y $|X_2|$. Si no se podría formar strings que tienen más de un símbolo que de ningún otro.
4. Si $A \Rightarrow^* X_1 A X_2$ y $A \Rightarrow^* X_3 A X_4$, entonces X_1 y X_3 consisten de los mismos símbolos. También X_2 y X_4 . Si no, la propiedad (1) sería violada.
5. Si $A \Rightarrow^* X_1 A X_2$, entonces a.- X_1 consiste sólo de a 's y X_2 sólo de b 's o de d 's b.- X_1 consiste sólo de b 's y X_2 sólo de c 's c.- X_1 consiste sólo de c 's y X_2 sólo de d 's

En cualquiera de los otros casos es fácil derivar un string que no pertenece a L . Por lo tanto, las variables que no sean S pueden agruparse en 4 clases, C_{ab}, C_{ad}, C_{bc} y C_{cd} . C_{ab} es el conjunto de todas las $A \in V$, tales que $A \Rightarrow^* X_1 A X_2$, con $X_1 \in a^*$ y $X_2 \in b^*$. C_{ad}, C_{bc} y C_{cd} se definen en forma análoga.

6. Una derivación que contiene un símbolo en C_{ab} o C_{cd} no puede contener un símbolo en C_{ad} o C_{bc} y viceversa. Si no, sería posible incrementar el número de tres de los tipos de símbolos de una sentencia en L , sin importar el cuarto. En ese caso habría un string en L para el cual un símbolo aparecería menos veces que todos los otros.

Nótese que si una derivación contiene una variable en C_{ab} o C_{cd} , entonces el string terminal generado debe estar en $\{a^N b^N c^M d^M / N \geq 1 \text{ y } M \geq 1\}$. Porque supóngase que una variable $A \in C_{ab}$ aparece en una derivación de un string X que no está en ese conjunto. Entonces X debe ser de la forma $a^N b^M c^M d^N$, con $M \neq N$. Ya que $A \in C_{ab}$, es posible generar una sentencia $a^{N+p} b^{M+p} c^M d^N$, con $M \neq N$ para algún $p > 0$, la que no pertenece a L . Un argumento similar se cumple si $A \in C_{cd}$. Un razonamiento análogo implica que si una derivación contiene una variable en C_{ad} o C_{bc} , entonces la sentencia generada debe estar en $\{a^N b^M c^M d^N / N \geq 1 \text{ y } M \geq 1\}$.

Se divide G en dos gramáticas,

$$G_1 = (\{S\} \cup C_{ab} \cup C_{cd}, T, P_1, S)$$

y

$$G_2 = (\{S\} \cup C_{ad} \cup C_{bc}, T, P_2, S)$$

en que P_1 contiene todas las producciones de P con una variable de C_{ad} o C_{bc} ya sea en su lado izquierdo o derecho, y P_2 todas aquellas con una variable de C_{ad} o C_{bc} ya sea en su lado izquierdo o derecho. Además P_1 contiene todas las producciones en P , de la forma $S \rightarrow a^N b^N c^M d^M$, $N \neq M$; y P_2 todas aquellas de la forma $S \rightarrow a^N b^M c^M d^N$, $N \neq M$. Producciones de la forma $S \rightarrow a^N b^N c^N d^N$ no están ni en P_1 ni en P_2 . Ya que G genera

$$\{a^N b^N c^M d^M / N \geq 1 \text{ y } M \geq 1\} \cup \{a^N b^M c^M d^N / N \geq 1 \text{ y } M \geq 1\},$$

G_1 debe generar todas las sentencias en

$$\{a^N b^N c^M d^M / N \geq 1, M \geq 1 \text{ y } N \neq M\}$$

más, posiblemente, algunos strings en $a^N b^N c^N d^N / N \geq 1$, y G_2 debe generar todos los strings en

$$\{a^N b^M c^M d^N / N \geq 1, M \geq 1 \text{ y } N \neq M\}$$

más, posiblemente, algunos strings en $\{a^N b^N c^N d^N / N \geq 1\}$. Se muestra que esto no puede ser así a menos que G_1 y G_2 generen ambos todos, excepto un conjunto finito de strings en $\{a^N b^N c^N d^N / N \geq 1\}$. Por lo tanto todos, excepto un número finito de strings en $\{a^N b^N c^N d^N / N \geq 1\}$ son generados por G_1 y G_2 y tienen entonces dos derivaciones diferentes en G . Esto contradice la hipótesis de que G no era ambigua, como se quería.

Para ver que G_1 y G_2 generan todos, excepto un número finito, de strings en $\{a^N b^N c^N d^N / N \geq 1\}$, se numera las producciones de P_1 de la forma $S \rightarrow \alpha$, de 1 a r . Para $1 \leq i \leq r$, si $S \rightarrow \alpha$ es la i -ésima producción, sea N_i el conjunto de todos los N tales que

$$S_{G_1} \xrightarrow{*} \alpha \Rightarrow_{G_1} a^N b^N c^M d^M$$

para algún M , y sea M_i el conjunto de todos los M tales que

$$S_{G_1} \xrightarrow{*} \alpha \Rightarrow_{G_1} a^N b^N c^M d^M$$

para algún N . Es fácil probar que para cualquier $N \in N_i$ y $M \in M_i$

$$S_{G_1} \xrightarrow{*} \alpha \Rightarrow_{G_1} a^N b^N c^M d^M$$

(Recuerde que las variables de α están en C_{ab} o C_{cd}). De donde se concluye, por el lema inicial, que G_1 debe generar todas, excepto un número finito, las sentencias en $\{a^N b^N c^N d^N / N \geq 1\}$. Un argumento similar es aplicable a G_2 . (Ver en el libro).

□

Chapter 6

PROPIEDADES DE LOS LENGUAJES LIBRES DE CONTEXTO

Este capítulo es, respecto de los lenguajes libres de contexto, lo que el capítulo 4 es respecto de los lenguajes regulares. En primer lugar se verá un lema de bombeo para probar que ciertos lenguajes no son libres de contexto. Luego se considerarán algunas propiedades de clausura y, finalmente, se verán algunos algoritmos para responder ciertas preguntas sobre lenguajes libres de contexto.

6.1 Lema de Bombeo para Lenguajes Libres de Contexto

El lema de bombeo para conjuntos regulares establece que todo string suficientemente largo de un conjunto regular contiene un substring corto que se puede bombear. Es decir, al insertar tantas copias del substring como se desee, se obtiene siempre un string en el conjunto regular. El lema de bombeo para lenguajes libres de contexto establece que hay siempre dos substrings cortos que pueden ser repetidos, el mismo número de veces ambos, tanto como se desee.

Lema 9 *Sea L un lenguaje libre de contexto. Entonces, hay una constante N , que sólo depende de L , tal que si $Z \in L$ y $|Z| \geq N$, entonces es posible escribir $Z = uvwxy$ tal que*

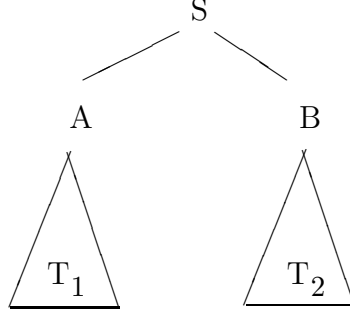
1. $|vx| \geq 1$
2. $|vwx| \leq N$
3. $\forall i \geq 0, uv^iwx^iy \in L$

Demostración : Sea G una gramática libre de contexto en la forma normal de Chomsky que genera $L - \{\varepsilon\}$. Obsérvese que si $Z \in L(G)$ y Z es largo, entonces cualquier árbol de derivación para Z debe contener un camino largo. Más precisamente, se muestra por inducción en i , que si el árbol de derivación de un string generado por una gramática en la forma normal de Chomsky no tiene caminos de largo mayor que i , entonces la palabra (string) es de longitud no mayor que 2^{i-1} . La base, $i = 1$, es trivial ya que el árbol debe tener la forma de la Figura 6.1.

Para la inducción, sea $i > 1$. Sea el árbol de derivación de la forma de la Figura 6.2

Si no hay caminos de largo mayor que $i - 1$ en los árboles T_1 y T_2 , entonces ellos generan strings de a lo sumo 2^{i-2} símbolos y, por lo tanto, el árbol completo genera strings de no más de 2^{i-1} símbolos.

Sean k las variables de G y sea $N = 2^k$. Si $Z \in L(G)$ y $|Z| \geq N$, como $|Z| > 2^{k-1}$, cualquier árbol de derivación para Z debe tener un camino de largo $k + 1$ al menos. Pero un camino de ese largo tiene al menos

Figure 6.1: Árbol de derivación para $i = 1$ Figure 6.2: Árbol de derivación para $i \geq 1$

$k + 2$ vértices, todos los cuales, excepto el último, son variables. Debe haber alguna variable que aparece dos veces en ese camino.

En realidad se puede precisar más. Alguna variable debe aparecer dos veces cerca del fin del camino. En particular, sea P un camino tan largo como el que más en el árbol. Debe haber dos vértices v_1 y v_2 en ese camino, que satisfacen las siguientes condiciones,

1. Los vértices v_1 y v_2 tienen la misma etiqueta, A .
2. El vértice v_1 está más cerca de la raíz que v_2 .
3. El camino entre v_1 y la hoja es de largo $k + 1$ a lo más.

Para ver que v_1 y v_2 existen, basta proceder hacia arriba por el camino P desde la hoja; de los primeros $k + 2$ vértices, sólo la hoja tiene un terminal como etiqueta. Los demás $k + 1$ no pueden tener todos etiquetas distintas.

El subárbol T_1 , con raíz v_1 , representa la derivación de un substring de largo 2^k a lo sumo. Esto es cierto pues P fue el camino más largo de todo el árbol. Sea Z_1 el rédito del árbol T_1 . Si T_2 es el subárbol con raíz en v_2 y Z_2 es su rédito, entonces se puede escribir Z_1 como $Z_3Z_2Z_4$. Además Z_3 y Z_4 no pueden ser ambos ε ya que la primera producción usada en la derivación de Z_1 es de la forma $A \rightarrow BC$ y el subárbol T_2 debe estar completamente dentro del árbol generado de B , o completamente dentro del generado de C .

Se sabe que

$$A \xRightarrow{*} Z_3AZ_4 \text{ y } A \xRightarrow{*}, \text{ con } |Z_3Z_2Z_4| \leq 2^k = N$$

Por lo tanto $A \xRightarrow{*} Z_3^iAZ_4^i, \xRightarrow{*} Z_3^iZ_2Z_4^i$ para todo $i \geq 0$. Claramente, el string Z puede ser escrito como $uZ_3Z_2Z_4y$ para algunos u e y . Si $Z_3 = v$, $Z_2 = w$ y $Z_4 = x$, el lema queda demostrado.

□

Este lema de bombeo puede utilizarse para probar que un número de lenguajes no son libres de contexto, utilizando un argumento con adversario similar al usado con el lema de bombeo para lenguajes regulares.

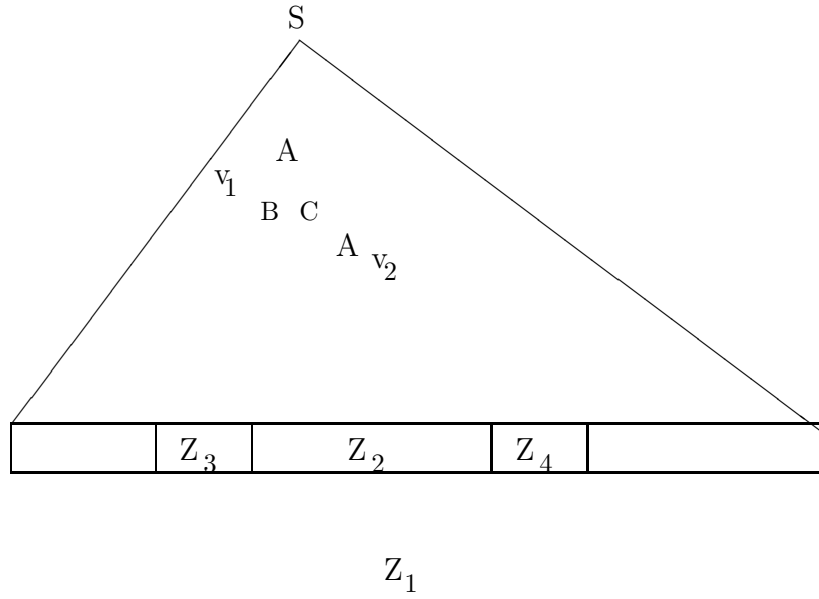


Figure 6.3: Derivación de un substring

Ejemplo 78 Considere el lenguaje $L_1 = \{a^i b^i c^i / i \geq 1\}$. Asuma que L es libre de contexto y sea N la constante del lema de bombeo. Considere el string $Z = a^N b^N c^N$. Por el lema, se puede escribir $Z = uvwxy$, con $|vx| \geq 1$ y $|vwx| \leq N$. Como $|vwx| \leq N$, no es posible que vx contenga a 's y c 's ya que hay $N + 1$ posiciones entre la última a y la primera c . Si v y x sólo contienen a 's, entonces uwy ($uv^i wx^i y$, con $i = 0$) tiene N b 's y N c 's, pero menos de N a 's, ya que $|vx| \geq 1$. Por lo tanto no es de la forma $a^j b^j c^j$, es decir, no pertenece a L_1 , contradiciendo el lema de bombeo. Los casos en que v y x sólo tienen b 's o c 's son similares.

Si vx tiene a 's y b 's, entonces uwy tiene más c 's que a 's o b 's y, por lo tanto, no está en L_1 , contradiciendo el lema de bombeo. Si vx contiene b 's y c 's, sucede algo similar.

En todos los casos posibles, se contradice el lema de bombeo, por lo tanto se concluye que L_1 no es un lenguaje libre de contexto.

□

Ejemplo 79 Sea $L_2 = \{a^i b^j c^i d^j / i \geq 1 \text{ y } j \geq 1\}$. Suponga que L_2 es un lenguaje libre de contexto y sea N la constante del lema de bombeo. Considere el string $Z = a^N b^N c^N d^N$. Por el lema, se puede escribir $Z = uvwxy$, con $|vx| \geq 1$ y $|vwx| \leq N$. Como $|vwx| \leq N$, vx puede tener a lo más dos símbolos diferentes, los que deben ser consecutivos (ab , bc , cd).

Si vx sólo tiene a 's, entonces uwy tiene menos a 's que c 's y no está en L_2 , contradiciendo el lema de bombeo. El mismo resultado se obtiene si vx contiene sólo b 's, sólo c 's o sólo d 's.

Si vx tiene a 's y b 's, entonces uwy tiene menos a 's que c 's. Una contradicción similar con el lema de bombeo ocurre si vx tiene b 's y c 's o c 's y d 's.

Ya que en todos los casos posibles se contradice el lema de bombeo, se concluye que L_2 no es un lenguaje libre de contexto.

□

Hay algunos lenguajes que no son libres de contexto, para los cuales el lema de bombeo no es suficiente. Por ejemplo

$$L_3 = \{a^i b^j c^k d^l / i = 0 \text{ ó } j = k = l\}$$

no es libre de contexto. Sin embargo, si se escoge $Z = b^j c^k d^l$ y se escribe $Z = uvwxy$, es siempre posible escoger u, v, w, x e y , tales que $uv^M wx^M y \in L_3, \forall M$. Por ejemplo, se escoge vw de manera que sólo tenga b 's. Si se escoge $Z = a^i b^j c^k d^l$, entonces v y x podrían tener sólo a 's, en cuyo caso $uv^M wx^M y \in L_3, \forall M$.

Se requiere una versión más poderosa del lema de bombeo que permita enfocar un número de posiciones en el string y luego bombearlas. Una extensión similar es simple para lenguajes regulares ya que en cualquier secuencia de $N + 1$ estados en un AFD de N estados, debe contener alguno dos veces; y el substring en el medio puede ser bombeado. El resultado para lenguajes libres de contexto es más difícil de obtener pero se puede mostrar. Se establece y prueba una versión simple de lo que se conoce como el lema de *Ogden*.

Lema 10 (Lema de Ogden) *Sea L un lenguaje libre de contexto. Entonces hay una constante N (que puede ser la misma que para el lema de bombeo), tal que si $Z \in L$ y se marcan N o más posiciones (símbolos) cualesquiera de Z como “distinguidas”, entonces se puede escribir $Z = uvwxy$, tal que*

1. vx tiene al menos una posición distinguida
2. vw tiene a lo más N posiciones distinguidas
3. $\forall i \geq 0; uv^i wx^i y \in L$

Demostración : Sea G una gramática en la forma normal de Chomsky que genera $L - \{\varepsilon\}$. Sean k las variables de G y sea $N = 2^{k+1}$. Se debe construir un camino P en el árbol, similar al de la prueba del lema de bombeo. Sin embargo, ya que estamos interesados sólo en las posiciones distinguidas, no interesarán todos los vértices, peor sólo los “puntos de quiebre” (branch points), que son vértices en que ambos hijos tienen descendientes distinguidos.

P se construye como sigue. La raíz pertenece a P . Si r es el último vértice incluido en P , se sigue como se indica a continuación. Si r tiene un hijo con descendientes distinguidos, ese hijo se agrega a P . Si r es una hoja, se termina el proceso. Si ambos hijos de r tienen descendientes distinguidos, r es un punto de quiebre y se agrega el hijo con el mayor número de descendientes distinguidos a P (en caso de empate, se escoge arbitrariamente).

Por lo tanto, cada punto de quiebre en P tiene al menos la mitad de descendientes distinguidos que el punto de quiebre anterior. Ya que hay al menos N posiciones distinguidas en Z , y todas son descendientes de la raíz, hay al menos $k + 1$ puntos de quiebre en P . Por lo tanto, entre los últimos $k + 1$ puntos de quiebre debe haber dos con igual etiqueta. Se escoge v_1 y v_2 como dichos puntos de quiebre y la demostración continúa exactamente como en el lema de bombeo.

□

Ejemplo 80 Sea $L_4 = \{a^i b^j c^k / i \neq j, j \neq k, i \neq k\}$. Asuma que L_4 es un lenguaje libre de contexto y sea N la constante del lema de Ogden y considere el string $Z = a^N b^{N+N!} c^{N+2N!}$. Sean las posiciones de las a 's distinguidas y sea $Z = uvwxy$, satisfaciendo las condiciones del lema de Ogden. Si v o x contienen símbolos diferentes, entonces $uv^2 wx^2 y \notin L_4$ ya que tendrá símbolos no en el orden correcto. Al menos uno de v y x debe tener a 's, ya que sólo las a 's han sido distinguidas. Por lo tanto si x está en b^* o c^* , v debe estar en a^+ . Si $x \in a^+$, entonces $v \in a^*$. Considere el caso en que $x \in b^*$, los demás son similares; entonces $v \in a^+$. Sea $p = |v|$. Entonces $1 \leq p \leq N$ y, por lo tanto, p divide $N!$, sea q tal que $pq = n!$. Entonces $z' = uv^{2q+1} wx^{2q+1} y$ debiera estar en L_4 . Pero $v^{2q+1} = a^{2pq+p} = a^{2N!+p}$. Como uvw tiene exactamente $(n - p)$ a 's, Z' tiene $(2N! + N)$ a 's; sin embargo como v y x no tienen c 's, Z' también tiene $(2N! + N)$ c 's y, por lo tanto, no está en L_4 . Una contradicción con el lema de Ogden. Una contradicción similar ocurre si x está en a^+ o c^* . Por lo tanto L_4 no es un lenguaje libre de contexto.

□

Debe notarse que el lema de bombeo es un caso especial del lema de Ogden en que todas las posiciones son distinguidas.

6.2 Propiedades de Clausura

En esta sección se consideran algunas operaciones que preservan los lenguajes libres de contexto. Las operaciones son útiles no sólo para construir o probar que ciertos lenguajes son libres de contexto, sino que para probar que algunos no lo son. Un lenguaje L puede probarse no libre de contexto construyendo, a partir de L , un lenguaje no libre de contexto, usando sólo operaciones que preserven los lenguajes libres de contexto.

Teorema 28 *Los lenguajes libres de contexto son cerrados bajo unión, concatenación y clausura de Kleene.*

Demostración : Sean L_1 y L_2 lenguajes libres de contexto generados por las gramáticas

$$G_1 = (V_1, T_1, P_1, S_1)$$

y

$$G_2 = (V_2, T_2, P_2, S_2)$$

respectivamente. Se asume que V_1 y V_2 son disjuntos y que S_3, S_4 y S_5 no están en $V_1 \cup V_2$.

Para $L_1 \cup L_2$ se construye la gramática $G_3 = (V_1 \cup V_2 \cup \{S_3\}, T_1 \cup T_2, P_3, S_3)$ en que P_3 es $P_1 \cup P_2$ más las producciones $S_3 \xrightarrow{*} S_1 \xrightarrow{*} w$ es también posible en G_3 ya que $P_1 \subseteq P_3$. En forma similar, todo string en L_2 tiene una derivación en G_3 que comienza con $S_3 \Rightarrow S_2$. Por lo tanto, $L_1 \cup L_2 \subseteq L(G_3)$. Ahora, sea $w \in L(G_3)$. Entonces la derivación $S_3 \xrightarrow{*} S_1 \xrightarrow{*} w$ o con $S_3 \xrightarrow{*} S_2 \xrightarrow{*} w$. En el primer caso, como V_1 y V_2 son disjuntos, sólo símbolos de G_1 aparecen en $S_1 \xrightarrow{*} w$. Como las únicas producciones de P_3 que usan sólo símbolos de G_1 son las de P_1 , se concluye que sólo producciones de P_1 son usadas en la derivación $S_1 \xrightarrow{*} w$. Por lo tanto, $S_1 \xrightarrow{*} w$ y, luego, $w \in L_1$. Análogamente, si la derivación comienza $S_3 \xrightarrow{*} S_2$, se concluye que $w \in L_2$. De aquí, $L_3 \subseteq L_1 \cup L_2$. Por lo tanto, $L(G_3) = L_1 \cup L_2$, como se deseaba.

Para la concatenación, sea $G_4 = (V_1 \cup V_2 \cup \{S_4\}, T_1 \cup T_2, P_4, S_4)$, en que P_4 es $P_1 \cup P_2$ más la producción $S_4 \rightarrow S_1 S_2$. La prueba de que $L(G_4) = L_1 L_2$ es similar a la anterior.

Para la clausura de Kleene, sea $G_5 = (V_1 \cup \{S_5\}, T_1, P_5, S_5)$, donde P_5 es P_1 más la producción $S_5 \rightarrow S_1 S_5 | \varepsilon$. La prueba de que $L(G_5) = L_1^*$ es también similar a las anteriores.

□

Teorema 29 *Los lenguajes libres de contexto son cerrados bajo sustitución por lenguajes libres de contexto.*

Demostración : Sea L un lenguaje libre de contexto, $L \subseteq \Sigma^*$, y por cada $a \in \Sigma$ sea $L_a = L(G_a)$. Asuma que las variables de G y de G_a son disjuntas. Construya una gramática G' de la siguiente forma. Las variables de G' son las de G y de las G_a 's. El símbolo inicial de G' es el símbolo inicial de G . Las producciones de G' son todas las producciones de las G_a 's junto a las producciones formadas tomando una producción $A \rightarrow \alpha$ de G y sustituyendo S_a , el símbolo inicial de G_a , por cada aparición de todo $a \in \Sigma$ en el lado derecho α .

□

Ejemplo 81 Sea L el conjunto de palabras con igual número de a 's y b 's y sean $L_a = \{0^N 1^N / N \geq 1\}$ y $L_b = \{w w^r / w \in (0+2)^*\}$.

Para G se puede escoger

$$S \rightarrow a S b S | b S a S | \varepsilon$$

Para G_a se toma

$$S_a \rightarrow 0 S_a 1 | 01$$

Para G_b se toma

$$S_b \rightarrow 0S_b0|2S_b2|\varepsilon$$

Para la sustitución $f(a) = L_a$ y $f_b = L_b$; entonces $f(L)$ es generado por la siguiente gramática

$$\begin{aligned} S &\rightarrow S_a S S_b S | S_b S S_a S | \varepsilon \\ S_a &\rightarrow 0S_a1|01 \\ S_b &\rightarrow 0S_b0|2S_b2|\varepsilon \end{aligned}$$

□

Debiera observarse que, ya sea que a, b, ab y a^* son lenguajes libres de contexto, la clausura de los lenguajes libres de contexto bajo sustitución por LLC, implica clausura bajo unión, concatenación y clausura de Kleene. La unión de L_a y L_b es simplemente la sustitución de L_a y L_b en $\{a, b\}$; similarmente, $L_a L_b$ y L_a^* son las sustituciones en $\{ab\}$ y a^* , respectivamente. Es decir, el primer teorema puede ser presentado como un corolario de este último.

Ya que un homomorfismo es un caso especial de una sustitución, se establece el siguiente corolario:

Corolario 1 *Los lenguajes libres de contexto son cerrados bajo homomorfismos.*

Teorema 30 *Los lenguajes libres de contexto son cerrados bajo el inverso de un homomorfismo.*

Demostración : Sea $h : \Sigma \rightarrow \Delta^*$ un homomorfismo y sea L un lenguaje libre de contexto. Sea $L = L(M)$ en que M es el AA $(Q, \Delta, \Gamma, \delta, q_0, Z_0, F)$. Se construye un AA, M' , que acepta $h^{-1}(L)$ como sigue. Dado un input a , M' genera $h(a)$ y simula a M en $h(a)$. Si M fuera un AF, todo lo que podría hacer en $h(a)$ sería cambiar estados y M' podría simularlo en una sola movida. Pero como M es un AA, puede hacer *pop* de muchos símbolos o (por ser no determinístico) hacer movidas que ponen un número arbitrario de símbolos en el stack. Es decir, M' no puede, necesariamente, simular las movidas de M en $h(a)$ con una (o cualquier número finito) de sus propias movidas.

Se da, entonces, a M' un buffer en que puede almacenar $h(a)$. M' puede entonces simular cualquier movida de M que desee, consumiendo un símbolo de $h(a)$ a la vez, como si fuera el input de M . Como el buffer es parte del control finito de M' , no se le puede permitir crecer en forma arbitraria. Para asegurar ésto, se permite que M' lea un símbolo del input sólo cuando el buffer está vacío. Es decir, el buffer siempre contiene un sufijo de $h(a)$ para algún a . M' acepta su input w si el buffer está vacío y M está en un estado final. Esto es, M ha aceptado $h(w)$. Es decir,

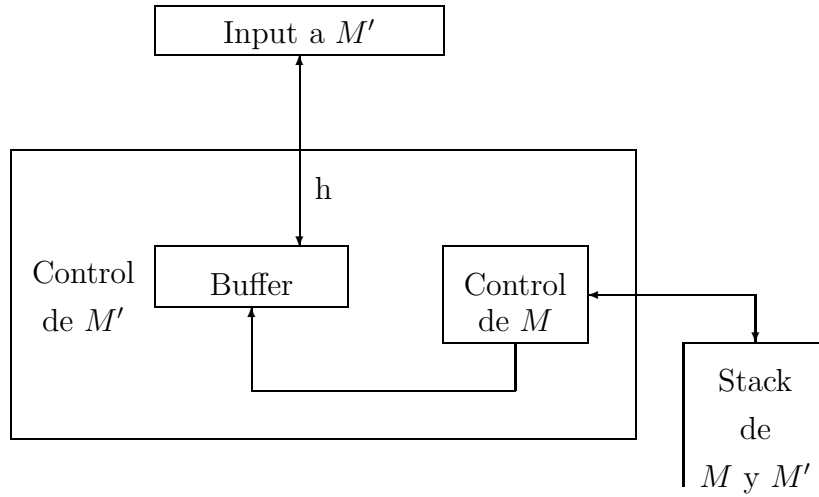
$$L(M') = \{w/h(w) \in L\} = h^{-1}(L(M))$$

Sea $M' = (Q', \Sigma, \Gamma, \delta', [q_0, \varepsilon], Z_0, F \times \{\varepsilon\})$ en que Q' consta de los pares $[q, x]$ tales que $q \in Q$ y x es un sufijo (no necesariamente propio) de $h(a)$ para algún $a \in \Sigma$. La función δ' se define como sigue:

1. $\delta'([q, x], \varepsilon, Y)$ contiene todos los $([p, x], \gamma)$ tales que $\delta(q, \varepsilon, Y)$ contiene (p, γ) . Simula las movidas- ε de M independientemente del contenido del buffer.
2. $\delta'([q, ax], \varepsilon, Y)$ contiene todos los $([p, x], \gamma)$ tales que $\delta(q, a, Y)$ contiene (p, γ) . Simula a M en input $a \in \Delta$, removiendo a del primer lugar del buffer.
3. $\delta'([q, \varepsilon], a, Y)$ contiene $([q, h(a)], Y)$ $\forall a \in \Sigma$ e $Y \in \Gamma$. Pone $h(a)$ en el buffer leyendo $a \in \Sigma$ desde el input de M' ; el estado y stack de M no cambian.

Para mostrar que $L(M') = h^{-1}(L(M))$ obsérvese primero que, por una aplicación de la regla (3) seguida por aplicaciones de las reglas (1) y (2), si $(q, h(a), \alpha) \vdash_M^* (p, \varepsilon, \beta)$, entonces

$$([q, \varepsilon], a, \alpha) \vdash_{M'}^* ([q, h(a)], \varepsilon, \alpha) \vdash_{M'}^* ([p, \varepsilon], \varepsilon, \beta)$$

Figure 6.4: Construcción de un AA que acepte $h^{-1}(L)$

Por lo tanto si M acepta $h(w)$, esto es,

$$(q_0, h(w), Z_0) \vdash_M^* (p, \varepsilon, \beta)$$

para $p \in F$ y $\beta \in \Gamma^*$, se concluye que

$$([q_0, \varepsilon], w, Z_0) \vdash_{M'}^* ([p, \varepsilon], \varepsilon, \beta)$$

es decir, M' acepta w . Por lo tanto $L(M') \supseteq h^{-1}(L(M))$.

Al revés, suponga que M' acepta $w = a_1 a_2 \dots a_N$. Como regla (3) sólo puede aplicarse con el buffer (segundo componente de Q') vacío, la secuencia de movidas de M' que conducen a aceptar w , puede escribirse como:

$$\begin{array}{lcl}
 ([q_0, \varepsilon], a_1 a_2 \dots a_N, Z_0) & \vdash_{M'}^* & ([p_1, \varepsilon], a_1 a_2 \dots a_N, \alpha_1) \\
 & \vdash_{M'} & ([p_1, h(a_1)], a_2 \dots a_N, \alpha_1) \\
 & \vdash_{M'}^* & ([p_2, \varepsilon], a_2 \dots a_N, \alpha_2) \\
 & \vdash_{M'} & ([p_2, h(a_2)], a_3 \dots a_N, \alpha_2) \\
 & \vdots & \\
 & \vdash_{M'}^* & ([p_{N-1}, \varepsilon], a_N, \alpha_N) \\
 & \vdash_{M'} & ([p_{N-1}, h(a_N)], \varepsilon, \alpha_N) \\
 & \vdash_{M'}^* & ([p_N, \varepsilon], \varepsilon, \alpha_{N+1})
 \end{array}$$

En que $p_N \in F$. Las transiciones de estados $[p_i, \varepsilon]$ a $[p_i, h(a_i)]$ son por regla (3); las demás, por reglas (1) y (2). Por lo tanto $(q_0, \varepsilon, Z_0) \vdash_M^* (p_1, \varepsilon, \alpha_1)$ y, para todo i ,

$$(p_i, h(a_i), \alpha_i) \vdash_M^* (p_{i+1}, \varepsilon, \alpha_{i+1})$$

lo que indica que

$$(q_0, h(a_1 a_2 \dots a_N), Z_0) \vdash_M^* (p_N, \varepsilon, \alpha_{N+1})$$

es decir, $h(a_1 a_2 \dots a_N) \in L(M)$. Luego, $L(M') \subseteq h^{-1}(L(M))$ y por lo tanto se concluye que $L(M') = h^{-1}(L(M))$.

□

Hay varias propiedades de clausura de los lenguajes regulares que los lenguajes libres de contexto no poseen. Notable es el caso de la intersección y de la complementación.

Teorema 31 *Los lenguajes libres de contexto no son cerrados bajo intersección.*

Demostración : Ya se mostró que $L_1 = \{a^i b^i c^i / i \geq 1\}$ no es un lenguaje libre de contexto. Se muestra que los siguientes lenguajes sí son libres de contexto.

$$L_2 = \{a^i b^j c^j / i \geq 1 \text{ y } j \geq 1\}$$

$$L_3 = \{a^i b^j c^j / i \geq 1 \text{ y } j \geq 1\}$$

Por ejemplo, las siguientes gramáticas los generan

$$\begin{array}{ll} S_2 \rightarrow AB & S_3 \rightarrow CD \\ A \rightarrow aAb|ab & C \rightarrow aC|a \\ B \rightarrow cB|c & D \rightarrow bDc|bc \end{array}$$

Sin embargo $L_1 = L_2 \cap L_3$ y entonces, si fueran cerrados bajo intersección, L_1 debería ser libre de contexto. Se concluye que los lenguajes libres de contexto no son cerrados bajo intersección.

□

Corolario 2 *Los lenguajes libres de contexto no son cerrados bajo complementación.*

Demostración : Ya que son cerrados bajo unión, si fueran cerrados bajo complementación serían, por la ley de De Morgan, $(L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}})$, cerrados bajo intersección.

□

Teorema 32 *Si L es un lenguaje libre de contexto y R es un conjunto regular, entonces $L \cap R$ es libre de contexto.*

Demostración : Sea $L = L(M)$ para un AA, $M = (Q_M, \Sigma, \Gamma, \delta_M, q_0, Z_0, F_M)$ y sea $R = L(A)$ para un AFD, $A = (Q_A, \Sigma, \delta_A, p_0, F_A)$. Se construye un AA M' para $L \cap R$ ejecutando M y A en paralelo. M' simula movidas de M en input ε sin cambiar el estado de A . Cuando M hace una movida en símbolo a , M' simula esa movida y también simula los cambios de estado de A en input a . M' acepta si y sólo si tanto A como M aceptan. Formalmente sea

$$M' = (Q_A \times Q_M, \Sigma, \Gamma, \delta, [p_0, q_0], Z_0, F_A \times F_M)$$

con δ definida por $\delta([p, q], a, X) \supseteq \{([p', q'], \gamma)\}$ ssi $\delta_A(p, a) = p'$ y $\delta_M(q, a, X) \supseteq \{(q', \gamma)\}$. Si $a = \varepsilon$, entonces $p' = p$.

Graficamente, la máquina se comporta como sigue

Una simple inducción en i muestra que

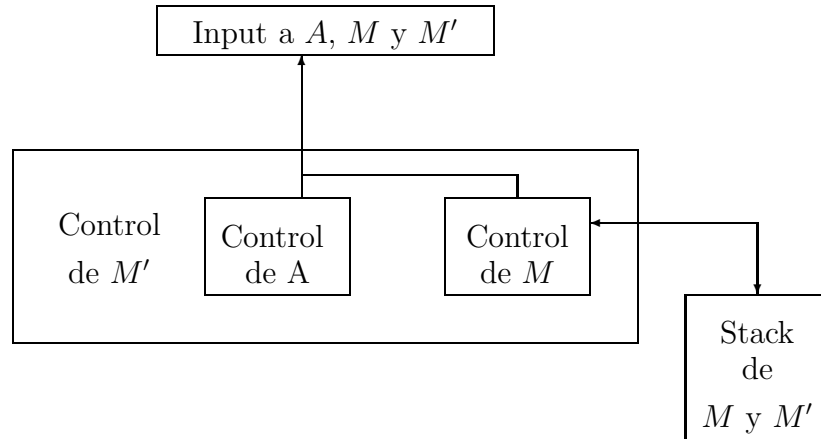
$$([p_0, q_0], w, Z_0)_{M'} \stackrel{i}{\vdash} ([p, q], \varepsilon, \gamma)$$

si y sólo si

$$(q_0, w, Z_0)_M \stackrel{i}{\vdash} (q, \varepsilon, \gamma) \quad \text{y} \quad \delta(p_0, w) = p$$

La base, $i = 0$, es trivial pues $p = p_0$ y $q = q_0$, $\gamma = Z_0$ y $w = \varepsilon$. Para la inducción, asuma que es verdad para $i - 1$, y sea

$$([p_0, q_0], xa, Z_0)_{M'} \stackrel{i-1}{\vdash} ([p', q'], a, \beta)_{M'} \stackrel{i}{\vdash} ([p, q], \varepsilon, \gamma)$$

Figure 6.5: Construcción de un AA para $L \cap R$

en que $w = xa$; con $a \in \Sigma \cup \{\varepsilon\}$. Por la hipótesis de inducción

$$\delta_A(p_0, x) = p' \quad \text{y} \quad (q_0, x, Z_0)_M \vdash^{i-1} (q', \varepsilon, \beta)$$

Por la definición de δ , el hecho de que $([p', q'], a, \beta)_M \vdash ([p, q], \varepsilon, \gamma)$ indica que $\delta_A(p', a) = p$ y $(q', a, \beta)_M \vdash (q, \varepsilon, \gamma)$. Por lo tanto $\delta_A(p_0, w) = p$, y

$$(q_0, w, Z_0)_M \vdash^i (q, \varepsilon, \gamma)$$

El converso, que $(q_0, w, Z_0)_M \vdash^i (q, \varepsilon, \gamma)$ y $\delta_A(p_0, w) = p$ implica $([p_0, q_0], w, Z_0)_M \vdash^i ([p, q], \varepsilon, \gamma)$ es similar.

□

Ejemplo 82 Sea $L = \{ww/w \in \{a, b\}^*\}$. Esto es, L consiste de todas las palabras cuyas primeras y segundas mitades son las mismas. Si L fuera libre de contexto, entonces $L_1 = L \cap a^+b^+a^+b^+$ debería serlo, ya que son cerrados bajo intersección con un lenguaje regular. Pero L_1 es claramente $\{a^ib^ja^ib^j/i \geq 1, j \geq 1\}$ casi idéntico a uno que ya se probó no era libre de contexto.

Sea h el homomorfismo $h(a) = h(c) = a$ y $h(b) = h(d) = b$. Entonces $h^{-1}(L_1)$ contiene strings de la forma $x_1x_2x_3x_4$ en que x_1 y x_3 tienen igual largo y pertenecen a $(a+c)^+$ y x_2 y x_4 tienen igual largo y están en $(b+d)^+$. Por lo tanto $h^{-1}(L_1) \cap a^*b^*c^*d^*$ es igual a $\{a^ib^jc^id^j/i \geq 1, j \geq 1\}$. Como este último no es libre de contexto, L tampoco lo es.

□

6.3 Algoritmos de Decisión

Hay varias preguntas sobre los lenguajes libres de contexto que se puede responder. Ellas incluyen determinar si un lenguaje dado es vacío, finito o infinito y si un string está en un lenguaje. Sin embargo, hay otras preguntas acerca de los lenguajes libres de contexto para los cuales no existe algoritmo que las responda. Entre éstas está el saber si dos gramáticas son equivalentes, si un lenguaje es cofinito, si el complemento de un lenguaje libre de contexto es también libre de contexto y si una cierta gramática es o no ambigua. En esta sección se verán algoritmos para algunas de las preguntas que tienen algoritmos.

Como en el caso de los lenguajes regulares, hay varias representaciones posibles para los lenguajes libres de contexto, es decir, gramáticas libres de contexto y autómatas apiladores que aceptan por stack vacío o

por estado final. Como las construcciones del capítulo 5 son todas efectivas, un algoritmo que usa una representación se puede hacer funcionar para cualquiera de las otras. En esta sección se usará la representación por gramáticas libres de contexto.

Teorema 33 *Existen algoritmos para determinar si un lenguaje libre de contexto es*

1. *vacío,*
2. *finito, o*
3. *infinito.*

Demostración : Ya se ha dado un algoritmo para probar si un lenguaje libre de contexto es vacío. Para una gramática $G = (V, T, P, S)$, el test del primer lema para remover símbolos inútiles determina si una variable genera algún string de terminales. Obviamente $L(G)$ es no vacío si y sólo si el símbolo inicial, S , genera algún string de terminales.

Para saber si $L(G)$ es finito, utilice el algoritmo del teorema correspondiente para construir una gramática $G' = (V', T, P', S)$ en la forma normal de Chomsky, que genera $L(G) - \{\varepsilon\}$. $L(G')$ es finito si y sólo si $L(G)$ es finito. Un test simple para la finitud de una gramática en forma normal de Chomsky sin símbolos inútiles, es construir un grafo dirigido con un vértice por variable y un arco de A a B , si hay una producción de la forma $A \rightarrow BC$ o $A \rightarrow CB$ para algún C . El lenguaje generado es finito si y sólo si este grafo no tiene ciclos. (Ver texto).

□

Otra pregunta que se puede responder es: dada una gramática libre de contexto, $G = (V, T, P, S)$ y un string x en T^* , ¿está $x \in L(G)$? Aquí se presentará un algoritmo simple de orden $\vartheta(|x|^3)$ conocido como el algoritmo de Cocke-Younger-Kasami o CYK. Dado x de longitud $N \geq 1$ y una gramática G , que se asume está en la forma normal de Chomsky, se determina para cada i , para cada j y para cada variable A , si $A \Rightarrow^* X_{ij}$, en que X_{ij} es el substring de x que tiene largo j y comienza en la posición i .

El proceso es por inducción en j . Para $j = 1$, $A \Rightarrow^* X_{ij}$ si y sólo si $A \rightarrow X_{ij}$ es una producción, ya que X_{ij} tiene largo 1. Para valores mayores de j , si $j > 1$, entonces $A \Rightarrow^* X_{ij}$ si y sólo si hay alguna producción $A \rightarrow BC$ y algún k , $1 \leq k \leq j$, tal que B deriva los primeros k símbolos de X_{ij} y C deriva los últimos $j - k$ símbolos de X_{ij} . Esto es, $B \Rightarrow^* X_{i, k}$ y $C \Rightarrow^* X_{i+k, j-k}$. Ya que tanto k como $j - k$ son menores que j , en el proceso ya se sabe si estas dos últimas derivaciones son posibles. Por lo tanto, se puede determinar si $A \Rightarrow^* X_{ij}$. Cuando $j = N$, se puede determinar si $S \Rightarrow^* X_{1N} = x$. Es decir, se puede saber si $x \in L(G)$.

Para definir el algoritmo de CYK en forma precisa, sea V_{ij} el conjunto de variables A , tales que $A \Rightarrow^* X_{ij}$. Se puede asumir que $1 \leq i \leq N - j + 1$ ya que no hay string más largo de $N - i + 1$ que comienza en posición i .

```

(1)      FOR  $i := 1$  TO  $N$  DO
(2)           $V_{i1} := \{A/A \rightarrow a \in P \text{ y } a \text{ es el } i\text{-ésimo símbolo de } x\}$ 
(3)      FOR  $j := 2$  TO  $N$  DO
(4)          FOR  $i := 1$  TO  $N - j + 1$  DO BEGIN
(5)               $V_{ij} := \emptyset$ ;
(6)              FOR  $k := 1$  TO  $j - 1$  DO
(7)                   $V_{ij} := V_{ij} \cup \{A/A \rightarrow BC \in P, B \in V_{ik} \text{ y } C \in V_{i+k, j-k}\}$ 
          END

```

El loop de líneas (1) y (2) inicializan para $j = 1$. Como la gramática es fija, línea (2) toma tiempo constante. Por lo tanto el ciclo toma $\vartheta(N)$ pasos.

Los loops anidados de líneas (3) y (4) hacen que las líneas (5) a (7) se ejecuten a lo más N^2 veces.

La línea (5) toma tiempo constante cada vez, es decir, en total se ejecuta $\vartheta(N^2)$ veces. El loop de la línea (6) hace que la línea (7) se ejecute $\vartheta(N^3)$ veces. Es decir el algoritmo es $\vartheta(N^3)$.

Ejemplo 83 Considere la gramática libre de contexto que se indica a continuación:

$$S \rightarrow AB|BC$$

$$A \rightarrow BA|a$$

$$B \rightarrow CC|b$$

$$C \rightarrow AB|a$$

y el string *baaba*.

		b	a	a	b	a
				i		
		1	2	3	4	5
V _{ij}	1	B	A, C	A, C	B	A, C
	2	S, A	B	S, C	S, A	
j	3	∅	B	B		
	4	∅	S, A, C			
	5	S, A, C				

Para calcular V_{24} :

$$V_{21} = \{A, C\} \quad V_{33} = \{B\} \quad \Rightarrow S, C$$

$$V_{22} = \{B\} \quad V_{42} = \{S, A\} \quad \Rightarrow A$$

$$V_{23} = \{B\} \quad V_{51} = \{A, C\} \quad \Rightarrow A, S$$

es decir, $V_{24} = \{S, A, C\}$.

Ya que $S \in V_{15}$, se concluye que el string *baaba* $\in L(G)$.

□

Chapter 7

ACEPTACIÓN Y GENERACIÓN DE LENGUAJES ENUMERABLES RECURSIVAMENTE Y LENGUAJES RECURSIVOS

En este capítulo se estudiarán las máquinas de Turing, un modelo matemático simple de lo que es un computador. A pesar de su simpleza, esta máquina modela la capacidad de computación de un computador de propósito general. Las máquinas de Turing son estudiadas tanto por la clase de lenguajes que definen (llamados *enumerables recursivamente*), como también por la clase de funciones enteras que pueden computar (llamadas *funciones recursivas parciales*). Un número de otros modelos de computación se presentan y se muestra que ellos son equivalentes a la máquina de Turing en su poder de computación.

7.1 Algoritmos

La noción intuitiva de algoritmo o procedimiento efectivo ha aparecido varias veces. Por ejemplo, se vio un procedimiento efectivo para determinar si el conjunto aceptado por un AF es vacío, finito o infinito. Inocentemente, se podría pensar que para cualquier clase de lenguajes con descripciones finitas, habría un procedimiento efectivo que respondiera tales preguntas. Sin embargo, no es así. Por ejemplo, no hay un algoritmo que indique si el complemento de un lenguaje libre de contexto es vacío; aún cuando sí se puede saber si el lenguaje en sí es vacío. Esta discusión no se refiere a un procedimiento que responda la pregunta para un lenguaje específico, sino que a un único procedimiento que responda correctamente la pregunta, cualquiera fuera el lenguaje.

Es obvio que si se tratara de responder si un lenguaje libre de contexto específico tiene un complemento vacío, entonces existe el algoritmo. Basta tener uno que responda siempre *SI* y otro que siempre responda *NO* ; uno de ellos es el algoritmo deseado en este caso. Por supuesto que puede no ser obvio cuál es el algoritmo que responde correctamente.

A comienzos de siglo, el matemático David Hilbert se embarca en la búsqueda de un algoritmo para determinar la veracidad o falsedad de cualquier proposición matemática. En particular, él buscaba un procedimiento para determinar si una fórmula arbitraria del cálculo de predicados de primer orden, aplicada a enteros, es verdadera. Como el cálculo de predicados de primer orden es suficientemente poderoso para expresar la sentencia de que el lenguaje generado por una gramática libre de contexto es igual a Σ^* , si Hilbert hubiese tenido éxito, el problema de decidir si el complemento de un lenguaje libre de contexto es vacío, se habría resuelto. Sin embargo, en 1931, Kurt Gödel publicó su famoso teorema de incompletitud, que probó

que dicho procedimiento efectivo no puede existir. Para ello, Gödel construyó una fórmula en cálculo de predicados aplicados a enteros, cuya misma definición establecía que no podía ser probada ni refutada en ese sistema lógico. La formalización de este argumento y la subsecuente clarificación y formalización del concepto intuitivo de lo que es un procedimiento efectivo es uno de los mayores logros de este siglo.

Una vez formalizada la noción de procedimiento efectivo, fue posible demostrar que no hay uno para computar muchas funciones específicas. En realidad, la existencia de estas funciones es fácil de ver usando un argumento de conteo. Hay, simplemente, demasiadas funciones, un número incontable y sólo hay un número contable de procedimientos. Por lo tanto, la existencia de tales funciones no debiera sorprender. Lo que sí es sorprendente es que algunos problemas y funciones de importancia en matemáticas, ciencias de la computación y otras disciplinas sean no computables.

Hoy en día, la máquina de Turing es la formalización aceptada de lo que es un procedimiento efectivo. Obviamente, no es posible demostrar que este modelo es equivalente a la noción intuitiva de lo que es un computador, pero hay fuertes argumentos para esta equivalencia, que se conoce como la hipótesis de Church. En particular, la máquina de Turing es equivalente, en poder de computación, a los computadores digitales como se los conoce hoy, y también a las nociones matemáticas más generales de lo que es computación.

7.2 Modelo de la Máquina de Turing

Un modelo para un procedimiento efectivo debiera proveer ciertas características. En primer lugar, cada procedimiento debe consistir de pasos discretos, cada uno de los cuales se puede efectuar mecánicamente. Un modelo como ése fue definido por Alan Turing en 1936. Aquí se presenta una variante de él.

El modelo básico tiene un control finito, una cinta dividida en celdas y una cabeza sobre la cinta que recorre una celda de la cinta a la vez. La cinta es finita por la izquierda, pero infinita por la derecha. Cada celda contiene exactamente uno, de entre un número finito de símbolos posibles. Inicialmente, las n celdas de más a la izquierda de la cinta (para algún $n \geq 0$) contienen el string de entrada, que es un string de símbolos tomados de un subconjunto de los símbolos de la cinta, llamados los símbolos de entrada. Las celdas restantes (infinitas), contienen el símbolo *blanco*, un símbolo especial de la cinta, que no es un símbolo de entrada.

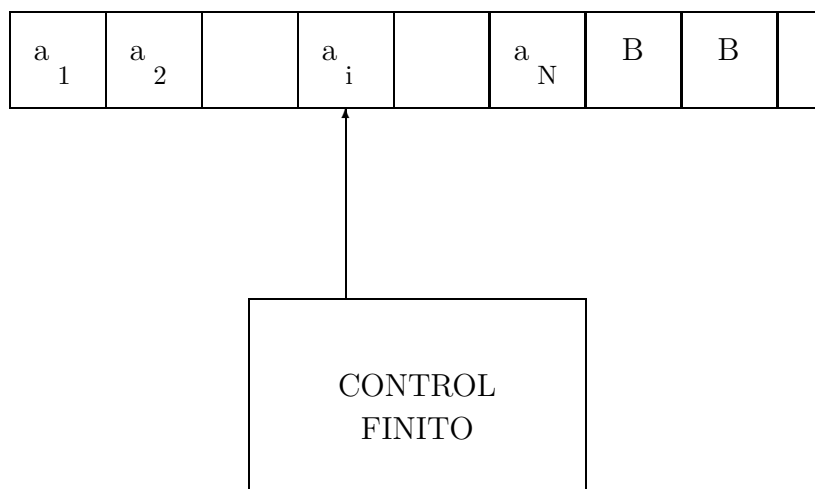


Figure 7.1: Modelo básico de una Máquina de Turing

En una movida, la máquina de Turing, dependiendo del símbolo en la cinta que está bajo la cabeza y del estado en el control finito, efectúa los siguientes cambios:

1. Cambia de estado.
2. Escribe un símbolo en la celda de la cinta que está bajo la cabeza, reemplazando lo que allí había.
3. Mueve la cabeza a la izquierda o la derecha, exactamente una celda.

Formalmente, una máquina de Turing (MT) se denota por la séxtupla

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

en que

Q es un conjunto finito de estados.

Γ es el conjunto finito de símbolos de la cinta posibles.

$B \in \Gamma$ es el símbolo *blanco*.

$\Sigma \subset \Gamma$, que no incluye B , es el conjunto de símbolos de entrada.

$q_0 \in Q$ es el estado inicial.

$F \subseteq Q$ es el conjunto de estados finales (o de aceptación).

δ es la función que determina las movidas.

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{I, D\}$$

y puede estar indefinida para algunos argumentos.

Una descripción instantánea (DI) de una máquina de Turing, M , se denota por $\alpha_1 q \alpha_2$. En ella, q es el estado en que se encuentra M y $\alpha_1 \alpha_2 \in \Gamma^*$ es el contenido de la cinta hasta el símbolo no blanco de más a la derecha o el símbolo a la izquierda de la cabeza, el que esté más a la derecha. Nótese que B puede estar en α_1 *alpha*₂. Para evitar confusión se supone que Γ y Q son disjuntos. Finalmente, se asume que la cabeza está sobre el símbolo de más a la izquierda de α_2 , o si $\alpha_2 = \varepsilon$, la cabeza está sobre un blanco.

Una movida de M se define como sigue. Sea $X_1 X_2 \dots X_{i-1} q X_i \dots X_n$ una DI y suponga que $\delta(q, X_i) = (p, Y, I)$, donde si $i - 1 = u$, entonces $X_i = B$. Si $i = 1$ entonces no hay una próxima DI, ya que la cabeza no puede caer hacia la izquierda de la cinta. Si $i > 1$, entonces se escribe.

$$X_1 X_2 \dots X_{i-1} q X_i \dots X_n \stackrel{\vdash}{M} X_1 X_2 \dots X_{i-2} p X_{i-1} Y X_{i+1} \dots X_n$$

sin embargo, si cualquier sufijo de $X_{i-1} Y X_{i+1} \dots X_n$ es completamente blanco, ese sufijo es eliminado.

Alternativamente, si $\delta(q, X_i) = (p, Y, D)$, entonces

$$X_1 X_2 \dots X_{i-1} q X_i \dots X_n \stackrel{\vdash}{M} X_1 X_2 \dots X_{i-1} Y p X_{i+1} \dots X_n$$

en el caso $i - 1 = n$, el string $X_i \dots X_n$ es vacío y la DI nueva ha alargado el string en la cinta.

Si dos DI están relacionadas por $\stackrel{\vdash}{M}$, se dice que la segunda resulta de la primera por una movida. Si una DI resulta de otra después de un número finito de movidas (incluidas cero movidas), ellas están en la relación $\stackrel{*}{\vdash} M$, la clausura refleja y transitiva de $\stackrel{\vdash}{M}$.

El lenguaje aceptado por M , $L(M)$, es el conjunto de strings en Σ^* , que hacen que M entre en un estado final, cuando se pone a la izquierda de la cinta, con M en q_0 y la cabeza en la celda de más a la izquierda. Formalmente, el lenguaje aceptado por $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ es el conjunto:

$$L(M) = \{w \in \Sigma^* / q_0 w \stackrel{*}{\vdash} \alpha_1 p \alpha_2 \quad \text{con } p \in F \text{ y } \alpha_1 \alpha_2 \in \Gamma^*\}$$

Dada una máquina de Turing que reconoce L , se puede asumir, sin pérdida de generalidad, que la MT se detiene, es decir, no tiene una próxima movida al aceptar un string. Sin embargo, para strings que no están en L , es posible que nunca se detenga.

Ejemplo 84 Una máquina de Turing, M , que acepta el lenguaje $L = \{0^N 1^N / N \geq 1\}$. Inicialmente, la cinta de M contiene $0^N 1^N$ seguido de un número infinito de *blancos*. En forma repetida, M reemplaza el 0 de más a la izquierda por X y se mueve hacia la derecha hasta el 1 de más a la izquierda y lo reemplaza por Y , luego se mueve hacia la izquierda hasta la X de más a la derecha y luego se mueve una celda a la derecha, hasta el 0 de más a la izquierda y repite el ciclo. Si al buscar un 1, M encuentra un *blanco*, entonces M se detiene sin aceptar. Si después de cambiar un 1 por Y , M no encuentra más ceros, entonces revisa que no hayan más 1's, en cuyo caso acepta.

Sea $Q = \{q_0, q_1, q_2, q_3, q_4\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, X, Y, B\}$ y $F = \{q_4\}$. Informalmente cada estado representa una o un grupo de sentencias de un programa. Al estado q_0 se entra inicialmente y también antes de cada reemplazo del 0 de más a la izquierda por una X . El estado q_1 es usado para buscar un 1 hacia la derecha, saltándose 0's e Y 's. Si encuentra un 1, M lo cambia por Y y entra en q_2 . En este estado busca una X hacia la izquierda y entra q_0 luego de encontrarlo, moviéndose una celda a la derecha al cambiar de estado. Si mientras M busca hacia la derecha en estado q_1 , encuentra una B o X antes de un 1, entonces el string es rechazado; hay demasiados ceros o el string no pertenece a 0^*1^* .

El estado q_0 juega también otro papel. Si, después que el estado q_2 encuentra la X de más a la derecha, entonces se han acabado los ceros. De q_0 , sobre Y , se entra q_3 para recorrer las Y 's y revisar que no quedan 1's. Si las Y 's son seguidas de B , se entra q_4 aceptando; si no, el string es rechazado. La función de transición se muestra a continuación:

ESTADO	SIMBOLO				
	0	1	X	Y	B
q_0	(q_1, X, D)			(q_3, Y, D)	
q_1	$(q_1, 0, D)$	(q_2, Y, I)		(q_1, Y, D)	
q_2	$(q_2, 0, I)$		(q_0, X, D)	(q_2, Y, I)	
q_3				(q_3, Y, D)	(q_4, B, D)
q_4					

Si el input es 0011 se producen las siguientes movidas:

$q_0 0011 \vdash X q_1 011 \vdash X 0 q_1 11 \vdash X q_2 0 Y 1 \vdash q_2 X 0 Y 1 \vdash$
 $X q_0 0 Y 1 \vdash X X q_1 Y 1 \vdash X X Y q_1 1 \vdash X X q_2 Y Y \vdash X q_2 X Y Y \vdash$
 $X X q_0 Y Y \vdash X X Y q_3 Y \vdash X X Y Y q_3 \vdash X X Y Y B q_4$

□

7.3 Técnicas para la construcción de Máquinas de Turing

El diseño de máquinas de Turing describiendo el conjunto completo de estados y movidas es bastante engoroso. Para describir máquinas complejas, se necesitan herramientas conceptualmente de más alto nivel. En esta sección se discutirán algunas de ellas.

7.3.1 Almacenamiento en el Control Finito

El control finito puede usarse para almacenar una cantidad finita de información. Para hacerlo, el estado es considerado un par de elementos, uno ejerciendo el control y el otro almacenando un símbolo. Debe notarse que este es un arreglo conceptual, no se ha modificado lo que es una MT. En general se puede permitir que los estados tengan k componentes, de los cuales todos menos uno, almacenan información.

Ejemplo 85 Considere una MT, M , que mire el primer símbolo de su input, lo almacene en su control finito y revise que dicho símbolo no aparezca en otra parte del input. Nótese que M acepta un lenguaje regular:

$$M = (Q, \{0, 1\}, \{0, 1, B\}, \delta, [q_0, B], B, F)$$

en que Q es $\{q_0, q_1\} \times \{0, 1, B\}$. El conjunto F es $\{[q_1, B]\}$. La intención es que la primera componente del estado controle la acción, mientras que la segunda recuerda un símbolo. La función δ se define como:

$$\begin{aligned} \delta([q_0, B], 0) &= ([q_1, 0], 0, D) & \delta([q_0, B], 1) &= ([q_1, 1], 1, D) \\ \delta([q_1, 0], 1) &= ([q_1, 0], 1, D) & \delta([q_1, 1], 0) &= ([q_1, 1], 0, D) \\ \delta([q_1, 0], B) &= ([q_1, B], B, I) & \delta([q_1, 1], B) &= ([q_1, B], B, I) \end{aligned}$$

□

7.3.2 Pistas Múltiples

Es también posible suponer que la cinta de la máquina de Turing está dividida en un número finito, k , de pistas. Por ejemplo, para $k = 3$

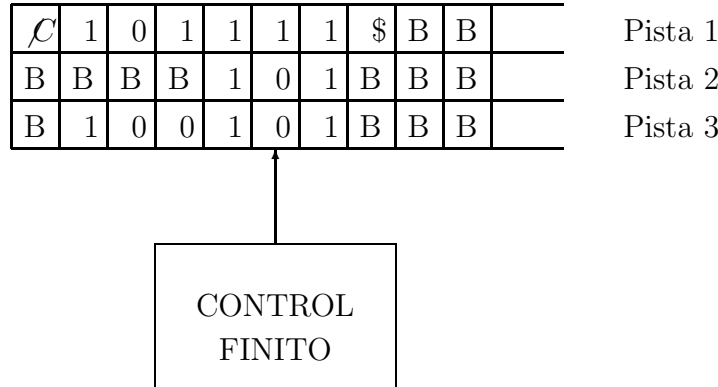


Figure 7.2: Máquina de Turing con pistas múltiples

Los símbolos en la cinta se consideran k -tuplas, con una componente por cada pista.

Ejemplo 86 La cinta de la figura 7.2 pertenece a una MT que toma un input binario mayor que 2, escrito en la primera pista y determina si es un número primo. El input está enmarcado por los símbolos $\mathcal{C}$ y $\$$. Por lo tanto los símbolos de entrada son las tuplas $[C, B, B]$, $[0, B, B]$, $[1, B, B]$ y $[\$, B, B]$. Estos símbolos se pueden identificar con C , 0, 1 y $\$$ respectivamente al verlos como símbolos de entrada. El *blanco*, se identifica con $[B, B, B]$. Para saber si el input es un número primo, la MT primero escribe el número 2 (en binario) en la segunda pista y copia la primera pista en la tercera. Luego, la segunda pista es sustraída tantas veces como sea posible de la tercera, dividiendo la tercera pista por la segunda y dejando en ella el resto.

Si el resto es cero, el número en la primera pista, el input, no es primo. Si el resto no es cero, se incrementa en 1 el número de la segunda pista. Si ella iguala a la primera, el número era primo, porque no puede ser dividido por ningún número entre 1 y sí mismo. Si el número de la segunda pista es menor que el de la primera, toda la operación se repite para el nuevo número en la segunda pista.

En la figura 7.2 la MT está chequeando si 47 es un primo, lo está dividiendo por 5, el que ya ha sido sustraído dos veces, por lo que el número 37 está en la tercera pista.

□

7.3.3 Marcar Símbolos

Poner marcas en algunos símbolos es una forma útil de visualizar cómo una MT reconoce lenguajes definidos por strings repetidos, tales como

$$\{ww/w \in \Sigma^*\}, \{wcy/w \text{ e } y \in \Sigma^* \text{ y } w \neq y\}, \{ww^r/w \in \Sigma^*\}$$

Y también es útil cuando deben compararse longitudes de algunos substrings, tales como en los lenguajes

$$\{a^i b^j / i \geq 1\}, \{a^i b^j c^k / i \neq j \text{ o } j \neq k\}$$

Para ello se usa una segunda pista en la cinta, la que sólo contiene un *blanco* o un $\surd$ (visto). El símbolo $\surd$ aparece bajo uno de la primera pista, que ya ha sido considerado por la MT en una de las comparaciones.

7.3.4 Correr Símbolos

Una máquina de Turing puede hacer espacio en su cinta al mover todos los símbolos no blancos un número finito de celdas hacia la derecha. Para ello, la cabeza se mueve hacia la derecha almacenando repetidamente los símbolos leídos en celdas de más a la izquierda. La MT puede entonces volver a las celdas vaciadas y escribir los símbolos que desee. Si hay espacio disponible, también es posible empujar grupos de símbolos hacia la izquierda de manera similar.

7.3.5 Subrutinas

Tal como sucede con programas, un diseño modular o “top-down” se facilita al usar subrutinas que definen procesos elementales. Una máquina de Turing puede simular cualquier tipo de subrutinas encontradas en lenguajes de programación, incluso procedimientos recursivos y cualquiera de los métodos conocidos para pasar parámetros. Aquí sólo se describirá el uso de subrutinas sin parámetros y no recursivas, pero aún éstas son bastante poderosas.

La idea general es escribir una parte de una MT que sirva como subrutina; ella tendrá un estado inicial y uno de regreso que momentáneamente no tendrá movidas y que se usará para efectuar el regreso a la rutina que la llamó. Para designar una MT que “llama” a la subrutina, un conjunto nuevo de estados para la subrutina se llama y se especifica una movida para el estado de regreso. La llamada se efectúa entrando al estado inicial de la subrutina y el regreso, por la movida definida para el estado de regreso.

7.4 Lenguajes y Funciones Computables

Un lenguaje aceptado por una máquina de Turing se llama *enumerable recursivamente* (recursively enumerable o r.e., en inglés). El término enumerable deriva del hecho que son precisamente estos lenguajes cuyos strings pueden ser enumerados (listados) por una máquina de Turing. Recursivamente, es un término matemático previo a la existencia de los computadores y su significado es similar a lo que se llama recursión en ciencia de la computación. La clase de los lenguajes enumerables recursivamente es muy amplia e incluye con propiedad a la clase de los lenguajes libres de contexto.

La clase de los lenguajes enumerables recursivamente incluye algunos lenguajes para los que no se puede determinar pertenencia en forma mecánica. Si $L(M)$ es uno de esos lenguajes, entonces cualquier máquina de Turing que reconozca $L(M)$ debe no detenerse en algunos strings que no pertenecen al lenguaje. Si $w \in L(M)$, M se detendrá eventualmente en input w . Sin embargo, mientras M esté ejecutando en algún input, no es posible saber si parará y aceptará si se la deja ejecutar lo suficiente, o si M no se detendrá nunca y correrá para siempre.

Es conveniente singularizar un subconjunto de los conjuntos enumerables recursivamente, llamados los *conjuntos recursivos*, que son aquellos lenguajes aceptados por al menos una máquina de Turing que se detiene en todos sus inputs, ya sea aceptando o no. Posteriormente se verá que los conjuntos recursivos son una subclase propia de los conjuntos enumerables recursivamente. Nótese también que por el algoritmo CYK, todo lenguaje libre de contexto es un conjunto recursivo.

Además de ser un dispositivo de aceptación, la máquina de Turing puede verse como un computador de funciones de enteros a enteros. La forma tradicional es representar los enteros en unario; es decir, $i \geq 0$ se representa por el string 0^i . Si una función tiene k argumentos, $i_1, i_2, \dots, i_k$, entonces estos enteros se ponen inicialmente en la cinta separados por 1's, como: $0^{i_1} 1 0^{i_2} 1 \dots 1 0^{i_k}$.

Si la máquina de Turing se detiene, aceptando o no, con una cinta que consiste de 0^M (para algún M), entonces se dice que $f(i_1, i_2, \dots, i_k) = M$, en que f es la función de k argumentos que computa esa máquina de Turing. Nótese que una única MT puede computar una función de un argumento, una diferente de dos argumentos, etcétera. También debe notarse que si una MT, M , computa una función de k argumentos, no es necesario que f tenga un valor para todas las diferentes k -tuplas de enteros que sean posibles argumentos.

Si $f(i_1, i_2, \dots, i_k)$ está definida para toda tupla $(i_1, i_2, \dots, i_k)$, entonces se dice que es una *función recursiva total*. Una función $f(i_1, i_2, \dots, i_k)$ computada por una máquina de Turing es llamada una *función recursiva parcial*. En cierto sentido, las funciones recursivas parciales son análogas a los lenguajes enumerables recursivamente, ya que son computadas por MT que pueden o no detenerse en ciertos inputs. Las funciones recursivas totales corresponden a los lenguajes recursivos, ya que son computadas por máquinas que siempre se detienen. Todas las funciones aritméticas comunes en enteros, tales como multiplicación, $n!$ y 2^N , son funciones recursivas totales.

Ejemplo 87 La sustracción propia, $m \overset{\circ}{-} n$, se define de la siguiente forma: $m \overset{\circ}{-} n = \begin{matrix} m - n & \text{si } m \geq n \\ 0 & \text{si } m < n \end{matrix}$

La siguiente máquina de Turing, inicialmente con el string $0^m 10^n$ en su cinta, se detiene con el string $0^{m \overset{\circ}{-} n}$ en ella.

$$M = (\{q_0, q_1, \dots, q_6\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_6\})$$

M reemplaza repetidamente el primer 0 por blanco y luego busca hacia la derecha un 1 seguido de un 0, y cambia el 0 por un 1. Luego, M se mueve a la izquierda hasta que encuentra un blanco y entonces repite el ciclo. La repetición termina si:

- (i) Buscando un 0 hacia la derecha, se encuentra un blanco. En ese caso, los n 0's de $0^m 10^n$ han sido cambiados a 1's y $n + 1$ de los m 0's a B . M reemplaza entonces los $n + 1$ 1's por un 0 y n blancos, dejando $m - n$ 0's en la cinta.
- (ii) Al comenzar el ciclo, M no encuentra un 0 que cambiar por un blanco, ya que los primeros m 0's han sido cambiados. Entonces $n \geq m$ y, por lo tanto, $m \overset{\circ}{-} n = 0$. En ese caso, M reemplaza todos los 1's y 0's que queden por blancos.

La función de transición δ se describe a continuación:

$$1. \delta(q_0, 0) = (q_1, B, D).$$

Comienza el ciclo reemplazando el cero inicial por un *blanco*.

$$2. \delta(q_1, 0) = (q_1, 0, D).$$

$$\delta(q_1, 1) = (q_2, 1, D).$$

Se mueve hacia la derecha buscando el primer 1.

$$3. \delta(q_2, 1) = (q_2, 1, D).$$

$$\delta(q_2, 0) = (q_3, 1, I).$$

Busca sobre los primeros 1's hasta encontrar un 0; lo cambia a un 1.

$$4. \delta(q_3, 1) = (q_3, 1, I).$$

$$\delta(q_3, 0) = (q_3, 0, I).$$

$$\delta(q_3, B) = (q_0, B, D).$$

Se mueve a la izquierda hasta un blanco y entra q_0 para repetir ciclo.

$$5. \delta(q_2, B) = (q_4, B, I).$$

$$\delta(q_4, 1) = (q_4, B, I).$$

$$\delta(q_4, 0) = (q_4, 0, I).$$

$$\delta(q_4, B) = (q_6, 0, D).$$

Si en estado q_2 se encuentra un B antes de un 0 , se está en el caso (i) descrito más arriba. Se entra estado q_4 y se mueve a la izquierda cambiando los 1 's a B 's, hasta encontrar una B , la que se cambia a 0 , se entra en estado q_6 y M para.

$$6. \delta(q_0, 1) = (q_5, B, D).$$

$$\delta(q_5, 0) = (q_5, B, D).$$

$$\delta(q_5, 1) = (q_5, B, D).$$

$$\delta(q_5, B) = (q_6, B, D).$$

Si en estado q_0 se encuentra un 1 en vez de un 0 , el primer bloque de 0 's se ha acabado y se está en el caso (ii) descrito anteriormente. M entra q_5 para borrar con blancos el resto de la cinta y luego entra q_6 y se detiene.

□

Notar que si m, n o ambos son 0 's, la función se comporta perfectamente bien.

7.5 Extensiones al Modelo

Una de las razones para aceptar que la máquina de Turing es un modelo general de computabilidad, es que el modelo que ya se ha visto es equivalente a muchas versiones modificadas que, de antemano, aparecerían incrementando la capacidad de computación. En esta sección se dan pruebas informales de estos teoremas de equivalencia.

7.5.1 Cinta Infinita en Ambas Direcciones

Una máquina de Turing con cinta infinita en ambas direcciones se denota como $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, como en el modelo original. Sin embargo, como su nombre lo indica, su cinta es infinita no sólo hacia la derecha, sino que también hacia la izquierda. Las DI se denotan en igual forma que antes, asumiendo que hay una infinidad de blancos, tanto a la izquierda como a la derecha del trozo actualmente no blanco.

La relación $\vdash^+$ entre DI que define las movidas, es como en el modelo original, con la excepción que si $\delta(q, X) = (p, Y, I)$, entonces $qX\alpha \vdash^+ pBY\alpha$ (en el modelo original no hay movida posible), y que si $\delta(q, X) = (p, B, D)$, entonces $qX\alpha \vdash^+ p\alpha$ (en el original, el símbolo B aparecería a la izquierda de p).

La DI inicial es q_0w . La relación $\vdash^* M$, como antes, relaciona dos DI, si la de la derecha se puede obtener de la de la izquierda en algún número (posiblemente cero) de movidas de la máquina.

Teorema 34 *L es reconocido por una máquina de Turing con cinta infinita en ambas direcciones si y sólo si es reconocido por una MT con cinta infinita en sólo una dirección.*

Demostración : La prueba de que una MT con cinta infinita en dos direcciones puede simular una MT con cinta infinita sólo hacia la derecha es fácil. Aquella marca la celda a la izquierda de la posición inicial de su cabeza y luego simula a la otra. Si durante la simulación aparece la celda marcada, la máquina se detiene sin aceptar.

En la otra dirección, sea $M_2 = (Q_2, \Sigma_2, \Gamma_2, \delta_2, q_2, B, F_2)$ una MT con cinta infinita en dos direcciones. Se construye una máquina de Turing M_1 , que simula M_2 y tiene cinta infinita sólo hacia la derecha. M_1 tendrá

	A ₋₅	A ₋₄	A ₋₃	A ₋₂	A ₋₁	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅	
--	-----------------	-----------------	-----------------	-----------------	-----------------	----------------	----------------	----------------	----------------	----------------	----------------	--

	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅		...
	$\mathcal{C}$	A ₋₁	A ₋₂	A ₋₃	A ₋₄	A ₋₅		

Figure 7.3: Reconocimiento del lenguaje aceptado por M_2 usando una MT con cinta infinita en una dirección

2 pistas, una representando las celdas de M_2 que están desde la celda inicial (inclusive) hacia la derecha; la otra pista representa (invertida) a las celdas que están a la izquierda de la celda inicial de M_2 .

La primera celda de M_1 tiene el símbolo $\mathcal{C}$ en su pista inferior, para indicar que es el de más a la izquierda. El control finito de M_1 “recuerda” si M_2 estaría sobre un símbolo que aparece en la pista superior o inferior de M_1 .

Debiera ser bastante obvio que M_1 puede simular a M_2 , en el sentido que si M_2 está a la derecha de su posición inicial, M_1 trabaja con la pista superior; mientras que si M_2 está a la izquierda, M_1 trabaja con la pista inferior, moviéndose en dirección opuesta a M_2 . Los símbolos de entrada a M_1 son símbolos con *blanco* en la pista inferior y un símbolo de entrada de M_2 en la pista superior; los que pueden identificarse con los de M_2 . B se identifica con $[B, B]$.

La construcción formal es la siguiente, $M_1 = (Q_1, \Sigma_1, \Gamma_1, \delta_1, q_1, B, F_1)$. Los estados en Q_1 son objetos de la forma $[q, S]$ o $[q, I]$, en que $q \in Q_2 \cup \{q_1\}$. La segunda componente indica si M_1 está trabajando en la pista superior (S) o inferior (I). $\Gamma_1 = \Gamma_2 \times (\Gamma_1 \cup \{\mathcal{C}\})$. $\Sigma_1 = \Sigma_2 \times \{B\}$. $F_1 = \{[q, S], [q, I] / q \in F_2\}$. La función δ_1 se define como sigue

1. $\forall a \in \Sigma_2 \cup \{B\}$
 $\delta_1(q_1, [a, B]) = ([q, S], [X, \mathcal{C}], D)$ si $\delta_2(q_2, a) = (q, X, D)$.
2. $\forall a \in \Sigma_2 \cup \{B\}$
 $\delta_1(q_1, [a, B]) = ([q, I], [X, \mathcal{C}], D)$ si $\delta_2(q_2, a) = (q, X, I)$.
3. $\forall [X, Y] \in \Gamma_1$ con $Y \neq \mathcal{C}$ y $A = I$ o D .
 $\delta_1([q, S], [X, Y]) = ([p, S], [Z, Y], A)$ si $\delta_2(q, X) = (p, Z, A)$.
4. $\forall [X, Y] \in \Gamma_1$ con $Y \neq \mathcal{C}$ y $A = I$ o D .
 $\delta_1([q, I], [X, Y]) = ([p, I], [X, Z], A)$ si $\delta_2(q, Y) = (p, Z, \overline{A})$.
 Con $\overline{A}$ representando la dirección contraria a la que representa A .
5. $\delta_1([q, S], [X, \mathcal{C}]) = \delta_1([q, I], [X, \mathcal{C}])$
 $= ([p, C], [Y, \mathcal{C}], D)$ si $\delta_2(q, X) = (p, Y, A)$

En que

$$C = S \text{ si } A = D$$

$$C = I \text{ si } A = I.$$

□

7.5.2 Máquinas de Turing con Varias Cintas

Una máquina de Turing con varias cintas consta de un control finito con k cabezas y k cintas infinitas en ambas direcciones. Por ejemplo, para $k = 3$:

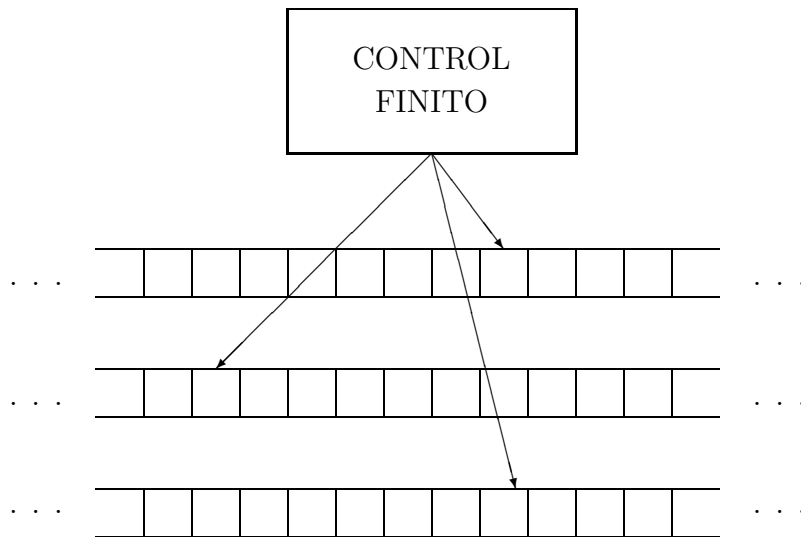


Figure 7.4: Máquina de Turing con varias cintas

En cada movida, dependiendo del estado en que se encuentre el control finito y del símbolo bajo cada una de las cabezas, la máquina puede:

1. Cambiar de estado.
2. Escribir un nuevo símbolo en cada celda bajo las cabezas.
3. Mover cada cabeza, independientemente, una celda a la izquierda, a la derecha o mantenerla inmóvil.

Inicialmente el input está en la primera cinta y las demás están en blanco.

Teorema 35 *Si un lenguaje L es aceptado por una MT con varias cintas, es aceptado por una MT con una sola cinta.*

Demostración : Sea L aceptado por M_1 , una MT con k cintas. Se construye M_2 , una máquina con una cinta dividida en $2k$ pistas; 2 pistas por cada cinta correspondiente de M_1 . Una pista contiene el símbolo donde está la cabeza correspondiente de M_1 . El control finito de M_2 almacena el estado de M_1 y un contador del número de cabezas de M_1 que están a la derecha de M_2 .

Cada movida de M_1 es simulada por un recorrido de izquierda a derecha y luego de derecha a izquierda por la cabeza de M_2 . Inicialmente, la cabeza de M_2 está en la celda de más a la izquierda que contiene una marca de cabeza. La cabeza de M_2 se mueve hacia la derecha visitando cada celda con marcas y recordando el símbolo leído por la cabeza correspondiente de M_1 . Cuando M_2 cruza una marca, debe actualizar el contador de marcas a su derecha. Cuando no quedan más, M_2 ha visto los símbolos leídos por cada cabeza de M_1 , con lo que M_2 tiene la información necesaria para determinar la movida de M_1 . Ahora M_2 hace una pasada hacia la izquierda, hasta que llega a la marca de más a la izquierda. El contador le permite saber hasta dónde llegar. A medida que M_2 pasa cada marca, cambia el símbolo correspondiente a esa cinta de M_1 , mueve la marca una celda a la izquierda o la derecha (o no la mueve) para simular la movida de M_1 en esa cinta. Por último, M_2 cambia el estado de M_1 que almacena en su control para finalizar la movida de M_1 . Si ese estado de M_1 es final, M_2 acepta.

□

Nótese que al simular la MT con cinta infinita en ambas direcciones por una MT con cinta infinita sólo hacia la derecha, la simulación fue movida por movida. En la que se acaba de presentar, cada movida de M_1 requiere de varias de M_2 para ser simulada. De hecho, para simular N movidas de M_1 , se requieren $\mathcal{O}(N^2)$ movidas de M_2 .

7.5.3 Movidas No Determinísticas

Una máquina de Turing no determinística es un dispositivo con un control finito y una cinta infinita sólo hacia la derecha. Dado un estado y símbolo bajo la cabeza, la máquina tiene un número finito de movidas posibles. Cada opción consiste de un nuevo estado, un símbolo para escribir y una dirección de movimiento de la cabeza. La máquina acepta un input si hay una secuencia de movidas que la lleve a un estado final.

Como en el caso de los autómatas finitos, el agregar no determinismo a la máquina de Turing no permite aceptar nuevos lenguajes. De hecho, la combinación de no determinismo con las otras extensiones de esta sección, no le añaden poder adicional.

Teorema 36 *Si L es aceptado por una MT no determinística M_1 , entonces L es aceptado por una MT determinística M_2 .*

Demostración : Para cada estado y símbolo de la cinta de M_1 hay un número finito de opciones para la próxima movida. Sea r el número máximo de opciones para todos los pares estado-símbolo.

Luego, cualquier secuencia finita de elecciones puede representarse por una secuencia de los dígitos 1 a r . Es posible que no todas dichas secuencias representen elecciones de movidas, ya que puede haber menos de r opciones en algunas situaciones.

M_2 tendrá tres cintas. La primera contendrá el input; en la segunda M_2 generará secuencias de dígitos de 1 a r en forma sistemática. Específicamente, las secuencias serán generadas con las más cortas primero. Secuencias del mismo largo son generadas en orden numérico.

Por cada secuencia generada en la segunda cinta, M_2 copia el input a la tercera cinta y simula a M_1 sobre la cinta 3; usando la secuencia definida en la cinta 2 para dictar las movidas de M_1 . Si M_1 entra a un estado de aceptación, M_2 también acepta. Si existe una secuencia de opciones que lleve a M_1 a aceptar, ella será eventualmente generada en la cinta 2. Cuando sea simulada, M_2 aceptará. Si no hay secuencia de elecciones que haga que M_1 acepte, M_2 no aceptará.

□

7.5.4 Máquinas Multidimensionales

Considérese otra modificación a las máquinas de Turing que tampoco les da poder adicional. Este dispositivo tiene un control finito, pero la cinta consiste de un arreglo k -dimensional de celdas infinitas en las $2k$ direcciones, para algún k fijo. Dependiendo del estado y símbolo, la máquina cambia de estado, escribe un símbolo y mueve la cabeza en alguna de las $2k$ direcciones. Inicialmente, el input está a lo largo de un eje y la cabeza en su primer símbolo a la izquierda.

En cualquier instante, sólo un número finito de filas en cualquier dimensión contiene símbolos no-blancos y de ellas cada una tiene sólo un número finito de estos símbolos. Se probará que una máquina uni-dimensional puede simular una MT de 2 dimensiones. La generalización se deja como ejercicio.

Teorema 37 *Si L es aceptado por una máquina de Turing de dos dimensiones, M_2 , entonces L es aceptado por una MT de una dimensión, M_1 .*

Demostración : M_1 representa la cinta de M_2 de la siguiente manera (ver Figura 7.5)

$$M_1 : ** BBBA_1 BBB * BBa_2 a_3 a_4 a_5 B * a_6 a_7 a_8 a_9 a_{10} B * \dots **$$

$M_2 :$

B	B	B	A_1	B	B	B
B	B	a_2	a_3	a_4	a_5	B
a_6	a_7	a_8	a_9	B	a_{10}	B
B	a_{11}	a_{12}	a_{13}	B	a_{14}	a_{15}
B	B	a_{16}	a_{17}	B	B	B

Figure 7.5: Representación de M_2 usando M_1

M_1 también tendrá una segunda cinta, ambas infinitas por ambos lados. Si M_2 hace una movida que no la saca del rectángulo ya representado en la cinta de M_1 , si la movida es horizontal, M_1 simplemente mueve el marcador de la cabeza un lugar; si es vertical, M_1 usa su segunda cinta para contar el número de celdas entre la posición de la cabeza y el * a su izquierda. Luego M_1 se mueve al * a la derecha, si la movida es hacia abajo, o al * de la izquierda si la movida es hacia arriba, y pone la cabeza en la posición correspondiente del nuevo bloque (región entre *'s), usando el contador de la segunda cinta.

Considérese ahora la situación cuando la cabeza de M_2 se mueve fuera del rectángulo representado por M_1 . Si la movida es vertical, se agrega un nuevo bloque de blancos a la izquierda o derecha, usando la segunda cinta para contar el largo actual de los bloques. Si la movida es horizontal, M_1 usa la técnica de correr símbolos para agregar un *blanco* en el extremo izquierdo o derecho de cada bloque. Como ** marca el final de la región usada para los bloques, M_1 sabe cuándo ha crecido todos los bloques. Luego de hacer el espacio necesario, M_1 simula la movida de M_2 como ya se ha descrito.

□

7.5.5 Máquinas de Varias Cabezas

Una MT de k -cabezas tiene un número fijo, k , de cabezas numeradas de 1 a k . Una movida depende del estado y del símbolo leído por cada cabeza. En una movida, las cabezas se pueden mover independientemente hacia la izquierda, derecha o permanecer estacionaria.

Teorema 38 Si L es aceptado por una MT de k cabezas, M_1 , es aceptado por una MT de una cabeza, M_2 .

Demostración : La prueba es similar a la hecha para el caso de máquinas de varias cintas. M_2 tiene $K + 1$ pistas en su cinta; la última tiene el contenido de la cinta de M_1 . La i -ésima pista ($1 \leq i \leq k$) tiene una marca indicando la posición de la i -ésima cabeza.

□

7.5.6 Máquinas Off-Line

Una MT off-line es una MT de varias cintas, cuya cinta con el string de entrada es sólo leíble (read-only). Usualmente se encierra el string de entrada entre los símbolos $\mathcal{C}$ (a la izquierda) y $\$$ (a la derecha). La máquina no puede mover la cabeza fuera de la región entre $\mathcal{C}$ y $\$$. Debería ser claro que éste es sólo un caso

especial de una máquina con varias cintas y, por lo tanto, no es más poderosa que ninguno de los modelos vistos. Al revés, una MT off-line puede simular cualquier MT, M , usando una cinta más que M . Lo primero que hará es copiar su input en esta cinta extra y simular a M como si ella fuera el input de M .

7.6 Hipótesis de Church

La suposición de que la noción intuitiva de “función computable” puede identificarse con la clase de funciones recursivas parciales, es conocida como la *Hipótesis de Church* o la *Tesis de Church-Turing*.

Aún cuando no se puede esperar tener una “prueba” de la hipótesis de *Church*, al menos mientras la noción informal de “computable” permanezca como noción informal, es sin embargo posible dar evidencia de porqué es una suposición razonable.

Si nuestra noción intuitiva de “computable” no posee límite en el número de pasos o la cantidad de almacenamiento necesaria, parece que las funciones recursivas parciales son (intuitivamente) computables. Aún cuando alguien podría argüir que una función no es “computable”, a menos que se pueda limitar la computación de antemano, o al menos saber si ella terminará o no.

Lo que es más discutible es si la clase de funciones recursivas parciales incluye a todas las funciones computables. Los lógicos-matemáticos han presentado muchos otros formalismos, como el cálculo- λ , sistemas de Post y funciones recursivas generales. Para todos ellos se ha demostrado que definen la misma clase de funciones, es decir las funciones recursivas parciales. Además, modelos abstractos de los computadores como la RAM (Random Access Machine) dan también lugar a las funciones recursivas parciales.

La RAM consiste de un número infinito de palabras de memoria, numeradas desde 0, cada una de las cuales puede almacenar un número entero; y un número finito de registros aritméticos, también capaces de almacenar un entero. Los enteros pueden ser decodificados como instrucciones en la forma usual de los computadores. No se definirá la RAM más formalmente, pero debiera ser claro que si se escoge un conjunto adecuado de instrucciones, la RAM puede simular cualquier computador existente.

Teorema 39 *Una máquina de Turing puede simular una RAM, provisto que las instrucciones de la RAM puedan ser simuladas por una MT.*

Demostración : Se usa una MT, M , de varias cintas para hacer la simulación. Una cinta de M tiene las palabras de memoria de la RAM, a las que se les ha dado valores. La cinta se ve como

$$\#0 * v_0 \#1 * v_1 \#10 * v_2 \# \dots \#i * v_i \# \dots$$

en que v_i es el contenido, en binario, de la i -ésima palabra. En todo momento, habrá algún número finito de palabras de la RAM que han sido usadas y M sólo necesita mantener los valores hasta la palabra de número mayor que se haya usado.

La RAM tiene un número finito de registros aritméticos. M usa una cinta para almacenar el contenido de cada registro; otra cinta contiene el “contador de posición”, que contiene el número de la palabra de memoria de donde se debe tomar la próxima instrucción y una cinta “memory address register” en que se puede poner el número de una palabra de memoria.

Supóngase que los primeros 10 bits de una instrucción denotan una de las operaciones estándar en los computadores, como *load*, *store*, *add*, etc., y que los bits restantes denotan la dirección del operando. Si bien no se discutirá los detalles de implementación para todas las instrucciones estándar, un ejemplo debiera poner las cosas claras. Supóngase que la cinta con el contador de posición tiene el número i en binario. M busca en su primera cinta desde la izquierda, buscando $\#i*$. Si se encuentra un blanco antes de encontrar $\#i*$, no hay instrucción en la palabra i y, por lo tanto, la RAM y M se detienen. Si $\#i*$ es encontrado, los bits que siguen a $*$, hasta el siguiente $\#$ (v_i) se examinan. Suponga que los primeros 10 bits están codificados para *add* al registro 2 y los bits restantes son un cierto número j en binario. M agrega 1 a i en el contador de posición y copia j en la “memory address register”. Luego M busca $\#j*$ en la primera cinta, comenzando desde la izquierda ($\#0*$ marca el final por la izquierda). Si $\#j*$ no se encuentra, se supone que j tiene 0 y

se sigue con la próxima instrucción de la RAM. Si $\#j * v_j\#$ es encontrado, v_j es sumado al registro 2, que está en su propia cinta, y se continúa con la próxima instrucción.

Obsérvese que aún cuando la simulación de la RAM hizo uso de una MT con varias cintas, por teorema 35, una MT con una cinta sería mucho más compleja.

□

7.7 Máquinas de Turing como Generadores

Se ha visto a las máquinas de Turing como reconocedoras de lenguajes y como computadoras de funciones en los enteros no negativos. Hay una tercera visión útil de las MT, como dispositivos generadores. Considere una MT, M , que usa una cinta como cinta de output, en la cual un símbolo, una vez escrito, no puede ser cambiado y cuya cabeza (escritora en este caso) nunca se mueve a la izquierda. Suponga también que en la cinta de output M escribe strings sobre algún alfabeto Σ , separados por un símbolo especial $\#$. Se puede definir $G(M)$, el lenguaje *generado* por M , como el conjunto de $w \in \Sigma^*$, tal que w es eventualmente escrito entre un par de $\#$'s en la cinta de output de M .

Nótese que a menos que M no pare, $G(M)$ es finito. Tampoco se requiere que las palabras sean generadas en algún orden en particular, o que cualquier palabra sea generada una sola vez. Si L es $G(M)$ para alguna MT, M , entonces L es un conjunto enumerable recursivamente y viceversa. Los conjuntos recursivos también tienen una caracterización en términos de generadores; ellos son exactamente los lenguajes cuyas palabras pueden ser generadas en orden creciente de tamaño.

Lema 11 Si L es $G(M_1)$ para alguna MT, M_1 , entonces L es un conjunto enumerable recursivamente.

Demostración : Se construye una MT, M_2 , con una cinta más que M_1 . M_2 simula a M_1 usando todo excepto la cinta de entrada de M_2 . Cada vez que M_1 imprime un $\#$ en su cinta de output, M_2 compara su input con el string recién generado. Si son el mismo, M_2 acepta; si no, sigue simulando a M_1 . Obviamente M_2 acepta un string X , si y sólo si $X \in G(M_1)$. Por lo tanto, $L(M_2) = G(M_1) = L$ es enumerable recursivamente.

□

El converso de este lema es algo más difícil. Suponga que M_1 reconoce a $L \subseteq \Sigma^*$. Nuestro primer (y poco exitoso) intento para diseñar un generador para L puede ser generar palabras en Σ^* , en algún orden, $w_1, w_2, \dots$, hacer correr a M_1 en w_1 y si M_1 acepta, generar w_1 en la cinta de output. Luego hacer correr a M_1 en w_2 , generándolo si M_1 acepta, etc. Este método funciona si M_1 está garantizado de parar en todos los inputs. Sin embargo, como se verá en el próximo capítulo, hay lenguajes enumerables recursivamente que no son recursivos. En esos casos, aparece la posibilidad que M_1 nunca se detenga en algún w_i . Luego M_2 nunca considerará $w_{i+1}, w_{i+2}, \dots$ y no puede generarlas aún cuando M_1 las aceptase.

Debe, por lo tanto, evitarse la simulación indefinida de M_1 en alguna palabra. Para ello se fija un orden en que enumerar strings en Σ^* . Luego se desarrolla un método para generar todos los pares de enteros positivos (i, j) . La simulación procede generando un par (i, j) y simulando a M_1 en la i -ésima palabra durante j pasos.

Se fija un orden canónico para Σ^* como sigue. Se listan los strings en orden de tamaño, con palabras del mismo largo en "orden numérico". Esto es, sea $\Sigma = \{a_0, a_1, \dots, a_{k-1}\}$, e imagine que a_i es el dígito i en base k . Es decir, las palabras de largo N son los números 0 a $k^N - 1$, escritos en base k . El diseño de una máquina de Turing que genere palabras en orden canónico no es difícil y se deja como ejercicio.

Ejemplo 88 Si $\Sigma = \{0, 1\}$, el orden canónico es $\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots$

□

Nótese que el orden aparentemente más simple en que usualmente se generan las representaciones ás cortas de los números en base k , 0, 1, 2, $\dots$, no sirve pues nunca se generan strings como $a_0a_0a_1$, que tienen ceros adelante.

Considérese ahora la generación de pares (i, j) en tal forma que cada par sea generado después de una cantidad finita de tiempo. La tarea no es tan simple como parece, el método ingenuo de generar: $(1,1)$, $(1,2)$, $(1,3)$, $\dots$, nunca genera pares en que $i \geq 1$. En lugar de esto, los pares se deben generar en orden de su suma, $i + j$, y entre los de igual suma, en orden creciente de i . Esto es, se genera $(1,1)$, $(1,2)$, $(2,1)$, $(1,3)$, $(2,2)$, $(3,1)$, $(1,4)$, $\dots$. El par (i, j) es el $\{[(i + j - 1)(i + j - 2)]/2 + i\}$ -ésimo par generado. Este orden tiene la propiedad deseada de que hay un tiempo finito en el cual cualquier par en particular es generado.

Una MT que genera pares (i, j) en este orden en binario, es fácil de diseñar y se deja al lector dicha labor. Tal MT será llamada el *generador de pares*. Incidentalmente, el orden usado por el generador de pares demuestra que los pares de enteros pueden ponerse en correspondencia 1 a 1 con los enteros, un resultado aparentemente paradójico descubierto por *Georg Cantor* cuando él mostró que los racionales (que en realidad son la razón entre dos enteros), eran equinumerosos con los enteros.

Teorema 40 *Un lenguaje es enumerable recursivamente si y sólo si es $G(M_2)$ para alguna MT, M_2 .*

Demostración : Con el lema anterior ya probado, sólo se necesita probar cómo un lenguaje enumerable recursivamente $L = L(M_1)$ puede ser generado por una MT, M_2 . M_2 simula al generador de pares. Cuando el par (i, j) es generado, M_2 produce la i -ésima palabra w_i , en orden canónico y simula j pasos de M_1 en w_i . Si M_1 acepta en el paso j , contando la DII como paso 1, entonces M_2 genera w_i .

Es claro que M_2 genera sólo strings en L . Si $w \in L$, sea w la i -ésima palabra en el orden canónico para el alfabeto de L y suponga que M_1 acepta w en j movidas. Como toma sólo un tiempo finito para que M_2 genere cualquier string en orden canónico o simular un número determinado de movidas de M_1 , es claro que M_2 eventualmente producirá el par (i, j) . En ese momento, w será generado por M_2 . Por lo tanto, $L = G(M_2)$. □

Corolario 3 *Si L es un conjunto enumerable recursivamente, entonces hay un generador para L que enumera cada string en L exactamente una vez.*

Demostración : La MT, M_2 , descrita en la demostración del teorema 40 tiene dicha propiedad ya que genera w_i sólo cuando considera el par (i, j) , en que j es exactamente el número de pasos que M_1 toma para aceptar w_i . □

Se mostrará ahora, que los conjuntos recursivos son precisamente aquellos conjuntos cuyos strings pueden ser generados en orden canónico.

Lema 12 *Si L es recursivo, entonces hay un generador para L que imprime los strings de L en orden canónico y no imprime otras palabras.*

Demostración : Sea $L = L(M_1 \subseteq \Sigma^*$, en que M_1 se detiene en todos sus inputs. Se construye M_2 para generar L , como sigue. M_2 genera (en una cinta de borrador) las palabras en Σ^* de una a la vez y en orden canónico. Después de generar algún string w , M_2 simula M_1 en w . Si M_1 acepta w , M_2 genera w . Como M_1 para siempre, se sabe que M_2 terminará de procesar cada string después de un tiempo finito y, por lo tanto, considerará eventualmente cada string en Σ^* . Obviamente, M_2 genera L en orden canónico. □

El converso de este lema, que si L puede ser generado en orden canónico, entonces L es recursivo, es también verdadero. Sin embargo, hay un detalle que debiera quedar claro. En el lema anterior fue posible construir M_2 a partir de M_1 . Sin embargo, dada una MT, M , que genera L en orden canónico, se sabe que existe una máquina de Turing que siempre para y que reconoce L , pero no hay algoritmo para construirla.

Supóngase que M_1 genera L en orden canónico. Lo natural es construir M_2 , tal que en input w simule M_1 hasta que M_1 genere w o una palabra posterior a w en el orden canónico. En el primer caso M_2 acepta

w , en el segundo, M_2 se detiene sin aceptar w . Sin embargo, si L es finito, M_1 puede no detenerse después de generar el último string en L , con lo que M_1 podría no generar w ni ningún string posterior. En esta situación M_2 no pararía. Esto sucede sólo cuando L es finito, aún cuando se sabe que todo conjunto finito es aceptado por una MT que siempre se detiene. Infortunadamente, no se puede determinar si una MT genera un conjunto finito o, si es finito, cuál conjunto es. Por lo tanto, se sabe que una MT que siempre para y acepta L , el lenguaje que genera M_1 , siempre existe; pero no hay algoritmo para construirla.

Teorema 41 *L es recursivo si y sólo si L es generado en orden canónico.*

Demostración : El lema 12 establece una dirección. Si L es infinito, la MT M_2 , descrita más arriba, es una MT que siempre se detiene y acepta L . Si L es finito, hay un autómata finito que acepta L y, por lo tanto, hay una MT que siempre se detiene y que acepta L . En general, no es posible exhibir una MT particular que acepte L , sólo se establece que ella debe existir.

□

Chapter 8

PROPIEDADES DE LOS LENGUAJES ENUMERABLES RECURSIVAMENTE Y RECURSIVOS

8.1 Algunas Propiedades

Un número de teoremas se demuestran reduciendo un problema a otro. Estas reducciones envuelven el uso de varias MT para formar una máquina compuesta. El estado de una MT compuesta tiene una componente por cada máquina individual. Similarmente, la máquina compuesta tiene cintas separadas para cada máquina. Los detalles son tediosos y aportan poco, por lo que las construcciones se describirán en forma más bien informal.

Dado un algoritmo (MT que siempre se detiene), se puede permitir que la máquina compuesta haga una acción si el algoritmo acepta y otra si no acepta. Esto no se puede hacer si en lugar de un algoritmo se tuviera una MT arbitraria, ya que si la MT no acepta puede no detenerse y, por lo tanto, la máquina compuesta nunca iniciaría su siguiente tarea.

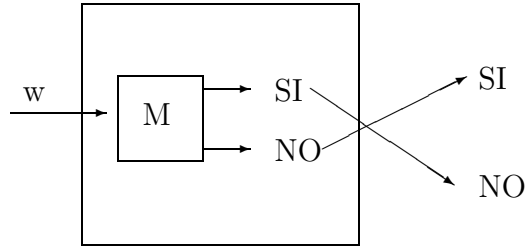
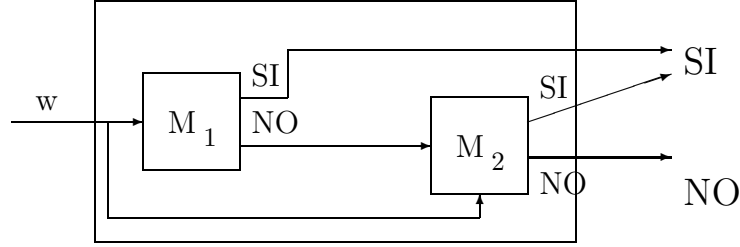
Teorema 42 *El complemento de un lenguaje recursivo es recursivo.*

Demostración : Sea L un lenguaje recursivo y M una MT que siempre se detiene y que acepta L . Se construye M' , a partir de M , de tal forma que si M entra a un estado final en input w , entonces M' se detiene sin aceptar. Si M se detiene sin aceptar, M' entra a un estado final. Ya que siempre sucede uno de estos dos eventos, M' es un algoritmo. Claramente $L(M')$ es el complemento de L y, por lo tanto, el complemento de L es un lenguaje recursivo. La Figura 8.1 ilustra la construcción de M' .

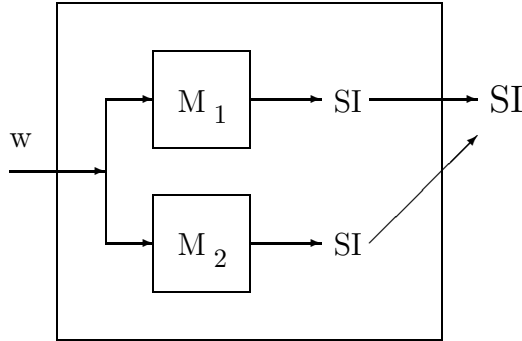
□

Teorema 43 *La unión de dos lenguajes recursivos es recursivo. La unión de dos lenguajes enumerables recursivamente es enumerable recursivamente.*

Demostración : Sean L_1 y L_2 lenguajes recursivos aceptados por los algoritmos M_1 y M_2 . Se construye M , que primero simula M_1 . Si M_1 acepta, M acepta. Si M_1 rechaza, M simula M_2 y acepta si y sólo si M_2 acepta. Ya que tanto M_1 como M_2 son algoritmos, M se detendrá. Claramente M acepta $L_1 \cup L_2$. (Ver Figura 8.2).

Figure 8.1: Construcción de M' , complemento de M Figure 8.2: Construcción de M , equivalente a la unión de dos MT, para el caso de lenguajes recursivos

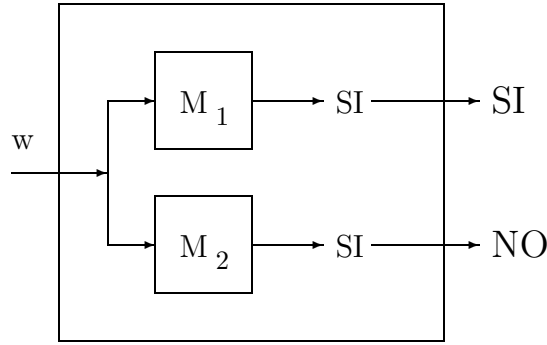
Para los lenguajes enumerables recursivamente, la construcción anterior no funciona, ya que M_1 puede no detenerse nunca. En su lugar, M puede simular simultáneamente a M_1 y M_2 en cintas separadas. Si cualquiera acepta, entonces M también acepta. (Ver Figura 8.3).

Figure 8.3: Construcción de M , equivalente a la unión de dos MT, para el caso de lenguajes enumerables recursivamente

□

Teorema 44 Si un lenguaje L y su complemento $\overline{L}$ son ambos enumerables recursivamente, entonces L y $\overline{L}$ son recursivos.

Demostración : Sean L y $\overline{L}$ aceptados por M_1 y M_2 respectivamente. Se construye M que simula simultáneamente a M_1 y M_2 . M acepta si M_1 acepta w y rechaza si M_2 acepta w . Ya que w está en L o está en $\overline{L}$, exactamente una de M_1 o M_2 lo aceptarán. Por lo tanto, M siempre dirá SI o NO, pero nunca ambas respuestas. Nótese que no hay un límite a priori en cuanto al tiempo que pasará hasta que M_1 o M_2 acepten, pero es claro que una de ellas lo hará. Como M es un algoritmo que acepta L , se concluye que L es recursivo. (Ver Figura 8.4).

Figure 8.4: Construcción de M , que simula simultáneamente a dos MT, M_1 y M_2

□

El primero y último de estos teoremas tienen una consecuencia muy importante. Sean L y $\bar{L}$ un par de lenguajes complementarios. Entonces una sola de las siguientes aserciones se cumple:

1. L y $\bar{L}$ son recursivos
2. Ni L ni $\bar{L}$ son enumerables recursivamente
3. Uno entre L y $\bar{L}$ es enumerable recursivamente, pero no recursivo; el otro no es enumerable recursivamente.

Una técnica importante para mostrar que un problema no es decidible es mostrar, por diagonalización, que el complemento del lenguaje para ese problema no es enumerable recursivamente. Por lo tanto, los casos (2) ó (3) anteriores no son aplicables. Esta técnica será esencial para probar el primer problema no-decidible. Después, varias formas de reducciones pueden emplearse para mostrar que otros problemas no son decidibles.

8.2 Máquina de Turing Universal

Ahora se usará la técnica de diagonalización para mostrar que un cierto problema no es decidible. El problema es: “¿Acepta una MT, M , un string de entrada, w ?” En este caso, tanto M como w son parámetros del problema.

Al formalizar el problema como un lenguaje, se restringirá w a ser sobre el alfabeto $\{0, 1\}$ y a que M tenga alfabeto de la cinta $\{0, 1, B\}$. Como el problema restringido es no-decidible, con toda seguridad el problema más general también lo es. Se escoge esta versión restringida para simplificar la codificación de instancias como strings.

Para comenzar, se codifican las máquinas de Turing con alfabetos restringidos como strings sobre el alfabeto $\{0, 1\}$. Sea

$$M = (Q, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$$

una máquina de Turing restringida como se desea. Además supóngase que $Q = \{q_1, q_2, \dots, q_N\}$ es el conjunto de estados, y que q_2 es el único estado final. Un teorema anterior asegura que si $L \subseteq \{0, 1\}^*$ es aceptado por una MT, entonces es aceptado por una con alfabeto $\{0, 1, B\}$. También, no hay necesidad de más de un estado final, ya que una vez que acepta puede parar.

Es conveniente llamar los símbolos $0, 1$ y B como X_1, X_2 y X_3 ; también las direcciones I y D serán llamadas D_1 y D_2 , respectivamente. Entonces una movida cualquiera $\delta(q_i, X_j) = (q_k, X_l, D_m)$ se codifica por el string binario

$$0^i 10^j 10^k 10^l 10^m$$

Un código binario para una máquina de Turing M , es

111 código₁11 código₂11...11 código _{r} 111

en que cada código _{i} es un string que codifica una movida de M y en que cada movida está codificada en alguno de los código _{i} . No es necesario que las movidas aparezcan en algún orden en particular, por lo que cada MT tiene en realidad muchos códigos. Cualquiera de esos códigos se denotará por $\langle M \rangle$.

Cada string binario representa el código de a lo más una MT; muchos strings binarios no representan MT. El par MT, w se representa por el código de M seguido por w , y se denota como $\langle M, w \rangle$.

Ejemplo 89 Sea $M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$, con movidas

$$\begin{aligned}\delta(q_1, 1) &= (q_3, 0, D) \\ \delta(q_3, 0) &= (q_1, 1, D) \\ \delta(q_3, 1) &= (q_2, 0, D) \\ \delta(q_3, B) &= (q_3, 1, I)\end{aligned}$$

Entonces el string denotado por $\langle M, 1011 \rangle$ es

111010010001010011000101010010011
0001001001010011
0001000100010010
111
1011

Note que muchos otros strings son también códigos para el par $\langle M, 1011 \rangle$ y que cualquiera de ellos es representado por la notación $\langle M, 1011 \rangle$.

□

Suponga que se tiene una lista de $\{0, 1\}^*$ en orden canónico, donde w_i es la i -ésima palabra y M_j es la MT cuyo código es el entero j escrito en binario.

Imagine una tabla infinita que indique para todo i y j si $w_i \in L(M_j)$. La Figura 8.5 sugiere cómo sería esa tabla; en ella, un 0 significa que $w_i \notin L(M_j)$ y un 1 que $w_i \in L(M_j)$. En realidad, como todas las MT de “numeración baja” aceptan el conjunto vacío, la porción mostrada de la tabla sólo debería tener ceros.

		$j \rightarrow$				
		1	2	3	4	...
1		0	1	1	0	...
2		1	1	0	0	...
3		0	0	1	0	...
4		0	1	0	1	...
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Diagonal

Figure 8.5: Construcción de tabla para diagonalización

Se construye un lenguaje L_D usando la diagonal de la tabla, para determinar si un string pertenece a L_D o no. Para garantizar que ninguna MT acepte L_D , se define que $w_i \in L_D$ si y sólo si la entrada (i, i) de la tabla es 0, esto es, si M_i no acepta w_i .

Suponga que alguna MT, M_j , acepta L_D , se produce la siguiente contradicción. Si $w_j \in L_D$, entonces la entrada (j, j) es 0 (por definición de L_D , implicando que $w_j \notin L(M_j)$ y contradiciendo $L_D = L(M_j)$. Si por el contrario, $w_j \notin L_D$, entonces la entrada (j, j) es 1, implicando que $w_j \in L(M_j)$, lo que de nuevo contradice $L_D = L(M_j)$. Como w_j está o no en L_D , se concluye que la suposición $L_D = L(M_j)$ es falsa. Por lo tanto, ninguna MT en la lista acepta L_D ; es decir ninguna MT acepta L_D .

Lema 13 L_D no es enumerable recursivamente

Demostración : Recién enunciada en la discusión anterior

□

Se define L_u , el lenguaje universal, como el conjunto

$$\{ \langle M, w \rangle / M \text{ acepta } w \}$$

Se le llama “universal”, pues la pregunta de si un string w en particular es aceptado por una máquina de Turing M en particular, es equivalente a la pregunta si $\langle M', w \rangle$ pertenece a L_u ; donde M' es la MT equivalente a M construida con una cinta semi-infinita y alfabeto $\{0, 1, B\}$ que acepte L_u .

Teorema 45 L_u es enumerable recursivamente.

Demostración : Se mostrará una MT con 3 cintas, M_1 , que acepta L_u . La primera cinta de M_1 es la cinta de entrada y es usada para buscar movidas de M cuando se le da el código $\langle M, w \rangle$ como input. La segunda cinta de M_1 simulará la cinta de M . La tercera cinta mantiene el estado de M , con q_i representado por 0^i . M_1 funciona de la siguiente manera:

1. Verifica el formato de la cinta 1 para ver que tiene un prefijo correspondiente al código de alguna MT y que no hay dos movidas codificadas que comiencen con $0^i 10^j$ para el mismo i y j . También verifica que si $0^i 10^j 10^k 10^l 10^m$ es un código, $1 \leq j \leq 3$, $1 \leq l \leq 3$, $1 \leq m \leq 2$. La tercera cinta puede usarse como “cinta borrador” para facilitar la comparación de códigos.
2. Inicializa la cinta 2 a contener w , la parte del input que sigue al segundo grupo de tres 1's consecutivos. Inicializa la cinta 3 con un solo 0, que simboliza q_1 . Las tres cabezas se posicionan en el símbolo de más a la izquierda. Esos símbolos pueden ser marcados para facilitar la vuelta de las cabezas a ellos.
3. Si la cinta 3 contiene ∞ , el código para el estado final, la máquina se detiene y acepta.
4. Sea X_j el símbolo bajo la cabeza en la segunda cinta y sea 0^i el contenido de la cinta 3. Se recorre la cinta 1 desde la izquierda hasta el segundo 111, buscando un substring que comience con $110^i 10^j 1$. Si no se encuentra, la máquina se detiene y rechaza; M no tiene próxima movida y no ha aceptado. Si se encuentra ese código, sea $0^i 10^j 10^k 10^l 10^m$. Se pone 0^k en la cinta 3, se escribe X_l en la celda de la segunda cinta y esa cabeza se mueve en dirección D_m . Nótese que ya se ha chequeado que $1 \leq l \leq 3$ y que $1 \leq m \leq 2$. Repetir después el paso (3).

Es simple ver que M_1 acepta $\langle M, w \rangle$ si y sólo si M acepta w . También es cierto que si M no se detiene en w , M_1 no se detiene en $\langle M, w \rangle$ y que si M se detiene sin aceptar w , M_1 se detiene sin aceptar $\langle M, w \rangle$.

□

La existencia de M_1 es suficiente para probar el teorema. Sin embargo, usando los teoremas del capítulo 7, se puede encontrar una MT con una cinta semi-infinita y alfabeto $\{0, 1, B\}$ que acepte L_u . Esa MT en particular se denominará M_U , la *Máquina de Turing Universal*, ya que ella hace el trabajo de cualquier MT con alfabeto de entrada $\{0, 1\}$.

Según el lema 13, el lenguaje diagonal L_D no es enumerable recursivamente y, por lo tanto, no es recursivo. Por un teorema anterior se concluye que $\overline{L_D}$ no es recursivo. Nótese que $\overline{L_D} = \{w_i / M_i \text{ acepta } w_i\}$. Se probará que el lenguaje universal $L_u = \{ \langle M, w \rangle / M \text{ acepta } w \}$ no es recursivo, reduciendo $\overline{L_D}$ a L_u . Por lo tanto L_u es un lenguaje enumerable recursivamente, pero no recursivo; en realidad, $\overline{L_D}$ es otro ejemplo de ese tipo.

Teorema 46 L_u no es recursivo.

Demostración : Supóngase que A fuera un algoritmo que reconoce L_u . Entonces se podría reconocer $\overline{L_D}$ de la siguiente manera. Dado un string $w \in (0+1)^*$, se determina (fácilmente) el valor de i , tal que $w = w_i$. Ese entero i , en binario, es el código para una MT M_i . Se alimenta a A con $\langle M_i, w_i \rangle$ y se acepta w si y sólo si M_i acepta w_i . Es fácil ver que el algoritmo así construido acepta w si y sólo si $w = w_i$ y $w_i \in L(M_i)$. Por lo tanto, se tiene un algoritmo para $\overline{L_D}$. Como dicho algoritmo no puede existir, se concluye que la suposición de que existe un algoritmo A para L_u es falsa. Por lo tanto, L_u es enumerable recursivamente, pero no recursivo. (Ver Figura 8.6).

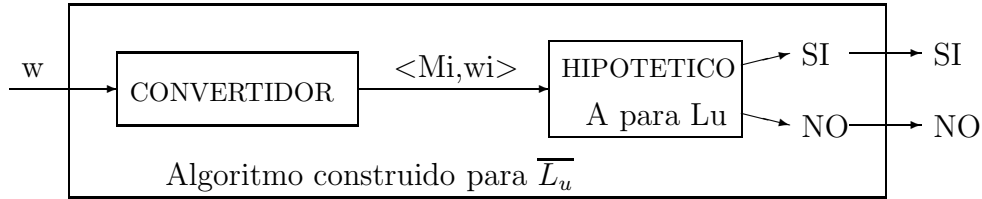


Figure 8.6: Construcción de $\overline{L_D}$

□

Chapter 9

INDECIDIBILIDAD

9.1 Problemas

Informalmente se usa la palabra *problema* para referirse a preguntas tales como: ¿Es una gramática libre de contexto dada, ambigua? En el caso del problema anterior, de la ambigüedad, una instancia del problema es una gramática en particular. En general, una *instancia* de un problema es una lista de argumentos, un argumento por cada parámetro del problema. Restringiendo la atención sólo a problemas cuya respuesta sea *SI* o *NO* y codificando instancias del problema por strings sobre un alfabeto finito, es posible transformar la pregunta de si existe un algoritmo para un problema, a saber si un lenguaje en particular es recursivo. Debe notarse que al considerar sólo problemas con respuesta *SI* o *NO*, no se está dejando de lado muchos problemas importantes, ya que muchos tienen versiones en *SI* o *NO* que son, demostrablemente, tan difíciles como el “problema general”.

Considérese el problema de la ambigüedad de las gramáticas libres de contexto. Denomínese AMB a la versión *SI* o *NO*. Una versión más general del problema, llamada *encuentre*, requiere producir un string con 2 ó más árboles de derivación, si existe, o responder “*NO*”, si no existe. Un algoritmo para *encuentre* puede usarse para resolver AMB. Si *encuentre* produce un string w , se responde *SI*; si *encuentre* responde *NO*, se responde *NO*. Por otro lado, dado un algoritmo para AMB, se puede producir un algoritmo para *encuentre*. El algoritmo primero aplica AMB a la gramática. Si AMB responde *NO*, se responde *NO*. Si AMB responde *SI*, el algoritmo comienza a generar sistemáticamente todos los strings sobre el alfabeto de G . Tan pronto como se genera un string w , se ve si tiene dos o más árboles de derivación. Nótese que el algoritmo empieza a generar strings sólo si G es ambigua, por lo tanto eventualmente encontrará el string deseado y lo escribirá. Por lo tanto, en realidad se tiene un algoritmo. La parte del algoritmo que chequea si w tiene 2 ó más árboles de derivación se deja como ejercicio.

El proceso por el cual se construye un algoritmo para un problema (como *encuentre*), usando un supuesto algoritmo para otro (AMB), es llamado una *reducción* (de *encuentre* a AMB). En general, cuando un problema A se reduce a un problema B , se está mostrando que B es al menos tan “difícil” como A . Por lo tanto en este caso, como en muchos otros, el problema *SI* o *NO* AMB no es más sencillo (fácil) que la versión más general del problema. Posteriormente se verá que no hay algoritmo para AMB. Por la reducción de AMB a *encuentre*, se concluye que tampoco hay un algoritmo para *encuentre*, ya que su existencia implicaría la existencia de un algoritmo para AMB, una contradicción.

Un punto instructivo adicional concierne a la codificación de la gramática G . Como todas las MT tienen un alfabeto fijo, no se puede considerar la notación de cuádrupla $G = (V, T, P, S)$ como la codificación de G sin modificarla. Pero es posible codificar cuádruplas como strings binarios. Los metasímbolos $(,), \{, \}, , , \rightarrow$ se codifican como 1, 10, 100, $\dots$, 10^5 , respectivamente. El i -ésimo símbolo de la gramática (en cualquier orden elegido), se codifica como 10^{i+5} . Con esta codificación no se distinguen los terminales ni los no-terminales. Por supuesto que renombrar los no-terminales no afecta el lenguaje generado, por lo que sus símbolos no son importantes. Aún cuando se piensa que la identidad de los terminales es importante, para

este problema los símbolos son irrelevantes ya que el renombrar terminales no afecta la ambigüedad de una gramática.

Un problema cuyo lenguaje es recursivo, se dice *decidible*, en otro caso el problema es *no-decidible*. Esto es, un problema es no-decidible si no hay un algoritmo que tome como input una instancia del problema y determine si la respuesta a esa instancia es *SI* o *NO*.

Una consecuencia poco intuitiva de la definición de no-decidible es que problemas con sólo una instancia son trivialmente decidibles. Considérese el siguiente problema basado en la *conjetura de Fermat*. ¿Hay solución entre los enteros positivos a la ecuación $x^i + y^i = z^i$, si $i \geq 3$? Nótese que x, y, z e i no son parámetros, sino que variables internas del problema. Hay una MT que acepta todo input y otra que los rechaza todos. Una de ellas responde correctamente a la *conjetura de Fermat*, aún cuando no se sabe cuál. De hecho, puede ni siquiera haber una resolución de la conjetura usando los axiomas de la aritmética. Esto es, la conjetura puede ser cierta y aún así puede que no haya una demostración aritmética de ella. La posibilidad de esto, aunque no en certeza, sigue del teorema de *Incompletitud de Gödel*, que establece que cualquier sistema formal consistente y suficientemente poderoso para describir teoría de números, debe tener sentencias verdaderas pero no demostrables dentro del sistema.

No debiera molestar que un problema como la *conjetura de Fermat* sea decidible. La teoría de no-decidibilidad concierne a la existencia o no existencia de algoritmos para resolver problemas con una infinidad de instancias.

9.2 Otros Problemas No Decidibles

Se tiene ahora un ejemplo de un lenguaje enumerable recursivamente que no es recursivo. El problema asociado a ese lenguaje, ¿Acepta M a w ?, es no decidible y se puede usar para mostrar que otros problemas son no decidibles.

Ejemplo 90 Considérese el problema: ¿Es $L(M) \neq \phi$? Sea $\langle M \rangle$ una codificación para M . Se define

$$L_{NV} = \{ \langle M \rangle / L(M) \neq \phi \}$$

$$L_V = \{ \langle M \rangle / L(M) = \phi \}$$

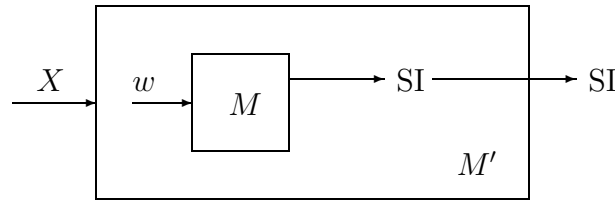
Nótese que L_V y L_{NV} son uno el complemento del otro, ya que cada string binario representa alguna MT; aquellos mal formados, denotan una MT sin movidas. Todos estos strings están en L_V . Se mostrará que L_{NV} es enumerable recursivamente, pero no recursivo y que L_V no es enumerable recursivamente.

Se muestra que L_{NV} es enumerable recursivamente, construyendo una MT, M , que reconoce códigos de MT's que aceptan conjuntos no vacíos. Dado un input $\langle M_i \rangle$, M en forma no determinística adivina un string X aceptado por M_i y verifica que M_i lo acepte, simulando M_i en input X . Este paso también puede ser ejecutado en forma determinística, usando el generador de pares. Para el par (j, k) , se simula M_i en el j -ésimo string durante k pasos. Si M_i acepta, M acepta $\langle M_i \rangle$.

Ahora se debe mostrar que L_V no es recursivo. Supóngase que sí lo fuera, entonces se podría construir un algoritmo para L_u . Sea A un algoritmo hipotético que acepta L_V . Hay un algoritmo B que, dado $\langle M, w \rangle$, construye una MT M' que acepta ϕ si M no acepta w y que acepta $\{0, 1\}^*$ si M acepta w . La idea se muestra en la Figura 9.1. M' ignora su entrada X y simula M en entrada w , aceptando si M acepta.

Note que M' no es B . Más bien, B es como un compilador que toma $\langle M, w \rangle$ como programa fuente y produce M' como programa objeto. Se ha descrito qué hace B , pero no cómo lo hace. La construcción es simple, toma $\langle M, w \rangle$ y separa w . Sea $w = a_1 a_2 \dots a_N$. B crea $N + 3$ estados $q_1, q_2, \dots, q_{N+3}$, con movidas

$$\begin{aligned} \delta(q_1, X) &= (q_2, \$, D) && \text{para todo } X && \text{(marca)} \\ \delta(q_i, X) &= (q_{i+1}, a_{i-1}, D) && \text{para todo } X \text{ y } && \text{(escribe } w) \\ &&& 2 \leq i \leq N+1 && \\ \delta(q_{N+2}, X) &= (q_{N+2}, B, D) && \text{para } X \neq B && \text{(borra cinta)} \\ \delta(q_{N+2}, B) &= (q_{N+3}, B, I) && && \\ \delta(q_{N+3}, X) &= (q_{N+3}, X, I) && \text{para } X \neq \$ && \text{(busca marca)} \end{aligned}$$

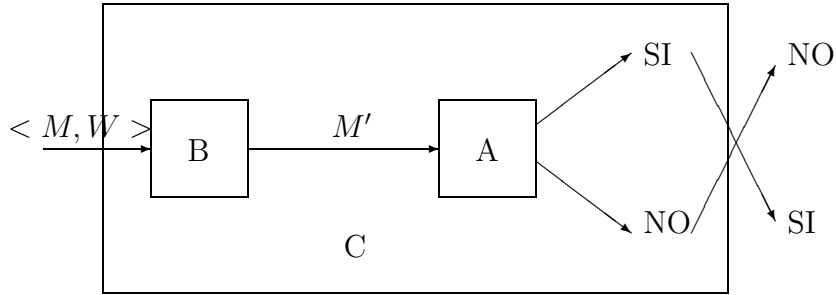
Figure 9.1: Construcción de M' , correspondiente al problema: ¿Es $L(M) \neq \phi$?

Habiendo producido el código para estas movidas, B agrega $N + 3$ a los índices de los estados de M e incluye la movida

$$\delta(q_{N+3}, \$) = (q_{N+4}, \$, D) \quad (\text{hace partir a } M)$$

y todas las de M en la MT que genera. La MT resultante tiene un símbolo extra, $\$$, pero por teorema del capítulo 7, se puede construir M' con alfabeto de cinta $\{0, 1, B\}$ y con seguridad se puede hacer que q_2 sea el estado de aceptación. Esto completa el algoritmo B y su salida es la máquina M' deseada.

Supóngase ahora que existe un algoritmo A que acepta L_V . Entonces se construye un algoritmo C para L_u como se indica en la Figura 9.2.

Figure 9.2: Construcción del algoritmo C

Si M acepta w , entonces $L(M') \neq \phi$; es decir, A dice NO y C dice SI . Si M no acepta w , entonces $L(M') = \phi$; A dice SI y C dice NO . Como C no puede existir, A no puede existir. Por lo tanto, L_V no es recursivo.

Si L_{NV} fuera recursivo, L_V también lo sería pues es su complemento. Por lo tanto L_{NV} es enumerable recursivamente pero no recursivo. Si L_V fuera enumerable recursivamente, L_V y L_{NV} serían recursivos. Por lo tanto L_V no es enumerable recursivamente.

□

Ejemplo 91 Considere los lenguajes

$$L_R = \{ \langle M \rangle / L(M) \text{ es recursivo} \}$$

$$L_{NR} = \{ \langle M \rangle / L(M) \text{ no es recursivo} \}.$$

Nótese que L_R no es $\{ \langle M \rangle / M \text{ siempre se detiene} \}$, aún cuando incluye a este último. Una MT M puede aceptar un lenguaje recursivo aunque puede que M no pare para algunos strings que no están en $L(M)$; alguna otra MT equivalente a M debe siempre detenerse. Se probará que ni L_R ni L_{NR} son enumerables recursivamente.

Suponga que L_R fuese enumerable recursivamente. Entonces se puede construir una MT para $\overline{L_u}$, que se sabe no puede existir. Sea M_R una MT que acepta L_R . Se puede construir un algoritmo A que tome $\langle M, w \rangle$ como input y produzca como output una MT M' , tal que

$$L(M') = \begin{cases} \phi & \text{si } M \text{ no acepta } w \\ L_u & \text{si } M \text{ acepta } w \end{cases}$$

Note que L_u no es recursivo, así que M' acepta un lenguaje recursivo si y sólo si M no acepta w . El plan para M' se indica en la Figura 9.3.

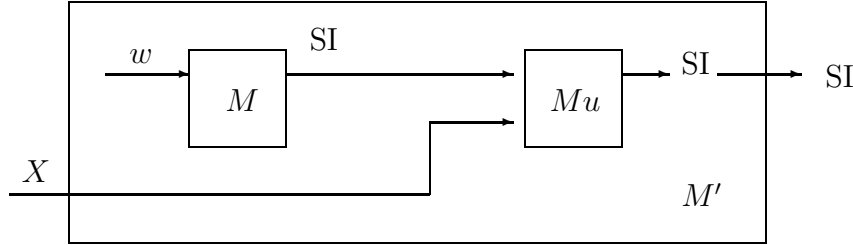


Figure 9.3: Construcción de M'

Como en el ejemplo anterior, se ha descrito el output de A . Se deja su construcción como ejercicio. Dado A y M_R se puede construir una MT que acepta $\overline{L_u}$. (Ver Figura 9.4).

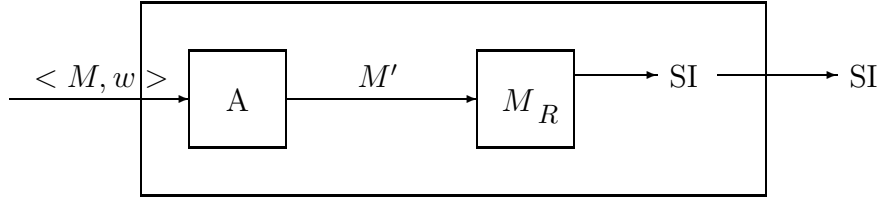


Figure 9.4: Construcción de una MT que acepta $\overline{L_u}$

En input $\langle M, w \rangle$ la MT usa A para producir M' , y usa M_R para determinar si el conjunto aceptado por M' es recursivo. Acepta si y sólo si $L(M')$ es recursivo, pero $L(M')$ es recursivo si y sólo si $L(M') = \phi$, lo que significa que M no acepta w . Por lo tanto acepta $\langle M, w \rangle$ si y sólo si $\langle M, w \rangle \in \overline{L_u}$.

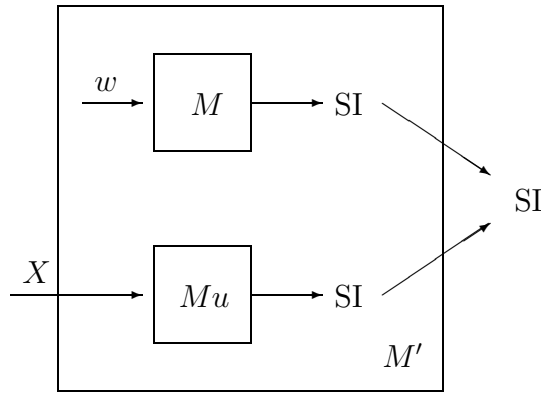
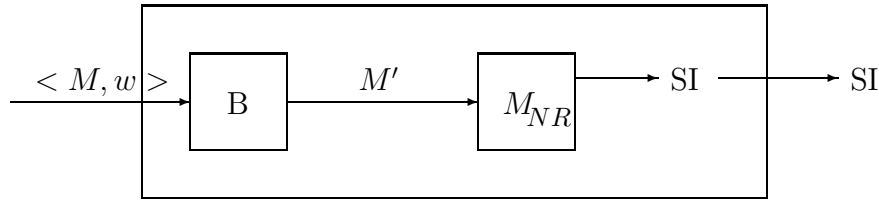
Se estudia ahora L_{NR} . Suponga que se tiene una MT, M_{NR} , que acepta L_{NR} . Se puede usar M_{NR} y un algoritmo B a ser construido por el lector, que acepta $\overline{L_u}$. B toma $\langle M, w \rangle$ como entrada y produce una MT M' (ver Figura 9.5), tal que

$$L(M') = \begin{cases} \Sigma^* & \text{si } M \text{ acepta } w \\ L_u & \text{si } M \text{ no acepta } w \end{cases}$$

Por lo tanto M' acepta un lenguaje recursivo si y sólo si M acepta w . Dados B y M_{NR} , la Figura 9.6 siguiente es una MT que acepta $\overline{L_u}$:

La MT acepta $\langle M, w \rangle$ si y sólo si $L(M')$ no es recursivo, o equivalentemente, si y sólo si M no acepta w . Esto es, la MT acepta $\langle M, w \rangle$ si y sólo si $\langle M, w \rangle \in \overline{L_u}$. Como ya se ha mostrado que no existe tal MT, se concluye que la suposición de que M_{NR} existe es falsa y, por lo tanto, L_{NR} no es enumerable recursivamente.

□

Figure 9.5: Construcción de M' Figure 9.6: Máquina de Turing que acepta $\overline{L_u}$

Los ejemplos anteriores muestran que no es decidible si el conjunto aceptado por una MT es vacío o recursivo. La técnica usada en las demostraciones se puede usar para probar que no se puede decidir si el conjunto aceptado es finito, infinito, regular, libre de contexto, tiene un número par de strings o satisface muchos otros predicados.

¿Qué puede ser decidido entonces sobre los conjuntos aceptados por una máquina de Turing? Sólo los predicados triviales, tales como ¿Acepta una MT un lenguaje enumerable recursivamente? que son verdaderos para todas las MT o falsos para todas ellas.

En lo que sigue se discutirán lenguajes que representan propiedades de los lenguajes enumerables recursivamente. Esto es, los lenguajes son conjuntos de códigos de MT tales que la pertenencia de $\langle M \rangle$ en el lenguaje depende sólo de $L(M)$ y no de M misma. Más adelante se considerarán lenguajes de códigos de MT que dependen de la MT misma, como “ M tiene 27 estados”, que pueden ser satisfechos para algunas, pero no todas las MT que aceptan un lenguaje dado.

Sea $\mathfrak{S}$ un conjunto de lenguajes enumerables recursivamente, cada uno sobre $\{0, 1\}$. $\mathfrak{S}$ es una propiedad de los lenguajes enumerables recursivamente. Un conjunto L tiene la propiedad $\mathfrak{S}$, si $L \in \mathfrak{S}$. Por ejemplo, la propiedad de ser infinito es $\{L/L \text{ es infinito}\}$. $\mathfrak{S}$ es una propiedad trivial si es vacío o consiste de todos los lenguajes enumerables recursivamente. Sea $L_{\mathfrak{S}}$ el conjunto $\{\langle M \rangle / L(M) \in im\}$.

Teorema 47 (Teorema de Rice) *Cualquier propiedad no trivial $\mathfrak{S}$ de los lenguajes enumerables recursivamente no es decidible.*

Demostración : Sin perder generalidad se asume que $\phi \notin \mathfrak{S}$ (si no, considérese $\overline{\mathfrak{S}}$). Como $\mathfrak{S}$ es no trivial, existe L con propiedad $\mathfrak{S}$. Sea M_L una MT que acepta L . Suponga que $\mathfrak{S}$ fuera decidible. Entonces existe un algoritmo $M_{\mathfrak{S}}$ que acepta $L_{\mathfrak{S}}$. Se usa M_L y $M_{\mathfrak{S}}$ para construir un algoritmo para L_u . Primero se construye un algoritmo A que toma $\langle M, w \rangle$ y produce $\langle M' \rangle$, en que $L(M') \in \mathfrak{S}$ si y sólo si M acepta w ($\langle M, w \rangle \in L_u$). (Ver Figura 9.7).

Primero M' ignora su input y simula M en w . Si M no acepta w , M' no acepta X . Si M acepta w , M' simula M_L en X y acepta X si y sólo si M_L acepta X . Luego M' acepta ϕ o L , dependiendo de si M acepta

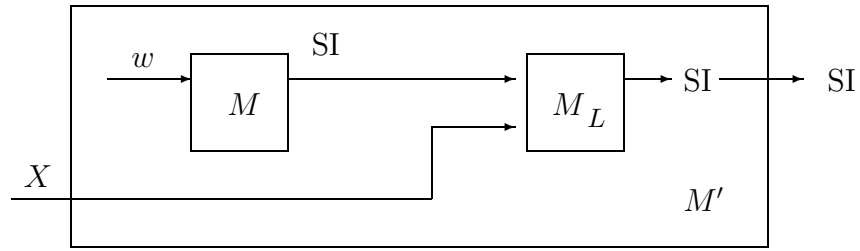


Figure 9.7: construcción de M' , correspondiente a la demostración del Teorema de Rice

w .

Se puede usar el algoritmo hipotético $M_{\mathfrak{S}}$ para determinar si $L(M') \in \mathfrak{S}$. Como $L(M') \in \mathfrak{S}$ si y sólo si $\langle M, w \rangle \in L_u$, se tiene un algoritmo que reconoce L_u , una contradicción. Por lo tanto, $\mathfrak{S}$ debe ser no decidable. Note cómo esta demostración generaliza el ejemplo 91.

□

Este teorema tiene varias consecuencias, algunas de las cuales se resumen en el siguiente corolario:

Corolario 4 *Las siguientes propiedades de los conjuntos enumerables recursivamente no son decidibles:*

1. *Ser vacío*
2. *Ser finito*
3. *Ser regular*
4. *Ser libre de contexto*

¿Implica el teorema anterior que cualquier cosa sobre las MT es no decidable? La respuesta es *NO*. Este teorema sólo tiene que ver con propiedades de los lenguajes aceptados, no con propiedades de las máquinas de Turing mismas. Por ejemplo, el problema: ¿Tiene una MT dada un número par de estados?, es claramente decidable. Al tratar propiedades de las MT mismas se debe usar el ingenio.